

سلام



دانشکده‌گان علوم
دانشکده ریاضی، آمار و علوم کامپیوتر

صوری سازی ترجمه حذف وابستگی در سیستم‌های نوع محض لایه‌ای

نگارش
حسین مددی

استاد راهنما
مجید علی زاده

پایان نامه برای دریافت درجه کارشناسی ارشد
در رشته علوم کامپیوتر

تابستان ۱۴۰۵

تعهدنامه اصالت اثر

اینجانب حسین مددی متعهد می شوم که مطالب مندرج در این پایان نامه/رساله حاصل کار پژوهشی اینجانب است و به دستاوردهای پژوهشی دیگران که در این پژوهش از آنها استفاده شده است، مطابق مقررات ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان نامه/رساله قبلاً برای احراز هیچ مدرک هم سطح یا بالاتر ارائه نشده است. در صورت اثبات تخلف (در هر زمان) مدرک تحصیلی صادر شده توسط دانشگاه از اعتبار ساقط خواهد شد. کلیه حقوق مادی و معنوی این اثر متعلق به دانشکده ریاضی، آمار و علوم کامپیوتر دانشکدگان علوم دانشگاه تهران می باشد.

حسین مددی



حق مالکیت معنوی

حق مالکیت معنوی این اثر متعلق به دانشگاه تهران می‌باشد. استفاده از مطالب این پایان‌نامه در فعالیتهای تحقیقاتی با ذکر منبع بلامانع می‌باشد. در صورت استفاده تجاری، مانند چاپ این پایان‌نامه، هماهنگی لازم و اجازه کتبی از دانشگاه و نگارنده پایان‌نامه الزامی می‌باشد.

چکیده

در این پایان نامه، ترجمه حذف وابستگی معرفی شده توسط Nathan Mull برای سیستم‌های نوع محض لایه‌ای [۱۰] به طور کامل در اثبات‌یار Coq صوری‌سازی و اثبات ماشینی شد. توابع ترجمه به همراه تمام لم‌های واسطه شامل لم جابه‌جایی جانشینی، لم حفظ مسیرهای کاهش و لم حفظ نوع، به صورت ماشینی اثبات گردیدند. قضیه اصلی Mull مبنی بر اینکه در تمام سیستم‌های نوع محض لایه‌ای با مجموعه قوانینی خاص، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی است، برای اولین بار به صورت کاملاً ماشینی بررسی شد. کتابخانه‌ای باز و قابل گسترش تولید شد که بستری مناسب برای تعمیم این ترجمه به سیستم‌های وابسته‌تر مانند حساب ساختمان‌ها فراهم می‌آورد. نتایج این پژوهش گامی مهم در جهت پیشرفت اثبات حدس BGK^۱ [۷] در سیستم‌های وابسته به شمار می‌رود.

کلمات کلیدی: نظریه انواع، سیستم‌های نوع محض لایه‌ای، ترجمه حذف وابستگی، نرمال‌سازی قوی، اثبات ماشینی، Coq، حدس BGK

پیشگفتار

نظریه انواع یکی از مهم‌ترین شاخه‌های علوم کامپیوتر نظری و منطق ریاضی در قرن بیستم به شمار می‌رود. این نظریه نه تنها پایه و اساس زبان‌های برنامه‌نویسی تابعی پیشرفته مانند Haskell، OCaml و Agda را تشکیل می‌دهد، بلکه هسته اصلی اثبات‌یارهای مطرحی نظیر Coq و Lean و نیز بر مبنای آن طراحی شده است.

یکی از مسائل کلاسیک و همچنان باز در این حوزه، رابطه میان نرمال‌سازی ضعیف و نرمال‌سازی قوی در سیستم‌های نوع وابسته است. حدس معروف BGK که بیش از سه دهه پیش مطرح شد، بیان می‌کند که در هر سیستم نوع محض، اگر نرمال‌سازی ضعیف برقرار باشد، آنگاه نرمال‌سازی قوی نیز برقرار خواهد بود [۷]. این حدس برای سیستم‌های غیروابسته از دهه ۱۹۹۰ اثبات شده است [۸]، اما در سیستم‌های وابسته مانند حساب ساختمان‌ها^۱ همچنان یک مسئله باز باقی مانده است.

Nathan Mull در سال ۲۰۲۳، در رساله دکتری خود در دانشگاه شیکاگو [۱۰]، خانواده جدیدی از سیستم‌های نوع به نام «سیستم‌های نوع محض لایه‌ای^۲» را معرفی کرد و با ارائه ترجمه‌ای هوشمندانه که وابستگی‌های غیرضروری را حذف می‌کند، توانست نشان دهد که در خانواده‌ای از این سیستم‌ها، حدس BGK برقرار است. این نتیجه یکی از بزرگ‌ترین پیشرفت‌های اخیر در مسیر اثبات این حدس به شمار می‌رود.

این پایان‌نامه به صورتی سازی کامل اثبات Mull در اثبات‌یار Coq اختصاص دارد. هدف، تبدیل این اثبات نظری قابل توجه، به یک اثبات ماشینی قابل اتکا و استفاده مجدد است که می‌توان از آن برای تعمیم به سیستم‌های پیچیده‌تر نیز استفاده کرد.

محتوا پایان‌نامه در پنج فصل تنظیم شده است؛ فصل اول مفاهیم و تعاریف اولیه را بیان می‌کند، فصل دوم سیستم‌های نوع محض لایه‌ای را معرفی می‌کند، فصل سوم ترجمه حذف وابستگی را تعریف و لم‌های آنان به همراه قضیه اصلی را بیان و اثبات می‌کند و فصل چهارم به جمع‌بندی و پیشنهادهایی برای پژوهش‌های آینده اختصاص دارد. تمامی کدهای توسعه داده‌شده به منظور صورتی سازی، در بخش پیوست موجود است.

^۱ هسته اثبات‌یار Coq

^۲ Tiered Pure Type Systems

فهرست مطالب

یک	چکیده
یک	پیشگفتار
چهار	فهرست مطالب
پنج	فهرست شکل‌ها
شش	فهرست جداول

اول مفاهیم و تعاریف مقدماتی هفت

۱	۱ مفاهیم مقدماتی
۱	۱.۱ مقدمه
۲	۲.۱ حساب لامبدا بدون نوع
۲	۱.۲.۱ نحو
۲	۲.۲.۱ متغیرهای آزاد و بسته
۳	۳.۲.۱ جانشینی
۴	۴.۲.۱ تبدیل α
۴	۵.۲.۱ کاهش β
۵	۶.۲.۱ مسیرهای کاهش و فرم نرمال
۶	۷.۲.۱ نرمال‌سازی ضعیف و قوی
۷	۳.۱ سیستم‌های کاهش انتزاعی
۷	۱.۳.۱ تعریف سیستم کاهش انتزاعی
۷	۲.۳.۱ دنباله و بستار کاهش
۷	۳.۳.۱ فرم نرمال و نرمال‌سازی

۸	 همگرایی	۴.۳.۱
۹	 حساب لامبدا نوع دار ساده	۴.۱
۹	 نحو	۱.۴.۱
۱۰	 زمینه و قضاوت نوع	۲.۴.۱
۱۱	 خواص	۳.۴.۱
۱۲	 سیستم‌های نوع محض	۵.۱
۱۲	 مشخصه و نحو	۱.۵.۱
۱۳	 قضاوت نوع	۲.۵.۱
۱۴	 مکعب لامبدا	۳.۵.۱
۱۶	 اهمیت	۴.۵.۱
۱۶	 خواص فرانظری سیستم‌های نوع محض	۵.۵.۱

دوم سیستم‌های نوع محض لایه‌ای و ترجمه حذف وابستگی ۱۹

۲۰	سیستم‌های نوع محض لایه‌ای	۲
۲۰	 انگیزه و ایده اصلی	۱.۲
۲۰	 انگیزه	۱.۱.۲
۲۰	 ایده اصلی	۲.۱.۲
۲۱	 سیستم‌های نوع محض لایه‌ای	۲.۲
۲۱	 لایه‌ها و شهود آن‌ها	۱.۲.۲
۲۲	 تعریف رسمی سیستم	۲.۲.۲
۲۳	 خواص	۳.۲.۲
۲۷	 گسترش سیستم	۴.۲.۲
۲۸	 درجه	۵.۲.۲

۳۰	ترجمه حذف وابستگی	۳
۳۰	 مقدمه	۱.۳
۳۱	 هدف	۲.۳
۳۳	 ترجمه سطح نوع	۳.۳
۴۳	 ترجمه سطح عبارت	۴.۳
۴۸	 نتیجه اصلی	۵.۳

۵۰ سوم نتایج و پیشنهادات آینده

۴ نتیجه‌گیری و جمع‌بندی ۵۱

۵۱ ۱.۴ جمع‌بندی

۵۱ ۲.۴ پیشنهادات

۵۴ مراجع

۵۵ پیوست: کدهای COQ

۵۵ مشخصه سیستم‌های نوع محض

۵۵ مشخصه سیستم‌های نوع محض لایه‌ای

۵۵ سیستم نوع محض و لم‌ها

۱۳۲ سیستم نوع محض لایه‌ای و ترجمه حذف وابستگی

فهرست تصاویر

۱.۱	مکعب لامبدا	۱۵
-----	-------------	----

فهرست جداول

۱۶	۱.۱ جدول بیان سیستم‌های نوع در مکعب لامبدا به کمک سیستم‌های نوع محض
----	---

بخش اول

مفاهیم و تعاریف مقدماتی

فصل ۱

مفاهیم مقدماتی

۱.۱ مقدمه

در این فصل، مفاهیم و تعاریف پایه‌ای مورد نیاز برای درک مباحث اصلی پایان‌نامه معرفی می‌شوند. این مفاهیم شامل حساب لامبدا بدون نوع، سیستم‌های کاهش انتزاعی، حساب لامبدا نوع‌دار ساده و سیستم‌های نوع محض هستند که به ترتیب از ساده به پیچیده ارائه می‌گردند. هدف از این فصل، ایجاد یک زبان مشترک و نمادگذاری واحد برای خواننده است تا زمینه لازم برای مطالعه فصل‌های بعدی، که به معرفی سیستم‌های نوع محض لایه‌ای، ترجمه حذف وابستگی و صوری‌سازی آن‌ها در اثبات‌یار Coq اختصاص دارد، فراهم آید.

فصل با بررسی حساب لامبدا بدون نوع آغاز می‌شود که پایه تاریخی و مفهومی تمام سیستم‌های بعدی را تشکیل می‌دهد. سپس چارچوب کلی سیستم‌های کاهش انتزاعی معرفی می‌گردد که ابزار ریاضی برای مطالعه دقیق خواص نرمال‌سازی و همگرایی را فراهم می‌کند. این چارچوب به دلیل عمومیت خود، امکان بیان یکپارچه نتایج مربوط به نرمال‌سازی ضعیف، قوی و همگرایی را فراهم می‌کند. در ادامه، حساب لامبدا نوع‌دار ساده به عنوان نخستین سیستم نوع‌دار وابسته بررسی می‌شود. این سیستم نه تنها مقدمه‌ای طبیعی برای ورود به سیستم‌های پیچیده‌تر است، بلکه نمونه‌ای کلاسیک از چگونگی جلوگیری انواع از حلقه‌های محاسباتی را نشان می‌دهد. در نهایت، به سیستم‌های نوع محض می‌پردازیم که چارچوب کلی و یکپارچه‌ای را برای بیان طیف وسیعی از سیستم‌های نوع، از حساب لامبدا نوع‌دار ساده تا حساب ساختمان‌ها، ارائه می‌کنند.

اثبات‌های تفصیلی بسیاری از قضایا و لم‌های این فصل به منابع استاندارد مانند [۲] و [۷] ارجاع داده شده و تنها صورت تعاریف و قضایای کلیدی همراه با توضیحات ضروری ارائه می‌گردد. این رویکرد از تکرار مطالب جلوگیری می‌کند و خواننده را به مطالعه منابع اصلی ترغیب می‌نماید.

با پایان این فصل، خواننده با ابزارهای نظری لازم برای درک سیستم‌های نوع محض لایه‌ای و ترجمه حذف وابستگی آشنا خواهد شد و آماده ورود به مباحث تخصصی فصل‌های بعدی می‌شود.

۲.۱ حساب لامبدا بدون نوع

حساب لامبدا بدون نوع که نخستین بار توسط Alonzo Church در [۴] معرفی شد، پایه تاریخی و مفهومی تمام سیستم‌های محاسباتی تابعی و نظریه انواع مدرن به شمار می‌رود. این دستگاه، محاسبه را به عنوان کاربرد توابع بر آرگومان‌ها مدل می‌کند و قدرت بیان معادلی با ماشین تورینگ دارد. این حساب، با وجود سادگی نحوی، مسائل عمیقی مانند حلقه‌های نامتناهی و تصمیم‌ناپذیری را نشان می‌دهد؛ که این موارد در سیستم‌های نوع‌دار با استفاده از انواع کنترل می‌شوند.

۱.۲.۱ نحو

تعریف ۱.۱ (نحو عبارات در حساب لامبدا بدون نوع). مجموعه عبارات حساب لامبدا بدون نوع، که معمولاً با Λ نشان داده می‌شود، به صورت بازگشتی زیر تعریف می‌گردد

$$M ::= V \mid \lambda V.M \mid MM$$

که در آن

- $V = \{x, y, z, \dots\}$ مجموعه متغیرهاست.
- $\lambda V.M$ یک انتزاع و بیانگر تعریف یک تابع است.
- MM یک کاربرد و بیانگر صدا زدن یک تابع است.

۲.۲.۱ متغیرهای آزاد و بسته

برای درک دقیق مفاهیم جانشینی و کاهش، معرفی متغیرهای آزاد و بسته ضروری است.

تعریف ۲.۱ (متغیرهای آزاد یک عبارت). برای یک عبارت M ، مجموعه متغیرهای آزاد

آن که با $FV(M)$ نشان داده می‌شود، به صورت بازگشتی زیر تعریف می‌گردد

$$FV(x) = \{x\}$$

$$FV(\lambda x.M) = FV(M) - \{x\}$$

$$FV(MN) = FV(M) \cup FV(N)$$

گزاره ۳.۰.۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $x \in FV(M)$

- متغیر x در عبارت M آزاد است.

- متغیر x در عبارت M دارای رخداد آزاد است.

گزاره ۴.۰.۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $x \notin FV(M)$

- متغیر x در عبارت M بسته است.

- متغیر x در عبارت M دارای رخداد بسته است.

گزاره ۵.۰.۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $FV(M) = \emptyset$

- عبارت M بسته است.

- عبارت M یک ترکیب‌گر است.

۳.۲.۱ جانشینی

جانشینی با نماد $M[N/x]$ نشان داده می‌شود و معنای آن جایگزینی همه رخدادهاى آزاد x در M با عبارت N است.

تذکره ۶.۰.۱. در برخی منابع مانند [۲]، جانشینی با نماد

$$M[x := N]$$

نشان داده می‌شود. ما در این پایان‌نامه از نماد متداول دیگری استفاده می‌کنیم که $Mull$ نیز در تر دکتري خود استفاده کرده است، یعنی

$$M[N/x]$$

تعریف ۷.۱. جانشینی به صورت بازگشتی زیر تعریف می شود

$$\begin{aligned}
 x[N/x] &= N \\
 y[N/x] &= y & (x \neq y) \\
 PQ[N/x] &= (P[N/x])(Q[N/x]) \\
 (\lambda x.M)[N/x] &= \lambda x.M \\
 (\lambda y.M)[N/x] &= \lambda y.(M[N/x]) & (x \neq y)
 \end{aligned}$$

۴.۲.۱ تبدیل α

تبدیل α به ما اجازه می دهد نام متغیرهای بسته را، بدون تغییر معنای عبارت، تغییر دهیم.

تعریف ۸.۱. اگر M یک عبارت باشد و $x \notin FV(M)$ ، آنگاه تبدیل α به صورت زیر بیان می شود

$$\lambda x.M \equiv_{\alpha} \lambda y.M[y/x]$$

۵.۲.۱ کاهش β

تعریف ۹.۱. کاهش β قاعده اصلی محاسبه در حساب لامبدا است و به صورت زیر تعریف می شود

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x]$$

که بستار بازتاب-تعدی آن با نماد $\twoheadrightarrow_{\beta}$ نشان داده می شود.

مثال

.۱

$$\begin{aligned}
 \lambda x.(\lambda xy.y)x &\equiv_{\alpha} \lambda x.(\lambda ry.y)x \\
 &\rightarrow_{\beta} \lambda x.(\lambda y.y)[x/r] \\
 &= \lambda x.(\lambda y.y)
 \end{aligned}$$

۲. اگر $M \equiv \lambda x.xx$ ، آنگاه

$$\begin{aligned} MM &\equiv (\lambda x.xx)M \\ &\rightarrow_{\beta} xx[M/x] \\ &= MM \\ &\equiv (\lambda x.xx)M \\ &\rightarrow_{\beta} xx[M/x] \\ &= \dots \end{aligned}$$

پس داریم

$$MM \rightarrow_{\beta} MM \rightarrow_{\beta} MM \rightarrow_{\beta} \dots$$

همانطور که در مثال ۲ مشاهده می‌کنید، دنباله کاهش عبارت MM نامتناهی است. این مثال ما را به مفاهیم بعدی سوق می‌دهد.

۶.۲.۱ مسیرهای کاهش و فرم نرمال

تعریف ۱۰.۱. یک مسیر کاهش، دنباله‌ای به شکل زیر است که می‌تواند متناهی یا نامتناهی باشد.

$$M_0 \rightarrow_{\beta} M_1 \rightarrow_{\beta} \dots \rightarrow_{\beta} M_n$$

تعریف ۱۱.۱. عبارت M در فرم نرمال (یا یک فرم نرمال) است، اگر هیچ گامی از کاهش β بر آن قابل اعمال نباشد. یعنی

$$M \in \text{NF} \iff \nexists N.M \rightarrow_{\beta} N$$

مثال

عبارات زیر در فرم نرمال هستند:

$$\begin{aligned} x \\ x(\lambda a.a) \\ \lambda x.x \\ \lambda y.x \end{aligned}$$

عبارات زیر در فرم نرمال نیستند:

$$(\lambda x.x)x$$

$$(\lambda y.xy)(\lambda z.r)$$

زیرا به ترتیب داریم:

$$(\lambda x.x)x \equiv_{\alpha} (\lambda y.y)x \rightarrow_{\beta} y[x/y] = x$$

$$(\lambda y.xy)(\lambda z.r) \rightarrow_{\beta} xy[(\lambda z.r)/y] = x(\lambda z.r)$$

۷.۲.۱ نرمال سازی ضعیف و قوی

تعریف ۱۲.۱ (نرمال سازی ضعیف). عبارت M دارای خاصیت نرمال سازی ضعیف است، اگر حداقل یک مسیر کاهش به یک فرم نرمال داشته باشد. یعنی

$$\exists N \in \text{NF}. M \twoheadrightarrow_{\beta} N$$

تعریف ۱۳.۱ (نرمال سازی قوی). عبارت M دارای خاصیت نرمال سازی قوی است، اگر هیچ مسیر کاهش نامتناهی ای از آن آغاز نشود.

قضیه ۱۴.۱. اگر یک عبارت دارای خاصیت نرمال سازی قوی باشد، آنگاه دارای خاصیت نرمال سازی ضعیف نیز است.

عکس قضیه فوق همان حدس BGK است که برای سیستم های وابسته، هنوز اثبات نشده است.

حدس ۱۵.۱ (BGK). اگر یک عبارت دارای خاصیت نرمال سازی ضعیف باشد، آنگاه دارای خاصیت نرمال سازی قوی نیز است.

قضیه ۱۶.۱. در حساب لامبدا بدون نوع، نرمال سازی ضعیف و قوی معادل هستند؛ به بیان دیگر، حدس ۱۵.۱ در حساب لامبدا بدون نوع، برقرار است.

مثال

همانطور که در ۲ دیدیم، عبارت زیر دارای خاصیت نرمال سازی قوی نیست

$$(\lambda x.xx)(\lambda x.xx)$$

۳.۱ سیستم‌های کاهش انتزاعی

سیستم‌های کاهش انتزاعی^۱ که نخستین بار توسط Max Newman در [۱۱] به منظور مطالعه همگرایی روابط کاهش معرفی شدند، چارچوبی ریاضی برای مطالعه روابط کاهش در سیستم‌های محاسباتی هستند. بسیاری از مفاهیم نظریه انواع (از جمله نرمال‌سازی ضعیف و قوی، همگرایی و یکتایی فرم نرمال) به‌طور ضمنی در این چارچوب بیان می‌شوند. در این بخش تعریف رسمی و ویژگی‌های بنیادی این چارچوب را بیان می‌کنیم.

۱.۳.۱ تعریف سیستم کاهش انتزاعی

تعریف ۱.۳.۱. یک سیستم کاهش انتزاعی، یک زوج $(A, \rightarrow)$ است که در آن:

- A مجموعه اشیا است.

- $\rightarrow \subseteq A \times A$ یک رابطه دودویی روی A است که رابطه کاهش نام دارد.

برای مثال در حساب لامبدا، مجموع عبارات Λ همراه با رابطه کاهش β ، یک ARS را تشکیل می‌دهند.

۲.۳.۱ دنباله و بستار کاهش

تعریف ۲.۳.۱. دنباله کاهش، زنجیره‌ای از کاهش‌ها به شکل زیر است که می‌تواند متناهی یا نامتناهی باشد.

$$a_0 \rightarrow a_1 \rightarrow \dots \rightarrow a_n$$

تعریف ۲.۳.۱. بستار تعدی رابطه $\rightarrow$ را با $a \rightarrow^+ b$ نشان می‌دهیم که به این معناست که حداقل یک گام کاهش از b به a وجود دارد.

بستار بازتابی-تعدی رابطه $\rightarrow$ را با $a \rightarrow^* b$ نشان می‌دهیم که یعنی $b = a$ یا یک دنباله کاهش از b به a وجود دارد.

۳.۳.۱ فرم نرمال و نرمال‌سازی

تعریف ۳.۳.۱. (فرم نرمال). شی a در فرم نرمال (یا یک فرم نرمال) است اگر هیچ کاهش بر آن قابل اعمال نباشد. یعنی

$$a \in \text{NF} \iff \nexists b. a \rightarrow b$$

Abstract Reduction Systems^۱ یا به اختصار ARS

تعریف ۲۱.۱ (نرمال سازی ضعیف). شی a دارای خاصیت نرمال سازی ضعیف است، اگر حداقل یک دنباله کاهش به یک فرم نرمال داشته باشد. یعنی

$$\exists b \in \text{NF}. a \rightarrow^* b$$

تعریف ۲۲.۱ (نرمال سازی قوی). شی a دارای خاصیت نرمال سازی قوی است، اگر هیچ دنباله کاهش نامتناهی از آن آغاز نشود.

قضیه ۲۳.۱. اگر یک شی دارای خاصیت نرمال سازی قوی باشد، آنگاه دارای خاصیت نرمال سازی ضعیف نیز است.

۴.۳.۱ همگرایی

تعریف ۲۴.۱ (همگرایی). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگراست، هرگاه هر دو دنباله کاهش که از یک شی منشعب می شوند، دوباره به شی مشترکی برسند. یعنی

$$\forall a, b_1, b_2. (a \rightarrow^* b_1 \wedge a \rightarrow^* b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

نتایجی مانند یکتایی فرم نرمال از قضیه پیش رو ثابت می شوند.

قضیه ۲۵.۱ (قضیه چرچ-راسر). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگراست، اگر و تنها اگر دارای خاصیت چرچ-راسر باشد؛ یعنی

$$\forall b_1, b_2. (b_1 = b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

نتیجه ۲۶.۱ (یکتایی فرم نرمال). فرم نرمال یک شی، در صورت وجود، یکتاست. یعنی

$$\forall M, N_1 \in \text{NF}, N_2 \in \text{NF}. (M \rightarrow^* N_1 \wedge M \rightarrow^* N_2) \implies N_1 = N_2$$

اثبات همگرایی معمولاً دشوار است، اما یک ویژگی محلی ساده تر وجود دارد.

تعریف ۲۷.۱ (همگرایی محلی). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگرای محلی است اگر

$$\forall a, b_1, b_2. (a \rightarrow b_1 \wedge a \rightarrow b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

لم ۲۸.۱ (لم نیومن). اگر یک سیستم کاهش انتزاعی، همگرای محلی و دارای خاصیت نرمال سازی قوی باشد، آنگاه همگراست.

تذکر ۲۹.۱. در برخی سیستم‌ها (مانند حساب لامبدا بدون نوع)، به دلیل همگرایی، نرمال‌سازی ضعیف معادل نرمال‌سازی قوی است. اما در سیستم‌های نوع‌دار وابسته، ممکن است عبارتی دارای خاصیت نرمال‌سازی ضعیف باشد اما در حداقل یک دنباله کاهش نامتناهی ظاهر شود. این دقیقاً مسئله‌ای است که حدس ۱۵.۱ به آن می‌پردازد و Mull در [۱۰] با ارائه سیستم‌های نوع محض لایه‌ای می‌کوشد که آن را حل کند.

۴.۱ حساب لامبدا نوع‌دار ساده

حساب لامبدا نوع‌دار ساده^۱ که نخستین بار توسط Alonzo Church در [۵] به عنوان پایه‌ای برای منطق معرفی شد، یکی از مهم‌ترین و کلاسیک‌ترین سیستم‌های نوع است. این سیستم پایه بسیاری از زبان‌های تابعی، سیستم‌های نوع پیشرفته و بخش مهمی از نظریه انواع به شمار می‌رود. همچنین ساده‌ترین نمونه از «سیستم‌های نوع محض» است و مقدمه‌ای طبیعی برای ورود به مباحث بخش بعدی به حساب می‌آید. سیستم مذکور نشان می‌دهد که افزودن انواع، باعث تضمین نرمال‌سازی قوی می‌شود و از کاهش‌های نامتناهی جلوگیری می‌کند.

۱.۴.۱ نحو

تعریف ۳۰.۱ (انواع). مجموعه انواع به صورت بازگشتی زیر تعریف می‌شود

$$T ::= B \mid T \rightarrow T$$

که در آن

• $B = \{\alpha, \beta, \gamma, \dots\}$ مجموعه انواع پایه است.

• $T \rightarrow T$ بیانگر نوع توابع است.

تعریف ۳۱.۱ (عبارات). مجموعه عبارات به صورت بازگشتی زیر تعریف می‌شود

$$M ::= V \mid \lambda V^T. M \mid MM$$

که در آن

• $V = \{x, y, z, \dots\}$ مجموعه متغیرهاست.

^۱ Simply Typed Lambda Calculus یا به اختصار STLC یا $\lambda \rightarrow$

- $\lambda V^T.M$ یک انتزاع و بیانگر تعریف یک تابع است.

- MM یک کاربرد و بیانگر صدا زدن یک تابع است.

۲.۴.۱ زمینه و قضاوت نوع

تعریف ۳۲.۱ (زمینه). یک زمینه مانند Γ ، دنباله‌ای متناهی از مفروضات به شکل زیر است

$$x_1 : A_1, \dots, x_n : A_n$$

که

- $x_i : A_i$ به این معناست که x_i از نوع A_i است.

- $\Gamma, x : A$ به معنای افزودن فرض $x : A$ به Γ است.

تعریف ۳۳.۱ (قضاوت نوع). یک قضاوت نوع، گزاره‌ای به شکل زیر است

$$\Gamma \vdash M : A$$

که در آن Γ یک زمینه، M یک عبارت و A یک نوع است و معنای آن این است که ”در زمینه Γ ، عبارت M از نوع A است”.

تعریف ۳۴.۱ (قواعد استنتاج نوع). قواعد استنتاج نوع در $STLC$ شامل سه قاعده زیر است:

- قاعده متغیر

$$\frac{}{\Gamma, x : A \vdash x : A}$$

- قاعده انتزاع

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x^A.M : A \rightarrow B}$$

- قاعده کاربرد

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MB : B}$$

تعریف ۳۵.۱ (نوع‌پذیری). یک عبارت نوع‌پذیر است هرگاه بتوان طبق قواعد **۳۴.۱** یک نوع برای آن یافت.
به بیان دیگر؛ عبارت M نوع‌پذیر است هرگاه

$$\exists \Gamma, A. \Gamma \vdash M : A$$

مثال

عبارات زیر نوع‌پذیر هستند؛

$$\begin{aligned} \lambda x. x \\ \lambda x y. x \\ \lambda f g x. (f(gx)) \end{aligned}$$

زیرا به ترتیب داریم؛

$$\begin{aligned} \lambda x^A. x : A \rightarrow A \\ \lambda x^A y^B. x : A \rightarrow B \rightarrow A \\ \lambda f^{B \rightarrow C} g^{A \rightarrow B} x^A. (f(gx)) : (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C \end{aligned}$$

عبارت زیر، که قبل‌تر در **۲** دیدیم، در STLC نوع‌پذیر نیست؛

$$(\lambda x. (xx))(\lambda x. (xx))$$

مثال فوق‌ما را به سمت قضیه مهم بعدی سوق می‌دهد.

۳.۴.۱ خواص

در این بخش، برخی خواص مهم STLC را مطالعه می‌کنیم.

قضیه ۳۶.۱ (حفظ نوع در کاهش). کاهش دادن یک عبارت، نوع آن را تغییر نمی‌دهد.
یعنی

$$(\Gamma \vdash M : A \wedge M \rightarrow_{\beta} N) \implies \Gamma \vdash N : A$$

قضیه بعدی بیان می‌کند که وجود انواع، از ایجاد عباراتی که دارای خاصیت نرمال‌سازی قوی نیستند، جلوگیری می‌کند.

قضیه ۳۷.۱ (قضیه نرمال‌سازی قوی). هر عبارت نوع‌پذیر در $STLC$ دارای خاصیت نرمال‌سازی قوی است.

۵.۱ سیستم‌های نوع محض

سیستم‌های نوع محض^۱ که نخستین بار توسط [۳] و [۱۴] معرفی شدند و سپس توسط [۱] تعمیم یافتند، یک چارچوب یکپارچه برای توصیف طیف وسیعی از سیستم‌های نوع مبتنی بر حساب لامبدا را فراهم می‌کنند. بسیاری از سیستم‌های مهم از جمله حساب لامبدا نوع‌دار ساده، سیستم‌های چندریخت و حساب ساختمان‌ها را می‌توان تنها با انتخاب مناسب مجموعه‌ای از ”رده‌ها”، ”اصول” و ”روابط” در قالب یک PTS بیان کرد. این چارچوب به دلیل سادگی نحوی و همچنین انعطاف بسیار بالا، نقش محوری در نظریه انواع دارد. از طرفی سیستمی که Mull در [۱۰] معرفی می‌کند که اساس کار این پایان‌نامه است، نسخه تغییر یافته همین سیستم است؛ لذا آشنایی با این سیستم برای ما ضروری است.

۱.۵.۱ مشخصه و نحو

تعریف ۳۸.۱ (مشخصه سیستم نوع محض). یک سیستم نوع محض با سه تایی (S, A, R) مشخص می‌شود که در آن

$$\bullet S = \{s_1, s_2, \dots\} \text{ مجموعه رده‌ها}$$

$$\bullet A \subseteq S \times S \text{ مجموعه اصول}$$

$$\bullet R \subseteq S \times S \times S \text{ مجموعه قوانین}$$

هستند.

عبارات و انواع در PTS از منظر نحو یکسان هستند؛ یعنی در این چارچوب، ”نوع” و ”عبارت” یک ساختار واحد دارند و فقط جایگاه آن‌ها در قضاوت نوع است که نقش متفاوتی به آن‌ها می‌دهد.

^۱ Pure Type Systems یا به اختصار PTS

تعریف ۳۹.۱ (نحو سیستم نوع محض). نحو سیستم نوع محض به صورت بازگشتی زیر تعریف می‌شود

$$T ::= S \mid V \mid \lambda V : T.T \mid \Pi V : T.T \mid TT$$

که در آن

• $S = \{s_1, s_2, \dots\}$ مجموعه رده‌ها

• $V = \{x, y, z, \dots\}$ مجموعه متغیرها

• $\lambda V : T.T$ انتزاع

• $\Pi V : T.T$ نوع تابع

• TT کاربرد

هستند.

در این پایان‌نامه، مطابق با [۱۰]، متغیرها را با رده خودشان نشانه‌گذاری می‌شوند.

تعریف ۴۰.۱ (نشانه‌گذاری متغیرها). برای هر $s \in S$ ، V_s را مجموعه شمارشی متغیرهای مربوط به رده s در نظر می‌گیریم. $s v_i$ به معنای i مین متغیر در V_s است و $V = \bigcup_{s \in S} V_s$.

۲.۵.۱ قضاوت نوع

تعریف ۴۱.۱ (زمینه). یک زمینه مانند Γ ، دنباله‌ای متناهی از مفروضات به شکل زیر است

$$x_1 : A_1, \dots, x_n : A_n$$

که در آن $x_i : A_i$ به این معناست که " x_i از نوع A_i است". همچنین $\Gamma, x : A$ به معنای افزودن فرض $x : A$ به Γ است.

تعریف ۴۲.۱ (قواعد استنتاج نوع در سیستم نوع محض). در یک سیستم نوع محض با مشخصه (S, A, R) قواعد زیر موجود است:

• قاعده اصل

$$\frac{}{\vdash s : t} \quad (s, t) \in \mathcal{A}$$

• قاعده شروع

$$\frac{\Gamma \vdash A : s}{\Gamma, {}^s x : A \vdash {}^s x : A} \quad {}^s x \notin \Gamma$$

• قاعده تضعیف

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s}{\Gamma, {}^s x : B \vdash M : A} \quad {}^s x \notin \Gamma$$

• قاعده تولید

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, {}^{s_1} x : A \vdash B : s_2}{\Gamma \vdash (\Pi^{s_1} x^A. B) : s_3} \quad (s_1, s_2, s_3) \in \mathcal{R}$$

• قاعده انتزاع

$$\frac{\Gamma, {}^s x : A \vdash M : B \quad \Gamma \vdash \Pi^s x^A. B : s}{\Gamma \vdash (\lambda^s x^A. M) : (\Pi^s x^A. B)}$$

• قاعده کاربرد

$$\frac{\Gamma \vdash M : (\Pi^s x^A. B) \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[N/{}^s x]}$$

• قاعده تبدیل

$$\frac{\Gamma \vdash M : A \quad A \equiv_\beta B \quad \Gamma \vdash B : s}{\Gamma \vdash M : B}$$

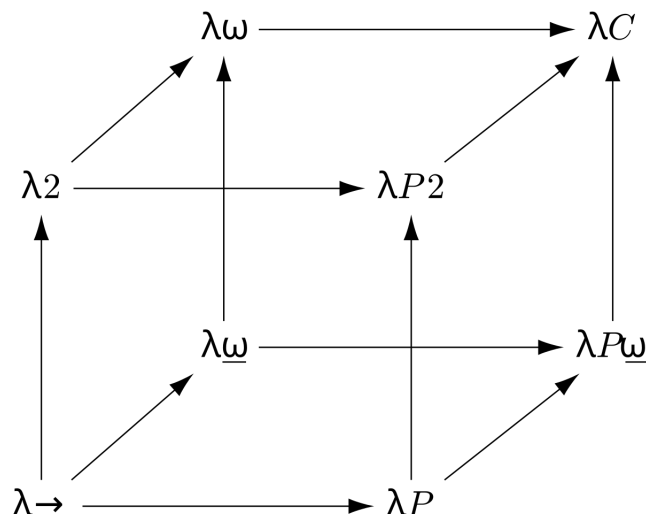
۳.۵.۱ مکعب لامبدا

توصیف

مکعب لامبدا یک چارچوب مفهومی و هندسی برای دسته‌بندی خانواده‌ای از سیستم‌های نوع است. این مکعب نشان می‌دهد که چگونه با افزودن یا حذف برخی قابلیت‌های نوعی، می‌توان به سیستم‌های مختلفی از حساب لامبدا دست یافت. مکعب لامبدا دارای سه بُعد مستقل است که هر بُعد متناظر با افزودن یک قابلیت به سیستم پایه است:

۱. چندریختی

این بُعد نشان‌دهنده امکان کمی‌سازی روی انواع است؛ به بیان دیگر، اجازه می‌دهد توابعی تعریف شوند که برای همه انواع معتبر باشند. وجود این قابلیت منجر به سیستم‌هایی مانند λ_2 می‌شود.



شکل ۱.۱: مکعب لامبدا

۲. اپراتورهای نوع

این بُعد امکان تعریف توابعی را فراهم می‌کند که روی انواع عمل می‌کنند، نه روی عبارات. در این حالت، انواع می‌توانند به عنوان ورودی توابع به کار روند و این امر منجر به سیستم‌هایی مانند $\lambda\omega$ می‌شود.

۳. انواع وابسته

این بُعد اجازه می‌دهد نوع یک عبارت به مقدار یک عبارت دیگر وابسته باشد. به این ترتیب می‌توان انواعی تعریف کرد که اطلاعاتی درباره مقدار عبارات در خود جای می‌دهند؛ این قابلیت، پایه سیستم‌هایی مانند حساب ساختمان‌هاست.

ترکیب یا عدم ترکیب هر یک از این سه قابلیت، هشت رأس مکعب را تشکیل می‌دهد که هر رأس متناظر با یک سیستم نوع مشهور است. برای مثال، حساب لامبدای نوع دار ساده در رأسی قرار دارد که هیچ‌یک از این قابلیت‌ها را ندارد، در حالی که حساب ساختمان‌ها در رأسی قرار می‌گیرد که هر سه قابلیت را به طور هم‌زمان داراست.

تعریف به کمک سیستم‌های نوع محض

حال با معرفی کامل سیستم‌های نوع محض، بسیاری از سیستم‌های مشهور تنها با انتخاب مناسب (S, A, R) قابل بیان‌اند. جدول ۱.۱ بیانگر تعریف هر هشت رأس مکعب لامبدا به کمک سیستم‌های نوع محض است.

	$\mathcal{S}$	$\mathcal{A}$	$\mathcal{R}$
$\lambda \rightarrow$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *)\}$
$\lambda 2$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, *, *)\}$
$\lambda \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (*, \square, \square)\}$
λP	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square)\}$
$\lambda P 2$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (\square, *, *)\}$
$\lambda \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (*, \square, \square), (\square, *, *)\}$
$\lambda P \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (*, \square, \square)\}$
λC	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (*, \square, \square), (\square, *, *)\}$

جدول ۱.۱: جدول بیان سیستم‌های نوع در مکعب لامبدا به کمک سیستم‌های نوع محض

۴.۵.۱ اهمیت

در ادامه این پایان‌نامه، سیستم‌های نوع محض نقطه آغاز برای تعریف «سیستم‌های نوع محض لایه‌ای^۱» سپس معرفی ترجمه حذف وابستگی Mull هستند. در واقع:

- TPTS گسترشی ساختاریافته از PTS است.
 - قواعد نوع TPTS نسخه تقویت‌شده همین قواعداند.
 - اثبات‌های نرمال‌سازی وابسته به ساختار PTS بیان می‌شوند.
- در نتیجه، این بخش چارچوب مفهومی لازم برای تعریف سیستم‌های لایه‌ای و صورتی‌سازی آن‌ها در Coq را فراهم می‌کند.

۵.۵.۱ خواص فرانظری سیستم‌های نوع محض

در این بخش برخی خواص فرانظری سیستم‌های نوع محض را مورد بررسی قرار می‌دهیم.

تذکر ۴۳.۱. از تکرار تعاریفی که قبلاً مشاهده کرده‌ایم (برای مثال فرم نرمال و نرمال‌سازی برای یک عبارت که در ۶.۲.۱ دیدیم) خودداری می‌کنیم.

تعریف ۴۴.۱ (نرمال‌سازی ضعیف و قوی در سیستم). یک سیستم نوع محض دارای خاصیت نرمال‌سازی ضعیف یا قوی است، هرگاه هر عبارت نوع‌پذیر در آن دارای این خاصیت باشد.

^۱TPTS یا Tiered Pure Type Systems

لم ۴۵.۱ (تولید).

• رده:

$$\forall s \in \mathcal{S}. \quad \Gamma \vdash s : A \implies \exists s' \in \mathcal{S}. (A =_{\beta} s') \wedge ((s, s') \in \mathcal{A})$$

• متغیر:

$$\forall {}^s x \in V. \quad \Gamma \vdash {}^s x : A \implies \exists B. (A =_{\beta} B) \wedge (({}^s x : B) \in \Gamma)$$

• عبارت Π :

$$\forall s \in \mathcal{S}, B, C. \quad \Gamma \vdash (\Pi {}^s x : B. C) : A \implies$$

$$\exists s', s'' \in \mathcal{S}. (\Gamma \vdash B : s) \wedge (\Gamma, {}^s x : B \vdash C : s') \wedge ((s, s', s'') \in \mathcal{R}) \wedge (A =_{\beta} s'')$$

• عبارت λ :

$$\forall s \in \mathcal{S}, B, M. \quad \Gamma \vdash (\lambda {}^s x : B. M) : A \implies$$

$$\exists s' \in \mathcal{S}, C. (\Gamma \vdash (\Pi {}^s x : B. C) : s') \wedge (\Gamma, {}^s x : B \vdash M : C) \wedge (A =_{\beta} \Pi {}^s x : B. C)$$

• کاربرد:

$$\forall M, N. \quad \Gamma \vdash MN : A \implies$$

$$\exists s \in \mathcal{S}, B, C. (\Gamma \vdash M : (\Pi {}^s x : B. C)) \wedge (\Gamma \vdash N : B) \wedge (A =_{\beta} C[N/{}^s x])$$

لم ۴۶.۱ (جانشینی).

$$\forall \Gamma, \Delta, M, N, A, B. \quad (\Gamma, x : A, \Delta \vdash M : B) \wedge (\Gamma \vdash N : A) \implies \Gamma, \Delta[N/x] \vdash M[N/x] : B[N/x]$$

لم ۴۷.۱ (صحت نوع).

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies (A \in \mathcal{S}) \vee (\exists s \in \mathcal{S}. \Gamma \vdash A : s)$$

لم ۴۸.۱ (تضعیف).

$$\forall \Gamma, \Delta, M, A. \quad (\Gamma \subset \Delta) \wedge (\Gamma \vdash M : A) \implies \Delta \vdash M : A$$

لم ۴۹.۱ (جابه جایی).

$$\begin{aligned} \forall \Gamma, \Delta, x, y, A, B, C, M. \quad & (x \notin FV(B)) \wedge (\Gamma, x : A, y : B, \Delta \vdash M : C) \\ & \implies \Gamma, y : B, x : A, \Delta \vdash M : C \end{aligned}$$

بخش دوم

سیستم‌های نوع محض لایه‌ای و ترجمه
حذف وابستگی

فصل ۲

سیستم‌های نوع محض لایه‌ای

۱.۲ انگیزه و ایده اصلی

۱.۱.۲ انگیزه

در فصل پیشین، مفاهیم پایه‌ای حساب لامبدا، سیستم‌های کاهش انتزاعی، حساب لامبدا نوع‌دار ساده و سیستم‌های نوع محض را بررسی کردیم. این چارچوب‌ها زمینه لازم برای درک مسائل پیشرفته‌تر نظریه انواع، به‌ویژه رابطه میان نرمال‌سازی ضعیف و قوی را فراهم آوردند.

سیستم‌های نوع محض، به‌عنوان چارچوبی یکپارچه برای توصیف سیستم‌های نوع مبتنی بر حساب لامبدا، بستری مناسب برای مطالعه خواص فرانظری نظیر نرمال‌سازی، یکتایی نوع و سازگاری منطقی فراهم می‌کنند. با این حال، بررسی رابطه میان نرمال‌سازی ضعیف و نرمال‌سازی قوی در این سیستم‌ها، که در قالب حدس ۱۵.۱ بیان می‌شود، در حضور انواع وابسته با دشواری‌های اساسی مواجه است.

مسئله اصلی از آنجا ناشی می‌شود که در سیستم‌های نوع وابسته، ساختار انواع می‌تواند به‌طور دلخواه به مقادیر عبارات وابسته باشد. این وابستگی باعث می‌شود که رفتار مسیرهای کاهش β به‌شدت پیچیده شده و تحلیل آن‌ها، به‌ویژه در اثبات نرمال‌سازی قوی، با موانع جدی روبه‌رو گردد. در چنین شرایطی، روش‌های کلاسیک که برای سیستم‌های غیروابسته به کار می‌رود، مانند تکنیک‌های مبتنی بر کاهش‌پذیری یا استدلال‌های ترتیبی، دیگر به‌سادگی قابل اعمال نیستند.

۲.۱.۲ ایده اصلی

Mull در رساله دکتری خود [۱۰] بر این اصل استوار است که وابستگی‌های موجود در

سیستم‌های نوع وابسته، لزوماً همگی برای بیان توان محاسباتی سیستم ضروری نیستند. به بیان دیگر، بخشی از این وابستگی‌ها را می‌توان به صورت کنترل‌شده حذف یا محدود کرد، بی‌آنکه به خوش‌نوعی یا قابلیت بیان سیستم آسیبی وارد شود.

بر همین اساس، Mull خانواده جدیدی از سیستم‌های نوع را تحت عنوان «سیستم‌های نوع محض لایه‌ای»^۱ معرفی می‌کند. در این سیستم‌ها، وابستگی میان انواع و عبارات به وسیله مفهوم «لایه» به طور صریح ساختاربندی می‌شود. لایه‌ها امکان تفکیک سطوح مختلف وابستگی را فراهم می‌کنند و از بروز وابستگی‌های چرخه‌ای یا کنترل‌نشده جلوگیری می‌نمایند.

مزیت کلیدی این لایه‌بندی آن است که امکان تعریف یک ترجمه حذف وابستگی از سیستم‌های نوع محض لایه‌ای به یک سیستم نوع محض فراهم می‌شود؛ به گونه‌ای که تحلیل نرمال‌سازی قوی در سیستم مقصد ساده‌تر بوده و می‌توان از آن برای استنتاج نرمال‌سازی قوی در سیستم مبدأ بهره گرفت.

در ادامه این فصل، سیستم‌های نوع محض لایه‌ای به‌طور رسمی معرفی می‌شوند، سپس در فصل‌های بعدی، ترجمه حذف وابستگی و خواص اساسی آن مورد بررسی قرار می‌گیرد. این مباحث، زمینه لازم برای صوری‌سازی کامل این ترجمه و اثبات‌های مرتبط با آن در اثبات‌یار Coq را که در فصل‌های بعدی ارائه می‌شوند، فراهم می‌سازند.

۲.۲ سیستم‌های نوع محض لایه‌ای

۱.۲.۲ لایه‌ها و شهود آن‌ها

ایده اصلی در سیستم‌های نوع محض لایه‌ای، افزودن ساختاری صریح برای کنترل وابستگی‌های نوعی است. این ساختار از طریق مفهوم «لایه» معرفی می‌شود. لایه‌ها سطوحی انتزاعی هستند که به متغیرها و عبارات نسبت داده می‌شوند و میزان وابستگی مجاز میان آن‌ها را محدود می‌کنند.

به‌طور شهودی، لایه‌ها را می‌توان به‌عنوان ابزاری برای مرتب‌سازی وابستگی‌ها در نظر گرفت؛ به گونه‌ای که یک عبارت در یک لایه بالاتر مجاز نیست به‌طور دلخواه به عبارات لایه‌های پایین‌تر وابسته باشد. این محدودیت مانع از شکل‌گیری وابستگی‌های چرخه‌ای و کنترل‌نشده در سطح انواع می‌شود، که یکی از موانع اصلی در تحلیل نرمال‌سازی قوی در سیستم‌های وابسته است.

لازم به تأکید است که لایه‌ها با «رده‌ها» متفاوت‌اند. رده‌ها نقش طبقه‌بندی نحوی انواع

^۱ Tiered Pure Type Systems یا به اختصار TPTS

و عبارات را ایفا می‌کنند، در حالی که لایه‌ها اطلاعاتی فرانظری درباره سطح وابستگی یک عبارت حمل می‌نمایند. به بیان دیگر، رده‌ها بخشی از ساختار نوعی سیستم هستند، اما لایه‌ها ابزاری ساختاری برای کنترل وابستگی‌ها محسوب می‌شوند.

در سیستم‌های نوع محض لایه‌ای، هر متغیر به صورت صریح به یک لایه نسبت داده می‌شود. این نشانه‌گذاری لایه‌ای، مشابه نشانه‌گذاری رده‌ای که پیش‌تر معرفی شد، صرفاً جنبه تزئینی ندارد، بلکه در قواعد نوع‌دهی نقش اساسی ایفا می‌کند. قیود لایه‌ای تعیین می‌کنند که در چه شرایطی تشکیل انواع تابع وابسته، انتزاع و کاربرد مجاز است.

هدف از معرفی لایه‌ها، کاهش توان بیان سیستم نیست، بلکه فراهم‌سازی بستری است که در آن بتوان وابستگی‌های نوعی را به گونه‌ای کنترل‌شده بازآرایی کرد. این کنترل دقیق، امکان تعریف ترجمه حذف وابستگی را فراهم می‌کند؛ ترجمه‌ای که نقش محوری در انتقال خواص نرمال‌سازی از سیستم‌های ساده‌تر به سیستم‌های نوع محض لایه‌ای ایفا می‌نماید.

۲.۲.۲ تعریف رسمی سیستم

تعریف ۱.۲ (سیستم نوع محض لایه‌ای). فرض کنید n یک عدد صحیح نامنفی باشد. یک سیستم نوع محض، n -لایه‌ای است؛ هرگاه مشخصات آن به شکل زیر باشد

$$\begin{aligned}\mathcal{S} &= \{s_i \mid i \in [n]\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in [n-1]\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}\end{aligned}$$

که در آن

$$[n] = \{1, \dots, n\}$$

مثال

۱. تنها سیستم نوع محض ۰-لایه‌ای، سیستم نوع محض خالی است؛

$$\mathcal{S} = \mathcal{A} = \mathcal{R} = \emptyset$$

۲. دو سیستم نوع محض ۱-لایه‌ای وجود دارد:

$$\begin{aligned}\mathcal{S} &= \{s_1\} & \mathcal{A} &= \emptyset & \mathcal{R} &= \emptyset \\ \mathcal{S} &= \{s_1\} & \mathcal{A} &= \emptyset & \mathcal{R} &= \{(s_1, s_1)\}\end{aligned}$$

۳. سیستم‌های نوع محض ۲- لایه‌ای، معادل کل مکعب لامبدا هستند.

۳.۲.۲ خواص

در این بخش برخی خواص مهم فرانظری سیستم‌های نوع محض را بررسی می‌کنیم، و خواهیم دید که زیرکلاسی که در بخش قبل معرفی کردیم، چه ویژگی‌های کاربردی‌ای را در اختیار ما قرار می‌دهد.

تعریف ۲.۲ (سیستم نوع محض تابعی). یک سیستم نوع محض را تابعی گوییم، هرگاه

• اگر $(s, t) \in A$ و $(s, t') \in A$ آنگاه $t = t'$.

• اگر $(s, t, u) \in R$ و $(s, t, u') \in A$ آنگاه $u = u'$.

لم ۳.۲ (یکتایی نوع). در یک سیستم نوع محض تابعی، نوع هر عبارت نوع‌پذیر، یکتاست. یعنی

$$\forall \Gamma, M, A, B. \quad (\Gamma \vdash M : A) \wedge (\Gamma \vdash M : B) \implies A =_{\beta} B$$

تعریف ۴.۲ (رده‌های بالا و پایین).

• رده s یک رده بالا است هرگاه

$$\nexists s' \in S. (s, s') \in A$$

• رده s یک رده پایین است هرگاه

$$\nexists s' \in S. (s', s) \in A$$

لم ۵.۲ (رده بالا). برای هر زمینه Γ ، متغیر x ، عبارات A و B و رده بالا s ، گزاره‌های زیر برقرار هستند:

$$\Gamma \not\vdash s : A$$

$$\Gamma \not\vdash x : s$$

$$\Gamma \not\vdash AB : s$$

تعریف ۶.۲ (سیستم نوع محض پایدار). یک سیستم نوع محض، پایدار است؛ هرگاه هر سه گزاره زیر برای آن برقرار باشد:

• تابعی باشد.

• اگر $(s, t) \in A$ و $(s', t) \in A$ آنگاه $s = s'$.

• $\mathcal{R} \subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}$

تذکر ۷.۲. از اینجا به بعد، از نماد (s, s') به جای قانون (s, s', s') استفاده می‌کنیم.

تعریف ۸.۲ (اجتماع مجزا دو سیستم نوع محض). اجتماع مجزا دو سیستم نوع محض $\lambda\mathcal{S}$ و $\lambda\mathcal{S}'$ ، که با نماد $\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'$ نشان داده می‌شود، یک سیستم نوع محض با مشخصات زیر است

$$\mathcal{S}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{S}_{\lambda\mathcal{S}} \sqcup \mathcal{S}_{\lambda\mathcal{S}'}$$

$$\mathcal{A}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{A}_{\lambda\mathcal{S}} \cup \mathcal{A}_{\lambda\mathcal{S}'}$$

$$\mathcal{R}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{R}_{\lambda\mathcal{S}} \cup \mathcal{R}_{\lambda\mathcal{S}'}$$

در تعریف بعد می‌خواهیم به $A \subset \mathcal{S} \times \mathcal{S}$ ، به عنوان یک رابطه نگاه کنیم.

تعریف ۹.۲ (رابطه A). در یک سیستم نوع محض، مجموعه $A(s)$ که به معنای اصول‌هایی است که s در آنان ظاهر شده، به صورت زیر تعریف می‌شود

$$A(s) = \{(s', t') \in A \mid s' = s \vee t' = s\}$$

نکات

• اگر A را به عنوان یک گراف تفسیر کنیم، $A(s)$ به معنای مجموعه یال‌هایی است که به s متصل هستند.

• در یک سیستم نوع محض پایدار، برای هر رده s داریم: $|A(s)| \leq 2$.

تعریف ۱۰.۲ (بستار رابطه A).

• بستار تعدی: $<_A$

• بستار بازتابی-تعدی: $\leq_A$

• بستار هم‌رزی: $\approx_A$

تعریف ۱۱.۲ (دنباله رده). طبق تعریف قبل، می‌نویسیم $s \leq_A t$ اگر و تنها اگر دنباله‌ای از رده‌ها به صورت زیر موجود باشد

$$\exists t_1, \dots, t_k \in \mathcal{S}. \quad (t_1 = s) \wedge (t_k = t) \wedge (\forall i \in [k-1]. (t_i, t_{i+1}) \in A)$$

دنباله مذکور را یک دنباله به طول $k-1$ از s به t می‌نامیم.

نکته: دقت شود که در تعریف فوق، لزومی ندارد که اعضای یک دنباله رده، منحصر به فرد باشند.

گزاره ۱۲.۲. یک سیستم نوع محض پایدار را در نظر بگیرید. برای هر رده $s \in S$ و هر k ؛

• حداکثر یک دنباله رده با طول k یافت می شود که از s شروع می شود.

• حداکثر یک دنباله رده با طول k یافت می شود که به s خاتمه می یابد.

تعریف ۱۳.۲ (سیستم نوع محض تفکیک پذیر). یک سیستم نوع محض را تفکیک پذیر گوئیم، هرگاه

$$\forall s, s' \in S. \quad (s, s') \in \mathcal{R} \implies s \approx_{\mathcal{A}} s'$$

تعریف ۱۴.۲ (سیستم نوع محض اتمی). یک سیستم نوع محض را اتمی گوئیم، هرگاه

$$\forall s, s' \in S. \quad s \approx_{\mathcal{A}} s'$$

تعریف ۱۵.۲ (سیستم نوع محض کراندار).

• یک سیستم نوع محض، شرط زنجیره صعودی را ارضا می کند، هرگاه دنباله نامتناهی

$$s, s', s'', \dots$$

یافت نشود به طوری که

$$s < s' < s'' < \dots$$

• یک سیستم نوع محض، شرط زنجیره نزولی را ارضا می کند، هرگاه دنباله نامتناهی

$$s, s', s'', \dots$$

یافت نشود به طوری که

$$s > s' > s'' > \dots$$

یک سیستم نوع محض کراندار است، اگر هر دو شرط زنجیره صعودی و نزولی را ارضا کند.

تعریف ۱۶.۲ (سیستم نوع محض ضعیفاً غیروابسته). یک سیستم نوع محض، ضعیفاً (یا به طور ضعیف) غیروابسته است؛ هرگاه

$$(s, s', s'') \in \mathcal{R} \implies s \geq s' \geq s''$$

اصطلاح «سلسله مراتبی» در ادامه به معنای دارا بودن ساختار ترتیبی روی وابستگی‌هاست و با مفهوم «لایه‌ای» که قبل‌تر دیدیم مرتبط است، اما عیناً یکسان نیست.

تعریف ۱۷.۲ (سیستم نوع محض سلسله مراتبی). یک سیستم نوع محض را سلسله مراتبی می‌نامیم هرگاه شرط زنجیره صعودی را ارضا کند و ضعیفاً غیروابسته باشد.

تعریف ۱۸.۲ (سیستم نوع محض غیروابسته تعمیم‌یافته). یک سیستم نوع محض، غیروابسته تعمیم‌یافته است؛ اگر سلسله مراتبی و پایدار باشد.

تعریف ۱۹.۲ (سیستم نوع محض غیروابسته کراندار). یک سیستم نوع محض، غیروابسته کراندار است؛ اگر سلسله مراتبی، پایدار و کراندار باشد.

تعریف ۲۰.۲ (سیستم نوع محض غیروابسته). یک سیستم نوع محض، غیروابسته است؛ اگر سلسله مراتبی، پایدار، کراندار و لایه‌ای باشد.

اکنون می‌توانیم سیستم‌های نوع محض لایه‌ای را به کمک خواصی که بررسی کردیم، مطالعه کنیم.

لم ۲۱.۲. یک سیستم نوع محض، لایه‌ای است؛ اگر و تنها اگر پایدار، کراندار و اتمی باشد.

لم ۲۲.۲. یک سیستم نوع محض، پایدار، کراندار و تفکیک‌پذیر است؛ اگر و تنها اگر اجتماع مجزا سیستم‌های نوع محض لایه‌ای باشد.

نتیجه ۲۳.۲. یک سیستم نوع محض، غیروابسته کراندار است؛ اگر و تنها اگر اجتماع مجزا سیستم‌های نوع محض لایه‌ای باشد.

Doorn و Roux در [۱۳] نشان دادند که نرمال‌سازی (قوی) اجتماع مجزا سیستم‌های نوع محض، معادل نرمال‌سازی (قوی) تک تک سیستم‌هاست. بنابراین برای بررسی خاصیت نرمال‌سازی در سیستم‌های نوع محض پایدار، کراندار و تفکیک‌پذیر، کافی است سیستم‌های نوع محض لایه‌ای را در نظر بگیریم.

گزاره ۲۴.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض پایدار، کراندار و تفکیک‌پذیر نیز چنین است.

به طور خاص، اگر برای همه سیستم‌های نوع محض غیروابسته، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض غیروابسته کراندار نیز چنین است.

۴.۲.۲ گسترش سیستم

در این بخش می‌خواهیم گسترش‌هایی از سیستم‌های نوع محض لایه‌ای را مطالعه کنیم.

گسترش نامتناهی

تعریف ۲۵.۲ (سیستم نوع محض $\mathbb{Z}^+$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}^+$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}^+\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in \mathbb{Z}^+\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

تعریف ۲۶.۲ (سیستم نوع محض $\mathbb{Z}^-$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}^-$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}^-\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \leq -2\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

تعریف ۲۷.۲ (سیستم نوع محض $\mathbb{Z}$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in \mathbb{Z}\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

گزاره ۲۸.۲. فرض کنید C مجموعه سیستم‌های نوع محض با ویژگی پیش‌رو باشد؛ اگر λS یک سیستم نوع محض لایه‌ای نامتناهی در C باشد، برای هر زیرسیستم نوع محض لایه‌ای متناهی $\lambda S'$ از λS ، یک زیرسیستم نوع محض لایه‌ای متناهی $\lambda S''$ یافت می‌شود به طوری که

$$(\lambda S'' \in C) \wedge (\lambda S' \subset \lambda S'' \subset \lambda S)$$

بنابراین هر سیستم در C دارای خاصیت نرمال‌سازی (قوی) است؛ اگر و تنها اگر هر سیستم متناهی در آن دارای این خاصیت باشد.

نتیجه ۲۹.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای غیروابسته، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض غیروابسته تعمیم‌یافته نیز چنین است.

گسترش دوردار

تعریف ۳۰.۲ (سیستم نوع محض لایه‌ای دوردار). یک سیستم نوع محض لایه‌ای دوردار، یک سیستم نوع محض لایه‌ای با مشخصات زیر است

$$\mathcal{S} = \{s_i \mid i \in [n]\}$$

$$\mathcal{A} = \{(s_i, s_{i+1}) \mid i \in [n-1]\} \cup \{(s_n, s_1)\}$$

$$\mathcal{R} \subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}$$

در واقع اصل (s_n, s_1) به مجموعه اصول یک سیستم نوع محض لایه‌ای اضافه شده است و گراف $\mathcal{A}$ به یک گراف دوردار تبدیل شده است.

گزاره ۳۱.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای متناهی بدون دور و دوردار، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض تفکیک‌پذیر پایدار نیز چنین است.

تذکره ۳۲.۲. در ادامه این پایان‌نامه؛ منظور از لایه‌ای، حالت متناهی بدون دور است.

۵.۲.۲ درجه

یکی از مزایای اصلی مطالعه سیستم‌های پایدار (به طور خاص سیستم‌های لایه‌ای) این است که عبارات را می‌توان بر اساس سطحی در سیستم که در آن قابل‌دستیابی هستند، طبقه‌بندی کرد. این ویژگی با تعریف یک معیار درجه برای عبارات و طبقه‌بندی عبارات بر اساس درجه آنها نشان داده می‌شود.

تعریف ۳۳.۲ (درجه عبارت). درجه یک عبارت با استفاده از تابع $\deg : T \rightarrow \mathbb{N}$ و به

صورت زیر تعریف می شود

$$\deg(s_i) = i + 1$$

$$\deg(s_i x) = i - 1$$

$$\deg(\Pi x^A . B) = \deg(B)$$

$$\deg(\lambda x^A . M) = \deg(M)$$

$$\deg(MN) = \deg(M)$$

تعریف ۳۴.۲.

$$T_j = \{M \in T \mid \deg(M) = j\}$$

$$T_{\geq j} = \{M \in T \mid \deg(M) \geq j\}$$

لم ۳۵.۲ (طبقه بندی). فرض کنید λS یک سیستم نوع محض n -لایه ای باشد. برای هر عبارت A ، گزاره های زیر برقرارند:

$$\deg(A) = n + 1 \iff A = s_n$$

$$\deg(A) = n \iff \exists \Gamma. \Gamma \vdash A : s_n$$

$$\forall i \in [n - 1]. \quad \deg(A) = i \iff \exists \Gamma, B. (\Gamma \vdash A : B) \wedge (\Gamma \vdash B : s_{i+1})$$

کاربرد خاص لم فوق چنین است که

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies \deg(A) = \deg(M) + 1$$

لم ۳۶.۲.

$$\deg(B) = j - 1 \implies \deg(A[B/s_j x]) = \deg(A)$$

$$A \twoheadrightarrow_\beta B \implies \deg(A) = \deg(B)$$

فصل ۳

ترجمه حذف وابستگی

۱.۳ مقدمه

در چند دهه اخیر، روش‌های متعددی برای اثبات نرمال‌سازی سیستم‌های موجود در مکعب λ و گسترش آنان ارائه شده است، اما بسیاری از این روش‌ها به سیستم‌های نوع محض تعمیم داده نشده‌اند. هدف ما در این بخش، تعمیم یکی از این روش‌ها به نام ترجمه حذف وابستگی است.

ایده کلی چنین است که به منظور نشان دادن آنکه یک سیستم دارای خاصیت نرمال‌سازی قوی است، کافی است یک ترجمه، با خواص حفظ نوع و حفظ مسیرهای کاهش نامتناهی، از آن سیستم به سیستم دیگری که می‌دانیم دارای خاصیت نرمال‌سازی قوی است، ارائه کنیم. Harper و همکارانش [۱۲] چنین ترجمه‌ای از λP به $\lambda \rightarrow$ را ارائه کردند و Guevers و Nederhof [۹] این ترجمه را به ترجمه‌ای از λC به $\lambda \omega$ تعمیم دادند. هر دو این ترجمه‌ها را می‌توان به عنوان حذف قانون وابسته در سیستم مربوطه در نظر گرفت، به طور دقیق‌تر قانون $(*, \square)$ که اجازه می‌دهد نوع به عبارت وابسته باشد.

برای سیستم‌های نوع محض که به اندازه کافی خوش ساختار هستند، این مفهوم وابستگی را می‌توان گسترش داد، همانطور که Barthe و همکارانش [۸] برای تعریف سیستم‌های نوع محض غیروابسته تعمیم‌یافته‌شان انجام داده‌اند (که مشابه تعریف سیستم‌های نوع محض غیروابسته منطقی است که توسط Coquand و Herbelin [۶] ارائه شده است). در ادامه قصد داریم ترجمه‌های مذکور را به سیستم‌های نوع محض به گونه‌ای گسترش دهیم که ویژگی حذف قاعده وابسته را حفظ کند. این ترجمه را می‌توان با ترجمه [۸] ترکیب کرد تا شواهدی را برای حدس BGK ارائه داد.

۲.۳ هدف

طبق ۱۶.۲، می‌دانیم یک سیستم نوع محض لایه‌ای، غیروابسته است؛ هرگاه قوانین آن غیروابسته باشد. یعنی

$$(s, s') \in \mathcal{R} \implies s \geq_A s'$$

ترجمه معرفی شده در این بخش، بخش وابسته یک سیستم نوع محض لایه‌ای را حذف می‌کند. این مورد ما را به سمت تعریف پیش‌رو سوق می‌دهد.

تعریف ۱.۳ (نسخه غیروابسته سیستم نوع محض لایه‌ای). نسخه غیروابسته یک سیستم نوع محض لایه‌ای λS که با λS^* نشان داده می‌شود، سیستمی با مشخصات زیر است

$$S_{\lambda S^*} = S_{\lambda S}$$

$$\mathcal{A}_{\lambda S^*} = \mathcal{A}_{\lambda S}$$

$$\mathcal{R}_{\lambda S^*} = \{(s, s', s') \in \mathcal{R}_{\lambda S} \mid (s \geq s')\}$$

می‌خواهیم یک ترجمه از سیستم نوع محض غیروابسته λS به نسخه غیروابسته آن، یعنی λS^* ارائه کنیم؛ به طوری که خواص حفظ نوع و حفظ مسیرهای کاهش نامتناهی را دارا باشد. الزامات روی ساختار غیروابسته کاملاً قوی خواهند بود.

ترجمه در سطح بالا، استفاده‌های وابسته از قاعده تولید در ۴۲.۱ را حذف می‌کند. اما چون باید مسیرهای کاهش نامتناهی را نیز حفظ کند، نمی‌تواند اطلاعات زیرعبارات را بیش از حد حذف کند؛ زیرا حذف این زیرعبارات ممکن است باعث حذف انتزاع‌هایی شود که مسئول مسیرهای کاهش در عبارت پیش از ترجمه هستند. ایده کلی این است که تمام رده‌ها را به پایین‌ترین رده (s_1) ترجمه کنیم، به طوری که وقتی عبارت M از نوع $\Pi x^A.B$ وجود دارد، شکل ترجمه‌شده آن، که اکنون با $\rho(B)$ نشان داده می‌شود، از رده s_1 است، که تضمین می‌کند هر تشکیل نوع با $\rho(B)$ بعنوان نوع هدف، قطعاً غیروابسته است (زیرا هیچ رده‌ای پایین‌تر از آن وجود ندارد).

مسئله اول این است که عبارات ترجمه‌شده، باید با استفاده از انواع ترجمه‌شده، نوع‌پذیر باشند؛ که نشان می‌دهد نیاز به دو ترجمه جداگانه است، یک ترجمه برای عبارات و یک ترجمه برای انواع. مسئله دوم این است که برخی از انواع، شامل عبارت هستند (مثل $(\lambda x^{s_i}.x).A$ زمانی که $\Gamma \vdash A : s_i$) که نشان می‌دهد به ترجمه‌ای مجزا برای این عبارات نیاز داریم. همچنین فرآیند مذکور باید تا بالاترین رده تکرار شود،

بنابراین ترجمه در دو گام تعریف می‌شود. اول، دنباله‌ای از ترجمه‌ها را تعریف می‌کنیم که در ترجمه ρ_0 برای انواع به پایان می‌رسد. هر ترجمه، رده‌ها را به پایین‌تر نگاشت

می‌کند و نهایتاً به یک نوع 0 (یک متغیر منحصر به فرد) از رده s_1 می‌رسیم. سپس ترجمه γ را برای عبارات تعریف می‌کنیم، به طوری که

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies \exists \Gamma^*. \Gamma^* \vdash \gamma(M) : \rho_0(A)$$

الزام قوی روی بخش غیروابسته λS از این واقعیت ناشی می‌شود که هر ترجمه بعدی متغیر جدیدی را در زمینه اضافه می‌کند، و با اضافه شدن متغیرهای بیشتر در زمینه، متغیرهای بیشتری نیز باید در انتزاع دخیل شوند تا اطلاعات زیرعبارات از دست نرود. این امر تعریف پیش‌رو را توجیح می‌کند.

تعریف ۲.۳ (سیستم نوع محض لایه‌ای کامل). یک سیستم نوع محض لایه‌ای، (i, j) -کامل^۱ است، اگر قوانین آن شامل

$$\{(s_l, s_k) \mid (l \leq i) \wedge (l \leq k \leq j)\}$$

باشد؛ و کامل است، هرگاه ویژگی بسته بودن زیر را برآورده کند

$$(s_i, s_j) \in \mathcal{R}_{\lambda S} \implies \lambda S \text{ is } (j, i) - full$$

می‌توانیم ببینیم که این تعریف چقدر محدودکننده است. سیستمی که برای $i \geq 2$ و $j \geq 3$ ، (i, j) -کامل است، شامل λU می‌شود و بنابراین ناسازگار خواهد بود. قوی‌ترین سیستم نوع محض n -لایه‌ای کامل سازگار، دارای قوانین زیر است

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

برای باقی این بخش، یک سیستم نوع محض n -لایه‌ای کامل λS را ثابت فرض کنید.

^۱ $(i, j) - full$

۳.۳ ترجمه سطح نوع

تعریف ۳.۳ (تابع ترجمه سطح نوع). برای هر $0 \leq i \leq n$ ، تابع $\rho_i : T_{\geq i+1} \rightarrow T_{i+1}$ به صورت استقرایی زیر تعریف می شود.

$$\rho_i(s_j) = \begin{cases} 0 & i = 0 \\ s_i & o.w. \end{cases}$$

$$\rho_i(s^j x) = s^{i+2} x \quad (j \geq i + 2)$$

$$\rho_i(\Pi x^A . B) = \Pi x^{\rho_{\deg A-1}(A)} \dots \Pi x^{\rho_i(A)} . \rho_i(B)$$

$$\rho_i(\lambda x^A . M) = \lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_{i+1}(A)} . \rho_i(M)$$

$$\rho_i(MN) = \rho_i(M) \rho_{\deg N-1}(N) \dots \rho_i(N)$$

که در آن 0 یک متغیر منحصر به فرد است.

تعریف هر ترجمه محدود به مواردی است که برای آن ها عباراتی در دامنه مورد نظر وجود دارد. برای مثال، پایه ای ترین ترجمه $\rho_n : T_{\geq n+1} \rightarrow T_{n+1}$ فقط بر روی $\{s_n\}$ با ضابطه $\rho_n(s_n) = s_n$ تعریف می شود و بقیه حالات نادیده گرفته می شوند. این ترجمه ها به زمینه ها نیز به شرح زیر گسترش می یابند.

$$\rho_i(\emptyset) = (0 : s_1)$$

$$\rho_i(\Gamma, s^j x : A) = \rho_i(\Gamma), s^j x : \rho_{j-1}(A), \dots, s^{i+1} x : \rho_i(A) \quad (j \geq i + 1)$$

توجه کنید که اگر $i < j$ آنگاه $\rho_j(\Gamma) \subset \rho_i(\Gamma)$ ، این مورد اغلب در ترکیب با لم تضعیف ۴۸.۱ استفاده می شود.

سه لم زیر ویژگی های کلیدی این ترجمه را توصیف می کنند.

نخست اینکه، ترجمه و عمل جانشینی، به نوعی دارای خاصیت جابه جایی هستند.

لم ۴.۳ (ترجمه با جانشینی). برای هر $0 \leq i \leq n$ و هر $1 \leq j \leq n$ و هر عبارت A و B ، اگر $A \in T_{\geq i+1}$ و $\deg B = j - 1$ ، آنگاه

$$\rho_i(A[B/s^j x]) = \rho_i(A)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]$$

اثبات. روی i استقرای معکوس می زنیم. حکم به وضوح برای ρ_n برقرار است.

برای یک i ثابت، روی ساختار A استقرا می زنیم.

• رده

فرض کنید A به فرم s_k باشد ($k \geq i$). از آنجایی که s_k متغیر آزاد ندارد و 0 یک متغیر منحصر به فرد است، آنگاه

$$\begin{aligned}\rho_i(s_k[B/s^j x]) &= \rho_i(s_k) \\ &= \rho_i(s_k)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

• متغیر

فرض کنید A به فرم $s_k y$ باشد ($k \geq i+2$). اگر $j < i+2$ ، آنگاه

$$\rho_i(s_k y[B/s^j x]) = \rho_i(s_k y)$$

اگر $j \geq i+2$ و $s_k y = s^j x$ ، آنگاه

$$\begin{aligned}\rho_i(s^j x[B/s^j x]) &= \rho_i(B) \\ &= s^{i+2} x [\rho_i(B)/s^{i+2} x] \\ &= \rho_i(s^j x)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

اگر $j \geq i+2$ و $s_k y \neq s^j x$ ، آنگاه

$$\begin{aligned}\rho_i(s_k y[B/s^j x]) &= \rho_i(s_k y) \\ &= \rho_i(s_k y)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

• عبارت Π

فرض کنید A به فرم $\Pi y^C.D$ باشد.

(حالت اول)

اگر $\deg C < i+1$ ، آنگاه

$$\begin{aligned}\rho_i((\Pi y^C.D)[B/s^j x]) &= \rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \rho_i(D[B/s^j x])\end{aligned}$$

اگر $j < i+2$ ، آنگاه طبق فرض استقرا داریم

$$\rho_i(D[B/s^j x]) = \rho_i(D) = \rho_i(\Pi y^C.D)$$

و اگر $j \geq i + 2$ ، به طور مشابه داریم

$$\begin{aligned}\rho_i(D[B/s^j x]) &= \rho_i(D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x] \\ &= \rho_i(\Pi y^C.D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

(حالت دوم)

اگر $\deg C \geq i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((\Pi y^C.D)[B/s^j x]) \\ &= \rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x])\end{aligned}$$

اگر $j < i + 2$ ، آنگاه طبق فرض استقرا داریم

$$\begin{aligned}\rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C)} \dots \Pi y^{\rho_i(C)}.\rho_i(D) \\ &= \rho_i(\Pi y^C.D)\end{aligned}$$

و اگر $j \geq i + 2$ ، به طور مشابه داریم

$$\begin{aligned}\rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]} \dots \Pi y^{\rho_i(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]} \\ &\quad .(\rho_i(D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]) \\ &= (\Pi y^{\rho_{\deg C-1}(C)} \dots \Pi y^{\rho_i(C)}.\rho_i(D))[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x] \\ &= \rho_i(\Pi y^C.D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

توجه داشته باشید که در اینجا ما به طور ضمنی از این لم استفاده کردیم که اگر

$$\rho_k(C[B/s^j x]) = \rho_k(C)$$

$$\rho_k(C) = \rho_k(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]$$

• عبارت λ

فرض کنید A به فرم $\lambda y^C.D$ باشد.

(حالت اول)

اگر $\deg C < i + 2$ ، آنگاه

$$\begin{aligned}\rho_i((\lambda y^C.D)[B/s_j x]) &= \rho_i(\lambda y^{C[B/s_j x]}.D[B/s_j x]) \\ &= \rho_i(D[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

(حالت دوم)

اگر $\deg C \geq i + 2$ ، آنگاه

$$\begin{aligned}\rho_i((\lambda y^C.D)[B/s_j x]) &= \rho_i(\lambda y^{C[B/s_j x]}.D[B/s_j x]) \\ &= \lambda y^{\rho_{\deg C-1}(C[B/s_j x])} \dots \lambda y^{\rho_i(C[B/s_j x])}.\rho_i(D[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

• کاربرد

فرض کنید A به فرم MN باشد.

(حالت اول)

اگر $\deg N < i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((MN)[B/s_j x]) &= \rho_i((M[B/s_j x])(N[B/s_j x])) \\ &= \rho_i(M[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

(حالت دوم)

اگر $\deg N \geq i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((MN)[B/s_j x]) &= \rho_i((M[B/s_j x])(N[B/s_j x])) \\ &= \rho_i(M[B/s_j x])\rho_{\deg N-1}(N[B/s_j x]) \dots \rho_i(N[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

□

لم بعدی بیان می‌کند که این ترجمه، کاهش‌های β را حفظ می‌کند. این ویژگی برای ترجمه قواعد تبدیل ضروری است؛ چرا که باید بتوانیم پس از اعمال ترجمه، نوع‌های β -هم‌ارز را جایگزین یکدیگر کنیم.

شایان ذکر است که این لم لزوماً به معنای حفظ مسیرهای کاهش نامتناهی توسط ترجمه نیست. دلیل این امر آن است که در جریان ترجمه، اطلاعات مربوط به زیرعبارت‌ها از دست می‌رود؛ برای مثال در مورد یک کاربرد، ممکن است داشته باشیم $\rho_i(MN) = \rho_i(M)$ ، و در نتیجه امیدی به حفظ یک دنباله کاهش نامتناهی مربوط به زیرعبارت N وجود نخواهد داشت.

لم ۵.۳ (حفظ مسیر کاهش). برای هر $0 \leq i \leq n$ و هر عبارت A و B ، اگر $A \in T_{\geq i+1}$ و $A \rightarrow_{\beta} B$ ، آنگاه

$$\rho_i(A) \rightarrow_{\beta} \rho_i(B)$$

اثبات. روی i استقرای معکوس می‌زنیم. حکم به وضوح برای ρ_n برقرار است. برای یک i ثابت، با استفاده از استقرا روی تعریف رابطه کاهش β چندگانه، پیش می‌رویم. در مورد کاهش به فرم

$$\rho_i((\lambda x^A.M)N) \rightarrow_{\beta} \rho_i(M[N/s_{\deg A} x])$$

اگر $\deg N < i + 1$ (و $\deg A < i + 2$)، آنگاه

$$\begin{aligned} \rho_i((\lambda x^A.M)N) &= \rho_i(\lambda x^A.M) \\ &= \rho_i(M) \\ &= \rho_i(M[N/s_{\deg A} x]) \end{aligned}$$

که تساوی آخر از لم ۴.۳ حاصل شده است.

و اگر $\deg N \geq i + 1$ (و $\deg A \geq i + 2$)، آنگاه

$$\begin{aligned} \rho_i((\lambda x^A.M)N) &= \rho_i(\lambda x^A.M) \rho_{\deg N-1}(N) \dots \rho_i(N) \\ &= (\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_{i+1}(A)}. \rho_i(M)) \rho_{\deg N-1}(N) \dots \rho_i(N) \\ &\rightarrow_{\beta} \rho_i(M) [\rho_{\deg N-1}(N)/s_{\deg A} x] \dots [\rho_i(N)/s_{i+2} x] \end{aligned}$$

توجه داشته باشید که در اینجا ما به طور ضمنی از این لم استفاده کردیم که $s_{\deg A} x$ در A ظاهر نمی‌شود، پس

$$\rho_k(A) [\rho_l(N)/s_{l+2} x] = \rho_k(A)$$

به طوری که $i + 1 \leq k \leq \deg A - 1$ و $i \leq l \leq \deg N - 1$. بنابراین هیچیک از کاهش‌های β بعدی، نوع‌ها را در عبارات λ مرتبط تغییر نمی‌دهند.

به سادگی می‌توان بررسی کرد که این ترجمه، کاهش‌های β را نسبت به سازگاری حفظ می‌کند. برای مثال، فرض کنید A به فرم $\Pi x^C.D$ و B به فرم $\Pi x^{C'}.D$ هستند، به طوری که $C \rightarrow_{\beta} C'$.

اگر $\deg C < i + 1$ ، آنگاه

$$\rho_i(\Pi x^C.D) = \rho_i(D) = \rho_i(\Pi x^{C'}.D)$$

اگر $\deg C \geq i + 1$ ، آنگاه

$$\begin{aligned} \rho_i(\Pi x^C.D) &= \Pi x^{\rho_{\deg C - 1}(C)} \dots \Pi x^{\rho_i(C)}. \rho_i(D) \\ &\rightarrow_{\beta} \Pi x^{\rho_{\deg C - 1}(C')} \dots \Pi x^{\rho_i(C')}. \rho_i(D) \\ &= \rho_i(\Pi x^{C'}.D) \end{aligned}$$

که

$$\rho_i(C) \rightarrow_{\beta} \rho_i(C')$$

از طریق استقرا روی تعریف رابطه کاهش β چندگامه و همچنین

$$\rho_k(C) \rightarrow_{\beta} \rho_k(C')$$

برای $k > i$ از طریق استقرا روی i . توجه داشته باشید که به طور ضمنی از این نکته استفاده کردیم که $\deg C = \deg C'$ (۳۶.۲).

□

در نهایت، ترجمه باید خاصیت نوع‌پذیری را حفظ کند. این امر تضمین می‌کند که این ترجمه، هنگام ترکیب با ترجمه عبارات که در بخش بعدی معرفی می‌شود، انواع خوش‌تعریفی را به وجود می‌آورد. به یاد داشته باشید که λS^* همان نسخه غیروابسته λS است.

لم ۶.۳ (حفظ نوع). برای هر $0 \leq i \leq n$ ، اگر Γ یک زمینه و A و B عباراتی باشند که $A \in T_{\geq i+1}$ و $\Gamma \vdash_{\lambda S} A : B$ ، آنگاه

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

اثبات. روی i استقرای معکوس می‌زنیم. حکم به وضوح برای ρ_n برقرار است. برای یک i ثابت، با استفاده از استقرا روی استنتاج نوع پیش می‌رویم.

• اصل

فرض کنید استنتاج به فرم

$$\vdash_{\lambda S} s_j : s_{j+1}$$

باشد که $j \geq i$. اگر $i \geq 1$ آنگاه حاصل ترجمه معادل است با

$$0 : s_1 \vdash_{\lambda S^*} s_i : s_{i+1}$$

و در غیر این صورت

$$0 : s_1 \vdash_{\lambda S^*} 0 : s_1$$

از آنجایی که هر سیستم کامل، شامل رابطه (s_1, s_2) است، این قضاوت قابل استنتاج است.

• شروع

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma' \vdash_{\lambda S} B : s_j}{\Gamma', {}^{s_j}x : B \vdash_{\lambda S} {}^{s_j}x : B}$$

باشد که $j \geq i + 2$ (پس $\deg({}^{s_j}x) \geq i + 1$). طبق فرض استقرار، برای هر $i \leq k \leq j - 1$ داریم

$$\rho_{k+1}(\Gamma') \vdash_{\lambda S^*} \rho_k(B) : s_{k+1}$$

و آنجا که زمینه $\rho_{i+1}(\Gamma')$ معتبر است، با استفاده از تضعیف، داریم

$$\rho_{i+1}(\Gamma') \vdash_{\lambda S^*} \rho_k(B) : s_{k+1}$$

اکنون با استفاده مکرر از تضعیف می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(B), \dots, {}^{s_{i+3}}x : \rho_{i+2}(B) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}$$

در نهایت با استفاده از قاعده شروع داریم

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(B), \dots, {}^{s_{i+2}}x : \rho_{i+1}(B) \vdash_{\lambda S^*} {}^{s_{i+2}}x : \rho_{i+1}(B)$$

• تضعیف

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma' \vdash_{\lambda S} A : B \quad \Gamma' \vdash_{\lambda S} C : s_j}{\Gamma', {}^{s_j}x : C \vdash_{\lambda S} A : B}$$

باشد.

اگر $j < i + 2$ ، با اعمال فرض استقرا بر حکم مقدم سمت چپ داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

دقت کنید که متغیر ${}^{s_j}x$ توسط ترجمه ρ_{i+1} حذف می‌شود.

اگر $j \geq i + 2$ ، از روشی مشابه آنچه در حالت شروع در بالا ذکر شد، استفاده می‌کنیم؛ یعنی برای هر k که $i \leq k \leq j - 1$ و سپس با استفاده مکرر از تضعیف

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

• تولید

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} C : s_j \quad \Gamma, {}^{s_j}x : C \vdash_{\lambda S} D : s_k}{\Gamma \vdash_{\lambda S} (\Pi x^C.D) : s_k}$$

باشد که $k \geq i + 1$.

اگر $j < i + 1$ ، با اعمال فرض استقرا بر حکم مقدم سمت راست، داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(D) : s_{i+1}$$

اگر $j \geq i + 1$ ، برای هر k که $i \leq k \leq j - 1$ داریم

$$\rho_{k+1}(\Gamma) \vdash_{\lambda S^*} \rho_k(C) : s_{k+1}$$

و هر یک از این استنتاج‌ها را می‌توان به صورت زیر تضعیف کرد

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{k+2}}x : \rho_{k+1}(C) \vdash_{\lambda S^*} \rho_k(C) : s_{k+1}$$

سپس با استفاده مکرر از قاعده تولید به همراه

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(D) : s_{i+1}$$

می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}.(\rho_i(C) \rightarrow \rho_i(D)) : s_{i+1}$$

اینجا از این فرض استفاده می‌کنیم که λS^* برای هر $k \geq i+1$ دارای قوانین (s_k, s_{i+1}, s_{i+1}) است. همچنین در مورد استفاده از نمادگذاری تابع توجه کنید که عبارت $\rho_i(C)$ در زمینه پیشین ظاهر نشده است.

• انتزاع

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma, {}^{s_j}x : C \vdash_{\lambda S} M : D \quad \Gamma \vdash_{\lambda S} \Pi x^C.D : s_k}{\Gamma \vdash_{\lambda S} (\lambda x^C.M) : (\Pi x^C.D)}$$

باشد که $k \geq i+2$.

اگر $\deg C < i+2$ (یعنی اگر $j < i+2$)، از آنجا که متغیر ${}^{s_j}x$ توسط ترجمه ρ_{i+1} حذف می‌شود، استنتاج متناظر عبارت است از

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(B)$$

اگر $\deg C \geq i+2$ ، می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(D)$$

و

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D) : s_{i+2}$$

بنابراین اگر $\deg C \geq i+3$ ، با استفاده مکرر از لم تولید (۴۵.۱) می‌توانیم برای هر $i+1 \leq k \leq j-2$ نتیجه بگیریم

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{k+2}}x : \rho_{k+1}(C) \vdash_{\lambda S^*} \Pi x^{\rho_k(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D) : s_{i+2}$$

و بدین ترتیب، با استفاده مکرر از قاعده انتزاع، می‌توانیم استنتاج کنیم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} (\lambda x^{\rho_{j-1}(C)} \dots \lambda x^{\rho_{i+1}(C)}. \rho_i(M)) : (\Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D))$$

• کاربرد

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} M : (\Pi x^C.D) \quad \Gamma \vdash_{\lambda S} N : C}{\Gamma \vdash_{\lambda S} MN : D[N/x]}$$

باشد.

چون $\deg(MN) = \deg M$ ، با اعمال فرض استقرا بر حکم مقدم سمت چپ داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(\Pi x^C.D)$$

اگر $\deg N < i + 1$ (و $\deg C < i + 2$)، آنگاه

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(D)$$

اگر $\deg N \geq i + 1$ (و $\deg C \geq i + 2$)، آنگاه داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \Pi x^{\rho_{\deg C-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D)$$

و برای هر $i \leq k \leq \deg N - 1$

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_k(N) : \rho_{k+1}(C)$$

اولین کاربرد $\rho_i(M)\rho_{\deg N-1}(N)$ دارای نوع زیر است

$$\Pi x^{\rho_{\deg C-2}(C)[\rho_{\deg C-2}(N)/^{s_{\deg C} x}]} \dots \Pi x^{\rho_{i+1}(C)[\rho_{\deg C-2}(N)/^{s_{\deg C} x}]} . (\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}])$$

عبارت $\rho_{\deg C-2}(C)$ متغیر $^{s_{\deg C} x}$ را ندارد، لذا این نوع ساده می شود به

$$\Pi x^{\rho_{\deg C-2}(C)} \dots \Pi x^{\rho_{i+1}(C)}. (\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}])$$

با تکرار این فرآیند، داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M)\rho_{j-1}(N) \dots \rho_i(N) : \rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}] \dots [\rho_i(N)/^{s_{i+2} x}]$$

از آنجایی که $^{s_{i+2} x}$ در $\rho_{i+1}(D)$ ظاهر نمی شود، نوع مذکور معادل است با

$$\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}] \dots [\rho_{i+1}(N)/^{s_{i+3} x}]$$

که طبق لم ۴.۳ معادل $\rho_{i+1}(D[N/x])$ است.

- تبدیل
فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} A : C \quad \Gamma \vdash_{\lambda S} B : s_k}{\Gamma \vdash_{\lambda S} A : B}$$

است که $k \geq i + 2$. آنگاه استنتاج متناظر به صورت زیر است

$$\frac{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(C) \quad \frac{\rho_{i+2}(\Gamma) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}}{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}}}{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)}$$

که در آن طبق لم ۵.۳ داریم: $\rho_{i+1}(B) = \rho_{i+1}(C)$

□

۴.۳ ترجمه سطح عبارت

در ادامه تابع ترجمه $\gamma : T \rightarrow T_0$ را بر روی همه عبارات تعریف می‌کنیم. این ترجمه، یک تفاوت اصلی با ترجمه Geuvers-Nederhof [۹] دارد. در ترجمه آن‌ها، متغیرها گاهی از طریق افزودن عبارات $\perp$ به زمینه، با عبارت‌های ساختگی جایگزین می‌شدند. از آنجا که $(s_2, s_1) \in \mathcal{R}_{\lambda\omega}$ ، مورد مذکور شدنی است. استفاده مستقیم از این تکنیک، به قوانین برای (s_{i+1}, s_i) نیاز دارد. با این حال، به دلیل کامل بودن، $(s_j, s_k) \in \mathcal{R}_{\lambda S}$ نیازمند این است که برای $l \in [k]$ داشته باشیم $(s_l, s_1) \in \mathcal{R}_{\lambda S}$ ، که به ما اجازه استفاده از متغیر $\text{prod} : \Pi A^{s_1}.0 \rightarrow A \rightarrow 0$ که فقط نیازمند قانون (s_2, s_1) است را می‌دهد. ما فرض می‌کنیم که این قانون در λS موجود است.

تعریف ۷.۳ (تابع ترجمه سطح عبارت). تابع $\gamma : T \rightarrow T$ به صورت استقرایی زیر تعریف می‌شود.

$$\begin{aligned} \gamma(s_i) &= \bullet \\ \gamma(s^i x) &= s^1 x \\ \gamma(\Pi x^A.B) &= \text{prod}(\Pi x^{\rho_{deg A-1}(A)} \dots \Pi x^{\rho_0(A)}.0) \gamma(A) (\lambda x^{\rho_{deg A-1}(A)} \dots \lambda x^{\rho_0(A)}. \gamma(B)) \\ \gamma(\lambda x^A.M) &= (\lambda z^0. \lambda x^{\rho_{deg A-1}(A)} \dots \lambda x^{\rho_0(A)}. \gamma(M)) \gamma(A) \\ \gamma(MN) &= \gamma(M) \rho_{deg N-1}(N) \dots \rho_0(N) \gamma(N) \end{aligned}$$

که در آن $\{\text{prod}, \bullet, z\}$ متغیرهای منحصر به فردی هستند.

سه لمی که در بخش قبل بررسی کردیم، برای ترجمه سطح عبارت نیز قابل بیان هستند.

لم ۸.۳ (حفظ نوع). برای هر عبارت A و B ، اگر $\Gamma \vdash_{\lambda S} A : B$ ، آنگاه

$$0 : s_1, \bullet : 0, \text{prod} : \Pi A^{s_1}. 0 \rightarrow A \rightarrow 0, \rho_0(\Gamma) \vdash_{\lambda S^*} \gamma(A) : \rho_0(B)$$

اثبات. بر روی استنتاج‌ها استقرا می‌زنیم. مواردی که در آن‌ها آخرین گام استنتاج، شروع یا تضعیف است، مشابه موارد متناظر در لم ۶.۳ هستند. در ادامه، فرض کنید

$$\Delta := (0 : s_1, \bullet : 0, \text{prod} : \Pi A^{s_1}. 0 \rightarrow A \rightarrow 0)$$

• اصل

اگر استنتاج به فرم

$$\vdash s_i : s_{i+1}$$

باشد، آنگاه استنتاج ترجمه‌شده به شکل زیر است

$$\Delta \vdash \bullet : 0$$

که به وضوح برقرار است.

• تولید

فرض کنید آخرین استنتاج به شکل زیر باشد

$$\frac{\Gamma \vdash C : s_i \quad \Gamma, {}^{s_i}x : C \vdash D : s_j}{\Gamma \vdash (\Pi x^C. D) : s_j}$$

بنا بر فرض استقرا

$$\Delta, \rho_0(\Gamma) \vdash \gamma(C) : 0$$

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_1}x : \rho_0(C) \vdash \gamma(D) : 0$$

به کمک تضعیف، داریم

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_1}x : \rho_0(C) \vdash 0 : s_1$$

توجه داشته باشید که به دلیل کامل بودن، از آنجا که $(s_i, s_j) \in \mathcal{R}_{\lambda\mathcal{S}}$ ، آنگاه

$$\forall k \in [i], (s_k, s_1) \in \mathcal{R}_{\lambda\mathcal{S}}$$

بدین ترتیب، با استفاده مکرر از تولید، که همگی با استفاده از ترجمه‌های از پیش تعریف شده صورت می‌گیرد؛

$$\rho_{k+1}(\Gamma) \vdash \rho_k(C) : s_{k+1}$$

می‌توانیم برای هر $1 \leq k \leq i$ بنویسیم

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_{k+1}}x : \rho_k(C) \vdash \Pi {}^{s_k}x^{\rho_{k-1}(C)} \dots \Pi {}^{s_1}x^{\rho_0(C)}.0 : s_1$$

همچنین

$$\Delta, \rho_0(\Gamma) \vdash \lambda {}^{s_i}x^{\rho_{i-1}(C)} \dots \lambda {}^{s_1}x^{\rho_0(C)}. \gamma(D) : 0$$

در نهایت با دو کاربرد با استفاده از استنتاج‌های تعریف شده‌ی قبلی، داریم

$$\Delta, \rho_0(\Gamma) \vdash \text{prod}(\Pi {}^{s_i}x^{\rho_{i-1}(C)} \dots \Pi {}^{s_1}x^{\rho_0(C)}.0) \gamma(A) (\lambda {}^{s_i}x^{\rho_{i-1}(C)} \dots \lambda {}^{s_1}x^{\rho_0(C)}. \gamma(D)) : 0$$

• انتزاع

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma, {}^{s_i}x : C \vdash M : D \quad \Gamma \vdash \Pi x^C.D : s_j}{\Gamma \vdash \lambda x^C.M : \Pi x^C.D}$$

مشابه مورد متناظر در اثبات لم ۶.۳، داریم

$$\rho_0(\Gamma) \vdash (\lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

و به کمک تضعیف، برای یک متغیر جدید z داریم

$$\rho_0(\Gamma), z : 0 \vdash (\lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

بنابراین

$$\frac{\rho_0(\Gamma) \vdash (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D)) : s_1 \quad \rho_0(\Gamma) \vdash 0 : s_1}{\rho_0(\Gamma) \vdash (0 \rightarrow \Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D)) : s_1}$$

پس با استفاده از انتزاع و کاربرد می‌توانیم بنویسیم

$$\rho_0(\Gamma) \vdash ((\lambda z^0. \lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) \gamma(C)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

• کاربرد

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma \vdash M : (\Pi x^C. D) \quad \Gamma \vdash N : C}{\Gamma \vdash MN : D[N/s_{\deg C} x]}$$

اگر $\deg N = 0$ ، آنگاه داریم

$$\frac{\rho_0(\Gamma) \vdash \gamma(M) : \rho_0(C) \rightarrow \rho_0(D) \quad \rho_0(\Gamma) \vdash \gamma(N) : \rho_0(C)}{\rho_0(\Gamma) \vdash \gamma(M)\gamma(N) : \rho_0(D)}$$

توجه کنید که به کمک لم ۴.۳، از این نکته استفاده کردیم که $\rho_0(D[N/s_1 x]) = \rho_0(D)$ حالتی که $\deg N \geq 1$ باشد، مشابه حالت متناظر در اثبات لم ۶.۳ است.

• تبدیل

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma \vdash A : C \quad \Gamma \vdash B : s_i}{\Gamma \vdash A : B}$$

طبق لم ۵.۳، می‌دانیم $\rho_0(C) = \rho_0(B)$. پس داریم

$$\frac{\rho_0(\Gamma) \vdash \gamma(A) : \rho_0(C) \quad \frac{\rho_1(\Gamma) \vdash \rho_0(B) : s_1}{\rho_0(\Gamma) \vdash \rho_0(B) : s_1}}{\rho_0(\Gamma) \vdash \gamma(A) : \rho_0(B)}$$

□

در ادامه، مجدد نشان می‌دهیم که عمل جانشینی با ترجمه جابه‌جا می‌شود. این نتیجه بیشتر جنبه کمکی دارد، چرا که برای لم بعدی که در مورد حفظ کاهش β است، به آن نیازمندیم.

لم ۹.۳ (ترجمه با جانشینی). برای هر $1 \leq j \leq n$ ، و هر عبارت A و B که $\deg(B) = j - 1$ ، داریم

$$\gamma(A[B/s_j^j x]) = \gamma(A)[\rho_{j-2}(B)/s_j^j x] \dots [\rho_0(B)/s_2^2 x][\gamma(B)/s_1^1 x]$$

اثبات. روی ساختار A استقرا می‌زنیم. حالاتی که در آن‌ها، A یک رده یا یک متغیر است، ساده هستند.

• عبارت Π

فرض کنید A به فرم $\Pi y^C.D$ باشد. در ادامه از نمادگذاری مشابه [۲] استفاده می‌کنیم:

$$M^+ \equiv M[B/s_j x]$$

داریم

$$\begin{aligned} & \gamma((\Pi y^C.D)^+) \\ &= \gamma(\Pi y^{C^+}.D^+) \\ &= \text{prod}(\Pi^{s_{\deg(C^+)}} x^{\rho_{\deg(C^+)-1}(C^+)} \dots \Pi^{s_1} x^{\rho_0(C^+)}.0) \\ & \quad \gamma(A)(\lambda^{s_{\deg(C^+)}} x^{\rho_{\deg(C^+)-1}(C^+)} \dots \lambda^{s_1} x^{\rho_0(C^+)}. \gamma(D^+)) \end{aligned}$$

از آنجا که تمام جانشینی‌ها جابه‌جا می‌شوند، چه بر اساس فرض استقرا و چه از طریق **لم ۴.۳**، حکم برقرار است.

• حالات عبارت λ و کاربرد، به شیوه مشابه اثبات می‌شوند.

□

لم ۱۰.۳ (حفظ مسیر کاهش). برای هر عبارت A و B ، اگر $A \rightarrow_\beta B$ ، آنگاه

$$\gamma(A) \twoheadrightarrow_\beta^+ \gamma(B)$$

اثبات. روی تعریف رابطه کاهش β چندگانه، استقرا می‌زنیم. در مورد کاهش به فرم

$$(\lambda x^A.M)N \rightarrow_\beta M[N/s_{\deg A} x]$$

اگر $\deg N = 0$ (و $\deg A = 1$)، آنگاه

$$\begin{aligned} \gamma((\lambda x^A.M)N) &= \gamma(\lambda x^A.M)\gamma(N) \\ &= (\lambda z^0.\lambda x^{\rho_0(A)}. \gamma(M))\gamma(A)\gamma(N) \\ &\rightarrow_\beta (\lambda x^{\rho_0(A)}. \gamma(M))\gamma(N) \\ &\rightarrow_\beta \gamma(M)[\gamma(N)/s_1 x] \\ &= \gamma(M[N/s_1 x]) \end{aligned}$$

اگر $\deg N \geq 1$ ، آنگاه

$$\begin{aligned}
 \gamma((\lambda x^A.M)N) &= \gamma(\lambda x^A.M)\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &= (\lambda z^0.\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_0(A)}.\gamma(M))\gamma(A)\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &\rightarrow_{\beta} (\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_0(A)}.\gamma(M))\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &\rightarrow_{\beta} \gamma(M)[\rho_{\deg N-1}(N)/^{s_{\deg A}}x] \dots [\rho_0(N)/^{s_2}x][\gamma(N)/^{s_1}x] \\
 &= \gamma(M[N/^{s_{\deg A}}x])
 \end{aligned}$$

مشابه اثبات لم ۵.۳، اثبات اینکه ترجمه، کاهش‌های β را از نظر سازگاری حفظ می‌کند، ساده و سرراست است. نکته کلیدی این است که γ بر تک تک زیرعبارت‌های عبارت پیش‌تر ترجمه‌شده اعمال می‌شود؛ بنابراین، سازگاری، کاهش‌های β غیرصفر را حفظ می‌کند.

□

۵.۳ نتیجه اصلی

قضیه ۱۱.۳. فرض کنید λS یک سیستم نوع محض n -لایه‌ای است که تا یک مقدار i کامل است ($i < n$). λS دارای خاصیت نرمال‌سازی قوی است، اگر و تنها اگر λS^* چنین باشد.

اثبات. از آنجایی که همه عبارات قابل استنتاج λS^* ، در λS نیز استنتاج می‌شوند، اگر λS دارای خاصیت نرمال‌سازی قوی باشد آنگاه λS^* نیز دارای این خاصیت خواهد بود. حال فرض کنید λS دارای خاصیت نرمال‌سازی قوی نباشد. پس یک دنباله کاهش نامتناهی مانند

$$M_1 \rightarrow_{\beta} M_2 \rightarrow_{\beta} \dots$$

در آن وجود دارد. طبق لم‌های ۸.۳ و ۱۰.۳، می‌توان گفت دنباله کاهش نامتناهی

$$\gamma(M_1) \twoheadrightarrow_{\beta}^+ \gamma(M_2) \twoheadrightarrow_{\beta}^+ \dots$$

در λS^* یافت می‌شود، که با فرض آنکه λS^* دارای خاصیت نرمال‌سازی قوی است، در تناقض است.

□

تنها سیستم واقعا جالبی که این قضیه بر آن قابل اعمال است، سیستم n -لایه‌ای با قوانین زیر است

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

و همچنین هر زیرسیستمی از آن که دارای همان قوانین غیروابسته باشد. قضیه ۱۱.۳ بیان می‌کند که خاصیت نرمال‌سازی قوی برای این سیستم، معادل همین خاصیت برای سیستم با قوانین زیر است.

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_2, s_2)\}$$

هرچند می‌توان این نتیجه را به صورت صوری نیز اثبات کرد، بررسی اجمالی این روش نشان می‌دهد که از دیدگاه فرانظری، این روش ضعیف است و می‌توان آن را در حساب پئانو اجرا کرد. از این رو، با ترکیب این نتیجه با نتیجه‌ی [۸]، به حالت خاصی از حدس قوی BGK دست می‌یابیم.

نتیجه ۱۲.۳. در سیستم نوع محض n -لایه‌ای با قوانین

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی است. همچنین، این اثبات را می‌توان در حساب پئانو انجام داد.

بخش سوم

نتایج و پیشنهادات آینده

فصل ۴

نتیجه‌گیری و جمع‌بندی

۱.۴ جمع‌بندی

در این پایان‌نامه، سیستم‌های نوع محض و تعمیم لایه‌ای آن‌ها، یعنی سیستم‌های نوع محض لایه‌ای، بر اساس رساله دکتری Mull [۱۰]، به‌طور کامل در اثبات‌یار Coq صوری‌سازی شدند. نحو عبارات، جانشینی، کاهش β ، نرمال‌سازی ضعیف و قوی و استنتاج نوع برای هر دو سیستم به همراه لم‌های مرتبط با آن‌ها صوری‌سازی و اثبات شدند.

در بخش نهایی کار، که به اثبات لم‌های مربوط به ترجمه حذف وابستگی اختصاص دارد، به دلیل محدودیت زمانی، برخی از گام‌های اثبات این لم‌ها به‌صورت Axiom در Coq فرض شده‌اند. تکمیل اثبات صوری این موارد، طبیعی‌ترین نقطه آغاز برای ادامه این کار است.

۲.۴ پیشنهادات

Mull در جمع‌بندی رساله خود [۱۰]، تعمیم ترجمه Geuvers–Nederhof [۹] به سیستم‌های ناپیش‌گویانه^۱ رده بالاتر را «یک آزمایش ناموفق در تعمیم‌دهی» می‌نامد. به گفته او، تمام تلاش‌ها برای تضعیف شرایط بخش غیروابسته سیستم پایه، معمولاً به دلیل عدم بسته بودن تحت جانشینی یا تساوی β با شکست مواجه شده‌اند. او این احتمال را مطرح می‌کند که شاید بتوان با تعدیلی جزئی در روش اثبات، همان نتیجه را برای سیستم‌های n -لایه‌ای با مجموعه قوانین

$$\{(s_1, s_k) \mid k \in [n]\} \cup \{(s_i, s_j) \mid 1 \leq i \leq j \leq n\}$$

impredicative^۱

به دست آورد؛ چرا که این مجموعه قوانین، نقش نتیجه Geuvers–Nederhof را بهتر بازتاب می‌دهد. این پرسش، به عنوان یک مسئله باز، هنوز پاسخ داده نشده است و می‌تواند موضوع پژوهش‌های آتی باشد.

بر این اساس، به عنوان جهت‌های پژوهشی آتی پیشنهاد می‌شود:

- بررسی دقیق‌تر مجموعه قوانین پیشنهادی Mull و تلاش برای یافتن تعدیلی در اثبات ترجمه حذف وابستگی که بتواند نتیجه اصلی رساله را برای این مجموعه قوانین نیز اثبات کند.
- بررسی این‌که آیا استدلال فرانظری Mull درباره قابل انجام بودن اثبات قضیه ۱۱.۳ در حساب پئانو، می‌تواند به‌طور دقیق و صوری تنظیم و اثبات شود؛ و در صورت امکان، صوری‌سازی این استدلال در Coq.
- صوری‌سازی کامل نتیجه ۱۲.۳ در Coq، به عنوان یک حالت خاص از حدس قوی BGK، به‌ویژه با تکیه بر صوری‌سازی موجود این پایان‌نامه از PTS و TPTS.
- تکمیل و جایگزینی Axiom های به کاررفته در این پایان‌نامه با اثبات‌های صوری کامل، به منظور دستیابی به یک صوری‌سازی کامل و بدون فرض اضافی از نتایج Mull.

مراجع

- [1] H. Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991.
- [2] H. Barendregt. *Lambda Calculi with Types*. Cambridge University Press, 2013.
- [3] S. Berardi. *Towards a mathematical analysis of the Coquand–Huet calculus of constructions and the other systems in Barendregt’s cube*. PhD thesis, University of Torino and Carnegie Mellon University, 1988.
- [4] A. Church. A set of postulates for the foundation of logic. *Annals of Mathematics*, 33(2):346–366, 1932.
- [5] A. Church. A formulation of the simple theory of types. *Journal of Symbolic Logic*, 5(2):56–68, 1940.
- [6] T. Coquand and H. Herbelin. A-translation and looping combinators in pure type systems. *Journal of Functional Programming*, 4(1):77–88, 1994.

- [7] H. Geuvers. *Logics and Type Systems*. PhD thesis, University of Nijmegen, 1993.
- [8] M. H. S. Gilles Barthe, John Hatcliff. Weak normalization implies strong normalization in a class of non-dependent pure type systems. *Theoretical Computer Science*, 269(1–2):317–361, 2001.
- [9] M.-J. N. Herman Geuvers. Modular proof of strong normalization for the calculus of constructions. *Journal of Functional Programming*, 1(2):155–189, 1991.
- [10] N. Mull. *Weak and String Normalization of Tiered Pure Type Systems via Type-Perserving Translation*. PhD thesis, The University of Chicago, 2023.
- [11] M. H. A. Newman. On theories with a combinatorial definition of equivalence. *Annals of Mathematics*, 43(2):223–243, 1942.
- [12] F. H. Robert Harper and G. Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, 1993.
- [13] C. Roux and F. van Doorn. The structural theory of pure type systems. In *Rewriting and Typed Lambda Calculi*, pages 364–378. Springer, 2014.
- [14] J. Terlouw. Een nadere bewijstheoretische analyse van gsts. Technical report, University of Nijmegen, 1989.

پوست: کدهای COQ

مشخصه سیستم‌های نوع محض

```
1 Module Type PTS_SIGNATURE.  
2   Parameter Sort : Type.  
3   Parameter A : Sort -> Sort -> Prop.  
4   Parameter R : Sort -> Sort -> Sort -> Prop.  
5 End PTS_SIGNATURE.
```

مشخصه سیستم‌های نوع محض لایه‌ای

```
1 Require Import Thesis.PTSSignature.  
2  
3 Module Type TPTS_SIGNATURE <: PTS_SIGNATURE.  
4   Include PTS_SIGNATURE.  
5  
6   Parameter n : nat.  
7   Parameter index_of : Sort -> nat.  
8  
9   Axiom n_range : n > 0.  
10  Axiom index_range : forall x, 0 < index_of x /\ index_of x < n + 1.  
11  Axiom index_inj : forall x y, index_of x = index_of y -> x = y.  
12  Axiom index_surj : forall i : nat, 0 < i /\ i < n + 1 -> exists x :  
    Sort, index_of x = i.  
13  
14  Axiom A_spec : forall x y, A x y <-> index_of x < n /\ index_of y =  
    S (index_of x).  
15  Axiom R_shape : forall x y z, R x y z -> y = z.  
16 End TPTS_SIGNATURE.
```

سیستم نوع محض و لم‌ها

```
1 From Stdlib Require Import List.  
2 From Stdlib Require Import Classical.
```

```

3  From Stdlib Require Import Logic.IndefiniteDescription.
4  From Stdlib.Program Require Import Program Wf.
5  From Stdlib Require Import Lia.
6  From Stdlib Require Import Arith.Arith.
7  From Stdlib Require Import ZArith.
8  Import ListNotations.
9
10 Require Import Thesis.PTSSignature.
11
12 Module PTS (Sig : PTS_SIGNATURE).
13
14 Import Sig.
15
16 (* =====
17 *)
18 (** * Syntax *)
19
20 Definition var := nat.
21
22 Definition eq_var_dec : forall x y : var, {x = y} + {x <> y} :=
23   Nat.eq_dec.
24
25 Inductive term : Type :=
26   | t_sort : Sort -> term
27   | t_bvar : Sort -> nat -> term
28   | t_fvar : Sort -> var -> term
29   | t_app  : term -> term -> term
30   | t_lam  : Sort -> term -> term -> term
31   | t_pi   : Sort -> term -> term -> term.
32
33 (* =====
34 *)
35 (** * Free variables *)
36
37 Fixpoint fv (t : term) : list var :=
38   match t with
39   | t_sort _      => []
40   | t_bvar _ _    => []
41   | t_fvar _ x    => [x]
42   | t_pi _ A B    => fv A ++ fv B
43   | t_lam _ A M    => fv A ++ fv M
44   | t_app M N     => fv M ++ fv N
45   end.
46
47 (* =====
48 *)
49 (** * Opening *)
50
51 Fixpoint open_rec (k : nat) (u : term) (t : term) : term :=

```

```

48   match t with
49   | t_sort s    => t_sort s
50   | t_bvar s n => if Nat.eqb n k then u else t_bvar s n
51   | t_fvar s x => t_fvar s x
52   | t_app M N  => t_app (open_rec k u M) (open_rec k u N)
53   | t_pi s A B  => t_pi s (open_rec k u A) (open_rec (S k) u B)
54   | t_lam s A M => t_lam s (open_rec k u A) (open_rec (S k) u M)
55   end.
56
57   Notation "t ^^ u" := (open_rec 0 u t) (at level 67).
58
59   Definition open_var (t : term) (s : Sort) (x : var) : term := open_rec
60   0 (t_fvar s x) t.
61
62   (* =====
63   *)
64   (** * Closing *)
65
66   Fixpoint close_rec (k : nat) (x : var) (t : term) : term :=
67   match t with
68   | t_sort s    => t_sort s
69   | t_bvar s n => t_bvar s n
70   | t_fvar s y => if eq_var_dec y x then t_bvar s k else t_fvar s y
71   | t_app M N  => t_app (close_rec k x M) (close_rec k x N)
72   | t_pi s A B  => t_pi s (close_rec k x A) (close_rec (S k) x B)
73   | t_lam s A M => t_lam s (close_rec k x A) (close_rec (S k) x M)
74   end.
75
76   Definition close (x : var) (t : term) : term := close_rec 0 x t.
77
78   (* =====
79   *)
80   (** * Substitution for a free variable *)
81
82   Reserved Notation "M [ x = N ]"
83   (at level 1, left associativity).
84
85   Fixpoint subst_term (x : var) (N : term) (t : term) : term :=
86   match t with
87   | t_sort s    => t_sort s
88   | t_bvar s n => t_bvar s n
89   | t_fvar s y => if eq_var_dec y x then N else t_fvar s y
90   | t_app P Q  => t_app (P[x = N]) (Q[x = N])
91   | t_pi s A B  => t_pi s (A[x = N]) (B[x = N])
92   | t_lam s A M => t_lam s (A[x = N]) (M[x = N])
93   end
94   where "M [ x = N ]" := (subst_term x N M).
95
96   Lemma subst_sort : forall s x N,

```

```

95   (t_sort s)[x = N] = t_sort s.
96 Proof. reflexivity. Qed.
97
98 Lemma subst_bvar : forall s n x N,
99   (t_bvar s n)[x = N] = t_bvar s n.
100 Proof. reflexivity. Qed.
101
102 Lemma subst_var : forall s y x N,
103   (t_fvar s y)[x = N] = if eq_var_dec y x then N else t_fvar s y.
104 Proof. reflexivity. Qed.
105
106 Lemma subst_app : forall M1 M2 x N,
107   (t_app M1 M2)[x = N] = t_app (M1[x = N]) (M2[x = N]).
108 Proof. reflexivity. Qed.
109
110 Lemma subst_pi : forall s A B x N,
111   (t_pi s A B)[x = N] = t_pi s (A[x = N]) (B[x = N]).
112 Proof. reflexivity. Qed.
113
114 Lemma subst_lam : forall s A M x N,
115   (t_lam s A M)[x = N] = t_lam s (A[x = N]) (M[x = N]).
116 Proof. reflexivity. Qed.
117
118 (* =====
119 *)
120 (** * Local closure *)
121 Inductive lc : term -> Prop :=
122   | lc_sort : forall s, lc (t_sort s)
123   | lc_fvar : forall s x, lc (t_fvar s x)
124   | lc_app : forall M N, lc M -> lc N -> lc (t_app M N)
125   | lc_pi : forall s A B L,
126     lc A ->
127     (forall x, ~ In x L -> lc (open_var B s x)) ->
128     lc (t_pi s A B)
129   | lc_lam : forall s A M L,
130     lc A ->
131     (forall x, ~ In x L -> lc (open_var M s x)) ->
132     lc (t_lam s A M).
133
134 (* =====
135 *)
136 (** * Bound-variable tag well-formedness. *)
137 Fixpoint bvar_tag_ok (k : nat) (s : Sort) (t : term) : Prop :=
138   match t with
139   | t_sort _ => True
140   | t_bvar s' n => n = k -> s' = s
141   | t_fvar _ _ => True
142   | t_app M _ => bvar_tag_ok k s M
143   | t_lam _ _ M => bvar_tag_ok (S k) s M

```

```

143 | t_pi _ _ B => bvar_tag_ok (S k) s B
144 end.
145
146 (* =====
147 *)
148 (** * Freshness of atoms *)
149
150 Fixpoint max_var_idx (xs : list var) : nat :=
151   match xs with
152   | [] => 0
153   | v :: xs => Nat.max v (max_var_idx xs)
154   end.
155
156 Definition fresh (xs : list var) : var := S (max_var_idx xs).
157
158 Lemma max_var_idx_ge : forall xs v,
159   In v xs -> v <= max_var_idx xs.
160 Proof.
161   induction xs as [| a xs IH]; intros v Hin.
162   - contradiction.
163   - simpl in Hin. destruct Hin as [Heq | Hin].
164     + subst. simpl. lia.
165     + simpl. specialize (IH v Hin). lia.
166 Qed.
167
168 Lemma fresh_notin : forall xs, ~ In (fresh xs) xs.
169 Proof.
170   intros xs Hin.
171   apply max_var_idx_ge in Hin.
172   unfold fresh in Hin. lia.
173 Qed.
174
175 (* =====
176 *)
177 (** * The open/subst/lc toolkit *)
178
179 Lemma not_in_app : forall (x : var) l1 l2,
180   ~ In x (l1 ++ l2) -> ~ In x l1 /\ ~ In x l2.
181 Proof.
182   intros x l1 l2 H. split; intro Hc; apply H; apply in_or_app; auto.
183 Qed.
184
185 Lemma not_in_cons_elim : forall (x : var) L y,
186   ~ In y (x :: L) -> y <> x /\ ~ In y L.
187 Proof.
188   intros x L y H. split.
189   - intro Hc. apply H. left. symmetry. exact Hc.
190   - intro Hc. apply H. right. exact Hc.
191 Qed.

```

```

191 Lemma open_rec_lc_core : forall t j v i u,
192   i <> j ->
193   open_rec j v t = open_rec i u (open_rec j v t) ->
194   t = open_rec i u t.
195 Proof.
196   induction t as [s | sn n | sx x | M IHM N IHN | sl A IHA M IHM | sp
    A IHA B IHB];
197   intros j v i u Hne Heq.
198   - reflexivity.
199   - destruct (Nat.eq_dec n j) as [Hnj | Hnj].
200     + subst n. simpl.
201       destruct (Nat.eq_dec j i) as [Hji | Hji].
202       * exfalso. apply Hne. symmetry. exact Hji.
203       * apply Nat.eqb_neq in Hji. rewrite Hji. reflexivity.
204     + simpl in Heq. apply Nat.eqb_neq in Hnj. rewrite Hnj in Heq.
205       simpl in Heq. exact Heq.
206   - reflexivity.
207   - simpl in Heq |- *.
208     injection Heq as HeqM HeqN. f_equal.
209     + apply (IHM j v i u Hne HeqM).
210     + apply (IHN j v i u Hne HeqN).
211   - simpl in Heq |- *.
212     injection Heq as HeqA HeqM. f_equal.
213     + apply (IHA j v i u Hne HeqA).
214     + apply (IHM (S j) v (S i) u (fun Hc => Hne (eq_add_S i j Hc))
        HeqM).
215   - simpl in Heq |- *.
216     injection Heq as HeqA HeqB. f_equal.
217     + apply (IHA j v i u Hne HeqA).
218     + apply (IHB (S j) v (S i) u (fun Hc => Hne (eq_add_S i j Hc))
        HeqB).
219 Qed.
220
221 Lemma subst_fresh : forall t x u,
222   ~ In x (fv t) -> t[x := u] = t.
223 Proof.
224   induction t as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp
    A IHA B IHB];
225   intros x u Hnotin; simpl in *.
226   - reflexivity.
227   - reflexivity.
228   - destruct (eq_var_dec y x) as [Heq | Hneq].
229     + subst y. exfalso. apply Hnotin. left. reflexivity.
230     + reflexivity.
231   - destruct (not_in_app x (fv M) (fv N) Hnotin) as [H1 H2]. f_equal;
auto.
232   - destruct (not_in_app x (fv A) (fv M) Hnotin) as [H1 H2]. f_equal;
auto.
233   - destruct (not_in_app x (fv A) (fv B) Hnotin) as [H1 H2]. f_equal;
auto.

```



```

234 Qed.
235
236 Lemma open_rec_lc : forall t, lc t -> forall k u, open_rec k u t = t.
237 Proof.
238   intros t H.
239   induction H as [ s | sx x | M N HM IHM HN IHN
240                 | sp A B L HA IHA HB IHB
241                 | sl A M L HA IHA HM IHM ];
242   intros k u; simpl.
243   - reflexivity.
244   - reflexivity.
245   - f_equal; auto.
246   - f_equal.
247     + auto.
248     + remember (fresh L) as x0 eqn:Hx0def.
249       assert (Hx0 : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
250       specialize (IHB x0 Hx0 (S k) u).
251       unfold open_var in IHB.
252       symmetry.
253       apply (open_rec_lc_core B 0 (t_fvar sp x0) (S k) u).
254       * lia.
255       * symmetry. exact IHB.
256   - f_equal.
257     + auto.
258     + remember (fresh L) as x0 eqn:Hx0def.
259       assert (Hx0 : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
260       specialize (IHM x0 Hx0 (S k) u).
261       unfold open_var in IHM.
262       symmetry.
263       apply (open_rec_lc_core M 0 (t_fvar sl x0) (S k) u).
264       * lia.
265       * symmetry. exact IHM.
266 Qed.
267
268 Lemma subst_open_rec : forall t1 t2 x u k,
269   lc u ->
270   (open_rec k t2 t1)[x = u] = open_rec k (t2[x = u]) (t1[x = u]).
271 Proof.
272   induction t1 as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp
273   A IHA B IHB];
274   intros t2 x u k Hlc; simpl.
275   - reflexivity.
276   - destruct (Nat.eqb n k) eqn:E; reflexivity.
277   - destruct (eq_var_dec y x) as [Heq | Hneq].
278     + subst y. symmetry. apply open_rec_lc. exact Hlc.
279     + reflexivity.
280   - f_equal; auto.
281   - f_equal.
282     + auto.
283     + rewrite IHM. reflexivity. exact Hlc.

```

```

283   - f_equal.
284   + auto.
285   + rewrite IHB. reflexivity. exact Hlc.
286 Qed.
287
288 Lemma subst_open : forall t1 t2 x u,
289   lc u ->
290   (t1 ^^ t2)[x = u] = (t1[x = u]) ^^ (t2[x = u]).
291 Proof. intros. apply subst_open_rec. exact H. Qed.
292
293 Lemma subst_open_var : forall t s x y u,
294   x <> y ->
295   lc u ->
296   (open_var t s x)[y = u] = open_var (t[y = u]) s x.
297 Proof.
298   intros t s x y u Hxy Hlc.
299   unfold open_var.
300   rewrite subst_open_rec by exact Hlc.
301   simpl. destruct (eq_var_dec x y) as [Heq | Hneq].
302   - exfalso. apply Hxy. exact Heq.
303   - reflexivity.
304 Qed.
305
306 Lemma subst_intro : forall s x t u,
307   ~ In x (fv t) ->
308   lc u ->
309   t ^^ u = (open_var t s x)[x = u].
310 Proof.
311   intros s x t u Hnotin Hlc.
312   unfold open_var.
313   rewrite subst_open_rec by exact Hlc.
314   simpl. destruct (eq_var_dec x x) as [_ | Hne].
315   - rewrite (subst_fresh t x u Hnotin). reflexivity.
316   - exfalso. apply Hne. reflexivity.
317 Qed.
318
319 Lemma subst_lc : forall t x u,
320   lc t -> lc u -> lc (t[x = u]).
321 Proof.
322   intros t x u Ht.
323   induction Ht as [ s | sy y | M N HM IHM HN IHN
324     | sp A B L HA IHA HB IHB
325     | sl A M L HA IHA HM IHM ]; intros Hu; simpl.
326   - constructor.
327   - destruct (eq_var_dec y x); [exact Hu | constructor].
328   - constructor; auto.
329   - apply (lc_pi sp (A[x=u]) (B[x=u]) (x :: L)).
330     + auto.
331     + intros y Hy.
332     destruct (not_in_cons_elim x L y Hy) as [Hyx HyL].

```

```

333     rewrite <- (subst_open_var B sp y x u Hyx Hu).
334     apply IHB; auto.
335 - apply (lc_lam sl (A[x=u]) (M[x=u]) (x :: L)).
336   + auto.
337   + intros y Hy.
338     destruct (not_in_cons_elim x L y Hy) as [Hyx HyL].
339     rewrite <- (subst_open_var M sl y x u Hyx Hu).
340     apply IHM; auto.
341 Qed.
342
343 Lemma lc_open_var_rename : forall t s x0 y,
344   x0 <> y ->
345   ~ In x0 (fv t) ->
346   lc (open_var t s x0) -> lc (open_var t s y).
347 Proof.
348   intros t s x0 y Hxy Hnotin Hlc.
349   assert (Heq : open_var t s y = (open_var t s x0)[x0 = t_fvar s y]).
350   { unfold open_var. apply subst_intro; auto. constructor. }
351   rewrite Heq. apply subst_lc; auto. constructor.
352 Qed.
353
354 Lemma fv_open_rec_incl : forall t k u,
355   incl (fv t) (fv (open_rec k u t)).
356 Proof.
357   induction t as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp
358     A IHA B IHB];
359   intros k u; simpl.
360 - apply incl_refl.
361 - apply incl_nil_l.
362 - apply incl_refl.
363 - apply incl_app.
364   + apply incl_appl. auto.
365   + apply incl_appr. auto.
366 - apply incl_app.
367   + apply incl_appl. auto.
368   + apply incl_appr. auto.
369 - apply incl_app.
370   + apply incl_appl. auto.
371   + apply incl_appr. auto.
372 Qed.
373
374 Lemma fv_open_var_incl : forall t s x,
375   incl (fv t) (fv (open_var t s x)).
376 Proof. intros. apply fv_open_rec_incl. Qed.
377
378 Lemma fv_open_rec_upper : forall t k u,
379   incl (fv (open_rec k u t)) (fv t ++ fv u).
380 Proof.
381   induction t as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp
382     A IHA B IHB];

```

```

381   intros k u; simpl.
382 - apply incl_nil_l.
383 - destruct (Nat.eqb n k); simpl.
384   + apply incl_refl.
385   + apply incl_nil_l.
386 - intros z Hz. simpl in Hz. destruct Hz as [Hz | []]. left. exact
    Hz.
387 - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
388   + apply (IHM k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz
    | Hz].
389     * apply in_or_app; left; apply in_or_app; left; exact Hz.
390     * apply in_or_app; right; exact Hz.
391   + apply (IHN k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz
    | Hz].
392     * apply in_or_app; left; apply in_or_app; right; exact Hz.
393     * apply in_or_app; right; exact Hz.
394 - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
395   + apply (IHA k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz
    | Hz].
396     * apply in_or_app; left; apply in_or_app; left; exact Hz.
397     * apply in_or_app; right; exact Hz.
398   + apply (IHM (S k) u) in Hz. apply in_app_or in Hz. destruct Hz as
    [Hz | Hz].
399     * apply in_or_app; left; apply in_or_app; right; exact Hz.
400     * apply in_or_app; right; exact Hz.
401 - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
402   + apply (IHA k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz
    | Hz].
403     * apply in_or_app; left; apply in_or_app; left; exact Hz.
404     * apply in_or_app; right; exact Hz.
405   + apply (IHB (S k) u) in Hz. apply in_app_or in Hz. destruct Hz as
    [Hz | Hz].
406     * apply in_or_app; left; apply in_or_app; right; exact Hz.
407     * apply in_or_app; right; exact Hz.
408 Qed.
409
410 Lemma subst_open_var_eq : forall t s x u,
411   ~ In x (fv t) ->
412   lc u ->
413   (open_var t s x) [x = u] = t ^^ u.
414 Proof. intros. symmetry. apply subst_intro; auto. Qed.
415
416 (* =====
417 *)
418 (** * Beta reduction *)
419 Definition beta (M N : term) : Prop :=
420   exists s Dom Body Arg,
421   M = t_app (t_lam s Dom Body) Arg /\

```

```

422     N = Body ^^ Arg.
423
424 Reserved Notation "M ->b N" (at level 70, no associativity).
425
426 Inductive beta_red : term -> term -> Prop :=
427
428   | beta_base : forall s A M N,
429     t_app (t_lam s A M) N ->b (M ^^ N)
430
431   | beta_app_l : forall M M' N,
432     M ->b M' ->
433     t_app M N ->b t_app M' N
434
435   | beta_app_r : forall M N N',
436     N ->b N' ->
437     t_app M N ->b t_app M N'
438
439   | beta_lam_A : forall s A A' M,
440     A ->b A' ->
441     t_lam s A M ->b t_lam s A' M
442
443   | beta_lam_M : forall s A M M',
444     M ->b M' ->
445     t_lam s A M ->b t_lam s A M'
446
447   | beta_pi_A : forall s A A' B,
448     A ->b A' ->
449     t_pi s A B ->b t_pi s A' B
450
451   | beta_pi_B : forall s A B B',
452     B ->b B' ->
453     t_pi s A B ->b t_pi s A B'
454
455 where "M ->b N" := (beta_red M N).
456
457 Reserved Notation "M ->>+b N" (at level 70, no associativity).
458
459 Inductive beta_trans : term -> term -> Prop :=
460
461   | bt_step : forall M N,
462     M ->b N ->
463     M ->>+b N
464
465   | bt_trans : forall M N P,
466     M ->>+b N ->
467     N ->>+b P ->
468     M ->>+b P
469
470 where "M ->>+b N" := (beta_trans M N).
471
472 Reserved Notation "M ->>b N" (at level 70, no associativity).

```

```

472
473 Inductive beta_rtrans : term -> term -> Prop :=
474   | brt_refl : forall M,
475     M ->>b M
476
477   | brt_step : forall M N,
478     M ->b N ->
479     M ->>b N
480
481   | brt_trans : forall M N P,
482     M ->>b N ->
483     N ->>b P ->
484     M ->>b P
485
486 where "M ->>b N" := (beta_rtrans M N).
487
488 Reserved Notation "M =b N" (at level 70, no associativity).
489
490 Inductive beta_eq : term -> term -> Prop :=
491   | beq_refl : forall M,
492     M =b M
493
494   | beq_step : forall M N,
495     M ->b N ->
496     M =b N
497
498   | beq_sym : forall M N,
499     M =b N ->
500     N =b M
501
502   | beq_trans : forall M N P,
503     M =b N ->
504     N =b P ->
505     M =b P
506
507 where "M =b N" := (beta_eq M N).
508
509 Lemma brt_from_trans : forall M N,
510   M ->>+b N -> M ->>b N.
511 Proof.
512   intros M N H.
513   induction H.
514   - apply brt_step; auto.
515   - apply brt_trans with N; auto.
516 Qed.
517
518 Lemma beq_from_rtrans : forall M N,
519   M ->>b N -> M =b N.
520 Proof.
521   intros M N H.

```

```

522 induction H.
523 - apply beq_refl.
524 - apply beq_step; auto.
525 - apply beq_trans with N; auto.
526 Qed.
527
528 Definition normal_form (M : term) : Prop :=
529   ~ exists N, M ->b N.
530
531 Fixpoint has_redex (M : term) : Prop :=
532   match M with
533   | t_sort _      => False
534   | t_bvar _ _    => False
535   | t_fvar _ _    => False
536   | t_app (t_lam _ _) _ => True
537   | t_app P Q     => has_redex P /\ has_redex Q
538   | t_lam _ A M   => has_redex A /\ has_redex M
539   | t_pi _ A B    => has_redex A /\ has_redex B
540   end.
541
542 Definition normal_form' (M : term) : Prop :=
543   ~ has_redex M.
544
545 Definition weakly_normalizing (M : term) : Prop :=
546   exists N, M ->>b N /\ normal_form N.
547
548 Definition strongly_normalizing (M : term) : Prop :=
549   Acc (fun N M => M ->b N) M.
550
551 Lemma sn_implies_wn : forall M,
552   strongly_normalizing M -> weakly_normalizing M.
553 Proof.
554   intros M Hacc.
555   induction Hacc as [M _ IH].
556   destruct (classic (exists N, M ->b N)) as [[N Hstep] | Hnf].
557   - destruct (IH N Hstep) as [P [HredP HnfP]].
558     exists P. split.
559     + apply brt_trans with N.
560       * apply brt_step; auto.
561       * auto.
562     + auto.
563   - exists M. split.
564     + apply brt_refl.
565     + unfold normal_form. auto.
566   Qed.
567
568 Lemma nf_is_sn : forall M,
569   normal_form M -> strongly_normalizing M.
570 Proof.
571   intros M Hnf.

```

```

572   constructor.
573   intros N Hstep.
574   exfalso. apply Hnf.
575   exists N. auto.
576 Qed.
577
578 (* =====
*)
579 (** * Contexts *)
580
581 Definition context := list (var * (Sort * term)).
582
583 Fixpoint lookup (c : context) (x : var) : option (Sort * term) :=
584   match c with
585   | [] => None
586   | (y, sA) :: c' =>
587     if eq_var_dec x y
588     then Some sA
589     else lookup c' x
590   end.
591
592 Definition dom (Γ : context) : list var :=
593   map fst Γ.
594
595 Definition is_fresh (x : var) (Γ : context) :=
596   ~ In x (dom Γ).
597
598 (* =====
*)
599 (** * Establishing [bvar_tag_ok] from [lc] *)
600 Fixpoint closed_at (k : nat) (t : term) : Prop :=
601   match t with
602   | t_sort _ => True
603   | t_bvar _ n => n < k
604   | t_fvar _ _ => True
605   | t_app M _ => closed_at k M
606   | t_lam _ _ M => closed_at (S k) M
607   | t_pi _ _ B => closed_at (S k) B
608   end.
609
610 Lemma closed_at_open_rec_inv : forall t k u,
611   closed_at k (open_rec k u t) ->
612   (forall s m, u <> t_bvar s m) ->
613   closed_at (S k) t.
614 Proof.
615   induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A
616     IHA B IHB];
617     intros k u Hc Hu; simpl in *; auto.
618   - destruct (Nat.eqb n k) eqn:Heqb.
619     + apply Nat.eqb_eq in Heqb. lia.

```



```

619   + apply Nat.eqb_neq in Heqb.
620   simpl in Hc. lia.
621 - eapply IHP; eauto.
622 - eapply IHM with (k := S k); eauto.
623 - eapply IHB with (k := S k); eauto.
624 Qed.
625
626 Lemma lc_closed_at : forall t, lc t -> closed_at 0 t.
627 Proof.
628   intros t Hlc.
629   induction Hlc as [ s | x | M N HM IHM HN IHN
630                     | s A B L HA IHA HB IHB
631                     | s A M L HA IHA HM IHM ]; simpl; auto.
632 - remember (fresh L) as x0 eqn:Hx0def.
633   assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
634   specialize (IHB x0 Hx0L).
635   unfold open_var in IHB.
636   apply (closed_at_open_rec_inv B 0 (t_fvar s x0) IHB).
637   intros sv m Hcontra. discriminate.
638 - remember (fresh L) as x0 eqn:Hx0def.
639   assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
640   specialize (IHM x0 Hx0L).
641   unfold open_var in IHM.
642   apply (closed_at_open_rec_inv M 0 (t_fvar s x0) IHM).
643   intros sv m Hcontra. discriminate.
644 Qed.
645
646 Lemma closed_at_bvar_tag_ok : forall t m,
647   closed_at m t -> forall k s, m <= k -> bvar_tag_ok k s t.
648 Proof.
649   induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A
650                 IHA B IHB];
651   intros m Hclosed k s Hmk; simpl in *; auto.
652 - intro Heq. subst n. lia.
653 - eapply IHP; eauto.
654 - eapply IHM; eauto. lia.
655 - eapply IHB; eauto. lia.
656 Qed.
657
658 Lemma lc_bvar_tag_ok : forall t, lc t -> forall k s, bvar_tag_ok k s
659   t.
660 Proof.
661   intros t Hlc k s.
662   apply (closed_at_bvar_tag_ok t 0); [apply lc_closed_at; exact Hlc |
663   lia].
664 Qed.
665
666 Lemma bvar_tag_ok_subst : forall t k s x N,
667   lc N -> bvar_tag_ok k s t -> bvar_tag_ok k s (t [ x = N ]).
668 Proof.

```

```

666 induction t as [s0 | s_n n | sy y | P IHP Q IHQ | s1 A IHA M IHM |
    s1 A IHA B IHB];
667 intros k s x N HlcN Htag; simpl in *; auto.
668 - destruct (eq_var_dec y x); auto.
669 apply lc_bvar_tag_ok. exact HlcN.
670 Qed.
671
672 (* =====
*)
673 (** * Typing *)
674
675 Reserved Notation "Γ ⊢ M ∈ A" (at level 70, no associativity).
676
677 Inductive typing : context -> term -> term -> Prop :=
678 | typing_axiom : forall s s',
679   A s s' ->
680   [] ⊢ t_sort s ∈ t_sort s'
681
682 | typing_var : forall Γ x A s,
683   is_fresh x Γ ->
684   Γ ⊢ A ∈ t_sort s ->
685   Γ ++ [(x, (s, A))] ⊢ t_fvar s x ∈ A
686
687 | typing_weak : forall Γ x B M A s,
688   is_fresh x Γ ->
689   Γ ⊢ M ∈ A ->
690   Γ ⊢ B ∈ t_sort s ->
691   Γ ++ [(x, (s, B))] ⊢ M ∈ A
692
693 | typing_pi : forall Γ A B s1 s2 s3 L,
694   Γ ⊢ A ∈ t_sort s1 ->
695   bvar_tag_ok 0 s1 B ->
696   (forall x, ~ In x L ->
697     Γ ++ [(x, (s1, A))] ⊢ (open_var B s1 x) ∈ t_sort s2) ->
698   R s1 s2 s3 ->
699   Γ ⊢ (t_pi s1 A B) ∈ (t_sort s3)
700
701 | typing_lam : forall Γ A M B s1 s3 L,
702   Γ ⊢ A ∈ t_sort s1 ->
703   bvar_tag_ok 0 s1 M ->
704   (forall x, ~ In x L ->
705     Γ ++ [(x, (s1, A))] ⊢ (open_var M s1 x) ∈ (open_var B s1 x))
706   ->
707   Γ ⊢ (t_pi s1 A B) ∈ (t_sort s3) ->
708   Γ ⊢ (t_lam s1 A M) ∈ (t_pi s1 A B)
709
710 | typing_app : forall Γ M N A B s1,
711   Γ ⊢ M ∈ t_pi s1 A B ->
712   Γ ⊢ N ∈ A ->
713   Γ ⊢ t_app M N ∈ (B ^^ N)

```

```

713 | typing_conv : forall  $\Gamma$  M A B s,
714    $\Gamma \vdash M \in A \rightarrow$ 
715    $A =_b B \rightarrow$ 
716    $\Gamma \vdash B \in t\_sort\ s \rightarrow$ 
717    $\Gamma \vdash M \in B$ 
718
719
720 where " $\Gamma \vdash M \in A$ " := (typing  $\Gamma$  M A).
721
722 Definition legal ( $\Gamma$  : context) : Prop :=
723   exists M N,  $\Gamma \vdash M \in N$ .
724
725 (* =====
726 *)
727 (** * Regularity *)
728
729 Lemma lc_pi_body : forall s A B,
730   lc (t_pi s A B) -> exists L, forall x, ~ In x L -> lc (open_var B s x).
731
732 Proof.
733   intros s A B H.
734   remember (t_pi s A B) as t eqn:Ht.
735   destruct H; try discriminate.
736   injection Ht as Hs HA HB. subst.
737   exists L. exact H0.
738
739 Qed.
740
741 Lemma lc_open_lc : forall B s x N,
742   ~ In x (fv B) -> lc (open_var B s x) -> lc N -> lc (B ^^ N).
743
744 Proof.
745   intros B s x N Hnotin Hlc HlcN.
746   rewrite (subst_intro s x B N Hnotin HlcN).
747   apply subst_lc; auto.
748
749 Qed.
750
751 Lemma typing_lc : forall  $\Gamma$  M A,  $\Gamma \vdash M \in A \rightarrow lc\ M \wedge lc\ A$ .
752
753 Proof.
754   intros  $\Gamma$  M A H.
755   induction H as
756   [ s s' HA
757   |  $\Gamma_0\ x\ A_0\ s_0\ Hfresh\ HA\ IHA$ 
758   |  $\Gamma_0\ x\ B_0\ M_0\ A_0\ s_0\ Hfresh\ HM\ IHM\ HB\ IHB$ 
759   |  $\Gamma_0\ A_0\ B_0\ s_1\ s_2\ s_3\ L\ HA\ IHA\ HTagB\ HB\ IHB\ HR$ 
760   |  $\Gamma_0\ A_0\ M_0\ B_0\ s_1\ s_3\ L\ HA\ IHA\ HTagM\ HM\ IHM\ HPi\ IHPi$ 
761   |  $\Gamma_0\ M_0\ N_0\ A_0\ B_0\ s_1a\ HM\ IHM\ HN\ IHN$ 
762   |  $\Gamma_0\ M_0\ A_0\ B_0\ s\ HM\ IHM\ Heq\ HB\ IHB$  ].
763   - split; constructor.
764   - split; [constructor | apply IHA].
765   - split; [apply IHM | apply IHM].
766   - split.

```

```

761 + remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
762 assert (Hx0L : ~ In x0 L).
763 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
764   apply in_or_app. left. exact Hc. }
765 assert (Hx0B : ~ In x0 (fv B0)).
766 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
767   apply in_or_app. right. exact Hc. }
768 destruct (IHB x0 Hx0L) as [Hlc_open _].
769 apply (lc_pi s1 A0 B0 (x0 :: nil)).
770 * apply IHA.
771 * intros y Hy.
772   assert (Hyx0 : y <> x0) by (intro Hc; apply Hy; left;
773     symmetry; exact Hc).
774   apply (lc_open_var_rename B0 s1 x0 y (not_eq_sym Hyx0) Hx0B
775     Hlc_open).
776 + constructor.
777 - split.
778 + remember (fresh (L ++ fv M0)) as x0 eqn:Hx0def.
779 assert (Hx0L : ~ In x0 L).
780 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
781   apply in_or_app. left. exact Hc. }
782 assert (Hx0M : ~ In x0 (fv M0)).
783 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
784   apply in_or_app. right. exact Hc. }
785 destruct (IHM x0 Hx0L) as [Hlc_open _].
786 apply (lc_lam s1 A0 M0 (x0 :: nil)).
787 * apply IHA.
788 * intros y Hy.
789   assert (Hyx0 : y <> x0) by (intro Hc; apply Hy; left;
790     symmetry; exact Hc).
791   apply (lc_open_var_rename M0 s1 x0 y (not_eq_sym Hyx0) Hx0M
792     Hlc_open).
793 + apply IHPI.
794 - split.
795 + constructor; [apply IHM | apply IHN].
796 + destruct (lc_pi_body sla A0 B0 (proj2 IHM)) as [L HL].
797 remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
798 assert (Hx0L : ~ In x0 L).
799 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
800   apply in_or_app. left. exact Hc. }
801 assert (Hx0B : ~ In x0 (fv B0)).
802 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
803   apply in_or_app. right. exact Hc. }
804 apply (lc_open_lc B0 sla x0 N0 Hx0B (HL x0 Hx0L) (proj1 IHN)).
805 - split; [apply IHM | apply IHB].
806 Qed.
807
808 Lemma typing_fv_subset : forall Γ M A,
809   Γ ⊢ M ∈ A -> incl (fv M) (dom Γ) /\ incl (fv A) (dom Γ).

```

```

806 Proof.
807   intros  $\Gamma$  M A H.
808   induction H as
809     [ s s' HA
810     |  $\Gamma_0$  x A0 s0 Hfresh HA IHA
811     |  $\Gamma_0$  x B0 M0 A0 s0 Hfresh HM IHM HB IHB
812     |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
813     |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
814     |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
815     |  $\Gamma_0$  M0 A0 B0 s HM IHM Heq HB IHB ].
816   - split; apply incl_nil_l.
817   - split.
818     + intros z Hz. destruct Hz as [Hz | []]. subst z.
819       unfold dom. rewrite map_app. apply in_or_app. right. simpl.
820       left. reflexivity.
821     + unfold dom in *. rewrite map_app. apply incl_appl. apply IHA.
822   - split.
823     + unfold dom in *. rewrite map_app. apply incl_appl. apply IHM.
824     + unfold dom in *. rewrite map_app. apply incl_appl. apply IHM.
825   - split.
826     + simpl. apply incl_app.
827       * apply IHA.
828       * remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
829         assert (Hx0L : ~ In x0 L).
830         { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
831           apply in_or_app. left. exact Hc. }
832         assert (Hx0B : ~ In x0 (fv B0)).
833         { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
834           apply in_or_app. right. exact Hc. }
835         destruct (IHB x0 Hx0L) as [Hfv_open _].
836         intros z Hz.
837         assert (Hz' : In z (fv (open_var B0 s1 x0))) by (apply
838           (fv_open_var_incl B0 s1 x0); exact Hz).
839         unfold dom in Hfv_open. rewrite map_app in Hfv_open.
840         specialize (Hfv_open z Hz'). apply in_app_or in Hfv_open.
841         destruct Hfv_open as [Hin | Hin].
842         -- exact Hin.
843         -- simpl in Hin. destruct Hin as [Hin | []]. subst z.
844         contradiction.
845     + apply incl_nil_l.
846   - split.
847     + simpl. apply incl_app.
848       * apply IHA.
849       * remember (fresh (L ++ fv M0)) as x0 eqn:Hx0def.
850         assert (Hx0L : ~ In x0 L).
851         { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
852           apply in_or_app. left. exact Hc. }
853         assert (Hx0M : ~ In x0 (fv M0)).
854         { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
855           apply in_or_app. right. exact Hc. }

```

```

853     destruct (IHM x0 Hx0L) as [Hfv_open _].
854     intros z Hz.
855     assert (Hz' : In z (fv (open_var M0 s1 x0))) by (apply
      (fv_open_var_incl M0 s1 x0); exact Hz).
856     unfold dom in Hfv_open. rewrite map_app in Hfv_open.
857     specialize (Hfv_open z Hz'). apply in_app_or in Hfv_open.
858     destruct Hfv_open as [Hin | Hin].
859     -- exact Hin.
860     -- simpl in Hin. destruct Hin as [Hin | []]. subst z.
      contradiction.
861   + apply IHPi.
862 - split.
863   + simpl. apply incl_app; [apply IHM | apply IHN].
864   + assert (HfvB0 : incl (fv B0) (dom Γ0)).
865     { intros z Hz. apply (proj2 IHM). simpl. apply in_or_app. right.
      exact Hz. }
866     intros z Hz.
867     apply (fv_open_rec_upper B0 0 N0) in Hz.
868     apply in_app_or in Hz. destruct Hz as [Hz | Hz].
869     * apply HfvB0. exact Hz.
870     * apply (proj1 IHN). exact Hz.
871   - split; [apply IHM | apply IHB].
872 Qed.
873
874 Lemma subst_fresh_typing : forall Γ M A x N,
875   is_fresh x Γ ->
876   Γ ⊢ M ∈ A ->
877   M [ x = N ] = M /\ A [ x = N ] = A.
878 Proof.
879   intros Γ M A x N Hfresh Htyp.
880   destruct (typing_fv_subset Γ M A Htyp) as [HfvM HfvA].
881   split.
882   - apply subst_fresh. intro Hc. apply Hfresh. apply HfvM. exact Hc.
883   - apply subst_fresh. intro Hc. apply Hfresh. apply HfvA. exact Hc.
884 Qed.
885
886 Lemma beta_red_subst : forall M N x u,
887   M ->b N -> lc u -> M[x = u] ->b N[x = u].
888 Proof.
889   intros M N x u H.
890   induction H; intros Hlc; simpl.
891   - rewrite subst_open by exact Hlc. apply beta_base.
892   - apply beta_app_l. apply IHbeta_red; auto.
893   - apply beta_app_r. apply IHbeta_red; auto.
894   - apply beta_lam_A. apply IHbeta_red; auto.
895   - apply beta_lam_M. apply IHbeta_red; auto.
896   - apply beta_pi_A. apply IHbeta_red; auto.
897   - apply beta_pi_B. apply IHbeta_red; auto.
898 Qed.
899

```

```

900 Lemma beta_eq_subst : forall M N x u,
901   M =b N -> lc u -> M[x = u] =b N[x = u].
902 Proof.
903   intros M N x u H.
904   induction H; intros Hlc.
905   - apply beq_refl.
906   - apply beq_step. apply beta_red_subst; auto.
907   - apply beq_sym. apply IHbeta_eq; auto.
908   - apply beq_trans with (N[x = u]); auto.
909 Qed.
910
911 (* =====
912 *)
913 (** * Basic context lemmas *)
914 Lemma start_axiom :
915   forall Γ s s',
916     legal Γ ->
917     A s s' ->
918     Γ ⊢ t_sort s ∈ t_sort s'.
919 Proof.
920   intros Γ s s' Hl Hs. destruct Hl as [M [N Htyp]].
921   induction Htyp; auto.
922   - apply typing_axiom. apply Hs.
923   - apply typing_weak.
924     + apply H.
925     + apply IHHtyp.
926     + apply Htyp.
927   - apply typing_weak.
928     + apply H.
929     + apply IHHtyp1.
930     + apply Htyp2.
931 Qed.
932
933 Lemma start_var :
934   forall Γ x s T,
935     legal Γ ->
936     In (x, (s, T)) Γ ->
937     Γ ⊢ t_fvar s x ∈ T.
938 Proof.
939   intros Γ x s T Hl Hin. destruct Hl as [M [N Htyp]].
940   induction Htyp as
941     [ s' s'' HA
942     | Γ0 x0 A0 s0 Hfresh0 HA0 IHA0
943     | Γ0 x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
944     | Γ0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
945     | Γ0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
946     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
947     | Γ0 M0 A0 B0 s3 HM IHM Heq HB IHB ].
948

```

```

949 - contradiction.
950
951 - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
952 + apply typing_weak.
953   * apply Hfresh0.
954   * apply IHA0. apply Hin.
955   * apply HA0.
956 + destruct Hin as [Hin | []]. inversion Hin. subst.
957   apply typing_var.
958   * apply Hfresh0.
959   * apply HA0.
960
961 - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
962 + apply typing_weak.
963   * apply Hfresh0.
964   * apply IHM0. apply Hin.
965   * apply HB0.
966 + destruct Hin as [Hin | []]. inversion Hin. subst.
967   apply typing_var.
968   * apply Hfresh0.
969   * apply HB0.
970
971 - auto.
972
973 - auto.
974
975 - auto.
976
977 - auto.
978 Qed.
979
980 Definition subcontext ( $\Gamma \Delta$  : context) : Prop :=
981   forall x T,
982     In (x, T)  $\Gamma$  ->
983     In (x, T)  $\Delta$ .
984
985 Notation " $\Gamma \subseteq \Delta$ " := (subcontext  $\Gamma \Delta$ ) (at level 70, no associativity).
986
987 Lemma subcontext_extend : forall  $\Gamma \Delta z A$ ,
988    $\Gamma \subseteq \Delta$  ->
989   is_fresh z  $\Delta$  ->
990    $\Gamma ++ [(z, A)] \subseteq \Delta ++ [(z, A)]$ .
991 Proof.
992   intros  $\Gamma \Delta z A$  Hsub Hfr y T Hin.
993   apply in_app_or in Hin.
994   destruct Hin.
995   - apply in_or_app. left. apply Hsub. assumption.
996   - simpl in H.
997     destruct H as [H | H].
998     + inversion H; subst.

```



```

999      apply in_or_app. right. left. reflexivity.
1000    + contradiction.
1001  Qed.
1002
1003  Lemma subcontext_extend_r : forall  $\Gamma$  x T,  $\Gamma \subseteq \Gamma ++ [(x, T)]$ .
1004  Proof.
1005    intros.
1006    unfold subcontext. intros.
1007    apply in_or_app. left.
1008    apply H.
1009  Qed.
1010
1011  Lemma thinning :
1012    forall  $\Gamma \Delta M N$ ,
1013      legal  $\Delta \rightarrow$ 
1014       $\Gamma \subseteq \Delta \rightarrow$ 
1015       $\Gamma \vdash M \in N \rightarrow$ 
1016       $\Delta \vdash M \in N$ .
1017  Proof.
1018    intros  $\Gamma \Delta M N$  Hleg Hsub Htyp.
1019    generalize dependent  $\Delta$ .
1020    induction Htyp as
1021      [ s s' HA
1022      |  $\Gamma_0 x_0 A_0 s_0$  Hfresh0  $HA_0 IHA_0$ 
1023      |  $\Gamma_0 x_0 B_0 M_0 A_0 s_0$  Hfresh0  $HM_0 IHM_0 HB_0 IHB_0$ 
1024      |  $\Gamma_0 A_0 B_0 s_1 s_2 s_3 L$  HA IHA HTagB HB IHB HR
1025      |  $\Gamma_0 A_0 M_0 B_0 s_1 s_3 L$  HA IHA HTagM HM IHM HPi IHPi
1026      |  $\Gamma_0 M_0 N_0 A_0 B_0 s_1 a$  HM IHM HN IHN
1027      |  $\Gamma_0 M_0 A_0 B_0 s$  HM IHM Heq HB IHB ];
1028    intros  $\Delta$  Hleg Hsub.
1029
1030    - apply start_axiom.
1031      + apply Hleg.
1032      + apply HA.
1033
1034    - apply (start_var  $\Delta x_0 s_0 A_0$  Hleg).
1035      unfold subcontext in Hsub. apply Hsub. apply in_or_app. right.
1036      simpl. left. reflexivity.
1037
1038    - apply IHM0.
1039      + apply Hleg.
1040      + unfold subcontext in *. intros y T Hin.
1041        apply Hsub. apply in_or_app. left. apply Hin.
1042
1043    - apply (typing_pi  $\Delta A_0 B_0 s_1 s_2 s_3 (L ++ \text{dom } \Delta)$ ).
1044      + apply IHA; auto.
1045      + exact HTagB.
1046      + intros x Hx.
1047        assert (HxL :  $\sim \text{In } x L$ ).
1048        { intro Hc. apply Hx. apply in_or_app. left. exact Hc. }

```

```

1049   assert (HxΔ : is_fresh x Δ).
1050   { intro Hc. apply Hx. apply in_or_app. right. exact Hc. }
1051   apply (IHB x HxL (Δ ++ [(x, (s1, A0))])).
1052   * exists (t_fvar s1 x). exists A0. apply typing_var.
1053     -- exact HxΔ.
1054     -- apply IHA; auto.
1055   * apply subcontext_extend; [exact Hsub | exact HxΔ].
1056   + exact HR.
1057
1058 - apply typing_lam with (s1 := s1) (s3 := s3) (L := L ++ dom Δ).
1059 + apply IHA; auto.
1060 + exact HTagM.
1061 + intros x Hx.
1062   assert (HxL : ~ In x L).
1063   { intro Hc. apply Hx. apply in_or_app. left. exact Hc. }
1064   assert (HxΔ : is_fresh x Δ).
1065   { intro Hc. apply Hx. apply in_or_app. right. exact Hc. }
1066   apply (IHM x HxL (Δ ++ [(x, (s1, A0))])).
1067   * exists (t_fvar s1 x). exists A0. apply typing_var.
1068     -- exact HxΔ.
1069     -- apply IHA; auto.
1070   * apply subcontext_extend; [exact Hsub | exact HxΔ].
1071   + apply IHPi; auto.
1072
1073 - apply typing_app with (A := A0) (s1 := sla); auto.
1074
1075 - apply typing_conv with A0 s; auto.
1076 Qed.
1077
1078 (* =====
1079 *)
1080 (** * Generation lemmas *)
1081
1082 Lemma app_singleton_injective :
1083   forall (Γ Δ : context)
1084     (x y : var)
1085     (M N : Sort * term),
1086   Γ ++ [(x, M)] = Δ ++ [(y, N)] ->
1087   Γ = Δ /\ x = y /\ M = N.
1088 Proof.
1089   induction Γ as [| [v T] Γ IH].
1090   - intros Δ x y M N H.
1091     destruct Δ as [| [v' T'] Δ].
1092     + simpl in H.
1093       inversion H. subst.
1094       repeat split; reflexivity.
1095     + simpl in H.
1096       destruct Δ.
1097       * discriminate.
1098       * discriminate.

```

```

1098
1099 - intros Δ x y M N H.
1100 destruct Δ as [| [v' T'] Δ].
1101 + simpl in H.
1102   destruct Γ.
1103   * discriminate.
1104   * discriminate.
1105
1106 + simpl in H.
1107   inversion H. subst.
1108   specialize (IH Δ x y M N H3).
1109   destruct IH as [HΓ [Hx HM]].
1110   subst.
1111   repeat split; reflexivity.
1112 Qed.
1113
1114 Lemma generation_sort :
1115   forall Γ s W,
1116     Γ ⊢ t_sort s ∈ W ->
1117     exists s',
1118       W =b t_sort s' /\ Sig.A s s'.
1119 Proof.
1120   intros Γ s W H.
1121   remember (t_sort s) as T eqn:HeqT.
1122   induction H as
1123     [ sa sb HA
1124     | Γ0 x0 A0 s0 Hfresh0 HA0 IHA0
1125     | Γ0 x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
1126     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1127     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1128     | Γ0 M0 N0 A0 B0 sla HM IHM HN IHN
1129     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1130   inversion HeqT; subst.
1131 - exists sb. split.
1132 + apply beq_refl.
1133 + apply HA.
1134 - apply IHM0. reflexivity.
1135 - destruct (IHM eq_refl) as [s' [HAeq HAx]]. exists s'. split.
1136 + apply beq_trans with (N := A0).
1137   * apply beq_sym. apply Heq.
1138   * apply HAeq.
1139 + apply HAx.
1140 Qed.
1141
1142 Lemma lookup_fresh :
1143   forall C y N, is_fresh y C -> lookup (C ++ [(y, N)]) y = Some N.
1144 Proof.
1145   intros C y N F. unfold is_fresh in F. induction C.
1146 - simpl. destruct (eq_var_dec y y).
1147   + reflexivity.

```

```

1148   + contradiction.
1149 -   destruct a as (v, T). simpl in *. destruct (eq_var_dec y v) as
    [Heq | Hneq].
1150   + exfalso. apply F. simpl. left. f_equal. symmetry. exact Heq.
1151   + apply IHC. intro Hc. apply F. simpl. right. exact Hc.
1152 Qed.
1153
1154 Lemma lookup_extend :
1155   forall C x y M N, (lookup C x = Some M) -> (lookup (C ++ [(y, N)]) x
    = Some M).
1156 Proof.
1157   intros C x y M N H. induction C.
1158   - simpl in H. discriminate H.
1159   - destruct a as (v, T). simpl in *. destruct (eq_var_dec x v) as
    [Heq | Hneq].
1160     + apply H.
1161     + apply IHC. apply H.
1162   Qed.
1163
1164 Lemma generation_var :
1165   forall Γ x s N,
1166     Γ ⊢ t_fvar s x ∈ N ->
1167     exists B,
1168       lookup Γ x = Some (s, B) /\
1169       Γ ⊢ B ∈ t_sort s /\
1170       N =b B.
1171 Proof.
1172   intros Γ x s N H.
1173   remember (t_fvar s x) as T eqn:HeqT.
1174   induction H as
1175     [ sa sb HA
1176     | Γ0 x0 A0 s0 Hfresh0 HA0 IHA0
1177     | Γ0 x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
1178     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1179     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1180     | Γ0 M0 N0 A0 B0 sla HM IHM HN IHN
1181     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1182   inversion HeqT; subst.
1183
1184 -   exists A0. repeat split; auto.
1185   + apply lookup_fresh; auto.
1186   + apply typing_weak; auto.
1187   + apply beq_refl.
1188
1189 -   destruct (IHM0 eq_refl) as [B' [Q1 [Q2 Q3]]]. exists B'. repeat
    split; auto.
1190   + apply lookup_extend; auto.
1191   + apply typing_weak; auto.
1192

```

```

1193 - destruct (IHM eq_refl) as [B' [Q1 [Q2 Q3]]]. exists B'. repeat
1194   split; auto.
1195   apply beq_trans with A0; auto. apply beq_sym; auto.
1196 Qed.
1197 Lemma generation_pi :
1198   forall  $\Gamma$  s0 B C W,
1199      $\Gamma \vdash t\_pi\ s0\ B\ C \in W \rightarrow$ 
1200     exists s2 s3 L,
1201        $\Gamma \vdash B \in t\_sort\ s0 \wedge$ 
1202       bvar_tag_ok 0 s0 C  $\wedge$ 
1203       (forall x, ~ In x L  $\rightarrow \Gamma ++ [(x, (s0, B))]$   $\vdash open\_var\ C\ s0\ x \in$ 
1204         t_sort s2)  $\wedge$ 
1205       Sig.R s0 s2 s3  $\wedge$ 
1206       W =b t_sort s3.
1207 Proof.
1208   intros  $\Gamma$  s0 B C W H.
1209   remember (t_pi s0 B C) as T eqn:HeqT.
1210   induction H as
1211     [ sa sb HA
1212     |  $\Gamma_0\ x_0\ A_0\ s_1\ Hfresh_0\ HA_0\ IHA_0$ 
1213     |  $\Gamma_0\ x_0\ B_0\ M_0\ A_0\ s_1\ Hfresh_0\ HM_0\ IHM_0\ HB_0\ IHB_0$ 
1214     |  $\Gamma_0\ A_0\ B_0\ sa\ sb\ sc\ L\ HA\ IHA\ HTagB\ HB\ IHB\ HR$ 
1215     |  $\Gamma_0\ A_0\ M_0\ B_0\ sa\ sc\ L\ HA\ IHA\ HTagM\ HM\ IHM\ HPi\ IHPi$ 
1216     |  $\Gamma_0\ M_0\ N_0\ A_0\ B_0\ sla\ HM\ IHM\ HN\ IHN$ 
1217     |  $\Gamma_0\ M_0\ A_0\ B_0\ sd\ HM\ IHM\ Heq\ HB\ IHB$  ];
1218   inversion HeqT; subst.
1219 - destruct (IHM0 eq_refl) as [s2 [s3 [L0 [HB1 [HTag1 [HB2 [HB3
1220   HB4]]]]]]].
1221   assert (HBthin :  $\Gamma_0 ++ [(x_0, (s_1, B_0))]$   $\vdash B \in t\_sort\ s_0$ )
1222     by (apply typing_weak; auto).
1223   exists s2. exists s3. exists (L0 ++ dom ( $\Gamma_0 ++ [(x_0, (s_1, B_0))]$ )).
1224   repeat split; auto.
1225 + intros y Hy.
1226   assert (HyL0 : ~ In y L0).
1227   { intro Hc. apply Hy. apply in_or_app. left. exact Hc. }
1228   assert (Hy $\Gamma$  : is_fresh y ( $\Gamma_0 ++ [(x_0, (s_1, B_0))]$ )).
1229   { intro Hc. apply Hy. apply in_or_app. right. exact Hc. }
1230   apply (thinning ( $\Gamma_0 ++ [(y, (s_0, B))]$ ) (( $\Gamma_0 ++ [(x_0, (s_1, B_0))]$ )
1231     ++ [(y, (s_0, B))])).
1232   * exists (t_fvar s0 y). exists B.
1233     apply typing_var.
1234     -- exact Hy $\Gamma$ .
1235     -- exact HBthin.
1236   * apply subcontext_extend.
1237     -- apply subcontext_extend_r.
1238     -- exact Hy $\Gamma$ .
1239   * apply HB2; auto.

```

```

1238
1239   - exists sb. exists sc. exists L.
1240     repeat split; auto. apply beq_refl.
1241
1242   - destruct (IHM eq_refl) as [s2 [s3 [L0 [HB1 [HTag1 [HB2 [HB3
1243     HB4]]]]]]].
1244     exists s2. exists s3. exists L0.
1245     repeat split; auto. apply beq_trans with A0; auto. apply beq_sym;
1246     auto.
1247 Qed.
1248
1249 Lemma pi_domain_valid : forall  $\Gamma$  s0 B C s' ,
1250    $\Gamma \vdash t\_pi\ s0\ B\ C \in t\_sort\ s' \rightarrow$ 
1251    $\Gamma \vdash B \in t\_sort\ s0$ .
1252 Proof.
1253   intros  $\Gamma$  s0 B C s' H.
1254   destruct (generation_pi  $\Gamma$  s0 B C (t_sort s') H) as [s2 [s3 [L [HB
1255     _]]]].
1256   exact HB.
1257 Qed.
1258
1259 Lemma generation_lam :
1260   forall  $\Gamma$  s0 B  $\bar{M}$  T,
1261    $\Gamma \vdash t\_lam\ s0\ B\ M \in T \rightarrow$ 
1262   exists C s3 L,
1263    $\Gamma \vdash B \in t\_sort\ s0 \wedge$ 
1264   (forall x,  $\sim In\ x\ L \rightarrow \Gamma ++ [(x, (s0, B))]$   $\vdash open\_var\ M\ s0\ x \in$ 
1265      $open\_var\ C\ s0\ x$ )  $\wedge$ 
1266    $\Gamma \vdash t\_pi\ s0\ B\ C \in t\_sort\ s3 \wedge$ 
1267    $T =_b t\_pi\ s0\ B\ C$ .
1268 Proof.
1269   intros  $\Gamma$  s0 B M T H.
1270   remember (t_lam s0 B M) as W eqn:HeqW.
1271   induction H as
1272     [ sa sb HA
1273     |  $\Gamma_0\ x_0\ A_0\ s_1\ Hfresh_0\ HA_0\ IHA_0$ 
1274     |  $\Gamma_0\ x_0\ B_0\ M_0\ A_0\ s_1\ Hfresh_0\ HM_0\ IHM_0\ HB_0\ IHB_0$ 
1275     |  $\Gamma_0\ A_0\ B_0\ sa\ sb\ sc\ L\ HA\ IHA\ HTagB\ HB\ IHB\ HR$ 
1276     |  $\Gamma_0\ A_0\ M_0\ B_0\ sa\ sc\ L\ HA\ IHA\ HTagM\ HM\ IHM\ HPi\ IHPi$ 
1277     |  $\Gamma_0\ M_0\ N_0\ A_0\ B_0\ s_1a\ HM\ IHM\ HN\ IHN$ 
1278     |  $\Gamma_0\ M_0\ A_0\ B_0\ sd\ HM\ IHM\ Heq\ HB\ IHB$  ];
1279   inversion HeqW; subst.
1280
1281   - destruct (IHM0 eq_refl) as [C [s3 [L0 [HB1 [HB2 [HB3 HB4]]]]]].
1282     assert (HBthin :  $\Gamma_0 ++ [(x_0, (s_1, B_0))]$   $\vdash B \in t\_sort\ s_0$ )
1283       by (apply typing_weak; auto).
1284     exists C. exists s3. exists (L0 ++ dom ( $\Gamma_0 ++ [(x_0, (s_1, B_0))]$ )).
1285     repeat split; auto.
1286     + intros y Hy.

```

```

1283   assert (HyL0 : ~ In y L0).
1284   { intro Hc. apply Hy. apply in_or_app. left. exact Hc. }
1285   assert (HyΓ : is_fresh y (Γ0 ++ [(x0, (s1, B0))])).
1286   { intro Hc. apply Hy. apply in_or_app. right. exact Hc. }
1287   apply (thinning (Γ0 ++ [(y, (s0, B))]) ((Γ0 ++ [(x0, (s1, B0))])
++ [(y, (s0, B))])).
1288   * exists (t_fvar s0 y). exists B.
1289     apply typing_var.
1290     -- exact HyΓ.
1291     -- exact HBthin.
1292   * apply subcontext_extend.
1293     -- apply subcontext_extend_r.
1294     -- exact HyΓ.
1295   * apply HB2; auto.
1296 + apply thinning with Γ0.
1297   * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1298   * apply subcontext_extend_r.
1299   * exact HB3.
1300
1301 - exists B0. exists sc. exists L.
1302   repeat split; auto. apply beq_refl.
1303
1304 - destruct (IHM eq_refl) as [C [s3 [L0 [HB1 [HB2 [HB3 HB4]]]]]].
1305   exists C. exists s3. exists L0.
1306   repeat split; auto. apply beq_trans with A0; auto. apply beq_sym;
auto.
1307 Qed.
1308
1309 Lemma generation_app :
1310   forall Γ M N T,
1311     Γ ⊢ t_app M N ∈ T ->
1312     exists s0 A B,
1313       Γ ⊢ M ∈ (t_pi s0 A B) /\
1314       Γ ⊢ N ∈ A /\
1315       T =b B ^^ N.
1316 Proof.
1317   intros Γ M N T H.
1318   remember (t_app M N) as W eqn:HeqW.
1319   induction H as
1320     [ sa sb HA
1321     | Γ0 x0 A0 s1 Hfresh0 HA0 IHA0
1322     | Γ0 x0 B0 M0 A0 s1 Hfresh0 HM0 IHM0 HB0 IHB0
1323     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1324     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1325     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
1326     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1327   inversion HeqW; subst.
1328
1329 - destruct (IHM0 eq_refl) as [s0 [A' [B' [Q1 [Q2 Q3]]]]].
1330   exists s0. exists A'. exists B'. repeat split; auto.

```

```

1331 + apply thinning with  $\Gamma_0$ ; auto.
1332 * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1333 * apply subcontext_extend_r.
1334 + apply thinning with  $\Gamma_0$ ; auto.
1335 * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1336 * apply subcontext_extend_r.
1337
1338 - exists sla. exists A0. exists B0. repeat split; auto. apply
1339   beq_refl.
1340
1341 - destruct (IHM eq_refl) as [s0 [A' [B' [Q1 [Q2 Q3]]]]].
1342   exists s0. exists A'. exists B'. repeat split; auto.
1343   apply beq_trans with A0; auto. apply beq_sym; auto.
1344 Qed.
1345
1346 (* =====
1347 *)
1348 (** * Substitution lemma*)
1349
1350 Definition subst_decl (x : var) (N : term) '(y,(s,T)) : (var * (Sort *
1351   term)) := (y, (s, T[x = N])).
1352
1353 Definition subst_context (x : var) (N : term) ( $\Gamma$  : context) := map
1354   (subst_decl x N)  $\Gamma$ .
1355
1356 Notation " $\Gamma$  [ x = N ]" := (subst_context x N  $\Gamma$ ) (at level 1, left
1357   associativity).
1358
1359 Lemma subst_context_snoc : forall x N  $\Delta$  y s A,
1360   ( $\Delta$  ++ [(y, (s, A))]) [ x = N ] =  $\Delta$  [ x = N ] ++ [(y, (s, A [ x = N
1361     ]))].
1362 Proof.
1363   intros. unfold subst_context. rewrite map_app. reflexivity.
1364 Qed.
1365
1366 Lemma map_fst_subst_decl : forall x N  $\Delta$ ,
1367   map fst (map (subst_decl x N)  $\Delta$ ) = map fst  $\Delta$ .
1368 Proof.
1369   intros x N  $\Delta$ .
1370   induction  $\Delta$  as [| [v [s A]]  $\Delta_{tl}$  IH].
1371   - reflexivity.
1372   - cbn [map subst_decl fst]. f_equal. exact IH.
1373 Qed.
1374
1375 Lemma substitution :
1376   forall ( $\Gamma$  : context) ( $\Delta$  : context) x sx C M B N,
1377      $\Gamma$  ++ [(x,(sx,C))] ++  $\Delta \vdash M \in B \rightarrow$ 
1378      $\Gamma \vdash N \in C \rightarrow$ 
1379      $\Gamma$  ++ ( $\Delta[x = N]$ )  $\vdash M[x = N] \in B[x = N]$ .

```



```

1374 Proof.
1375 intros  $\Gamma$   $\Delta$  x sx C M B N H1 H2.
1376 assert (HlcN : lc N) by (apply (typing_lc  $\Gamma$  N C H2)).
1377 remember ( $\Gamma$  ++ [(x, (sx, C))] ++  $\Delta$ ) as Z eqn:HZ.
1378 generalize dependent  $\Delta$ .
1379 induction H1 as
1380   [ sa sb HA
1381   |  $\Gamma_0$  x0 A0 s0 Hfresh0 HA0 IHA0
1382   |  $\Gamma_0$  x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
1383   |  $\Gamma_0$  A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1384   |  $\Gamma_0$  A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1385   |  $\Gamma_0$  M0 N0 A0 B0 sla HM IHM HN IHN
1386   |  $\Gamma_0$  M0 A0 B0 sd HM IHM Heq HB IHB ].
1387
1388 - intros  $\Delta$  HZ. destruct  $\Gamma$ .
1389   + discriminate.
1390   + discriminate.
1391
1392 - intros  $\Delta$  HZ. destruct  $\Delta$  as [| (y, E)  $\Delta_1$ ].
1393   + rewrite app_nil_r in HZ. apply app_singleton_injective in HZ.
1394     destruct HZ as [HZ1 [HZ2 HZ3]]. subst.
1395     inversion HZ3; subst.
1396     unfold subst_context. simpl map. rewrite app_nil_r.
1397     rewrite subst_var. destruct (eq_var_dec x x) as [_ | Hne]; [ |
1398       contradiction Hne; reflexivity].
1399     destruct (subst_fresh_typing  $\Gamma$  C (t_sort sx) x N Hfresh0 HA0) as
1400       [HC _].
1401     rewrite HC. exact H2.
1402   + assert (Hnonempty : (y, E) ::  $\Delta_1$  <> []) by discriminate.
1403     destruct (exists_last Hnonempty) as [ $\Delta_1'$  [[y0 E0] Hlast]].
1404     rewrite Hlast in *.
1405     rewrite app_assoc in HZ. rewrite app_assoc in HZ.
1406     apply app_singleton_injective in HZ. destruct HZ as [Q1 [Q2
1407       Q3]]. subst.
1408     unfold subst_context in *.
1409     rewrite map_app in *.
1410     simpl map in *.
1411     rewrite subst_var. destruct (eq_var_dec y0 x).
1412     * subst. ex falso. apply Hfresh0.
1413     unfold dom. rewrite map_app. rewrite map_app. simpl.
1414     apply in_or_app. left. apply in_or_app. right. simpl. left.
1415     reflexivity.
1416     * rewrite app_assoc. apply typing_var.
1417     -- unfold is_fresh in *. intros Hin. apply Hfresh0.
1418       unfold dom in *. repeat rewrite map_app in *. simpl in *.
1419       rewrite map_fst_subst_decl in Hin.
1420       apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1421       ++ apply in_or_app. left. apply in_or_app. left. exact Hin.
1422       ++ apply in_or_app. right. exact Hin.

```

```

1418      -- rewrite subst_sort in IHA0. apply IHA0. rewrite app_assoc.
      reflexivity.
1419
1420 - intros Δ HZ. destruct Δ as [| (y, E) Δ1].
1421 + rewrite app_nil_r in HZ. apply app_singleton_injective in HZ.
1422   destruct HZ as [HZ1 [HZ2 HZ3]]. subst.
1423   simpl in *. rewrite app_nil_r.
1424   destruct (subst_fresh_typing Γ M0 A0 x N Hfresh0 HM0) as [HM'
      HA0'].
1425   rewrite HM', HA0'. apply HM0.
1426 + assert (Hnonempty : (y, E) :: Δ1 <> []) by discriminate.
1427   destruct (exists_last Hnonempty) as [Δ1' [[y1 E1] Hlast]].
1428   rewrite Hlast in *.
1429   rewrite app_assoc in HZ. rewrite app_assoc in HZ.
1430   apply app_singleton_injective in HZ.
1431   destruct HZ as [HΓ0 [Hy1 HE1]]. subst.
1432   unfold subst_context in *. rewrite map_app. simpl map.
1433   rewrite app_assoc.
1434   apply typing_weak.
1435   * unfold is_fresh in *. intro Hin. apply Hfresh0.
1436     unfold dom in *. repeat rewrite map_app in *.
1437     rewrite map_fst_subst_decl in Hin.
1438     apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1439     -- apply in_or_app. left. apply in_or_app. left. exact Hin.
1440     -- apply in_or_app. right. exact Hin.
1441   * apply IHM0. rewrite <- app_assoc. reflexivity.
1442   * rewrite subst_sort in IHB0. apply IHB0. rewrite <- app_assoc.
      reflexivity.
1443
1444 - intros Δ HZ.
1445   apply (typing_pi (Γ ++ Δ [ x = N ]) (A0 [ x = N ]) (B0 [ x = N ])
      sa sb sc
1446     (x :: L ++ fv N)).
1447 + rewrite <- (subst_sort sa x N). apply IHA. exact HZ.
1448 + apply bvar_tag_ok_subst; [exact HlcN | exact HTagB].
1449 + intros y Hy.
1450   assert (Hyx : y <> x).
1451   { intro Heq. subst y. apply Hy. simpl. left. reflexivity. }
1452   assert (HyL' : ~ In y L).
1453   { intro Hc. apply Hy. simpl. right. apply in_or_app. left. exact
      Hc. }
1454   rewrite <- (subst_open_var B0 sa y x N Hyx HlcN).
1455   rewrite <- (subst_sort sb x N).
1456   rewrite <- app_assoc. rewrite <- subst_context_snoc.
1457   apply (IHB y HyL' (Δ ++ [(y, (sa, A0))])).
1458   rewrite HZ. rewrite <- app_assoc. reflexivity.
1459 + exact HR.
1460
1461 - intros Δ HZ.

```

```

1462   apply (typing_lam ( $\Gamma \mathrel{++} \Delta \mid x = N$ ]) ( $A0 \mid x = N$ ]) ( $M0 \mid x = N$ ])
1463         (B0  $\mid x = N$ ])
1464         sa sc (x :: L ++ fv N)).
1465 + rewrite <- (subst_sort sa x N). apply IHA. exact HZ.
1466 + apply bvar_tag_ok_subst; [exact HlcN | exact HTagM].
1467 + intros y Hy.
1468   assert (Hyx : y <> x).
1469   { intro Heq. subst y. apply Hy. simpl. left. reflexivity. }
1470   assert (HyL' : ~ In y L).
1471   { intro Hc. apply Hy. simpl. right. apply in_or_app. left. exact
1472     Hc. }
1473   rewrite <- (subst_open_var M0 sa y x N Hyx HlcN).
1474   rewrite <- (subst_open_var B0 sa y x N Hyx HlcN).
1475   rewrite <- app_assoc. rewrite <- subst_context_snoc.
1476   apply (IHM y HyL' ( $\Delta \mathrel{++} [(y, (sa, A0))]$ )).
1477   rewrite HZ. rewrite <- app_assoc. reflexivity.
1478 + rewrite <- (subst_pi sa A0 B0 x N). apply IHPi. exact HZ.
1479
1480 - intros  $\Delta$  HZ.
1481   rewrite subst_open by exact HlcN.
1482   apply typing_app with (A := A0  $\mid x = N$ ]) (s1 := s1a).
1483 + rewrite <- (subst_pi s1a A0 B0 x N). apply IHM. exact HZ.
1484 + apply IHN. exact HZ.
1485
1486 - intros  $\Delta$  HZ.
1487   apply typing_conv with (A0  $\mid x = N$ ]) sd.
1488 + apply IHM. exact HZ.
1489 + apply beta_eq_subst; auto.
1490 + rewrite <- (subst_sort sd x N). apply IHB. exact HZ.
1491
1492 Qed.
1493
1494 Lemma type_correctness:
1495   forall  $\Gamma$  M N,
1496      $\Gamma \vdash M \in N \rightarrow$ 
1497     ( $\exists$  s, N = t_sort s) /\
1498     ( $\exists$  s,  $\Gamma \vdash N \in$  (t_sort s)).
1499
1500 Proof.
1501   intros  $\Gamma$  M N Htyp.
1502   induction Htyp as
1503   [ sa sb HA
1504   |  $\Gamma0$  x0 A0 s0 Hfresh0 HA0 IHA0
1505   |  $\Gamma0$  x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
1506   |  $\Gamma0$  A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1507   |  $\Gamma0$  A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1508   |  $\Gamma0$  M0 N0 A0 B0 s1a HM IHM HN IHN
1509   |  $\Gamma0$  M0 A0 B0 sd HM IHM Heq HB IHB ].
1510
1511 - left. exists sb. reflexivity.
1512
1513 - right. exists s0. apply typing_weak; auto.

```

```

1510 - destruct IHM0 as [[s Hs] | [s Hs]].
1511 + left. exists s. apply Hs.
1512 + right. exists s. apply thinning with  $\Gamma_0$ ; auto.
1513 * exists (t_fvar s0 x0). exists B0. apply typing_var; auto.
1514 * apply subcontext_extend_r.
1515
1516 - left. exists sc. reflexivity.
1517
1518 - right. exists sc. apply HPi.
1519
1520
1521 - destruct IHM as [[s Hs] | [s Hs]].
1522 + discriminate Hs.
1523 + right.
1524   destruct (generation_pi  $\Gamma_0$  sla A0 B0 (t_sort s) Hs)
1525     as [s2 [s3 [L [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
1526   exists s2.
1527   remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
1528   assert (Hx0L :  $\sim$  In x0 L).
1529   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
1530     apply in_or_app. left. exact Hc. }
1531   assert (Hx0B :  $\sim$  In x0 (fv B0)).
1532   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
1533     apply in_or_app. right. exact Hc. }
1534   assert (Hopen :  $\Gamma_0 ++ [(x0, (sla, A0))] ++ [] \vdash \text{open\_var } B0 \text{ sla}$ 
1535      $x0 \in t\_sort \ s2$ ).
1536   { rewrite app_nil_r. apply HB2; auto. }
1537   pose proof (substitution  $\Gamma_0 [] x0 sla A0 (\text{open\_var } B0 \text{ sla } x0)$ 
1538     (t_sort s2) N0 Hopen HN)
1539     as Hsub.
1540   simpl in Hsub.
1541   assert (Heq : (open_var B0 sla x0) [ x0 = N0 ] = B0 ^^ N0).
1542   { symmetry. apply subst_intro; auto. apply (typing_lc  $\Gamma_0$  N0 A0
1543     HN). }
1544   rewrite Heq in Hsub. rewrite app_nil_r in Hsub. exact Hsub.
1545
1546 - right. exists sd. exact HB.
1547 Qed.
1548
1549 (* =====
1550 *)
1551 (** * Sort-order metatheory *)
1552
1553 Axiom permutation :
1554   forall  $\Gamma \Delta x y sx sy Tx Ty M C$ ,
1555      $\sim$  In x (fv Ty) ->
1556      $\Gamma ++ [(x, (sx, Tx))] ++ [(y, (sy, Ty))] ++ \Delta \vdash M \in C \rightarrow$ 
1557      $\Gamma ++ [(y, (sy, Ty))] ++ [(x, (sx, Tx))] ++ \Delta \vdash M \in C$ .

```

```

1555 Definition functional : Prop :=
1556   (forall s t t', A s t -> A s t' -> t = t') /\
1557   (forall s t u u', R s t u -> R s t u' -> u = u').
1558
1559 Axiom type_unicity :
1560   functional ->
1561   forall  $\Gamma$  M T1 T2,
1562    $\Gamma \vdash M \in T1$  ->
1563    $\Gamma \vdash M \in T2$  ->
1564   T1 = T2.
1565
1566 Definition top_sort (s : Sort) : Prop := ~ (exists s', A s s').
1567
1568 Definition bottom_sort (s : Sort) : Prop := ~ (exists s', A s' s).
1569
1570 Axiom top_sort_lemma :
1571   forall  $\Gamma$  x y M N s,
1572   top_sort s ->
1573   (~ ( $\Gamma \vdash t\_sort\ s \in M$ )) /\
1574   (~ ( $\Gamma \vdash t\_fvar\ x\ y \in t\_sort\ s$ )) /\
1575   (~ ( $\Gamma \vdash t\_app\ M\ N \in t\_sort\ s$ )).
1576
1577 Definition persistent : Prop :=
1578   functional /\
1579   (forall s s' t , A s t -> A s' t -> s = s') /\
1580   (forall s t u, R s t u -> t = u).
1581
1582 Definition tiered (n : nat) : Prop :=
1583   exists index_of : Sort -> nat,
1584   (forall x : Sort, 0 < index_of x /\ index_of x < n + 1) /\
1585   (forall x y : Sort, index_of x = index_of y -> x = y) /\
1586   (forall i : nat, 0 < i /\ i < n + 1 -> exists x : Sort, index_of x
1587     = i) /\
1588   (forall x y : Sort, A x y <-> index_of x < n /\ index_of y = S
1589     (index_of x)) /\
1590   (forall x y z : Sort, R x y z -> y = z).
1591
1592 Definition A_neighbor (s : Sort) (p : Sort * Sort) : Prop :=
1593   A (fst p) (snd p) /\ (fst p = s \/ snd p = s).
1594
1595 Lemma persistent_neighbor_bound :
1596   persistent ->
1597   forall s p1 p2 p3,
1598   A_neighbor s p1 -> A_neighbor s p2 -> A_neighbor s p3 ->
1599   p1 = p2 \/ p1 = p3 \/ p2 = p3.
1600
1601 Proof.
1602   intros [[Hfunc _] [Hpers _]] s [x1 y1] [x2 y2] [x3 y3]
1603   [HA1 [Hx1|Hy1]] [HA2 [Hx2|Hy2]] [HA3 [Hx3|Hy3]].
1604   - subst. simpl in *. subst. left. f_equal. eauto using Hfunc,
1605     Hpers.

```

```

1602   - subst. simpl in *. subst. left. f_equal. eauto using Hfunc,
1603     Hpers.
1604   - subst. simpl in *. subst. right. left. f_equal. eauto using
1605     Hfunc, Hpers.
1606   - subst. simpl in *. subst. right. right. f_equal. eauto using
1607     Hfunc, Hpers.
1608   - subst. simpl in *. subst. right. left. f_equal. eauto using
1609     Hfunc, Hpers.
1610   - subst. simpl in *. subst. left. f_equal. eauto using Hfunc,
1611     Hpers.
1612   - subst. simpl in *. subst. left. f_equal. eauto using Hfunc,
1613     Hpers.
1614 Qed.
1615
1616 Reserved Notation " $s <_A t$ " (at level 70, no associativity).
1617
1618 Inductive A_lt : Sort -> Sort -> Prop :=
1619   | A_lt_step : forall s t, A s t -> s <_A t
1620   | A_lt_trans : forall s t u, A s t -> t <_A u -> s <_A u
1621
1622 where " $s <_A t$ " := (A_lt s t).
1623
1624 Reserved Notation " $s \leq_A t$ " (at level 70, no associativity).
1625
1626 Inductive A_le : Sort -> Sort -> Prop :=
1627   | A_le_refl : forall s, s ≤_A s
1628   | A_le_step : forall s t u, A s t -> t ≤_A u -> s ≤_A u
1629
1630 where " $s \leq_A t$ " := (A_le s t).
1631
1632 Reserved Notation " $s \approx_A t$ " (at level 70, no associativity).
1633
1634 Inductive A_eq : Sort -> Sort -> Prop :=
1635   | A_eq_refl : forall s, s ≈_A s
1636   | A_eq_step : forall s t, A s t -> s ≈_A t
1637   | A_eq_sym : forall s t, s ≈_A t -> t ≈_A s
1638   | A_eq_trans : forall s t u, s ≈_A t -> t ≈_A u -> s ≈_A u
1639
1640 where " $s \approx_A t$ " := (A_eq s t).
1641
1642 Lemma A_le_of_lt : forall s t, s <_A t -> s ≤_A t.
1643 Proof. induction 1; econstructor; eauto. apply A_le_refl. Qed.
1644
1645 Lemma A_lt_trans' : forall s t u, s <_A t -> t <_A u -> s <_A u.
1646 Proof.
1647   intros s t u H1 H2.
1648   induction H1.

```

```

1644 - apply A_lt_trans with t; auto.
1645 - apply IHA_lt in H2. apply A_lt_trans with t; auto.
1646 Qed.
1647
1648 Fixpoint chain (c : list Sort) : Prop :=
1649   match c with
1650   | [] | [_] => True
1651   | t1 :: (t2 :: _) as rest => A t1 t2 /\ chain rest
1652   end.
1653
1654 Definition chain_from_to (s t : Sort) (c : list Sort) : Prop :=
1655   chain c /\ hd_error c = Some s /\ List.last c s = t.
1656
1657 Lemma last_default_irrelevant :
1658   forall (c : list Sort) (d1 d2 : Sort),
1659     c <> [] -> last c d1 = last c d2.
1660 Proof.
1661   induction c as [| a c IH]; intros d1 d2 Hne.
1662   - contradiction.
1663   - destruct c as [| b c'].
1664     + reflexivity.
1665     + simpl. apply IH. discriminate.
1666 Qed.
1667
1668 Lemma A_le_iff_chain : forall s t,
1669   s ≤A t <-> exists c, chain_from_to s t c.
1670 Proof.
1671   intros s t. repeat split.
1672   - intros H. induction H as [| s t u H1 H2 [c IH]].
1673     + exists [s]. repeat split.
1674     + destruct IH as [Q1 [Q2 Q3]]. exists (s :: c). repeat split.
1675       * destruct c.
1676         ** reflexivity.
1677         ** repeat split; auto. simpl in Q2. injection Q2 as Q2. subst.
1678           apply H1.
1679       * destruct c as [| c0 c'] eqn:Hc.
1680         ** discriminate.
1681         ** simpl. simpl in Q3.
1682           injection Q2 as Q2. subst c0.
1683           destruct c' as [| c1 c''].
1684             *** exact Q3.
1685             *** rewrite (last_default_irrelevant (c1 :: c'') s t).
1686               **** exact Q3.
1687               **** discriminate.
1688   - intros [c Hc0]. generalize dependent s.
1689     induction c as [| a c IH]; intros s [H1 [H2 H3]].
1690     + discriminate.
1691     + injection H2 as H2. subst a.
1692       destruct c as [| c0 c'].

```

```

1693 * simpl in H3. subst t. apply A_le_refl.
1694 * simpl in H1. destruct H1 as [HA Hchain]. apply A_le_step with
    (t := c0).
1695   -- exact HA.
1696   -- apply IH. repeat split.
1697   ++ exact Hchain.
1698   ++ destruct c' as [| c1 c''] eqn:E'.
1699       ** simpl in H3 |- *. exact H3.
1700       ** simpl in H3 |- *.
1701           destruct c'' as [| c2 c3].
1702           --- exact H3.
1703           --- rewrite (last_default_irrelevant (c2 :: c3) c0
                s).
1704               +++ exact H3.
1705               +++ discriminate.
1706 Qed.
1707
1708 Lemma persistent_chain_unique_from :
1709   persistent ->
1710   forall c1 c2 s,
1711     chain c1 -> chain c2 ->
1712     hd_error c1 = Some s -> hd_error c2 = Some s ->
1713     length c1 = length c2 ->
1714     c1 = c2.
1715 Proof.
1716   intros Hpers c1.
1717   induction c1 as [| a1 c1 IH]; intros c2 s H1 H2 Hh1 Hh2 Hlen.
1718   - discriminate Hh1.
1719   - injection Hh1 as Hh1. subst a1. destruct c2 as [| a2 c2].
1720   + discriminate Hh2.
1721   + injection Hh2 as Hh2. subst a2. destruct c1 as [| b1 c1'].
1722     * destruct c2 as [| b2 c2']; auto. discriminate Hlen.
1723     * destruct c2 as [| b2 c2'].
1724       -- discriminate Hlen.
1725       -- simpl in H1, H2.
1726         destruct H1 as [HA1 Hc1'], H2 as [HA2 Hc2'].
1727         assert (Hb : b1 = b2).
1728         { destruct Hpers as [[Hfunc1 _] _].
1729           eapply Hfunc1; eauto. }
1730         subst b2.
1731         f_equal.
1732         apply IH with (s := b1); auto.
1733 Qed.
1734
1735 Lemma chain_last :
1736   forall c x y,
1737     chain (c ++ [x;y]) ->
1738     A x y.
1739 Proof.
1740   induction c as [|a c IH].

```



```

1741 - simpl. tauto.
1742 - destruct c.
1743   + simpl. tauto.
1744   + intros. destruct H as [_ H]; auto.
1745 Qed.
1746
1747 Lemma chain_prefix :
1748   forall c x y,
1749     chain (c ++ [x; y]) ->
1750     chain (c ++ [x]).
1751 Proof.
1752   induction c as [| a c IH].
1753   - reflexivity.
1754   - destruct c as [| b c].
1755     + simpl. tauto.
1756     + simpl. intros x y [Hab Hchain]. split; auto. apply IH with y;
      auto.
1757 Qed.
1758
1759 Lemma chain_snoc_inv : forall l x,
1760   chain (l ++ [x]) ->
1761   chain l /\ (l <> [] -> A (last l x) x).
1762 Proof.
1763   induction l as [| a l IH]; intros x Hc.
1764   - simpl. split; [exact I | intros []; reflexivity].
1765   - destruct l as [| b l'].
1766     + simpl in Hc. destruct Hc as [HA _].
1767       simpl. split; [exact I | intros _; simpl; exact HA].
1768     + simpl in Hc. destruct Hc as [HAab Hc'].
1769       destruct (IH x Hc') as [IHchain IHlast].
1770       split.
1771       * simpl. split; [exact HAab | exact IHchain].
1772       * intros _.
1773         specialize (IHlast ltac:(discriminate)).
1774         simpl in IHlast.
1775         simpl.
1776         exact IHlast.
1777 Qed.
1778
1779 Lemma persistent_chain_unique_to :
1780   persistent ->
1781   forall c1 t s1 c2 s2,
1782     chain_from_to s1 t c1 -> chain_from_to s2 t c2 ->
1783     length c1 = length c2 ->
1784     c1 = c2.
1785 Proof.
1786   intros Hpers c1.
1787   induction c1 as [| x l IH] using rev_ind; intros t s1 c2 s2 H1 H2
      Hlen.
1788   - destruct H1 as [_ [Hh _]]. discriminate Hh.

```

```

1789 - destruct H1 as [Hc1 [Hh1 Hlast1]].
1790 rewrite last_last in Hlast1. subst x.
1791 destruct c2 as [| c2h c2t].
1792 + destruct H2 as [_ [Hh2 _]]. discriminate Hh2.
1793 + assert (Hne2 : c2h :: c2t <> []) by discriminate.
1794   destruct (exists_last Hne2) as [l2 [x2 Hl2eq]].
1795   rewrite Hl2eq in H2.
1796   destruct H2 as [Hc2 [Hh2 Hlast2]].
1797   rewrite last_last in Hlast2. subst x2.
1798   pose proof (chain_snoc_inv l t Hc1) as [Hcl Hal].
1799   pose proof (chain_snoc_inv l2 t Hc2) as [Hcl2 Hal2].
1800   destruct l as [| a l'].
1801 ++ destruct l2 as [| b l2'].
1802   * rewrite Hl2eq. reflexivity.
1803   * exfalso.
1804     assert (Hlen2 : length (c2h :: c2t) = length ((b :: l2') ++
1805       [t])).
1806     { rewrite Hl2eq. reflexivity. }
1807     rewrite length_app in Hlen2.
1808     simpl in Hlen2, Hlen.
1809     lia.
1810 ++ destruct l2 as [| b l2'].
1811   * exfalso.
1812     assert (Hlen2 : length (c2h :: c2t) = length ([] ++ [t])).
1813     { rewrite Hl2eq. reflexivity. }
1814     simpl in Hlen2.
1815     rewrite length_app in Hlen.
1816     simpl in Hlen.
1817     lia.
1818   * assert (Hne : a :: l' <> []) by discriminate.
1819     assert (Hne2' : b :: l2' <> []) by discriminate.
1820     specialize (Hal Hne). specialize (Hal2 Hne2').
1821     assert (Ht' : last (a :: l') t = last (b :: l2') t).
1822     { destruct Hpers as [_ [Hpred _]]. exact (Hpred _ _ t Hal
1823       Hal2). }
1824     assert (Hlen' : length (a :: l') = length (b :: l2')).
1825     { simpl. rewrite Hl2eq, !length_app in Hlen. simpl in Hlen.
1826       lia. }
1827     assert (Heq : a :: l' = b :: l2').
1828     { apply (IH (last (a :: l') t) a (b :: l2') b).
1829       - split; [exact Hcl|]. split; [reflexivity|].
1830         apply (last_default_irrelevant (a :: l') a t).
1831         discriminate.
1832       - split; [exact Hcl2|]. split; [reflexivity|].
1833         rewrite Ht'.
1834         apply (last_default_irrelevant (b :: l2') b t).
1835         discriminate.
1836       - exact Hlen'. }
1837     rewrite Hl2eq. f_equal. exact Heq.

```

```

1833 Qed.
1834
1835 Definition separable : Prop :=
1836   forall s s', R s s' s' -> s ≈A s'.
1837
1838 Definition atomic : Prop :=
1839   forall s s', s ≈A s'.
1840
1841 Definition ascending_chain_condition : Prop :=
1842   forall s : Sort, Acc (fun u v => u <A v) s.
1843
1844 Definition descending_chain_condition : Prop :=
1845   forall s : Sort, Acc (fun u v => v <A u) s.
1846
1847 Definition bounded : Prop :=
1848   ascending_chain_condition /\ descending_chain_condition.
1849
1850 Definition weakly_non_dependent : Prop :=
1851   forall s t u, R s t u -> u ≤A t /\ t ≤A s.
1852
1853 Definition stratified : Prop :=
1854   ascending_chain_condition /\ weakly_non_dependent.
1855
1856 Definition generalized_non_dependent : Prop :=
1857   stratified /\ persistent.
1858
1859 Definition bounded_non_dependent : Prop :=
1860   generalized_non_dependent /\ bounded.
1861
1862 Lemma wf_by_measure :
1863   forall (T : Type) (Rel : T -> T -> Prop) (f : T -> nat),
1864     (forall u v, Rel u v -> f u < f v) ->
1865     forall s, Acc (fun u v => Rel u v) s.
1866 Proof.
1867   intros T Rel f Hmono s.
1868   remember (f s) as k eqn:Hk.
1869   generalize dependent s.
1870   induction k as [k IH] using (well_founded_induction Wf_nat.lt_wf).
1871   intros s Hk.
1872   constructor.
1873   intros y Hy.
1874   apply (IH (f y)).
1875   - rewrite Hk. apply Hmono. exact Hy.
1876   - reflexivity.
1877 Qed.
1878
1879 Lemma acc_irrefl :
1880   forall (T : Type) (Rel : T -> T -> Prop) (x : T),
1881     Acc Rel x -> ~ Rel x x.
1882 Proof.

```

```

1883   intros T Rel x Hacc.
1884   induction Hacc as [x _ IH].
1885   intro Hxx.
1886   exact (IH x Hxx Hxx).
1887 Qed.
1888
1889 Lemma A_lt_irrefl : bounded -> forall s, ~ (s <A s).
1890 Proof.
1891   intros Hbnd s Hlt.
1892   destruct Hbnd as [Hasc _].
1893   exact (acc_irrefl Sort (fun u v => u <A v) s (Hasc s) Hlt).
1894 Qed.
1895
1896 Lemma chain_skipn : forall c k, chain c -> chain (skipn k c).
1897 Proof.
1898   induction c as [| a c IH]; intros k Hc.
1899   - destruct k; simpl; exact I.
1900   - destruct k as [| k].
1901     + simpl. exact Hc.
1902     + simpl. apply IH.
1903       destruct c as [| b c']; simpl in Hc.
1904       * exact I.
1905       * destruct Hc as [_ Hc]. exact Hc.
1906 Qed.
1907
1908 Lemma chain_firstn : forall c k, chain c -> chain (firstn k c).
1909 Proof.
1910   induction c as [| a c IH]; intros k Hc.
1911   - destruct k; simpl; exact I.
1912   - destruct k as [| k].
1913     + simpl. exact I.
1914     + destruct c as [| b c'].
1915       * simpl. destruct k; exact I.
1916       * simpl in Hc |- *. destruct Hc as [HAab Hc'].
1917         assert (Hfk : chain (firstn k (b :: c')) by (apply IH; simpl;
1918         assumption)).
1919         destruct (firstn k (b :: c')) as [| t2 rest] eqn:Efk.
1920         -- exact I.
1921         -- split.
1922           ++ destruct k as [| k'].
1923             ** simpl in Efk. discriminate Efk.
1924             ** simpl in Efk. injection Efk as Ht2. subst t2. exact
1925             HAab.
1926           ++ exact Hfk.
1927 Qed.
1928
1929 Lemma hd_error_skipn : forall c k a,
1930   nth_error c k = Some a -> hd_error (skipn k c : list Sort) = Some a.
1931 Proof.
1932   induction c as [| x c IH]; intros k a Hnth.

```

```

1931 - destruct k; discriminate.
1932 - destruct k as [| k].
1933   + simpl in Hnth |- *. exact Hnth.
1934   + simpl in Hnth |- *. apply IH. exact Hnth.
1935 Qed.
1936
1937 Lemma last_skipn : forall c k d,
1938   k < length c -> last (skipn k c : list Sort) d = last c d.
1939 Proof.
1940   induction c as [| x c IH]; intros k d Hk.
1941   - simpl in Hk. lia.
1942   - destruct k as [| k].
1943     + reflexivity.
1944     + simpl. destruct c as [| y c'].
1945       * simpl in Hk. lia.
1946       * apply IH. simpl in *. lia.
1947 Qed.
1948
1949 Lemma hd_error_firstn : forall c k a,
1950   hd_error c = Some a -> 0 < k -> hd_error (firstn k c : list Sort) =
1951     Some a.
1952 Proof.
1953   intros c k a Hhd Hk.
1954   destruct c as [| x c]; [discriminate |].
1955   simpl in Hhd. injection Hhd as Hhd. subst x.
1956   destruct k as [| k]; [lia |]. reflexivity.
1957 Qed.
1958
1959 Lemma last_firstn_nth : forall c k a d,
1960   nth_error c k = Some a -> last (firstn (S k) c : list Sort) d = a.
1961 Proof.
1962   induction c as [| x c IH]; intros k a d Hnth.
1963   - destruct k as [| k']; simpl in Hnth; discriminate Hnth.
1964   - destruct k as [| k].
1965     + simpl in Hnth. injection Hnth as Hnth. subst a. simpl.
1966       destruct c; reflexivity.
1967     + simpl in Hnth. simpl.
1968       destruct c as [| y c'].
1969       * destruct k as [| k']; simpl in Hnth; discriminate Hnth.
1970       * apply (IH k a d Hnth).
1971 Qed.
1972
1973 Lemma length_skipn : forall (c : list Sort) k,
1974   k <= length c -> length (skipn k c) = length c - k.
1975 Proof. intros c k Hk. apply length_skipn. Qed.
1976
1977 Lemma length_firstn_le : forall (c : list Sort) k,
1978   k <= length c -> length (firstn k c) = k.
1979 Proof. intros c k Hk. rewrite length_firstn. lia. Qed.

```

```

1980 Lemma chain_pos_length_A_lt :
1981   forall c a b, chain_from_to a b c -> 2 <= length c -> a <A b.
1982 Proof.
1983   induction c as [| x l IH] using rev_ind; intros a b [Hc [Hh Hl]]
1984   Hlen.
1985   - simpl in Hlen. lia.
1986   - rewrite last_last in Hl. subst x.
1987     destruct l as [| y l'].
1988     + simpl in Hlen. lia.
1989     + destruct (chain_snoc_inv (y :: l') b Hc) as [Hcl Hal].
1990       specialize (Hal ltac:(discriminate)).
1991       destruct l' as [| z l''].
1992       * simpl in Hh. injection Hh as Hh. subst y.
1993       * apply A_lt_step. exact Hal.
1994       * apply A_lt_trans' with (last (y :: z :: l'') b).
1995         -- apply IH.
1996         ++ split; [exact Hcl |]. split; [exact Hh |].
1997         ++ apply last_default_irrelevant. discriminate.
1998         ++ simpl. lia.
1999         -- apply A_lt_step. exact Hal.
2000 Qed.
2001 Lemma nth_error_exists : forall (c : list Sort) k,
2002   k < length c -> exists a, nth_error c k = Some a.
2003 Proof.
2004   induction c as [| x c IH]; intros k Hk.
2005   - simpl in Hk. lia.
2006   - destruct k as [| k'].
2007     + exists x. reflexivity.
2008     + simpl. apply IH. simpl in Hk. lia.
2009 Qed.
2010 Lemma chain_same_end_relates :
2011   persistent ->
2012   forall s u t c1 c2,
2013     chain_from_to s t c1 -> chain_from_to u t c2 ->
2014     s <A u \ / s = u \ / u <A s.
2015 Proof.
2016   intros Hpers s u t c1 c2 H1 H2.
2017   assert (Htri : length c1 < length c2 \ / length c1 = length c2 \ /
2018     length c2 < length c1) by lia.
2019   destruct Htri as [Hlt | [Heq | Hgt]].
2020
2021   - (* c1 shorter: u <A s *)
2022     right. right.
2023     destruct H1 as [Hc1 [Hh1 Hl1]].
2024     destruct H2 as [Hc2 [Hh2 Hl2]].
2025     remember (length c2 - length c1) as k eqn:Hkeq.
2026     assert (Hclne : 1 <= length c1).
2027     { destruct c1 as [| ? ?]; simpl.

```

```

2028   - discriminate Hh1.
2029   - lia. }
2030 assert (Hk1 : 0 < k) by lia.
2031 assert (Hklt : k < length c2) by lia.
2032 destruct (nth_error_exists c2 k Hklt) as [a Hnth].
2033 pose proof (hd_error_skipn c2 k a Hnth) as Hheadskip.
2034 pose proof (chain_skipn c2 k Hc2) as Hchainskip.
2035 pose proof (last_skipn c2 k a Hklt) as Hlastskip.
2036 assert (Hlast_a_u : last c2 a = last c2 u).
2037 { apply last_default_irrelevant. destruct c2; [discriminate Hh2 |
discriminate]. }
2038 assert (Hlastskip' : last (skipn k c2) a = t).
2039 { rewrite Hlastskip, Hlast_a_u. exact Hl2. }
2040 assert (Hsuffix : chain_from_to a t (skipn k c2)).
2041 { split; [exact Hchainskip | split; [exact Hheadskip | exact
Hlastskip']] . }
2042 assert (Hlensuf : length (skipn k c2) = length c1).
2043 { rewrite (length_skipn c2 k ltac:(lia)). lia. }
2044 assert (Heqc : c1 = skipn k c2).
2045 { apply (persistent_chain_unique_to Hpers c1 t s (skipn k c2) a).
2046   - split; [exact Hc1 | split; [exact Hh1 | exact Hl1]].
2047   - exact Hsuffix.
2048   - symmetry. exact Hlensuf. }
2049 assert (Hsa : s = a).
2050 { assert (Hh1' : hd_error c1 = Some s) by exact Hh1.
2051   rewrite Heqc in Hh1'. rewrite Hheadskip in Hh1'. injection Hh1'
2052   as Hh1'. symmetry. exact Hh1'. }
2053 assert (Hprefix : chain_from_to u s (firstn (S k) c2)).
2054 { split.
2055   - apply chain_firstn. exact Hc2.
2056   - split.
2057     + apply hd_error_firstn; [exact Hh2 | lia].
2058     + pose proof (last_firstn_nth c2 k a u Hnth) as Hlf.
2059       rewrite Hsa. exact Hlf. }
2059 assert (Hlenpref : length (firstn (S k) c2) = S k).
2060 { apply length_firstn_le. lia. }
2061 apply (chain_pos_length_A_lt (firstn (S k) c2) u s Hprefix).
2062 rewrite Hlenpref. lia.
2063
2064 - (* equal length: s = u *)
2065 right. left.
2066 destruct H1 as [Hc1 [Hh1 Hl1]].
2067 destruct H2 as [Hc2 [Hh2 Hl2]].
2068 assert (Heqc : c1 = c2).
2069 { apply (persistent_chain_unique_to Hpers c1 t s c2 u).
2070   - split; [exact Hc1 | split; [exact Hh1 | exact Hl1]].
2071   - split; [exact Hc2 | split; [exact Hh2 | exact Hl2]].
2072   - exact Heq. }
2073 rewrite Heqc in Hh1. rewrite Hh1 in Hh2. injection Hh2 as Hh2.
exact Hh2.

```

```

2074 - (* c2 shorter: s <A u, mirror of the first case *)
2075 left.
2076 destruct H1 as [Hc1 [Hh1 Hl1]].
2077 destruct H2 as [Hc2 [Hh2 Hl2]].
2078 remember (length c1 - length c2) as k eqn:Hkeq.
2079 assert (Hc2ne : 1 <= length c2).
2080 { destruct c2 as [| ? ?]; simpl.
2081   - discriminate Hh2.
2082   - lia. }
2083 assert (Hk1 : 0 < k) by lia.
2084 assert (Hklt : k < length c1) by lia.
2085 destruct (nth_error_exists c1 k Hklt) as [a Hnth].
2086 pose proof (hd_error_skipn c1 k a Hnth) as Hheadskip.
2087 pose proof (chain_skipn c1 k Hc1) as Hchainskip.
2088 pose proof (last_skipn c1 k a Hklt) as Hlastskip.
2089 assert (Hlast_a_s : last c1 a = last c1 s).
2090 { apply last_default_irrelevant. destruct c1; [discriminate Hh1 |
2091   discriminate]. }
2092 assert (Hlastskip' : last (skipn k c1) a = t).
2093 { rewrite Hlastskip, Hlast_a_s. exact Hl1. }
2094 assert (Hsuffix : chain_from_to a t (skipn k c1)).
2095 { split; [exact Hchainskip | split; [exact Hheadskip | exact
2096   Hlastskip']]]. }
2097 assert (Hlensuf : length (skipn k c1) = length c2).
2098 { rewrite (length_skipn c1 k ltac:(lia)). lia. }
2099 assert (Heqc : c2 = skipn k c1).
2100 { apply (persistent_chain_unique_to Hpers c2 t u (skipn k c1) a).
2101   - split; [exact Hc2 | split; [exact Hh2 | exact Hl2]].
2102   - exact Hsuffix.
2103   - symmetry. exact Hlensuf. }
2104 assert (Hua : u = a).
2105 { assert (Hh2' : hd_error c2 = Some u) by exact Hh2.
2106   rewrite Heqc in Hh2'. rewrite Hheadskip in Hh2'. injection Hh2'
2107   as Hh2'. symmetry. exact Hh2'. }
2108 assert (Hprefix : chain_from_to s u (firstn (S k) c1)).
2109 { split.
2110   - apply chain_firstn. exact Hc1.
2111   - split.
2112     + apply hd_error_firstn; [exact Hh1 | lia].
2113     + pose proof (last_firstn_nth c1 k a s Hnth) as Hlf.
2114       rewrite Hua. exact Hlf. }
2115 assert (Hlenpref : length (firstn (S k) c1) = S k).
2116 { apply length_firstn_le. lia. }
2117 apply (chain_pos_length_A_lt (firstn (S k) c1) s u Hprefix).
2118 rewrite Hlenpref. lia.
2119 Qed.
2120 Lemma chain_same_start_relates :
2121   persistent ->

```



```

2121   forall s t u c1 c2,
2122     chain_from_to s t c1 -> chain_from_to s u c2 ->
2123     t <A u \ / t = u \ / u <A t.
2124 Proof.
2125   intros Hpers s t u c1 c2 H1 H2.
2126   assert (Htri : length c1 < length c2 \ / length c1 = length c2 \ /
length c2 < length c1) by lia.
2127   destruct Htri as [Hlt | [Heq | Hgt]].
2128
2129   - (* c1 shorter: t <A u *)
2130     left.
2131     destruct H1 as [Hc1 [Hh1 Hl1]].
2132     destruct H2 as [Hc2 [Hh2 Hl2]].
2133     assert (Hclne : 1 <= length c1).
2134     { destruct c1 as [| ? ?]; simpl.
2135       - discriminate Hh1.
2136       - lia. }
2137     remember (length c1 - 1) as j eqn:Hjeq.
2138     assert (Hjlt : j < length c2) by lia.
2139     destruct (nth_error_exists c2 j Hjlt) as [a Hnth].
2140     assert (Hlenpref : length (firstn (length c1) c2) = length c1).
2141     { apply length_firstn_le. lia. }
2142     assert (Hchainpref : chain (firstn (length c1) c2)).
2143     { apply chain_firstn. exact Hc2. }
2144     assert (Hheadpref : hd_error (firstn (length c1) c2) = Some s).
2145     { apply hd_error_firstn; [exact Hh2 | lia]. }
2146     assert (Heqc : c1 = firstn (length c1) c2).
2147     { apply (persistent_chain_unique_from Hpers c1 (firstn (length c1)
c2) s).
2148       - exact Hc1.
2149       - exact Hchainpref.
2150       - exact Hh1.
2151       - exact Hheadpref.
2152       - symmetry. exact Hlenpref. }
2153     assert (Hat : a = t).
2154     { pose proof (last_firstn_nth c2 j a s Hnth) as Hlf.
2155       assert (HSj : S j = length c1) by lia.
2156       rewrite HSj in Hlf.
2157       rewrite <- Heqc in Hlf.
2158       rewrite Hl1 in Hlf.
2159       symmetry. exact Hlf. }
2160     pose proof (hd_error_skipn c2 j a Hnth) as Hheadskip.
2161     pose proof (chain_skipn c2 j Hc2) as Hchainskip.
2162     pose proof (last_skipn c2 j a Hjlt) as Hlastskip.
2163     assert (Hlast_a_s : last c2 a = last c2 s).
2164     { apply last_default_irrelevant. destruct c2; [discriminate Hh2 |
discriminate]. }
2165     assert (Hlastskip' : last (skipn j c2) a = u).
2166     { rewrite Hlastskip, Hlast_a_s. exact Hl2. }
2167     assert (Hsuffix : chain_from_to t u (skipn j c2)).

```

```

2168 { rewrite <- Hat.
2169   split; [exact Hchainskip | split; [exact Hheadskip | exact
    Hlastskip']]. }
2170 assert (Hlensuf : length (skipn j c2) = length c2 - j).
2171 { apply length_skipn. lia. }
2172 apply (chain_pos_length_A_lt (skipn j c2) t u Hsuffix).
2173 rewrite Hlensuf. lia.
2174
2175 - (* equal length: t = u *)
2176 right. left.
2177 destruct H1 as [Hc1 [Hh1 Hl1]].
2178 destruct H2 as [Hc2 [Hh2 Hl2]].
2179 assert (Heqc : c1 = c2).
2180 { apply (persistent_chain_unique_from Hpers c1 c2 s).
2181   - exact Hc1.
2182   - exact Hc2.
2183   - exact Hh1.
2184   - exact Hh2.
2185   - exact Heq. }
2186 rewrite Heqc in Hl1.
2187 rewrite Hl1 in Hl2.
2188 exact Hl2.
2189
2190 - (* c2 shorter: u <A t, mirror of first case *)
2191 right. right.
2192 destruct H1 as [Hc1 [Hh1 Hl1]].
2193 destruct H2 as [Hc2 [Hh2 Hl2]].
2194 assert (Hc2ne : 1 <= length c2).
2195 { destruct c2 as [| ? ?]; simpl.
2196   - discriminate Hh2.
2197   - lia. }
2198 remember (length c2 - 1) as j eqn:Hjeq.
2199 assert (Hjlt : j < length c1) by lia.
2200 destruct (nth_error_exists c1 j Hjlt) as [a Hnth].
2201 assert (Hlenpref : length (firstn (length c2) c1) = length c2).
2202 { apply length_firstn_le. lia. }
2203 assert (Hchainpref : chain (firstn (length c2) c1)).
2204 { apply chain_firstn. exact Hc1. }
2205 assert (Hheadpref : hd_error (firstn (length c2) c1) = Some s).
2206 { apply hd_error_firstn; [exact Hh1 | lia]. }
2207 assert (Heqc : c2 = firstn (length c2) c1).
2208 { apply (persistent_chain_unique_from Hpers c2 (firstn (length c2)
    c1) s).
2209   - exact Hc2.
2210   - exact Hchainpref.
2211   - exact Hh2.
2212   - exact Hheadpref.
2213   - symmetry. exact Hlenpref. }
2214 assert (Hau : a = u).
2215 { pose proof (last_firstn_nth c1 j a s Hnth) as Hlf.

```

```

2216   assert (HSj : S j = length c2) by lia.
2217   rewrite HSj in Hlf.
2218   rewrite <- Heqc in Hlf.
2219   rewrite Hl2 in Hlf.
2220   symmetry. exact Hlf. }
2221   pose proof (hd_error_skipn c1 j a Hnth) as Hheadskip.
2222   pose proof (chain_skipn c1 j Hc1) as Hchainskip.
2223   pose proof (last_skipn c1 j a Hjlt) as Hlastskip.
2224   assert (Hlast_a_s : last c1 a = last c1 s).
2225   { apply last_default_irrelevant. destruct c1; [discriminate Hh1 |
discriminate]. }
2226   assert (Hlastskip' : last (skipn j c1) a = t).
2227   { rewrite Hlastskip, Hlast_a_s. exact Hl1. }
2228   assert (Hsuffix : chain_from_to u t (skipn j c1)).
2229   { rewrite <- Hau.
2230     split; [exact Hchainskip | split; [exact Hheadskip | exact
Hlastskip']]]. }
2231   assert (Hlensuf : length (skipn j c1) = length c1 - j).
2232   { apply length_skipn. lia. }
2233   apply (chain_pos_length_A_lt (skipn j c1) u t Hsuffix).
2234   rewrite Hlensuf. lia.
2235 Qed.
2236
2237 Lemma A_eq_lt_case :
2238   persistent -> bounded ->
2239   forall s t, s ≈A t -> s <A t ∨ s = t ∨ t <A s.
2240 Proof.
2241   intros Hpers Hbnd s t Heq.
2242   induction Heq as
2243     [ s
2244     | s t Hst
2245     | s t Heq IH
2246     | s t u Heq1 IH1 Heq2 IH2 ].
2247
2248   - right. left. reflexivity.
2249
2250   - left. apply A_lt_step. exact Hst.
2251
2252   - destruct IH as [IH | [IH | IH]].
2253     + right. right. exact IH.
2254     + right. left. symmetry. exact IH.
2255     + left. exact IH.
2256
2257   - destruct IH1 as [Hst | [Hst | Hst]]; destruct IH2 as [Htu | [Htu |
Htu]].
2258     + left. apply A_lt_trans' with t; auto.
2259     + subst u. left. exact Hst.
2260     + destruct (A_le_iff_chain s t) as [Hfwd1 _].
2261       destruct (A_le_iff_chain u t) as [Hfwd2 _].

```

```

2262   assert (Hc1 : exists c, chain_from_to s t c) by (apply Hfwd1;
2263   apply A_le_of_lt; exact Hst).
2264   assert (Hc2 : exists c, chain_from_to u t c) by (apply Hfwd2;
2265   apply A_le_of_lt; exact Htu).
2266   destruct Hc1 as [c1 Hc1]. destruct Hc2 as [c2 Hc2].
2267   apply (chain_same_end_relates Hpers s u t c1 c2 Hc1 Hc2).
2268   + subst t. left. exact Htu.
2269   + subst t. subst u. right. left. reflexivity.
2270   + subst t. right. right. exact Htu.
2271   + destruct (A_le_iff_chain t s) as [Hfwd1 _].
2272   destruct (A_le_iff_chain t u) as [Hfwd2 _].
2273   assert (Hc1 : exists c, chain_from_to t s c) by (apply Hfwd1;
2274   apply A_le_of_lt; exact Hst).
2275   assert (Hc2 : exists c, chain_from_to t u c) by (apply Hfwd2;
2276   apply A_le_of_lt; exact Htu).
2277   destruct Hc1 as [c1 Hc1]. destruct Hc2 as [c2 Hc2].
2278   apply (chain_same_start_relates Hpers t s u c1 c2 Hc1 Hc2).
2279   + subst u. right. right. exact Hst.
2280   + right. right. apply A_lt_trans' with t; auto.
2281 Qed.
2282
2283 Lemma A_lt_total :
2284   persistent -> bounded -> atomic ->
2285   forall s t, s <A t \/ s = t \/ t <A s.
2286 Proof.
2287   intros Hpers Hbnd Hato s t.
2288   apply (A_eq_lt_case Hpers Hbnd s t (Hato s t)).
2289 Qed.
2290
2291 Axiom exists_top :
2292   bounded ->
2293   forall s0 : Sort, exists top, s0 ≤A top /\ ~ exists t, top <A t.
2294
2295 Axiom exists_bottom :
2296   bounded ->
2297   forall s0 : Sort, exists bottom, bottom ≤A s0 /\ ~ exists t, t <A
2298   bottom.
2299
2300 Lemma A_le_split : forall s t, s ≤A t -> s = t \/ s <A t.
2301 Proof.
2302   intros s t H. induction H as [s | s t u Hst H IH].
2303   - left. reflexivity.
2304   - destruct IH as [IH | IH].
2305     + subst. right. apply A_lt_step. exact Hst.
2306     + right. apply A_lt_trans' with t; [apply A_lt_step; exact Hst |
2307     exact IH].
2308 Qed.
2309
2310 Lemma A_le_lt_antisym :

```

```

2305     bounded ->
2306     forall s t, s ≤A t -> t <A s -> False.
2307 Proof.
2308   intros Hbnd s t Hle Hlt.
2309   destruct (A_le_split s t Hle) as [Heq | Hlt2].
2310   - subst. exact (A_lt_irrefl Hbnd t Hlt).
2311   - exact (A_lt_irrefl Hbnd s (A_lt_trans' s t s Hlt2 Hlt)).
2312 Qed.
2313
2314 Lemma top_unique :
2315   persistent -> bounded -> atomic ->
2316   forall top1 top2,
2317   (forall s, s ≤A top1) ->
2318   (forall s, s ≤A top2) ->
2319   top1 = top2.
2320 Proof.
2321   intros Hpers Hbnd Hato top1 top2 H1 H2.
2322   destruct (A_lt_total Hpers Hbnd Hato top1 top2) as [Hlt | [Heq |
Hgt]].
2323   - exfalso. apply (A_le_lt_antisym Hbnd top2 top1 (H1 top2) Hlt).
2324   - exact Heq.
2325   - exfalso. apply (A_le_lt_antisym Hbnd top1 top2 (H2 top1) Hgt).
2326 Qed.
2327
2328 Lemma bottom_unique :
2329   persistent -> bounded -> atomic ->
2330   forall bottom1 bottom2,
2331   (forall s, bottom1 ≤A s) ->
2332   (forall s, bottom2 ≤A s) ->
2333   bottom1 = bottom2.
2334 Proof.
2335   intros Hpers Hbnd Hato bottom1 bottom2 H1 H2.
2336   destruct (A_lt_total Hpers Hbnd Hato bottom1 bottom2) as [Hlt | [Heq
| Hgt]].
2337   - exfalso. apply (A_le_lt_antisym Hbnd bottom2 bottom1 (H2 bottom1)
Hlt).
2338   - exact Heq.
2339   - exfalso. apply (A_le_lt_antisym Hbnd bottom1 bottom2 (H1 bottom2)
Hgt).
2340 Qed.
2341
2342 Lemma nth_error_lt_length : forall (l : list Sort) i a,
2343   nth_error l i = Some a -> i < length l.
2344 Proof.
2345   induction l as [| x l IH]; intros i a H.
2346   - destruct i; discriminate.
2347   - destruct i as [| i].
2348     + simpl. lia.
2349     + simpl in H. simpl. specialize (IH i a H). lia.

```

```

2350 Qed.
2351
2352 Lemma chain_nth_A : forall c i a b,
2353   chain c -> nth_error c i = Some a -> nth_error c (S i) = Some b -> A
2354   a b.
2355 Proof.
2356   induction c as [| x c IH]; intros i a b Hc Ha Hb.
2357   - destruct i as [| i]; simpl in Ha; discriminate Ha.
2358   - destruct i as [| i].
2359     + simpl in Ha. injection Ha as Ha. subst a.
2360       simpl in Hb.
2361       destruct c as [| y c'].
2362       * discriminate Hb.
2363       * simpl in Hc. destruct Hc as [HAxy _].
2364         injection Hb as Hb. subst b.
2365         exact HAxy.
2366     + simpl in Ha, Hb.
2367       destruct c as [| y c'].
2368       * destruct i as [| i']; simpl in Ha; discriminate Ha.
2369       * simpl in Hc. destruct Hc as [_ Hc'].
2370         apply (IH i a b Hc' Ha Hb).
2371 Qed.
2372
2373 Lemma last_nth_error : forall (l : list Sort) d,
2374   l <> [] -> nth_error l (length l - 1) = Some (last l d).
2375 Proof.
2376   induction l as [| x l IH]; intros d Hne.
2377   - contradiction.
2378   - destruct l as [| y l'].
2379     + reflexivity.
2380     + simpl. specialize (IH d ltac:(discriminate)). simpl in IH.
2381       rewrite <- IH. f_equal. lia.
2382 Qed.
2383
2384 Lemma chain_lt_A : forall c i j a b,
2385   chain c -> i < j -> nth_error c i = Some a -> nth_error c j = Some b
2386   -> a <A b.
2387 Proof.
2388   intros c i j a b Hc Hij.
2389   revert a b.
2390   induction j as [j IH] using (well_founded_induction Wf_nat.lt_wf).
2391   intros a b Ha Hb.
2392   destruct j as [| j'].
2393   - lia.
2394   - destruct (Nat.eq_dec i j') as [Heq | Hneq].
2395     + subst i. apply A_lt_step.
2396       apply (chain_nth_A c j' a b Hc Ha Hb).
2397     + assert (Hij' : i < j') by lia.
2398       destruct (nth_error_exists c j' ltac:(apply (nth_error_lt_length
2399         c (S j') b) in Hb; lia)) as [m Hm].

```

```

2396     apply A_lt_trans' with m.
2397     * apply (IH j' ltac:(lia) ltac:(lia) a m Ha Hm).
2398     * apply A_lt_step. apply (chain_nth_A c j' m b Hc Hm Hb).
2399 Qed.
2400
2401 Lemma chain_pos_unique :
2402   persistent -> bounded ->
2403   forall c i j s, chain c -> nth_error c i = Some s -> nth_error c j =
2404     Some s -> i = j.
2405 Proof.
2406   intros Hpers Hbnd c i j s Hc Hi Hj.
2407   destruct (Nat.lt_trichotomy i j) as [Hlt | [Heq | Hgt]]; auto.
2408   - exfalso. apply (A_lt_irrefl Hbnd s).
2409     apply (chain_lt_A c i j s s Hc Hlt Hi Hj).
2410   - exfalso. apply (A_lt_irrefl Hbnd s).
2411     apply (chain_lt_A c j i s s Hc Hgt Hj Hi).
2412 Qed.
2413
2414 Lemma persistent_chain_prefix :
2415   persistent ->
2416   forall c1 c2 s,
2417     chain c1 -> chain c2 ->
2418     hd_error c1 = Some s -> hd_error c2 = Some s ->
2419     length c1 <= length c2 ->
2420     c1 = firstn (length c1) c2.
2421 Proof.
2422   intros Hpers c1 c2 s Hc1 Hc2 Hh1 Hh2 Hlen.
2423   apply (persistent_chain_unique_from Hpers c1 (firstn (length c1) c2)
2424     s).
2425   - exact Hc1.
2426   - apply chain_firstn. exact Hc2.
2427   - exact Hh1.
2428   - apply hd_error_firstn; [exact Hh2 |].
2429     destruct c1; [discriminate Hh1 | simpl; lia].
2430   - symmetry. apply length_firstn_le. exact Hlen.
2431 Qed.
2432
2433 Lemma app_nonempty_r : forall (l1 l2 : list Sort), l2 <> [] -> l1 ++
2434   l2 <> [].
2435 Proof.
2436   intros l1 l2 Hne Heq.
2437   apply Hne. destruct l1; simpl in Heq; auto; discriminate.
2438 Qed.
2439
2440 Lemma last_app_r : forall (l1 l2 : list Sort) d, l2 <> [] -> last (l1
2441   ++ l2) d = last l2 d.
2442 Proof.
2443   induction l1 as [| x l1 IH]; intros l2 d Hne.
2444   - reflexivity.

```

```

2441 - simpl. destruct (l1 ++ l2) as [| y ys] eqn:E.
2442 + exfalso. apply (app_nonempty_r l1 l2 Hne). exact E.
2443 + rewrite <- E. apply IH. exact Hne.
2444 Qed.
2445
2446 Lemma hd_error_app_l : forall (l1 l2 : list Sort) a,
2447   hd_error l1 = Some a -> hd_error (l1 ++ l2) = Some a.
2448 Proof.
2449   intros l1 l2 a H. destruct l1; simpl in H; [discriminate | simpl;
2450     exact H].
2451 Qed.
2452
2453 Lemma nth_error_app_l : forall (l1 l2 : list Sort) n,
2454   n < length l1 -> nth_error (l1 ++ l2) n = nth_error l1 n.
2455 Proof.
2456   induction l1 as [| x l1 IH]; intros l2 n Hn.
2457   - simpl in Hn. lia.
2458   - destruct n as [| n'].
2459     + reflexivity.
2460     + simpl. apply IH. simpl in Hn. lia.
2461 Qed.
2462
2463 Lemma nth_error_firstn : forall (l : list Sort) k i,
2464   i < k -> nth_error (firstn k l) i = nth_error l i.
2465 Proof.
2466   induction l as [| x l IH]; intros k i Hik.
2467   - destruct k; simpl; destruct i; reflexivity.
2468   - destruct k as [| k'].
2469     + lia.
2470     + destruct i as [| i'].
2471       * reflexivity.
2472       * simpl. apply IH. lia.
2473 Qed.
2474
2475 Lemma chain_app_general : forall (l1 l2 : list Sort) (m : Sort),
2476   chain (l1 ++ [m]) ->
2477   chain (m :: l2) ->
2478   chain (l1 ++ m :: l2).
2479 Proof.
2480   induction l1 as [| x l1 IH]; intros l2 m H1 H2.
2481   - simpl. exact H2.
2482   - simpl in H1 |- *.
2483     destruct l1 as [| y l1'].
2484     + simpl in H1. destruct H1 as [HAXm _].
2485       simpl. split; [exact HAXm | exact H2].
2486     + simpl in H1. destruct H1 as [HAXy Hrest].
2487       simpl. split; [exact HAXy | apply IH; [exact Hrest | exact H2]].
2488 Qed.
2489
2490 Lemma chain_from_to_app :

```



```

2490 forall c1 c2 b s t,
2491   chain_from_to b s c1 ->
2492   chain_from_to s t c2 ->
2493   chain_from_to b t (c1 ++ tl c2).
2494 Proof.
2495 intros c1 c2 b s t [Hc1 [Hh1 Hl1]] [Hc2 [Hh2 Hl2]].
2496 destruct c2 as [| x2 c2'].
2497 - discriminate Hh2.
2498 - simpl in Hh2. injection Hh2 as Hh2. subst x2.
2499   simpl.
2500   assert (Hc1ne : c1 <> []).
2501   { destruct c1; [discriminate Hh1 | discriminate]. }
2502   destruct (exists_last Hc1ne) as [l1 [e Hcleq]].
2503   assert (He : e = s).
2504   { assert (Hl : last c1 b = e).
2505     { rewrite Hcleq. rewrite last_last. reflexivity. }
2506     rewrite Hl1 in Hl. symmetry. exact Hl. }
2507   subst e.
2508   destruct c2' as [| x2' c2''].
2509   + assert (Ht : t = s).
2510     { simpl in Hl2. symmetry. exact Hl2. }
2511     subst t.
2512     rewrite app_nil_r.
2513     split; [exact Hc1 | split; [exact Hh1 | exact Hl1]].
2514   + set (c2' := x2' :: c2'') in *.
2515     assert (Hc2'ne : c2' <> []) by (subst c2'; discriminate).
2516     split.
2517     * rewrite Hcleq.
2518       rewrite <- app_assoc. simpl.
2519       apply (chain_app_general l1 c2' s).
2520       -- rewrite <- Hcleq. exact Hc1.
2521       -- exact Hc2.
2522     * split.
2523       -- apply (hd_error_app_l c1 c2' b Hh1).
2524       -- rewrite (last_app_r c1 c2' b Hc2'ne).
2525       assert (Hl2' : last c2' s = t).
2526       { simpl in Hl2. exact Hl2. }
2527       rewrite <- Hl2'.
2528       apply last_default_irrelevant.
2529       exact Hc2'ne.
2530 Qed.
2531
2532 Lemma chain_length_eq_of_same_ends :
2533   persistent -> bounded ->
2534   forall e c b t,
2535     chain_from_to b t e -> chain_from_to b t c ->
2536     length e = length c.
2537 Proof.
2538   intros Hpers Hbnd e c b t He Hc.
2539   destruct He as [Hce [Hhe Hle]].

```

```

2540 destruct Hc as [Hcc [Hhc Hlc]].
2541 assert (Hene : e <> []) by (destruct e; [discriminate Hhe |
discriminate]).
2542 assert (Hcne : c <> []) by (destruct c; [discriminate Hhc |
discriminate]).
2543 assert (Hene1 : 1 <= length e) by (destruct e; [contradiction Hene;
reflexivity | simpl; lia]).
2544 assert (Hcne1 : 1 <= length c) by (destruct c; [contradiction Hcne;
reflexivity | simpl; lia]).
2545 destruct (Nat.lt_trichotomy (length e) (length c)) as [Hlt | [Heq |
Hgt]]; auto.
2546 - exfalso.
2547   assert (Hpre : e = firstn (length e) c).
2548   { apply (persistent_chain_prefix Hpers e c b); [exact Hce | exact
Hcc | exact Hhe | exact Hhc | lia]. }
2549   assert (Hte : nth_error e (length e - 1) = Some t).
2550   { rewrite (last_nth_error e b Hene). rewrite Hle. reflexivity. }
2551   assert (Htc : nth_error c (length c - 1) = Some t).
2552   { rewrite (last_nth_error c b Hcne). rewrite Hlc. reflexivity. }
2553   assert (Hte'' : nth_error (firstn (length e) c) (length e - 1) =
Some t).
2554   { rewrite <- Hpre. exact Hte. }
2555   assert (Hte' : nth_error c (length e - 1) = Some t).
2556   { rewrite (nth_error_firstn c (length e) (length e - 1)) in Hte''.
- exact Hte''.
- lia. }
2559   assert (Heq2 : length e - 1 = length c - 1).
2560   { apply (chain_pos_unique Hpers Hbnd c (length e - 1) (length c -
1) t Hcc Hte' Htc). }
2561   lia.
2562 - exfalso.
2563   assert (Hpre : c = firstn (length c) e).
2564   { apply (persistent_chain_prefix Hpers c e b); [exact Hcc | exact
Hce | exact Hhc | exact Hhe | lia]. }
2565   assert (Hte : nth_error e (length e - 1) = Some t).
2566   { rewrite (last_nth_error e b Hene). rewrite Hle. reflexivity. }
2567   assert (Htc : nth_error c (length c - 1) = Some t).
2568   { rewrite (last_nth_error c b Hcne). rewrite Hlc. reflexivity. }
2569   assert (Htc'' : nth_error (firstn (length c) e) (length c - 1) =
Some t).
2570   { rewrite <- Hpre. exact Htc. }
2571   assert (Htc' : nth_error e (length c - 1) = Some t).
2572   { rewrite (nth_error_firstn e (length c) (length c - 1)) in Htc''.
- exact Htc''.
- lia. }
2575   assert (Heq2 : length c - 1 = length e - 1).
2576   { apply (chain_pos_unique Hpers Hbnd e (length c - 1) (length e -
1) t Hce Htc' Hte). }
2577   lia.

```

```

2578 Qed.
2579
2580 Lemma tiered_iff_persistent_bounded_atomic :
2581   (exists n, tiered n) <-> persistent /\ bounded /\ atomic.
2582 Proof.
2583   split.
2584
2585   - intros [n [index_of [index_r [index_i [index_t [HA HR]]]]]].
2586     split.
2587       + repeat split; auto.
2588         * intros s t u HA1 HA2.
2589           apply HA in HA1 as [_ HA1].
2590           apply HA in HA2 as [_ HA2].
2591           apply index_i. rewrite HA1, HA2. reflexivity.
2592         * intros s t u v HR1 HR2.
2593           apply HR in HR1. apply HR in HR2. rewrite <- HR1. exact HR2.
2594         * intros s t u HA1 HA2.
2595           apply HA in HA1 as [_ HA1].
2596           apply HA in HA2 as [_ HA2].
2597           apply index_i. apply eq_add_S. rewrite <- HA1. exact HA2.
2598       + split.
2599
2600         * unfold bounded.
2601         assert (A_lt_index_1 : forall s t, s <A t -> index_of s <
2602           index_of t).
2603         intros s t H. induction H as [s t Hst | s t u Hst _ IH].
2604           apply HA in Hst as [_ Heq]. lia.
2605           apply HA in Hst as [_ Heq]. lia.
2606         assert (A_lt_index_2 : forall s t, t <A s -> n - index_of s < n
2607           - index_of t).
2608         intros t s H.
2609         induction H as [s t Hst | s t u Hst _ IH].
2610           apply HA in Hst as [_ Heq]. destruct (index_r s) as [_ Hsn],
2611             (index_r t) as [_ Htn]. lia.
2612           apply HA in Hst as [_ Heq]. destruct (index_r s) as [_ Hsn],
2613             (index_r t) as [_ Htn]. lia.
2614         split.
2615         ** unfold ascending_chain_condition. intros s.
2616         apply (wf_by_measure Sort (fun u v => u <A v) index_of
2617           A_lt_index_1).
2618         ** unfold descending_chain_condition. intros s.
2619         apply (wf_by_measure Sort (fun u v => v <A u) (fun s => n -
2620           index_of s) A_lt_index_2).
2621
2622         * assert (chain_eq : forall k i, i + k < n + 1 -> 0 < i ->
2623           forall x y, index_of x = i -> index_of y = i + k -> x
2624             ≈A y).
2625         { induction k as [| k IH]; intros i Hbound Hipos x y Hx Hy.

```

```

2619 - assert (Hy0 : index_of y = i) by lia.
2620   assert (Heq : index_of x = index_of y) by lia.
2621   apply index_i in Heq. subst y. apply A_eq_refl.
2622 - assert (Hrange : 0 < i + k /\ i + k < n + 1) by lia.
2623   destruct (index_t (i + k) Hrange) as [z Hz].
2624   assert (Hxz : x ≈A z).
2625   { apply IH with i; auto; lia. }
2626   assert (Hzy : z ≈A y).
2627   { apply A_eq_step. apply HA. split.
2628     - rewrite Hz. lia.
2629     - rewrite Hz, Hy. lia. }
2630   apply A_eq_trans with z; auto. }
2631
2632 intros s s'.
2633 pose proof (index_r s) as [Hs1 Hs2].
2634 pose proof (index_r s') as [Hs1' Hs2'].
2635 assert (Hcase : index_of s <= index_of s' \/ index_of s' <=
index_of s) by lia.
2636 destruct Hcase as [Hle | Hge].
2637   ** assert (Hk : index_of s' = index_of s + (index_of s' -
index_of s)) by lia.
2638   apply (chain_eq (index_of s' - index_of s) (index_of s)); lia
|| auto.
2639
2640   ** apply A_eq_sym.
2641   assert (Hk : index_of s = index_of s' + (index_of s - index_of
s')) by lia.
2642   apply (chain_eq (index_of s - index_of s') (index_of s')); lia
|| auto.
2643
2644 - intros [Hpers [Hbnd Hato]].
2645 destruct (classic (exists s : Sort, True)) as [[s0 _] | Hempty].
2646 + destruct (classic (exists s1 s2 : Sort, s1 <> s2)) as [[s1 [s2
Hne]] | Hone].
2647   * (* |S| >= 2 *)
2648   destruct (exists_top Hbnd s1) as [top [_ Htopmax]].
2649   destruct (exists_bottom Hbnd s1) as [bot [_ Hbotmin]].
2650   assert (Htop : forall s, s ≤A top).
2651   { intros s. destruct (A_lt_total Hpers Hbnd Hato s top) as
[H][H|H]].
2652     - apply A_le_of_lt; exact H.
2653     - subst; apply A_le_refl.
2654     - exfalso; apply Htopmax; exists s; exact H. }
2655   assert (Hbot : forall s, bot ≤A s).
2656   { intros s. destruct (A_lt_total Hpers Hbnd Hato s bot) as
[H][H|H]].
2657     - exfalso; apply Hbotmin; exists s; exact H.
2658     - subst; apply A_le_refl.
2659     - apply A_le_of_lt; exact H. }

```

```

2660 destruct (A_le_iff_chain bot top) as [Hfwd _].
2661 destruct (Hfwd (Hbot top)) as [c Hc].
2662 exists (length c).
2663
2664 assert (Hpos : forall s : Sort, exists! i, nth_error c i = Some
s).
2665 { intros s.
2666   pose proof (Hbot s) as Hbs.
2667   pose proof (Htop s) as Hst.
2668   destruct (proj1 (A_le_iff_chain bot s) Hbs) as [c1 Hc1].
2669   destruct (proj1 (A_le_iff_chain s top) Hst) as [c2 Hc2].
2670   assert (He : chain_from_to bot top (c1 ++ tl c2)).
2671   { apply (chain_from_to_app c1 c2 bot s top Hc1 Hc2). }
2672   assert (Hlen : length (c1 ++ tl c2) = length c).
2673   { apply (chain_length_eq_of_same_ends Hpers Hbnd (c1 ++ tl c2)
c bot top He Hc). }
2674   assert (Heqec : c1 ++ tl c2 = c).
2675   { apply (persistent_chain_unique_from Hpers (c1 ++ tl c2) c
bot).
2676     - destruct He as [Hce _]. exact Hce.
2677     - destruct Hc as [Hcc _]. exact Hcc.
2678     - destruct He as [_ [Hhe _]]. exact Hhe.
2679     - destruct Hc as [_ [Hhc _]]. exact Hhc.
2680     - exact Hlen. }
2681   destruct Hc1 as [Hcc1 [Hhc1 Hlc1]].
2682   assert (Hclne : c1 <> []) by (destruct c1; [discriminate Hhc1
| discriminate]).
2683   assert (Hpos_s : nth_error c1 (length c1 - 1) = Some s).
2684   { rewrite (last_nth_error c1 bot Hclne). rewrite Hlc1.
reflexivity. }
2685   assert (Hclge1 : 1 <= length c1)
2686     by (destruct c1; [contradiction Hclne; reflexivity | simpl;
lia]).
2687   assert (Hpos_e : nth_error (c1 ++ tl c2) (length c1 - 1) =
Some s).
2688   { rewrite (nth_error_app_l c1 (tl c2) (length c1 - 1)).
2689     - exact Hpos_s.
2690     - lia. }
2691   rewrite Heqec in Hpos_e.
2692   exists (length c1 - 1).
2693   split.
2694   - exact Hpos_e.
2695   - intros i' Hi'. symmetry.
2696     apply (chain_pos_unique Hpers Hbnd c i' (length c1 - 1) s).
2697     + destruct Hc as [Hcc _]; exact Hcc.
2698     + exact Hi'.
2699     + exact Hpos_e. }
2700   assert (Hchoice : forall s, {i | nth_error c i = Some s}).
2701   { intros s. apply constructive_indefinite_description.

```

```

2702   destruct (Hpos s) as [i [Hi _]]. exists i. exact Hi. }
2703   set (index_of := fun s => S (proj1_sig (Hchoice s))).
2704   exists index_of.
2705
2706   split.
2707
2708   (* range *)
2709   intros x. unfold index_of.
2710   pose proof (proj2_sig (Hchoice x)) as Hnth.
2711   pose proof (nth_error_lt_length c (proj1_sig (Hchoice x)) x
Hnth) as Hlt.
2712   lia.
2713
2714   split.
2715
2716   (* injectivity *)
2717   intros x y Heq. unfold index_of in Heq.
2718   assert (Hixy : proj1_sig (Hchoice x) = proj1_sig (Hchoice y)) by
lia.
2719   pose proof (proj2_sig (Hchoice x)) as Hx.
2720   pose proof (proj2_sig (Hchoice y)) as Hy.
2721   rewrite Hixy in Hx.
2722   assert (Hsome : Some x = Some y) by (rewrite <- Hx; exact Hy).
2723   injection Hsome as Hsome. exact Hsome.
2724
2725   split.
2726
2727   (* surjectivity *)
2728   intros i [Hi1 Hi2].
2729   assert (Hilt : i - 1 < length c) by lia.
2730   destruct (nth_error_exists c (i - 1) Hilt) as [x Hx].
2731   exists x.
2732   destruct (Hpos x) as [i0 [Hi0 Huniq]].
2733   assert (Heq0 : i0 = i - 1) by (apply Huniq; exact Hx).
2734   assert (Heqchoice : proj1_sig (Hchoice x) = i0).
2735   { symmetry. apply Huniq. exact (proj2_sig (Hchoice x)). }
2736   unfold index_of. rewrite Heqchoice. lia.
2737
2738   split.
2739
2740   (* A clause *)
2741   intros x y. split.
2742
2743   assert (Hcne : c <> []).
2744   { destruct Hc as [_ [Hh _]]. destruct c; [discriminate Hh |
discriminate]. }
2745
2746   (* -> *)
2747   intro HAx.
2748   assert (Hxlt : x <A y) by (apply A_lt_step; exact HAx).

```

```

2749 assert (Hxnetop : x <> top).
2750 { intro Heq. subst x. apply Htopmax. exists y. exact Hxlt. }
2751
2752 destruct (Hchoice x) as [ix Hix] eqn:Ex.
2753 assert (Hix_index : index_of x = S ix).
2754 { unfold index_of. simpl. rewrite Ex. reflexivity. }
2755 assert (Hixlt : ix < length c) by (apply (nth_error_lt_length c
ix x); exact Hix).
2756
2757 assert (Hixne : ix <> length c - 1).
2758 { intro Heq.
2759   apply Hxnetop.
2760   assert (Hlast : nth_error c (length c - 1) = Some top).
2761   { destruct Hc as [_ [_ Hl]].
2762     rewrite (last_nth_error c bot Hcne). rewrite Hl.
2763     reflexivity. }
2764   clear Ex.
2765   rewrite Heq in Hix.
2766   rewrite Hix in Hlast.
2767   injection Hlast as Hlast.
2768   exact Hlast. }
2769
2770 assert (Hsix : S ix < length c) by lia.
2771 destruct (nth_error_exists c (S ix) Hsix) as [z Hz].
2772 assert (Hxaz : A x z).
2773 { apply (chain_nth_A c ix x z);
2774   [ destruct Hc as [Hcc _]; exact Hcc | exact Hix | exact Hz ].
2775 }
2776 assert (Hzy : z = y).
2777 { destruct Hpers as [[Hfunc1 _] _]. apply (Hfunc1 x); [exact
Hxaz | exact Hxay]. }
2778 subst z.
2779
2780 destruct (Hchoice y) as [iy Hiy] eqn:Ey.
2781 assert (Hiy_index : index_of y = S iy).
2782 { unfold index_of. simpl. rewrite Ey. reflexivity. }
2783 assert (Hiyeq : iy = S ix).
2784 { destruct (Hpos y) as [i0 [Hi0 Huniq0]].
2785   assert (E1 : i0 = iy) by (apply Huniq0; exact Hiy).
2786   assert (E2 : i0 = S ix) by (apply Huniq0; exact Hz).
2787   lia. }
2788
2789 split; lia.
2790
2791 (* <- *)
2792 intros [Hlt Heqsucc].
2793 destruct (Hchoice x) as [ix Hix] eqn:Ex.
2794 assert (Hix_index : index_of x = S ix).
2795 { unfold index_of. simpl. rewrite Ex. reflexivity. }

```

```

2794   assert (Hsix : S ix < length c).
2795   { rewrite Hix_index in Hlt. exact Hlt. }
2796   destruct (nth_error_exists c (S ix) Hsix) as [z Hz].
2797   assert (Hxaz : A x z).
2798   { apply (chain_nth_A c ix x z);
2799     [destruct Hc as [Hcc _]; exact Hcc | exact Hix | exact Hz].
2800   }

2801   destruct (Hchoice y) as [iy Hiy] eqn:Ey.
2802   assert (Hiy_index : index_of y = S iy).
2803   { unfold index_of. simpl. rewrite Ey. reflexivity. }
2804   assert (Hiyeq : iy = S ix).
2805   { rewrite Hix_index, Hiy_index in Heqsucc. lia. }
2806   assert (Hzy : z = y).
2807   { assert (Hiy' : nth_error c (S ix) = Some y).
2808     { rewrite <- Hiyeq. exact Hiy. }
2809     rewrite Hz in Hiy'.
2810     injection Hiy' as Hiy'.
2811     exact Hiy'. }
2812   subst z.
2813   exact Hxaz.

2814
2815   (* R shape *)
2816   intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.
2817
2818
2819   * (* |S| = 1 *)
2820   exists 1, (fun _ => 1).
2821   split. lia.
2822   split. intros x y _. destruct (classic (x = y)) as [|Hxy];
2823     [auto|].
2824     exfalso. apply Hone. exists x, y. exact Hxy.
2825     split. intros i [Hi1 Hi2]. exists s0. lia.
2826     split. intros x y. split.
2827       ** intros Hxxy. exfalso. assert (Hxy : x = y).
2828       { destruct (classic (x = y)) as [|Hxy]; [auto|].
2829         exfalso. apply Hone. exists x, y. exact Hxy. }
2830       subst y. apply (A_lt_irrefl Hbnd x). apply A_lt_step. exact
2831         Hxxy.
2832       ** intros [Hlt _]. lia.
2833       ** intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.
2834
2835   + (* |S| = 0 *)
2836   exists 0, (fun _ => 0). split.
2837   intros x. exfalso. apply Hempty. exists x. apply I. split.
2838   intros x y _. exfalso. apply Hempty. exists x. apply I. split.
2839   intros i [Hi1 Hi2]. lia. split.
2840   intros x y. split.
2841   intro Hxxy. exfalso. apply Hempty. exists x. apply I.
2842   intros [Hlt _]. lia.
2843   intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.

```



```

2841 Qed.
2842
2843 Definition disjoint_union_of_tiered : Prop :=
2844   exists (n_of : Sort -> nat) (index_of : Sort -> nat),
2845     (forall s t, A s t -> s ≈A t) /\
2846     (forall s t u, R s t u -> s ≈A t) /\
2847     (forall s t, s ≈A t -> n_of s = n_of t) /\
2848     (forall s, 0 < index_of s /\ index_of s < n_of s + 1) /\
2849     (forall s t, s ≈A t -> index_of s = index_of t -> s = t) /\
2850     (forall s i, 0 < i -> i <= n_of s -> exists t, t ≈A s /\ index_of
2851       t = i) /\
2852     (forall s t, A s t <-> (s ≈A t /\ index_of s < n_of s /\ index_of
2853       t = S (index_of s))) /\
2854     (forall s t u, R s t u -> t = u).
2855
2856 Lemma A_lt_in_class :
2857   forall (HAclass : forall s t, A s t -> s ≈A t),
2858   forall u v, u <A v -> u ≈A v.
2859 Proof.
2860   intros HAclass u v Huv.
2861   induction Huv as [u v Huv | u v w Huv _ IH].
2862   - apply A_eq_step. exact Huv.
2863   - apply A_eq_trans with v.
2864     + apply A_eq_step. exact Huv.
2865     + exact IH.
2866 Qed.
2867
2868 Lemma A_lt_index_incr :
2869   forall (index_of : Sort -> nat) (n_of : Sort -> nat),
2870   forall (HA : forall s t, A s t <-> (s ≈A t /\ index_of s < n_of s /\
2871     index_of t = S (index_of s))),
2872   forall u v, u <A v -> index_of u < index_of v.
2873 Proof.
2874   intros index_of n_of HA u v Huv.
2875   induction Huv as [u v Huv | u v w Huv _ IH].
2876   - apply HA in Huv as [_ [_ Heq]]. lia.
2877   - apply HA in Huv as [_ [_ Heq]]. lia.
2878 Qed.
2879
2880 Lemma A_lt_n_of_const :
2881   forall (n_of : Sort -> nat),
2882   forall (HAclass : forall s t, A s t -> s ≈A t)
2883     (Hnconst : forall s t, s ≈A t -> n_of s = n_of t),
2884   forall u v, u <A v -> n_of u = n_of v.
2885 Proof.
2886   intros n_of HAclass Hnconst u v Huv.
2887   apply Hnconst.
2888   apply (A_lt_in_class HAclass u v Huv).
2889 Qed.

```

```

2888 Lemma A_le_in_class : forall s t, s ≤A t -> s ≈A t.
2889 Proof.
2890   intros s t H. induction H as [s | s t u Hst H IH].
2891   - apply A_eq_refl.
2892   - apply A_eq_trans with t; [apply A_eq_step; exact Hst | exact IH].
2893 Qed.
2894
2895 Lemma chain_elem_in_class : forall c a,
2896   chain (a :: c) -> forall i x, nth_error (a :: c) i = Some x -> x ≈A a.
2897 Proof.
2898   induction c as [| y c IH]; intros a Hc i x Hn.
2899   - destruct i as [| i'].
2900     + simpl in Hn. injection Hn as Hn. subst x. apply A_eq_refl.
2901     + destruct i' as [| i'']; simpl in Hn; discriminate Hn.
2902   - simpl in Hc. destruct Hc as [HAay Hc'].
2903     destruct i as [| i'].
2904     + simpl in Hn. injection Hn as Hn. subst x. apply A_eq_refl.
2905     + simpl in Hn.
2906       assert (Hxy : x ≈A y) by (apply (IH y Hc' i' x Hn)).
2907       apply A_eq_trans with y. exact Hxy. apply A_eq_sym. apply
2908       A_eq_step. exact HAay.
2909 Qed.
2910
2911 Lemma chain_pos_in_class : forall c s0 bot,
2912   chain c -> hd_error c = Some bot -> bot ≈A s0 ->
2913   forall i a, nth_error c i = Some a -> a ≈A s0.
2914 Proof.
2915   intros c s0 bot Hc Hh Hbs i a Hn.
2916   destruct c as [| x c'].
2917   - discriminate Hh.
2918   - simpl in Hh. injection Hh as Hh. subst x.
2919     assert (Ha_bot : a ≈A bot) by (apply (chain_elem_in_class c' bot
2920     Hc i a Hn)).
2921     apply A_eq_trans with bot; [exact Ha_bot | exact Hbs].
2922 Qed.
2923
2924 Lemma class_tiered :
2925   persistent -> bounded ->
2926   forall s0 : Sort,
2927   exists n index_of,
2928     (forall t, t ≈A s0 -> 0 < index_of t /\ index_of t < n + 1) /\
2929     (forall t u, t ≈A s0 -> u ≈A s0 -> index_of t = index_of u -> t =
2930     u) /\
2931     (forall i, 0 < i -> i < n + 1 -> exists t, t ≈A s0 /\ index_of t =
2932     i) /\
2933     (forall t u, t ≈A s0 -> (A t u <-> (index_of t < n /\ index_of u =
2934     S (index_of t)))).
2935 Proof.

```

```

2931 intros Hpers Hbnd s0.
2932 destruct (exists_top Hbnd s0) as [top [Hs0top Htopmax]].
2933 destruct (exists_bottom Hbnd s0) as [bot [Hbots0 Hbotmin]].
2934 assert (Htops0 : top  $\approx_A$  s0) by (apply A_eq_sym; apply A_le_in_class;
exact Hs0top).
2935 assert (Hbots0' : bot  $\approx_A$  s0) by (apply A_le_in_class; exact Hbots0).
2936
2937 assert (Htop : forall s, s  $\approx_A$  s0 -> s  $\leq_A$  top).
2938 { intros s Hs.
2939   assert (Hstop : s  $\approx_A$  top) by (apply A_eq_trans with s0; [exact Hs
2940     | apply A_eq_sym; exact Htops0]).
2941   destruct (A_eq_lt_case Hpers Hbnd s top Hstop) as [Hlt | [Heq |
2942     Hgt]].
2943   - apply A_le_of_lt; exact Hlt.
2944   - subst; apply A_le_refl.
2945   - exfalso; apply Htopmax; exists s; exact Hgt. }
2946
2947 assert (Hbot : forall s, s  $\approx_A$  s0 -> bot  $\leq_A$  s).
2948 { intros s Hs.
2949   assert (Hsbot : s  $\approx_A$  bot) by (apply A_eq_trans with s0; [exact Hs
2950     | apply A_eq_sym; exact Hbots0']).
2951   destruct (A_eq_lt_case Hpers Hbnd s bot Hsbot) as [Hlt | [Heq |
2952     Hgt]].
2953   - exfalso; apply Hbotmin; exists s; exact Hlt.
2954   - subst; apply A_le_refl.
2955   - apply A_le_of_lt; exact Hgt. }
2956
2957 destruct (A_le_iff_chain bot top) as [Hfwd _].
2958 assert (Hbotop : bot  $\leq_A$  top) by (apply Hbot; exact Htops0).
2959 destruct (Hfwd Hbotop) as [c Hc].
2960
2961 assert (Hcne : c  $\neq$  []).
2962 { destruct Hc as [_ [Hh _]]. destruct c; [discriminate Hh |
2963   discriminate]. }
2964
2965 assert (Hpos : forall s : Sort, s  $\approx_A$  s0 -> exists! i, nth_error c i
2966   = Some s).
2967 { intros s Hs.
2968   pose proof (Hbot s Hs) as Hbs.
2969   pose proof (Htop s Hs) as Hst.
2970   destruct (proj1 (A_le_iff_chain bot s) Hbs) as [c1 Hc1].
2971   destruct (proj1 (A_le_iff_chain s top) Hst) as [c2 Hc2].
2972   assert (He : chain_from_to bot top (c1 ++ tl c2)) by (apply
2973     (chain_from_to_app c1 c2 bot s top Hc1 Hc2)).
2974   assert (Hlen : length (c1 ++ tl c2) = length c)
2975     by (apply (chain_length_eq_of_same_ends Hpers Hbnd (c1 ++ tl c2)
2976       c bot top He Hc)).
2977   assert (Heqec : c1 ++ tl c2 = c).
2978   { apply (persistent_chain_unique_from Hpers (c1 ++ tl c2) c bot).

```

```

2971   - destruct He as [Hce _]; exact Hce.
2972   - destruct Hc as [Hcc _]; exact Hcc.
2973   - destruct He as [_ [Hhe _]]; exact Hhe.
2974   - destruct Hc as [_ [Hhc _]]; exact Hhc.
2975   - exact Hlen. }
2976 destruct Hc1 as [Hcc1 [Hhc1 Hlc1]].
2977 assert (Hc1ne : c1 <> []) by (destruct c1; [discriminate Hhc1 |
discriminate]).
2978 assert (Hpos_s : nth_error c1 (length c1 - 1) = Some s)
2979   by (rewrite (last_nth_error c1 bot Hc1ne); rewrite Hlc1;
reflexivity).
2980 assert (Hclge1 : 1 <= length c1) by (destruct c1; [contradiction
Hc1ne; reflexivity | simpl; lia]).
2981 assert (Hpos_e : nth_error (c1 ++ tl c2) (length c1 - 1) = Some
s).
2982 { rewrite (nth_error_app_l c1 (tl c2) (length c1 - 1)); [exact
Hpos_s | lia]. }
2983 rewrite Heqec in Hpos_e.
2984 exists (length c1 - 1). split.
2985 - exact Hpos_e.
2986 - intros i' Hi'. symmetry.
2987   apply (chain_pos_unique Hpers Hbnd c i' (length c1 - 1) s).
2988   + destruct Hc as [Hcc _]; exact Hcc.
2989   + exact Hi'.
2990   + exact Hpos_e. }
2991
2992 assert (Hchoice : forall s, {i : nat |
2993   (s ≈A s0 -> nth_error c i = Some s) /\ (~ s ≈A s0 -> i = 0)}).
2994 { intros s.
2995   apply constructive_indefinite_description.
2996   destruct (classic (s ≈A s0)) as [Hs | Hs].
2997   - destruct (Hpos s Hs) as [i [Hi _]]. exists i.
2998     split; [intros _; exact Hi | intros Hcontra; contradiction].
2999   - exists 0. split; [intros Hcontra; contradiction | intros _;
reflexivity]. }
3000
3001 set (index_of := fun s => S (proj1_sig (Hchoice s))).
3002 exists (length c), index_of.
3003 split.
3004
3005 - (* range *)
3006   intros t Ht. unfold index_of.
3007   destruct (Hchoice t) as [it Hit] eqn:E. simpl.
3008   pose proof (proj1 Hit Ht) as Hnth.
3009   pose proof (nth_error_lt_length c it t Hnth) as Hlt.
3010   lia.
3011
3012 - split. (* injectivity *)
3013   intros t u Ht Hu Heq. unfold index_of in Heq.

```

```

3014 destruct (Hchoice t) as [it Hit] eqn:Et.
3015 destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3016 simpl in Heq.
3017 assert (Hitu : it = iu) by lia.
3018 pose proof (proj1 Hit Ht) as Hntht.
3019 pose proof (proj1 Hiu Hu) as Hnthu.
3020 rewrite Hitu in Hntht.
3021 rewrite Hntht in Hnthu.
3022 injection Hnthu as Hnthu. exact Hnthu.
3023 split.
3024
3025 (* surjectivity *)
3026 intros i Hi1 Hi2.
3027 assert (Hilt : i - 1 < length c) by lia.
3028 destruct (nth_error_exists c (i - 1) Hilt) as [t Ht].
3029 assert (Hclass : t ≈A s0).
3030 { destruct Hc as [Hcc [Hhc _]].
3031   apply (chain_pos_in_class c s0 bot Hcc Hhc Hbots0' (i - 1) t
    Ht). }
3032 exists t. split; [exact Hclass |].
3033 unfold index_of.
3034 destruct (Hchoice t) as [it Hit] eqn:E. simpl.
3035 destruct (Hpos t Hclass) as [i0 [Hi0 Huniq0]].
3036 assert (Heqchoice : it = i0) by (symmetry; apply Huniq0; exact
    (proj1 Hit Hclass)).
3037 assert (Heq2 : i - 1 = i0) by (symmetry; apply Huniq0; exact Ht).
3038 lia.
3039
3040 (* A-iff *)
3041 intros t u Ht. split.
3042 + intro HAtu.
3043   assert (Htlt : t <A u) by (apply A_lt_step; exact HAtu).
3044   assert (Htnetop : t <> top).
3045   { intro Heq. subst t. apply Htopmax. exists u. exact Htlt. }
3046   destruct (Hchoice t) as [it Hit] eqn:Et.
3047   assert (Hit_index : index_of t = S it) by (unfold index_of;
    rewrite Et; reflexivity).
3048   pose proof (proj1 Hit Ht) as Hntht.
3049   assert (Hitlt : it < length c) by (apply (nth_error_lt_length c
    it t); exact Hntht).
3050   assert (Hitne : it <> length c - 1).
3051   { intro Heq. apply Htnetop.
3052     assert (Hlast : nth_error c (length c - 1) = Some top).
3053     { destruct Hc as [_ [_ Hl]]; rewrite (last_nth_error c bot
    Hcne); rewrite Hl; reflexivity. }
3054     assert (Hit' : nth_error c (length c - 1) = Some t) by
    (rewrite <- Heq; exact Hntht).
3055     rewrite Hit' in Hlast. injection Hlast as Hlast. exact Hlast.
    }

```

```

3056   assert (Hsit : S it < length c) by lia.
3057   destruct (nth_error_exists c (S it) Hsit) as [z Hz].
3058   assert (HAtz : A t z).
3059   { apply (chain_nth_A c it t z); [destruct Hc as [Hcc _]; exact
Hcc | exact Hntht | exact Hz]. }
3060   assert (Hzu : z = u).
3061   { destruct Hpers as [[Hfunc1 _] _]. apply (Hfunc1 t); [exact
HAtz | exact HAtu]. }
3062   subst z.
3063   assert (Hu0 : u ≈A s0).
3064   { apply A_eq_trans with t; [apply A_eq_sym; apply A_eq_step;
exact HAtu | exact Ht]. }
3065   destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3066   assert (Hiu_index : index_of u = S iu) by (unfold index_of;
rewrite Eu; reflexivity).
3067   assert (Hiueq : iu = S it).
3068   { destruct (Hpos u Hu0) as [i0 [Hi0 Huniq0]].
3069     assert (E1 : i0 = iu) by (apply Huniq0; exact (proj1 Hiu
Hu0)).
3070     assert (E2 : i0 = S it) by (apply Huniq0; exact Hz).
3071     lia. }
3072   split; lia.
3073 + intros [Hlt Heqsucc].
3074   destruct (Hchoice t) as [it Hit] eqn:Et.
3075   assert (Hit_index : index_of t = S it) by (unfold index_of;
rewrite Et; reflexivity).
3076   assert (Hsit : S it < length c) by (rewrite Hit_index in Hlt;
exact Hlt).
3077   destruct (nth_error_exists c (S it) Hsit) as [z Hz].
3078   assert (HAtz : A t z).
3079   { apply (chain_nth_A c it t z); [destruct Hc as [Hcc _]; exact
Hcc | exact (proj1 Hit Ht) | exact Hz]. }
3080   destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3081   assert (Hiu_index : index_of u = S iu) by (unfold index_of;
rewrite Eu; reflexivity).
3082   assert (Hiueq : iu = S it) by (rewrite Hit_index, Hiu_index in
Heqsucc; lia).
3083   assert (Hu0 : u ≈A s0).
3084   { destruct (classic (u ≈A s0)) as [H | H]; [exact H | ].
3085     exfalso.
3086     assert (Hiu0 : iu = 0) by (apply (proj2 Hiu); exact H).
3087     lia. }
3088   assert (Hzu : z = u).
3089   { assert (Hiu' : nth_error c (S it) = Some u) by (rewrite <-
Hiueq; exact (proj1 Hiu Hu0)).
3090     rewrite Hz in Hiu'. injection Hiu' as Hiu'. exact Hiu'. }
3091   subst z. exact HAtz.
3092 Qed.
3093

```

```

3094 Lemma A_lt_pred : forall t u, t <A u -> exists w, A w u.
3095 Proof.
3096   intros t u H. induction H as [t u Htu | t v u Htv _ IH].
3097   - exists t. exact Htu.
3098   - exact IH.
3099 Qed.
3100
3101 Lemma no_pred_unique_in_class :
3102   persistent -> bounded ->
3103   forall s t u,
3104     t ≈A s -> u ≈A s ->
3105     (~ exists v, v ≈A s /\ A v t) ->
3106     (~ exists v, v ≈A s /\ A v u) ->
3107     t = u.
3108 Proof.
3109   intros Hpers Hbnd s t u Ht Hu Hnpt Hnpu.
3110   destruct (classic (t = u)) as [Heq | Hneq]; [exact Heq |].
3111   assert (Htu : t ≈A u) by (apply A_eq_trans with s; [exact Ht | apply
A_eq_sym; exact Hu]).
3112   destruct (A_eq_lt_case Hpers Hbnd t u Htu) as [Hlt | [Heq | Hgt]].
3113   - exfalso. destruct (A_lt_pred t u Hlt) as [w Hw].
3114     apply Hnpu. exists w. split; [| exact Hw].
3115     apply A_eq_trans with u; [apply A_eq_step; exact Hw | exact Hu].
3116   - exact Heq.
3117   - exfalso. destruct (A_lt_pred u t Hgt) as [w Hw].
3118     apply Hnpt. exists w. split; [| exact Hw].
3119     apply A_eq_trans with t; [apply A_eq_step; exact Hw | exact Ht].
3120 Qed.
3121
3122 Lemma class_tiered_unique :
3123   persistent -> bounded ->
3124   forall s n1 n2 f1 f2,
3125     (forall t, t ≈A s -> 0 < f1 t /\ f1 t < n1 + 1) ->
3126     (forall t u, t ≈A s -> u ≈A s -> f1 t = f1 u -> t = u) ->
3127     (forall i, 0 < i -> i < n1 + 1 -> exists t, t ≈A s /\ f1 t = i) ->
3128     (forall t u, t ≈A s -> (A t u <-> (f1 t < n1 /\ f1 u = S (f1 t)))) ->
3129     (forall t, t ≈A s -> 0 < f2 t /\ f2 t < n2 + 1) ->
3130     (forall t u, t ≈A s -> u ≈A s -> f2 t = f2 u -> t = u) ->
3131     (forall i, 0 < i -> i < n2 + 1 -> exists t, t ≈A s /\ f2 t = i) ->
3132     (forall t u, t ≈A s -> (A t u <-> (f2 t < n2 /\ f2 u = S (f2 t)))) ->
3133     n1 = n2 /\ forall t, t ≈A s -> f1 t = f2 t.
3134 Proof.
3135   intros Hpers Hbnd s n1 n2 f1 f2
3136     Hr1 Hi1 Hs1 HA1 Hr2 Hi2 Hs2 HA2.
3137
3138   assert (Hnp1 : forall i, i = 1 -> forall t, t ≈A s -> f1 t = i ->
3139     ~ exists v, v ≈A s /\ A v t).
3140   { intros i Hileq t Ht Hf1t [v [Hv HAvt]].

```



```

3141   apply (HA1 v t Hv) in HAvt as [_ Heq].
3142   destruct (Hr1 v Hv) as [Hf1vpos _].
3143   lia. }
3144
3145   assert (Hmain : forall i, 0 < i -> i < n1 + 1 -> i < n2 + 1 ->
3146           forall t, t ≈A s -> f1 t = i -> f2 t = i).
3147   { intros i.
3148     induction i as [i IH] using (well_founded_induction Wf_nat.lt_wf).
3149     intros Hi0 Hi1lt Hi2lt t Ht Hf1t.
3150     destruct i as [| i'].
3151     - lia.
3152     - destruct i' as [| i''].
3153       + (* i = 1: base case via no-predecessor uniqueness *)
3154         destruct (Hs2 1 ltac:(lia) Hi2lt) as [u [Hu Hf2u]].
3155         assert (Ht_np : ~ exists v, v ≈A s /\ A v t).
3156         { intros [v [Hv HAvt]].
3157           apply (HA1 v t Hv) in HAvt as [_ Heq].
3158           destruct (Hr1 v Hv) as [Hf1vpos _].
3159           rewrite Hf1t in Heq. lia. }
3160         assert (Hu_np : ~ exists v, v ≈A s /\ A v u).
3161         { intros [v [Hv HAvu]].
3162           apply (HA2 v u Hv) in HAvu as [_ Heq].
3163           destruct (Hr2 v Hv) as [Hf1vpos _].
3164           rewrite Hf2u in Heq.
3165           lia. }
3166         assert (Htu : t = u) by (apply (no_pred_unique_in_class Hpers
3167   Hbnd s t u Ht Hu Ht_np Hu_np)).
3168         rewrite Htu. exact Hf2u.
3169       + (* successor case *)
3170         destruct (Hs1 (S i'') ltac:(lia) ltac:(lia)) as [v [Hv Hf1v]].
3171         assert (HAvt : A v t).
3172         { apply (HA1 v t Hv). split; [lia | rewrite Hf1v, Hf1t;
3173   reflexivity]. }
3174         assert (Hf2v : f2 v = S i'') by (apply (IH (S i'') ltac:(lia)
3175   ltac:(lia) ltac:(lia) ltac:(lia) v Hv Hf1v)).
3176         apply (HA2 v t Hv) in HAvt as [_ Hf2t].
3177         rewrite Hf2v in Hf2t. exact Hf2t. }
3178
3179   assert (Hmain2 : forall i, 0 < i -> i < n1 + 1 -> i < n2 + 1 ->
3180           forall t, t ≈A s -> f2 t = i -> f1 t = i).
3181   { intros i.
3182     induction i as [i IH] using (well_founded_induction Wf_nat.lt_wf).
3183     intros Hi0 Hi1lt Hi2lt t Ht Hf2t.
3184     destruct i as [| i'].
3185     - lia.
3186     - destruct i' as [| i''].
3187       + destruct (Hs1 1 ltac:(lia) Hi1lt) as [u [Hu Hf1u]].
3188         assert (Ht_np : ~ exists v, v ≈A s /\ A v t).
3189         { intros [v [Hv HAvt]].
3190           apply (HA2 v t Hv) in HAvt as [_ Heq].

```



```

3188     destruct (Hr2 v Hv) as [Hf1vpos _].
3189     rewrite Hf2t in Heq.
3190     lia. }
3191   assert (Hu_np : ~ exists v, v ≈A s /\ A v u).
3192   { intros [v [Hv HAvu]].
3193     apply (HA1 v u Hv) in HAvu as [_ Heq].
3194     destruct (Hr1 v Hv) as [Hf1vpos _].
3195     rewrite Hf1u in Heq.
3196     lia. }
3197   assert (Htu : t = u) by (apply (no_pred_unique_in_class Hpers
Hbnd s t u Ht Hu Ht_np Hu_np)).
3198   rewrite Htu. exact Hf1u.
3199 + destruct (Hs2 (S i'') ltac:(lia) ltac:(lia)) as [v [Hv Hf2v]].
3200   assert (HAvt : A v t).
3201   { apply (HA2 v t Hv). split; [lia | rewrite Hf2v, Hf2t;
reflexivity]. }
3202   assert (Hf1v : f1 v = S i'') by (apply (IH (S i'') ltac:(lia)
ltac:(lia) ltac:(lia) ltac:(lia) v Hv Hf2v)).
3203   apply (HA1 v t Hv) in HAvt as [_ Hf1t].
3204   rewrite Hf1v in Hf1t. exact Hf1t. }
3205
3206   assert (Hn : n1 = n2).
3207   { destruct (Nat.lt_trichotomy n1 n2) as [Hlt | [Heq | Hgt]]; auto.
3208     - exfalso.
3209       destruct (Hs2 (n1+1) ltac:(lia) ltac:(lia)) as [u [Hu Hf2u]].
3210       destruct (Hr2 u Hu) as [_ Hf2u_lt].
3211       destruct (Nat.lt_trichotomy (f1 u) (n1+1)) as [Hc1 | [Hc2 |
Hc3]].
3212       + assert (Hf1lt : f1 u < n1 + 1) by lia.
3213       destruct (Hr1 u Hu) as [Hf1pos _].
3214       assert (Hf2u' : f2 u = f1 u) by (apply (Hmain (f1 u) Hf1pos
Hf1lt ltac:(lia) u Hu eq_refl)).
3215       rewrite Hf2u in Hf2u'. lia.
3216       + destruct (Hr1 u Hu) as [_ Hf1u_lt]. lia.
3217       + destruct (Hr1 u Hu) as [_ Hf1u_lt]. lia.
3218     - exfalso.
3219       destruct (Hs1 (n2+1) ltac:(lia) ltac:(lia)) as [u [Hu Hf1u]].
3220       destruct (Hr1 u Hu) as [_ Hf1u_lt].
3221       destruct (Nat.lt_trichotomy (f2 u) (n2+1)) as [Hc1 | [Hc2 |
Hc3]].
3222       + assert (Hf2lt : f2 u < n2 + 1) by lia.
3223       destruct (Hr2 u Hu) as [Hf2pos _].
3224       assert (Hf1u' : f1 u = f2 u) by (apply (Hmain2 (f2 u) Hf2pos
ltac:(lia) ltac:(lia) u Hu eq_refl)).
3225       rewrite Hf1u in Hf1u'. lia.
3226       + destruct (Hr2 u Hu) as [_ Hf2u_lt]. lia.
3227       + destruct (Hr2 u Hu) as [_ Hf2u_lt]. lia. }
3228
3229   split; [exact Hn |].

```

```

3230   intros t Ht.
3231   destruct (Hr1 t Ht) as [Hpos Hlt]. symmetry.
3232   apply (Hmain (f1 t) Hpos ltac:(lia) ltac:(lia) t Ht eq_refl).
3233 Qed.
3234
3235 Lemma persistent_bounded_seperable_iff_disjoint_union_of_tiered :
3236   (persistent /\ bounded /\ separable) <-> disjoint_union_of_tiered.
3237 Proof.
3238   split.
3239
3240 - intros [Hpers [Hbnd Hsep]].
3241   assert (Hwit : forall s0 : Sort, {p : nat * (Sort -> nat) |
3242     (forall t, t ≈A s0 -> 0 < snd p t /\ snd p t < fst p + 1) /\
3243     (forall t u, t ≈A s0 -> u ≈A s0 -> snd p t = snd p u -> t = u)
3244     /\
3245     (forall i, 0 < i -> i < fst p + 1 -> exists t, t ≈A s0 /\ snd p
3246       t = i) /\
3247     (forall t u, t ≈A s0 -> (A t u <-> (snd p t < fst p /\ snd p u =
3248       S (snd p t))))}).
3249   { intros s0.
3250     apply constructive_indefinite_description.
3251     destruct (class_tiered Hpers Hbnd s0) as [n [index_of H]].
3252     exists (n, index_of). exact H. }
3253
3254   set (n_of := fun s => fst (proj1_sig (Hwit s))).
3255   set (index_of := fun s => snd (proj1_sig (Hwit s)) s).
3256
3257   assert (Hprop : forall s,
3258     (forall t, t ≈A s -> 0 < snd (proj1_sig (Hwit s)) t /\ snd
3259       (proj1_sig (Hwit s)) t < fst (proj1_sig (Hwit s)) + 1) /\
3260     (forall t u, t ≈A s -> u ≈A s -> snd (proj1_sig (Hwit s)) t =
3261       snd (proj1_sig (Hwit s)) u -> t = u) /\
3262     (forall i, 0 < i -> i < fst (proj1_sig (Hwit s)) + 1 -> exists
3263       t, t ≈A s /\ snd (proj1_sig (Hwit s)) t = i) /\
3264     (forall t u, t ≈A s -> (A t u <-> (snd (proj1_sig (Hwit s)) t <
3265       fst (proj1_sig (Hwit s)) /\ snd (proj1_sig (Hwit s)) u = S (snd
3266       (proj1_sig (Hwit s)) t))))).
3267   { intros s. exact (proj2_sig (Hwit s)). }
3268
3269   assert (Hagree : forall s t, s ≈A t -> forall u, u ≈A s ->
3270     fst (proj1_sig (Hwit s)) = fst (proj1_sig (Hwit t)) /\
3271     snd (proj1_sig (Hwit s)) u = snd (proj1_sig (Hwit t)) u).
3272   { intros s t Hst u Hu.
3273     destruct (Hprop s) as [Hr1 [Hi1 [Hs1 HA1]]].
3274     destruct (Hprop t) as [Hr2 [Hi2 [Hs2 HA2]]].
3275     assert (Hr2' : forall v, v ≈A s -> 0 < snd (proj1_sig (Hwit t))
3276       v /\ snd (proj1_sig (Hwit t)) v < fst (proj1_sig (Hwit t)) + 1).
3277     { intros v Hv. apply Hr2. apply A_eq_trans with s; [exact Hv |
3278       exact Hst]. }

```

```

3269   assert (Hi2' : forall v w, v ≈A s -> w ≈A s -> snd (proj1_sig
3270     (Hwit t)) v = snd (proj1_sig (Hwit t)) w -> v = w).
3271   { intros v w Hv Hw. apply Hi2; apply A_eq_trans with s; auto. }
3272   assert (Hs2' : forall i, 0 < i -> i < fst (proj1_sig (Hwit t)) +
3273     1 -> exists v, v ≈A s /\ snd (proj1_sig (Hwit t)) v = i).
3274   { intros i Hi0 Hilt. destruct (Hs2 i Hi0 Hilt) as [v [Hv Hvi]].
3275     exists v. split; [| exact Hvi].
3276     apply A_eq_trans with t; [exact Hv | apply A_eq_sym; exact
3277       Hst]. }
3278   assert (HA2' : forall v w, v ≈A s -> (A v w <-> (snd (proj1_sig
3279     (Hwit t)) v < fst (proj1_sig (Hwit t)) /\ snd (proj1_sig (Hwit
3280     t)) w = S (snd (proj1_sig (Hwit t)) v))))).
3281   { intros v w Hv. apply HA2. apply A_eq_trans with s; [exact Hv |
3282     exact Hst]. }
3283   destruct (class_tiered_unique Hpers Hbnd s
3284     (fst (proj1_sig (Hwit s))) (fst (proj1_sig (Hwit
3285     t))))
3286     (snd (proj1_sig (Hwit s))) (snd (proj1_sig (Hwit
3287     t)))
3288     Hr1 Hi1 Hs1 HA1 Hr2' Hi2' Hs2' HA2') as [Hn Hf].
3289   split; [exact Hn | apply Hf; exact Hu]. }
3290
3291   exists n_of, index_of.
3292   split.
3293
3294   (* A s t -> s ≈A t *)
3295   intros s t HAst.
3296   destruct (Hprop s) as [_ [_ [_ HAiff]]].
3297   assert (Hss : s ≈A s) by apply A_eq_refl.
3298   apply A_eq_step; exact HAst.
3299   split.
3300
3301   (* R s t u -> s ≈A t *)
3302   intros s t u HR.
3303   destruct Hpers as [_ [_ Hshape]].
3304   assert (Htu : t = u) by (apply (Hshape s t u); exact HR).
3305   apply (Hsep s t). rewrite <- Htu in HR. exact HR.
3306   split.
3307
3308   (* n_of constant on class *)
3309   intros s t Hst. unfold n_of.
3310   destruct (Hagree s t Hst s) as [Hn _]; [apply A_eq_refl |]. exact
3311   Hn.
3312   split.
3313
3314   (* range *)
3315   intros s. unfold index_of.
3316   destruct (Hprop s) as [Hrange _].
3317   apply (Hrange s). apply A_eq_refl.

```

```

3308 split.
3309
3310 (* injectivity *)
3311 intros s t Hst Heq. unfold index_of, n_of in *.
3312 assert (Hs_ind : snd (proj1_sig (Hwit s)) s = snd (proj1_sig (Hwit
t)) t) by exact Heq.
3313 destruct (Hagree s t Hst s) as [_ Hval]; [apply A_eq_refl |].
3314 rewrite Hval in Hs_ind.
3315 destruct (Hprop t) as [_ [Hinj _]].
3316 apply (Hinj s t).
3317 apply A_eq_trans with s; [apply A_eq_refl | exact Hst].
3318 apply A_eq_refl.
3319 exact Hs_ind.
3320 split.
3321
3322 (* surjectivity *)
3323 intros s i H1 H2. unfold n_of, index_of in *.
3324 destruct (Hprop s) as [_ [_ [Hsurj _]]].
3325 assert (H2' : i < fst (proj1_sig (Hwit s)) + 1) by lia.
3326 destruct (Hsurj i H1 H2') as [t [Ht Hti]].
3327 exists t. split; [exact Ht |]. apply A_eq_sym in Ht.
3328 destruct (Hagree s t Ht t) as [_ Hval]. apply A_eq_sym. apply Ht.
3329 rewrite <- Hval. exact Hti.
3330 split.
3331
3332 (* A iff *)
3333 intros s t. split.
3334
3335 intro Hst.
3336 assert (Hst_class : s ≈A t) by (apply A_eq_step; exact Hst).
3337 destruct (Hprop s) as [_ [_ [HAiff]]].
3338 apply (HAiff s t (A_eq_refl s)) in Hst as [H1 H2].
3339 split; [exact Hst_class |].
3340 unfold index_of, n_of.
3341 destruct (Hagree s t Hst_class t) as [Hn Hval]. apply A_eq_sym.
3342 apply Hst_class.
3343 split; [exact H1 | rewrite <- Hval; exact H2].
3344
3345 intros [Hclass [Hlt Heq]].
3346 unfold index_of, n_of in *.
3347 destruct (Hprop s) as [_ [_ [HAiff]]].
3348 apply (HAiff s t (A_eq_refl s)).
3349 destruct (Hagree s t Hclass t) as [Hn Hval]; [apply A_eq_sym;
exact Hclass |].
3350 split; [exact Hlt | rewrite Hval; exact Heq].
3351
3352 (* R shape *)
3353 intros s t u HR.
3354 destruct Hpers as [_ [_ Hshape]]. eauto.

```

```

3355 - intros [n_of [index_of [HAcClass [HRclass [Hnconst [Hrange [Hinj
      [Hsurj [HA HR]]]]]]]]].
3356   split.
3357
3358 + (* persistent *)
3359   repeat split.
3360   * (* unique A-successor *)
3361     intros x y y' Hxy Hxy'.
3362     apply HA in Hxy as [Hxy1 [Hxy2 Hxy3]].
3363     apply HA in Hxy' as [Hxy1' [Hxy2' Hxy3']].
3364     apply Hinj.
3365     -- (* y ≈A y' *)
3366       apply A_eq_trans with x.
3367       ++ apply A_eq_sym. apply A_eq_step. apply HA. auto.
3368       ++ apply A_eq_step. apply HA. auto.
3369     -- rewrite Hxy3, Hxy3'. reflexivity.
3370   * (* unique R-shape (functional's 2nd component) *)
3371     intros s t u u' Hu Hu'.
3372     rewrite <- (HR s t u Hu), (HR s t u' Hu'). reflexivity.
3373   * (* unique A-predecessor *)
3374     intros x x' y Hxy Hxy'.
3375     apply HA in Hxy as [Hxy1 [Hxy2 Hxy3]].
3376     apply HA in Hxy' as [Hxy1' [Hxy2' Hxy3']].
3377     apply Hinj.
3378     -- apply A_eq_trans with y.
3379     ++ apply A_eq_step. apply HA. auto.
3380     ++ apply A_eq_sym. apply A_eq_step. apply HA. auto.
3381     -- assert (Heq : index_of x = index_of x') by lia.
3382     exact Heq.
3383   * (* R shape *)
3384     intros x y z HRxyz. exact (HR x y z HRxyz).
3385
3386 + split. split.
3387
3388   (* ascending chain condition *)
3389   intro s.
3390   apply (wf_by_measure Sort (fun u v => u <A v) index_of).
3391   intros u v Huv.
3392   apply (A_lt_index_incr index_of n_of HA u v Huv).
3393
3394   (* descending chain condition *)
3395   intro s.
3396   apply (wf_by_measure Sort (fun u v => v <A u) (fun s => n_of s -
      index_of s)).
3397   intros u v Huv.
3398   pose proof (A_lt_index_incr index_of n_of HA v u Huv) as Hidx.
3399   pose proof (A_lt_n_of_const n_of HAcClass Hnconst v u Huv) as Hn.
3400   pose proof (Hrange u) as [_ Hu2].
3401   pose proof (Hrange v) as [_ Hv2].
3402   lia.

```

```

3403
3404      (* separable *)
3405      intros s s' HR'.
3406      apply HRclass in HR' as Hclass.
3407      pose proof (HR s s' s' HR') as Ht.
3408      exact Hclass.
3409 Qed.
3410
3411 Definition non_dependent_tiered (n : nat) : Prop :=
3412   exists index_of : Sort -> nat,
3413     (forall x, 0 < index_of x /\ index_of x < n + 1) /\
3414     (forall x y, index_of x = index_of y -> x = y) /\
3415     (forall i, 0 < i -> i < n + 1 -> exists x, index_of x = i) /\
3416     (forall x y, A x y <-> index_of x < n /\ index_of y = S (index_of
3417       x)) /\
3418     (forall x y z, R x y z -> y = z /\ index_of y <= index_of x).
3419
3420 Definition disjoint_union_of_non_dependent_tiered : Prop :=
3421   exists (n_of : Sort -> nat) (index_of : Sort -> nat),
3422     (forall s t, A s t -> s ≈A t) /\
3423     (forall s t u, R s t u -> s ≈A t) /\
3424     (forall s t, s ≈A t -> n_of s = n_of t) /\
3425     (forall s, 0 < index_of s /\ index_of s < n_of s + 1) /\
3426     (forall s t, s ≈A t -> index_of s = index_of t -> s = t) /\
3427     (forall s i, 0 < i -> i <= n_of s -> exists t, t ≈A s /\ index_of
3428       t = i) /\
3429     (forall s t, A s t <-> (s ≈A t /\ index_of s < n_of s /\ index_of
3430       t = S (index_of s))) /\
3431     (forall s t u, R s t u -> t = u /\ index_of t <= index_of s).
3432
3433 (* Corollary 1 *)
3434 Axiom bounded_non_dependent_iff_disjoint_union_of_non_dependent_tiered
3435 :
3436   bounded_non_dependent <-> disjoint_union_of_non_dependent_tiered.
3437
3438 (* Proposition 2 *)
3439 Axiom weak_implies_strong_normalization_lift :
3440   (forall n, non_dependent_tiered n /\ tiered n ->
3441     (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3442   (persistent /\ bounded /\ separable ->
3443     (forall M, weakly_normalizing M -> strongly_normalizing M)).
3444
3445 Open Scope Z_scope.
3446
3447 Definition Zpos_tiered : Prop :=
3448   exists index_of : Sort -> Z,
3449     (forall x, 0 < index_of x) /\
3450     (forall x y, index_of x = index_of y -> x = y) /\
3451     (forall i : Z, 0 < i -> exists x, index_of x = i) /\

```

```

3448     (forall x y, A x y <-> index_of y = index_of x + 1) /\
3449     (forall x y z, R x y z -> y = z).
3450
3451 Definition Zneg_tiered : Prop :=
3452   exists index_of : Sort -> Z,
3453     (forall x, index_of x < 0) /\
3454     (forall x y, index_of x = index_of y -> x = y) /\
3455     (forall i : Z, i < 0 -> exists x, index_of x = i) /\
3456     (forall x y, A x y <-> index_of x <= -2 /\ index_of y = index_of x
3457       + 1) /\
3458     (forall x y z, R x y z -> y = z).
3459
3460 Definition Z_tiered : Prop :=
3461   exists index_of : Sort -> Z,
3462     (forall x y, index_of x = index_of y -> x = y) /\
3463     (forall i : Z, exists x, index_of x = i) /\
3464     (forall x y, A x y <-> index_of y = index_of x + 1) /\
3465     (forall x y z, R x y z -> y = z).
3466
3467 Close Scope Z_scope.
3468
3469 Axiom finite_or_Zneg_tiered_atomic_gen_non_dependent :
3470   forall n, tiered n \ / Zneg_tiered ->
3471   atomic /\ generalized_non_dependent.
3472
3473 Axiom weak_implies_strong_normalization_n_tiered_lift :
3474   (forall n, non_dependent_tiered n ->
3475     (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3476   (generalized_non_dependent ->
3477     (forall M, weakly_normalizing M -> strongly_normalizing M)).
3478
3479 Definition cyclic_tiered (n : nat) : Prop :=
3480   exists index_of : Sort -> nat,
3481     (forall x, 0 < index_of x /\ index_of x < n + 1) /\
3482     (forall x y, index_of x = index_of y -> x = y) /\
3483     (forall i, 0 < i -> i < n + 1 -> exists x, index_of x = i) /\
3484     (forall x y, A x y <->
3485       (index_of x < n /\ index_of y = S (index_of x)) \ /
3486       (index_of x = n /\ index_of y = 1)) /\
3487     (forall x y z, R x y z -> y = z).
3488
3489 (* Proposition 4 *)
3490 Axiom weak_implies_strong_normalization_persistent_separable_lift :
3491   (forall n, (tiered n \ / cyclic_tiered n) ->
3492     (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3493   (persistent /\ separable ->
3494     (forall M, weakly_normalizing M -> strongly_normalizing M)).
3495
3496 End PTS.

```


سیستم نوع محض لایه‌ای و ترجمه حذف وابستگی

```

1  From Stdlib Require Import List.
2  From Stdlib Require Import Classical.
3  From Stdlib Require Import Logic.IndefiniteDescription.
4  From Stdlib.Program Require Import Program Wf.
5  From Stdlib Require Import Lia.
6  From Stdlib Require Import Arith.Arith.
7  From Stdlib Require Import ZArith.
8  Import ListNotations.
9
10 Require Import Thesis.TPTSSignature.
11 Require Import Thesis.PTS.
12
13 Module TPTS (Sig : TPTS_SIGNATURE).
14
15 Import Sig.
16
17 Module Base := PTS Sig.
18
19 Import Base.
20
21 (* =====
22 *)
23 (* Retag *)
24 Definition retag (x : var) (s : Sort) : var := x * (n + 2) + index_of
    s.
25
26 Lemma retag_inj : forall x1 s1 x2 s2,
27   retag x1 s1 = retag x2 s2 -> x1 = x2 /\ s1 = s2.
28 Proof.
29   intros x1 s1 x2 s2 H. unfold retag in H.
30   pose proof (index_range s1) as [Hs1a Hs1b].
31   pose proof (index_range s2) as [Hs2a Hs2b].
32   assert (Hx : x1 = x2).
33   { assert (E1 : (x1 * (n + 2) + index_of s1) / (n + 2) = x1).
34     { rewrite Nat.div_add_l by lia. rewrite Nat.div_small by lia. lia.
35     }
36     assert (E2 : (x2 * (n + 2) + index_of s2) / (n + 2) = x2).
37     { rewrite Nat.div_add_l by lia. rewrite Nat.div_small by lia. lia.
38     }
39     rewrite H in E1. rewrite E1 in E2. exact E2. }
40   subst x1.
41   assert (Hi : index_of s1 = index_of s2) by lia.
42   split; [reflexivity | apply index_inj; exact Hi].
43 Qed.

```



```

43 Lemma retag_neq_of_atom_neq : forall x1 s1 x2 s2,
44   x1 <> x2 -> retag x1 s1 <> retag x2 s2.
45 Proof. intros x1 s1 x2 s2 Hne Heq. apply retag_inj in Heq. tauto. Qed.
46
47 Lemma retag_neq_of_sort_neq : forall x s1 s2,
48   s1 <> s2 -> retag x s1 <> retag x s2.
49 Proof. intros x s1 s2 Hne Heq. apply retag_inj in Heq. tauto. Qed.
50
51 Lemma retag_neq_mult : forall k x s, retag x s <> k * (n + 2).
52 Proof.
53   intros k x s Heq. unfold retag in Heq.
54   pose proof (index_range s) as [Hlo Hhi].
55   assert (Hm : (x * (n + 2) + index_of s) mod (n + 2) = index_of s).
56   { rewrite Nat.add_comm. rewrite Nat.Div0.mod_add by lia. apply
    Nat.mod_small; lia. }
57   rewrite Heq in Hm.
58   rewrite Nat.Div0.mod_mul in Hm by lia.
59   lia.
60 Qed.
61
62 (* =====
63 *)
64 (* Degree *)
65
66 Fixpoint deg (t : term) : nat :=
67   match t with
68   | t_sort s      => index_of s + 1
69   | t_bvar s _    => index_of s - 1
70   | t_fvar s _    => index_of s - 1
71   | t_app M _     => deg M
72   | t_lam _ _ M   => deg M
73   | t_pi _ _ B    => deg B
74   end.
75
76 Definition T_eq (j : nat) (M : term) : Prop := deg M = j.
77
78 Definition T_geq (j : nat) (M : term) : Prop := j <= deg M.
79
80
81 Notation "'T_[' j ]'" := (T_eq j) (at level 0).
82 Notation "'T_≥[' j ]'" := (T_geq j) (at level 0).
83
84 Definition derivable (M : term) : Prop := exists Γ N, Γ ⊢ M ∈ N.
85
86 Fixpoint closed_at (k : nat) (t : term) : Prop :=
87   match t with
88   | t_sort _      => True
89   | t_bvar _ n    => n < k
90   | t_fvar _ _    => True
91   | t_app M _     => closed_at k M
92   | t_lam _ _ M   => closed_at (S k) M
93   | t_pi _ _ B    => closed_at (S k) B

```

```

91   end.
92
93   Lemma closed_at_open_rec_inv : forall t k u,
94     closed_at k (open_rec k u t) ->
95     (forall s m, u <> t_bvar s m) ->
96     closed_at (S k) t.
97   Proof.
98     induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A
99       IHA B IHB];
100     intros k u Hc Hu; simpl in *; auto.
101     - destruct (Nat.eqb n k) eqn:Heqb.
102       + apply Nat.eqb_eq in Heqb. lia.
103       + apply Nat.eqb_neq in Heqb.
104         simpl in Hc. lia.
105     - eapply IHP; eauto.
106     - eapply IHM with (k := S k); eauto.
107     - eapply IHB with (k := S k); eauto.
108   Qed.
109
110   Lemma lc_closed_at : forall t, lc t -> closed_at 0 t.
111   Proof.
112     intros t Hlc.
113     induction Hlc as [ s | x | M N HM IHM HN IHN
114       | s A B L HA IHA HB IHB
115       | s A M L HA IHA HM IHM ]; simpl; auto.
116     - remember (fresh L) as x0 eqn:Hx0def.
117       assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
118       specialize (IHB x0 Hx0L).
119       unfold open_var in IHB.
120       apply (closed_at_open_rec_inv B 0 (t_fvar s x0) IHB).
121       intros sv m Hcontra. discriminate.
122     - remember (fresh L) as x0 eqn:Hx0def.
123       assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
124       specialize (IHM x0 Hx0L).
125       unfold open_var in IHM.
126       apply (closed_at_open_rec_inv M 0 (t_fvar s x0) IHM).
127       intros sv m Hcontra. discriminate.
128   Qed.
129
130   Lemma deg_open_rec_gen : forall B k x s,
131     bvar_tag_ok k s B ->
132     deg B = deg (open_rec k (t_fvar s x) B).
133   Proof.
134     induction B as [s0 | s_n n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A
135       IHA B IHB];
136     intros k x s Htag; simpl in *.
137     - reflexivity.
138     - destruct (Nat.eqb n k) eqn:Heqb.
139       + apply Nat.eqb_eq in Heqb. subst n.
140       specialize (Htag eq_refl). subst s_n.

```

```

139     reflexivity.
140   + reflexivity.
141   - reflexivity.
142   - apply (IHP k x s Htag).
143   - apply (IHM (S k) x s Htag).
144   - apply (IHB (S k) x s Htag).
145 Qed.
146
147 Lemma deg_open : forall B x s,
148   bvar_tag_ok 0 s B ->
149   deg B = deg (open_var B s x).
150 Proof.
151   intros B x s Htag.
152   unfold open_var.
153   apply deg_open_rec_gen. exact Htag.
154 Qed.
155
156 Lemma deg_open_rec_subst : forall B k u s,
157   bvar_tag_ok k s B ->
158   deg u = index_of s - 1 ->
159   deg (open_rec k u B) = deg B.
160 Proof.
161   induction B as [s0 | s_n n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A
162     IHA B IHB];
163     intros k u s Htag Hdeg; simpl in *.
164   - reflexivity.
165   - destruct (Nat.eqb n k) eqn:Heqb.
166     + apply Nat.eqb_eq in Heqb. subst n.
167       specialize (Htag eq_refl). subst s_n.
168       exact Hdeg.
169     + reflexivity.
170   - reflexivity.
171   - apply (IHP k u s Htag Hdeg).
172   - apply (IHM (S k) u s Htag Hdeg).
173   - apply (IHB (S k) u s Htag Hdeg).
174 Qed.
175
176 Lemma deg_subst_open : forall B x s N,
177   ~ In x (fv B) ->
178   bvar_tag_ok 0 s B ->
179   lc N ->
180   deg N = index_of s - 1 ->
181   deg ((open_var B s x) [ x = N ]) = deg (open_var B s x).
182 Proof.
183   intros B x s N HxB Htag HlcN HdegN.
184   rewrite (subst_open_var_eq B s x N HxB HlcN).
185   pose proof (deg_open_rec_subst B 0 N s Htag HdegN) as Hgen.
186   rewrite Hgen.
187   unfold open_var.
188   apply deg_open_rec_gen. exact Htag.

```

```

188 Qed.
189
190 Axiom deg_sort_typed : forall  $\Gamma$  A s,
191    $\Gamma \vdash A \in \text{t\_sort } s \rightarrow \text{deg } A = \text{index\_of } s$ .
192
193 Axiom deg_conv_invariant : forall  $\Gamma$  M A B s,
194    $\Gamma \vdash M \in A \rightarrow A =_b B \rightarrow \Gamma \vdash B \in \text{t\_sort } s \rightarrow \text{deg } A = \text{deg } B$ .
195
196 Lemma typing_pi_subject_bvar_tag_ok : forall  $\Gamma$  M T,
197    $\Gamma \vdash M \in T \rightarrow \text{forall } s \text{ A B, } M = \text{t\_pi } s \text{ A B} \rightarrow \text{bvar\_tag\_ok } 0 \text{ s B}$ .
198 Proof.
199   intros  $\Gamma$  M T Htyp.
200   induction Htyp as
201     [ sa sb HA
202     |  $\Gamma_0 \text{ x0 A0 s0 Hfresh0 HA0 IHA0}$ 
203     |  $\Gamma_0 \text{ x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0}$ 
204     |  $\Gamma_0 \text{ A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR}$ 
205     |  $\Gamma_0 \text{ A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi}$ 
206     |  $\Gamma_0 \text{ M0 N0 A0 B0 s1a HM IHM HN IHN}$ 
207     |  $\Gamma_0 \text{ M0 A0 B0 sd HM IHM Heq HB IHB } ]$ ;
208   intros s A B HeqM; try discriminate.
209   - apply (IHM0 s A B HeqM).
210   - injection HeqM as Hs HA' HB'. subst. exact HTagB.
211   - apply (IHM s A B HeqM).
212 Qed.
213
214 Lemma deg_typing_succ : forall  $\Gamma$  M N,  $\Gamma \vdash M \in N \rightarrow \text{deg } N = \text{deg } M + 1$ .
215 Proof.
216   intros  $\Gamma$  M N Htyp.
217   induction Htyp as
218     [ sa sb HA
219     |  $\Gamma_0 \text{ x0 A0 s0 Hfresh0 HA0 IHA0}$ 
220     |  $\Gamma_0 \text{ x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0}$ 
221     |  $\Gamma_0 \text{ A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR}$ 
222     |  $\Gamma_0 \text{ A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi}$ 
223     |  $\Gamma_0 \text{ M0 N0 A0 B0 s1a HM IHM HN IHN}$ 
224     |  $\Gamma_0 \text{ M0 A0 B0 sd HM IHM Heq HB IHB } ]$ ;
225   simpl in *.
226
227   - (* axiom *)
228     apply A_spec in HA as [Hlt Heq2]. lia.
229
230   - (* var *)
231     unfold deg in IHA0. simpl in IHA0.
232     pose proof (index_range s0) as [Hr1 Hr2]. unfold deg. lia.
233
234   - (* weak *)
235     exact IHM0.
236
237   - (* pi *)

```

```

238 remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
239 assert (Hx0L : ~ In x0 L).
240 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
241   apply in_or_app. left. exact Hc. }
242 specialize (IHB x0 Hx0L). unfold deg in IHB. simpl in IHB.
243 rewrite (deg_open B0 x0 s1 HTagB).
244 pose proof (R_shape s1 s2 s3 HR) as Hshape. subst s3. unfold deg.
245 lia.
246
247 - (* lam *)
248 remember (fresh (L ++ fv M0 ++ fv B0)) as x0 eqn:Hx0def.
249 assert (Hx0L : ~ In x0 L).
250 { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0 ++ fv
251   B0)).
252   apply in_or_app. left. exact Hc. }
253 assert (HtypM :  $\Gamma_0 ++ [(x0, (s1, A0))]$   $\vdash$  open_var M0 s1 x0  $\in$ 
254   open_var B0 s1 x0)
255   by (apply HM; auto).
256 specialize (IHM x0 Hx0L). unfold deg in IHM. simpl in IHM.
257 rewrite (deg_open M0 x0 s1 HTagM).
258 assert (HTagB0 : bvar_tag_ok 0 s1 B0)
259   by (apply (typing_pi_subject_bvar_tag_ok  $\Gamma_0$  (t_pi s1 A0 B0)
260     (t_sort s3) HPi
261       s1 A0 B0 eq_refl)).
262 rewrite (deg_open B0 x0 s1 HTagB0). unfold deg.
263 lia.
264
265 - (* app *)
266 pose proof (type_correctness  $\Gamma_0$  M0 (t_pi sla A0 B0) HM) as Hcor.
267 destruct Hcor as [[s Hs] | [s Hs]].
268 + discriminate Hs.
269 + destruct (generation_pi  $\Gamma_0$  sla A0 B0 (t_sort s) Hs)
270   as [s2 [s3 [L [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
271 assert (Hdeg_A0 : deg A0 = index_of sla) by (apply
272   (deg_sort_typed  $\Gamma_0$  A0 sla HB1)).
273 assert (Hdeg_N0 : deg N0 = index_of sla - 1) by (unfold deg in
274   *; lia).
275 pose proof (typing_lc  $\Gamma_0$  N0 A0 HN) as [HlcN0 _].
276 remember (fresh (fv B0)) as x0 eqn:Hx0def.
277 assert (Hx0B : ~ In x0 (fv B0)).
278 { rewrite Hx0def. apply fresh_notin. }
279 assert (Heqterm : B0 ^^ N0 = (open_var B0 sla x0) [ x0 = N0 ])
280   by (apply subst_intro; [exact Hx0B | exact HlcN0]).
281 assert (Hdegsub : deg ((open_var B0 sla x0) [ x0 = N0 ]) = deg
282   (open_var B0 sla x0))
283   by (apply deg_subst_open; [exact Hx0B | exact HTag1 | exact
284     HlcN0 | exact Hdeg_N0]).
285 assert (Hdegopen : deg B0 = deg (open_var B0 sla x0))
286   by (apply deg_open; exact HTag1).

```

```

280     unfold deg. rewrite Heqterm.
281     unfold deg in Hdegsub, Hdegopen.
282     rewrite Hdegsub. rewrite <- Hdegopen.
283     exact IHM.
284
285 - (* conv *)
286     pose proof (deg_conv_invariant  $\Gamma_0$  M0 A0 B0 sd HM Heq HB) as Hcv.
287     unfold deg in *. lia.
288 Qed.
289
290 Lemma deg_le_top : forall t, deg t <= n + 1.
291 Proof.
292     induction t as [s | s_n0 n0 | sv v | P IHP Q IHQ | s A IHA M IHM | s
293     A IHA B IHB];
294     simpl; auto.
295 - pose proof (index_range s) as [_ H]. lia.
296 - pose proof (index_range s_n0) as [_ H]. lia.
297 - pose proof (index_range sv) as [_ H]. lia.
298 Qed.
299
300 Lemma deg_top : forall M, (derivable M) ->
301 (deg M = n + 1 <-> exists s, M = t_sort s /\ index_of s = n).
302 Proof.
303     intros M [ $\Gamma$  [N Htyp]]. split.
304
305 - intros Hdeg. induction Htyp as
306     [ sa sb HA
307     |  $\Gamma_0$  x0 A0 s0 Hfresh0 HA0 IHA0
308     |  $\Gamma_0$  x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
309     |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
310     |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
311     |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
312     |  $\Gamma_0$  M0 A0 B0 sd HM IHM Heq HB IHB ].
313 + exists sa. split; [reflexivity |]. unfold deg in Hdeg. simpl in
314 Hdeg. lia.
315 + unfold deg in Hdeg. simpl in Hdeg.
316     pose proof (index_range s0) as [H1 H2]. lia.
317 + apply IHM0. exact Hdeg.
318 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
319     assert (Hsucc : deg (t_sort s3) = deg (t_pi s1 A0 B0) + 1)
320     by (apply deg_typing_succ with  $\Gamma_0$ ;
321         apply (typing_pi  $\Gamma_0$  A0 B0 s1 s2 s3 L HA HTagB HB HR)).
322     unfold deg in Hsucc. simpl in Hsucc.
323     pose proof (index_range s3) as [_ Hs2]. lia.
324 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
325     assert (Hsucc1 : deg (t_pi s1 A0 B0) = deg (t_lam s1 A0 M0) + 1)
326     by (apply deg_typing_succ with  $\Gamma_0$ ;
327         apply (typing_lam  $\Gamma_0$  A0 M0 B0 s1 s3 L HA HTagM HM HPi)).
328     unfold deg in Hsucc1. simpl in Hsucc1.
329     pose proof (deg_le_top B0) as HB'. unfold deg in *. lia.

```

```

328 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
329   assert (Hsucc : deg (t_pi s1a A0 B0) = deg M0 + 1)
330     by (apply deg_typing_succ with  $\Gamma_0$ ; exact HM).
331   unfold deg in Hsucc. simpl in Hsucc.
332   pose proof (deg_le_top B0) as HB'. unfold deg in *. lia.
333 + apply IHM. exact Hdeg.
334
335 - intros [s [Heq Hidx]]. subst M. unfold deg. simpl. lia.
336 Qed.
337
338 Lemma deg_top_type : forall M, (derivable M) ->
339   deg M = n <-> exists  $\Gamma$  s, ( $\Gamma \vdash M \in \text{t\_sort } s \wedge \text{index\_of } s = n$ ).
340 Proof.
341   intros M [ $\Gamma$  [N Htyp]]. split.
342
343 - intros Hdeg. induction Htyp as
344   [ sa sb HA
345   |  $\Gamma_0$  x0 A0 s0 Hfresh0 HA0 IHA0
346   |  $\Gamma_0$  x0 B0 M0 A0 s0 Hfresh0 HM0 IHM0 HB0 IHB0
347   |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
348   |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
349   |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
350   |  $\Gamma_0$  M0 A0 B0 sd HM IHM Heq HB IHB ].
351
352 + exists [], sb. split.
353   * apply typing_axiom. exact HA.
354   * pose proof (A_spec sa sb) as [HAiff _]. apply HAiff in HA as
355     [_ HAeq].
356     unfold deg in Hdeg. simpl in Hdeg. lia.
357
358 + pose proof (deg_typing_succ  $\Gamma_0$  A0 (t_sort s0) HA0) as Hs.
359   unfold deg in *. simpl in *.
360   pose proof (index_range s0). lia.
361
362 + apply IHM0. apply Hdeg.
363
364 + exists  $\Gamma_0$ , s3. split.
365   * apply (typing_pi  $\Gamma_0$  A0 B0 s1 s2 s3 L HA HTagB HB HR).
366   * pose proof (deg_typing_succ  $\Gamma_0$  (t_pi s1 A0 B0) (t_sort s3)
367     (typing_pi  $\Gamma_0$  A0 B0 s1 s2 s3 L HA HTagB HB HR))
368     as Hsucc.
369     unfold deg in Hsucc, Hdeg. simpl in Hsucc, Hdeg. lia.
370
371 + exfalso.
372   assert (Hsucc1 : deg (t_pi s1 A0 B0) = deg (t_lam s1 A0 M0) + 1)
373     by (apply deg_typing_succ with  $\Gamma_0$ ;
374       apply (typing_lam  $\Gamma_0$  A0 M0 B0 s1 s3 L HA HTagM HM HPi)).
375   unfold deg in Hsucc1, Hdeg. simpl in Hsucc1, Hdeg.
376   assert (Htop : deg (t_pi s1 A0 B0) = n + 1) by (unfold deg;
377     simpl; lia).

```

```

375   assert (Hderiv : derivable (t_pi s1 A0 B0)).
376   { unfold derivable. exists  $\Gamma$ 0. exists (t_sort s3). exact HPi. }
377   destruct (proj1 (deg_top (t_pi s1 A0 B0) Hderiv) Htop) as [s'
    [Heqs' _]].
378   discriminate Heqs'.
379
380 + exfalso.
381   unfold deg in Hdeg. simpl in Hdeg.
382   assert (Hsucc : deg (t_pi sla A0 B0) = deg M0 + 1)
383     by (apply deg_typing_succ with  $\Gamma$ 0; exact HM).
384   assert (Htop : deg (t_pi sla A0 B0) = n + 1) by (unfold deg in
    *; simpl in *; lia).
385   pose proof (type_correctness  $\Gamma$ 0 M0 (t_pi sla A0 B0) HM) as Hcor.
386   destruct Hcor as [[s Hs] | [s Hs]].
387   * discriminate Hs.
388   * assert (Hderiv : derivable (t_pi sla A0 B0)).
389     { unfold derivable. exists  $\Gamma$ 0. exists (t_sort s). exact Hs. }
390     destruct (proj1 (deg_top (t_pi sla A0 B0) Hderiv) Htop) as [s'
        [Heqs' _]].
391     discriminate Heqs'.
392
393 + apply IHM. exact Hdeg.
394
395 - intros [ $\Gamma'$  [s [H1 H2]]].
396   pose proof (deg_typing_succ  $\Gamma'$  M (t_sort s) H1). unfold deg in *.
    simpl in *. lia.
397 Qed.
398
399 Lemma deg_typing : forall M, derivable M -> forall i, (i > 0) -> (i <
    n) ->
400   (deg M = i <-> (exists  $\Gamma$  N s, ( $\Gamma \vdash M \in N$ ) /\ ( $\Gamma \vdash N \in$  t_sort s) /\
    (index_of s = i + 1))).
401 Proof.
402   intros M [ $\Gamma$  [T Htyp]] i Hi1 Hi2. split.
403
404 - intros Hdeg. pose proof (deg_typing_succ  $\Gamma$  M T Htyp).
405   pose proof (type_correctness  $\Gamma$  M T Htyp) as Hcor. destruct Hcor as
    [[s Hs] | [s Hs]].
406   + subst T. unfold deg in *. simpl in *. pose proof (index_range
    s). assert (Gi : index_of s = i) by lia.
407     assert (Hi : 0 < i + 1 < n + 1) by lia.
408     pose proof (index_surj (i + 1) Hi) as Hs'. destruct Hs' as [s'
        Hs'].
409     assert (Hss' : A s s') by (apply A_spec; lia).
410     exists  $\Gamma$ , (t_sort s), s'. repeat split; auto. apply start_axiom;
        auto.
411     exists M, (t_sort s); auto.
412   + pose proof (deg_typing_succ  $\Gamma$  T (t_sort s) Hs). unfold deg in *.
    simpl in *.

```



```

413     exists  $\Gamma$ , T, s. repeat split; auto; lia.
414
415   - intros [ $\Gamma'$  [N [s [H1 [H2 H3]]]]].
416     pose proof (deg_typing_succ  $\Gamma'$  M N H1) as Hdeg1.
417     pose proof (deg_typing_succ  $\Gamma'$  N (t_sort s) H2) as Hdeg2.
418     unfold deg in *. simpl in *. lia.
419 Qed.
420
421 Axiom deg_beta : forall M N, (M ->>b N) -> (deg M = deg N).
422
423 (* =====
424 *)
425 (* Full System *)
426 Definition full_with (i j : nat) : Prop :=
427   forall l k : Sort,
428     index_of l <= i ->
429     index_of l <= index_of k ->
430     index_of k <= j ->
431     R l k k.
432
433 Definition full : Prop :=
434   forall s1 s2 s3, R s1 s2 s3 -> full_with (index_of s2) (index_of
435     s1).
436
437 (* For the remainder of this section, fix a full n-tiered pure type
438 system  $\lambda S$ . *)
439 Axiom is_full : full.
440
441 Definition R_star (s s' s'' : Sort) : Prop :=
442   R s s' s'' /\ index_of s' <= index_of s.
443
444 (* =====
445 *)
446 (* The non-dependent typing judgement *)
447
448 Reserved Notation " $\Gamma \vdash^* M \in A$ " (at level 70, no associativity).
449
450 Inductive typing_star : context -> term -> term -> Prop :=
451   | typing_star_axiom : forall s s',
452     A s s' ->
453     []  $\vdash^*$  t_sort s  $\in$  t_sort s'
454   | typing_star_var : forall  $\Gamma$  x A s,
455     is_fresh x  $\Gamma$  ->
456      $\Gamma \vdash^*$  A  $\in$  t_sort s ->
457      $\Gamma \mathrel{++} [(x, (s, A))]$   $\vdash^*$  t_fvar s x  $\in$  A
458   | typing_star_weak : forall  $\Gamma$  x B M A s,

```

```

458   is_fresh x  $\Gamma$  ->
459    $\Gamma \vdash^* M \in A$  ->
460    $\Gamma \vdash^* B \in \text{t\_sort } s$  ->
461    $\Gamma ++ [(x, (s, B))] \vdash^* M \in A$ 
462
463 | typing_star_pi : forall  $\Gamma$  A B s1 s2 s3 L,
464    $\Gamma \vdash^* A \in \text{t\_sort } s1$  ->
465   (forall x, ~ In x L ->
466      $\Gamma ++ [(x, (s1, A))] \vdash^* (\text{open\_var } B \text{ s1 } x) \in \text{t\_sort } s2$ ) ->
467   R_star s1 s2 s3 ->
468    $\Gamma \vdash^* (\text{t\_pi } s1 \text{ A } B) \in (\text{t\_sort } s3)$ 
469
470 | typing_star_lam : forall  $\Gamma$  A M B s1 s3 L,
471    $\Gamma \vdash^* A \in \text{t\_sort } s1$  ->
472   (forall x, ~ In x L ->
473      $\Gamma ++ [(x, (s1, A))] \vdash^* (\text{open\_var } M \text{ s1 } x) \in (\text{open\_var } B \text{ s1 } x)$ )
474     ->
475      $\Gamma \vdash^* (\text{t\_pi } s1 \text{ A } B) \in (\text{t\_sort } s3)$  ->
476      $\Gamma \vdash^* (\text{t\_lam } s1 \text{ A } M) \in (\text{t\_pi } s1 \text{ A } B)$ 
477
478 | typing_star_app : forall  $\Gamma$  M N A B s1,
479    $\Gamma \vdash^* M \in \text{t\_pi } s1 \text{ A } B$  ->
480    $\Gamma \vdash^* N \in A$  ->
481    $\Gamma \vdash^* \text{t\_app } M \text{ N} \in (B \text{ ^^ } N)$ 
482
483 | typing_star_conv : forall  $\Gamma$  M A B s,
484    $\Gamma \vdash^* M \in A$  ->
485   A =b B ->
486    $\Gamma \vdash^* B \in \text{t\_sort } s$  ->
487    $\Gamma \vdash^* M \in B$ 
488
489 where " $\Gamma \vdash^* M \in A$ " := (typing_star  $\Gamma$  M A).
490
491 Definition legal_star ( $\Gamma$  : context) : Prop :=
492   exists M N,  $\Gamma \vdash^* M \in N$ .
493
494 Definition derivable_star (M : term) : Prop :=
495   exists  $\Gamma$  N,  $\Gamma \vdash^* M \in N$ .
496
497 Definition partial_sort_of (i : nat) (Hi : 0 < i < n + 1) : Sort :=
498   proj1_sig (
499     constructive_indefinite_description
500       (fun s => index_of s = i)
501       (index_surj i Hi)
502   ).
503
504 Lemma one_in_range : 0 < 1 /\ 1 < n + 1.
505 Proof. pose proof n_range. lia. Qed.
506
507 Definition sort_of (i : nat) : Sort :=

```

```

507   match (lt_dec 0 i, lt_dec i (n+1)) with
508   | (left H1, left H2) => partial_sort_of i (conj H1 H2)
509   | _ => partial_sort_of 1 one_in_range
510   end.
511
512   (* =====
513   *)
514   (* Type-Level Translation *)
515   (*  $\rho \ i : T_{\geq i+1} \rightarrow T_{i+1}$  *)
516   Definition zero_var : var := 0.
517
518   Lemma zero_var_retag_ne : forall (x : var) (s : Sort),
519     retag x s <> zero_var.
520   Proof. intros x s. unfold zero_var. apply (retag_neq_mult 0). Qed.
521
522   Axiom typing_avoids_zero_var :
523     forall  $\Gamma$  M N,  $\Gamma \vdash M \in N \rightarrow$ 
524     forall x sA A, In (x, (sA, A))  $\Gamma \rightarrow x <> zero\_var$ .
525
526   Fixpoint rho_pi_tel (r : nat -> term -> term) (A B : term) (i k : nat)
527   : term :=
528     match k with
529     | 0      => t_pi (sort_of i) (r i A) (r i B)
530     | S k'  => t_pi (sort_of (i + S k')) (r (i + S k') A) (rho_pi_tel r A
531       B i k')
532     end.
533
534   Fixpoint rho_lam_tel (r : nat -> term -> term) (A M' : term) (i k :
535     nat) : term :=
536     match k with
537     | 0      => t_lam (sort_of (i + 1)) (r (i + 1) A) (r i M')
538     | S k'  => t_lam (sort_of (i + 1 + S k')) (r (i + 1 + S k') A)
539       (rho_lam_tel r A M' i k')
540     end.
541
542   Fixpoint rho_app_tel (r : nat -> term -> term) (M' N : term) (i k :
543     nat) : term :=
544     match k with
545     | 0      => t_app (r i M') (r i N)
546     | S k'  => t_app (rho_app_tel r M' N i k') (r (i + S k') N)
547     end.
548
549   Fixpoint rho (i : nat) (M : term) : term :=
550     match M with
551     | t_sort s =>
552       match i with
553       | 0 => t_fvar (sort_of 2) zero_var
554       | _ => t_sort (sort_of i)

```

```

550   end
551   | t_bvar s0 k => t_bvar s0 k
552   | t_fvar s0 x =>
553     t_fvar (sort_of (i + 2)) (retag x (sort_of (i + 2)))
554   | t_pi s A B =>
555     match le_lt_dec (i + 1) (deg A) with
556     | left _ => rho_pi_tel rho A B i (deg A - 1 - i)
557     | right _ => rho i B
558     end
559   | t_lam s A M' =>
560     match le_lt_dec (i + 2) (deg A) with
561     | left _ => rho_lam_tel rho A M' i (deg A - 1 - (i + 1))
562     | right _ => rho i M'
563     end
564   | t_app M' N =>
565     match le_lt_dec (i + 1) (deg N) with
566     | left _ => rho_app_tel rho M' N i (deg N - 1 - i)
567     | right _ => rho i M'
568     end
569   end.
570
571 Lemma rho_pi_named : forall i s Dom B, i + 1 <= deg Dom ->
572   rho i (t_pi s Dom B) = rho_pi_tel rho Dom B i (deg Dom - 1 - i).
573 Proof.
574   intros i s Dom B Hdeg. simpl.
575   destruct (le_lt_dec (i + 1) (deg Dom)); [reflexivity | lia].
576 Qed.
577
578 Lemma rho_lam_named : forall i s Dom B, i + 2 <= deg Dom ->
579   rho i (t_lam s Dom B) = rho_lam_tel rho Dom B i (deg Dom - 1 - (i + 1)).
580 Proof.
581   intros i s Dom B Hdeg. simpl.
582   destruct (le_lt_dec (i + 2) (deg Dom)); [reflexivity | lia].
583 Qed.
584
585 Lemma rho_app_named : forall i P Q, i + 1 <= deg Q ->
586   rho i (t_app P Q) = rho_app_tel rho P Q i (deg Q - 1 - i).
587 Proof.
588   intros i P Q Hdeg. simpl.
589   destruct (le_lt_dec (i + 1) (deg Q)); [reflexivity | lia].
590 Qed.
591
592 Fixpoint rho_ctx_tel (x : var) (s : Sort) (T : term) (k : nat) :
593   context :=
594   let j := index_of s in
595   match k with
596   | 0 => [(retag x (sort_of j), (sort_of j, rho (j - 1) T))]
597   | S k' => (rho_ctx_tel x s T k')

```

```

597         ++ [(retag x (sort_of (j - k)), (sort_of (j - k), rho (j -
598             k - 1) T))]
599     end.
600 Fixpoint rho_ctx (i : nat) (Γ : context) : context :=
601     match Γ with
602     | [] => [(zero_var, (sort_of 2, t_sort (sort_of 1)))]
603     | (x, (s, T)) :: Γ' => (rho_ctx_tel x s T (index_of s - i - 1)) ++
        rho_ctx i Γ'
604     end.
605
606 Lemma subcontext_append :
607     forall Γ1 Δ1 Γ2 Δ2,
608     (Γ1 ⊆ Δ1) -> (Γ2 ⊆ Δ2) -> ((Γ1 ++ Γ2) ⊆ (Δ1 ++ Δ2)).
609 Proof.
610     intros Γ1 Δ1 Γ2 Δ2 H1 H2 x T Hin.
611     apply in_app_or in Hin. destruct Hin; apply in_or_app.
612     - left; auto.
613     - right; auto.
614 Qed.
615
616 Lemma rho_ctx_tel_step : forall x s T k,
617     rho_ctx_tel x s T k ⊆ rho_ctx_tel x s T (S k).
618 Proof.
619     intros x s T k y T' Hin.
620     simpl. apply in_or_app. left. exact Hin.
621 Qed.
622
623 Lemma rho_ctx_tel_sub :
624     forall x s T p q,
625     p < q -> rho_ctx_tel x s T p ⊆ rho_ctx_tel x s T q.
626 Proof.
627     intros x s T p q Hpq.
628     remember (q - p - 1) as d eqn:Hd.
629     assert (Hq : q = p + S d) by lia.
630     subst q. clear Hpq Hd.
631     induction d as [| d IH].
632     - replace (p + 1) with (S p) by lia.
633       apply rho_ctx_tel_step.
634     - replace (p + S (S d)) with (S (p + S d)) by lia.
635       intros y T' Hin.
636       apply rho_ctx_tel_step.
637       apply IH. exact Hin.
638 Qed.
639
640 Definition ctx_above (b : nat) (Γ : context) : Prop :=
641     forall x s T, In (x, (s, T)) Γ -> b < index_of s.
642
643 Lemma rho_ctx_sub : forall Γ i j,
644     i < j -> ctx_above j Γ -> rho_ctx j Γ ⊆ rho_ctx i Γ.

```

```

645 Proof.
646   induction  $\Gamma$  as [| [y [s T]]  $\Gamma'$  IH]; intros i j Hij Habove.
647   - simpl. unfold subcontext. auto.
648   - simpl. apply subcontext_append.
649     + apply rho_ctx_tel_sub.
650       assert (Hy : j < index_of s).
651       { apply Habove with y T. left. reflexivity. }
652       lia.
653     + apply IH; auto. intros x' s' T' Hin. apply Habove with x' T'.
        right. exact Hin.
654 Qed.
655
656 (* =====
657 *)
658 (* rho commutes with substitution *)
659 Fixpoint rho_subst_tel (M N : term) (varidx i k : nat) : term :=
660   match k with
661   | 0 => M [ retag varidx (sort_of (i + 2)) = rho i N ]
662   | S k' => rho_subst_tel (M [ retag varidx (sort_of (k + i + 2)) =
        rho (k + i) N ]) N varidx i k'
663   end.
664
665 Definition rho_subst (M N : term) (x : var) (s : Sort) (i : nat) :
term :=
666   let j := index_of s in
667   match lt_dec j (i + 2) with
668   | left _ => rho i M
669   | right _ => rho_subst_tel (rho i M) (N) (x) (i) (j - i - 2)
670   end.
671
672 Lemma sort_of_correct : forall i, 0 < i -> i < n + 1 -> index_of
(sort_of i) = i.
673 Proof.
674   intros i H1 H2. unfold sort_of.
675   destruct (lt_dec 0 i) as [Hd1|Hd1]; [| lia].
676   destruct (lt_dec i (n+1)) as [Hd2|Hd2]; [| lia].
677   unfold partial_sort_of.
678   destruct (constructive_indefinite_description (fun s => index_of s =
        i)
        (index_surj i (conj Hd1 Hd2))) as [s Hs].
679   exact Hs.
680 Qed.
681
682 Lemma sort_of_inj : forall p q,
683   0 < p -> p < n + 1 -> 0 < q -> q < n + 1 ->
684   sort_of p = sort_of q -> p = q.
685 Proof.
686   intros p q Hp1 Hp2 Hq1 Hq2 Heq.

```

```

688   assert (Hp : index_of (sort_of p) = p) by (apply sort_of_correct;
689   lia).
689   assert (Hq : index_of (sort_of q) = q) by (apply sort_of_correct;
690   lia).
690   rewrite Heq in Hp. rewrite Hp in Hq. exact Hq.
691 Qed.
692
693 Lemma index_of_sort_of_any : forall p, index_of (sort_of p) = 1 \ /
694 index_of (sort_of p) = p.
694 Proof.
695   intros p. unfold sort_of.
696   destruct (lt_dec 0 p) as [H1 | H1].
697   - destruct (lt_dec p (n + 1)) as [H2 | H2].
698     + right. unfold partial_sort_of.
699       destruct (constructive_indefinite_description (fun s => index_of
700         s = p)
701         (index_surj p (conj H1 H2))) as [s Hs].
702       exact Hs.
703     + left. unfold partial_sort_of.
704       destruct (constructive_indefinite_description (fun s => index_of
705         s = 1)
706         (index_surj 1 one_in_range)) as [s Hs].
707       exact Hs.
708     - left. unfold partial_sort_of.
709       destruct (constructive_indefinite_description (fun s => index_of s
710         = 1)
711         (index_surj 1 one_in_range)) as [s Hs].
712       exact Hs.
710 Qed.
711
712 Lemma sort_of_tier_neq : forall D m, m < D -> m + 2 <= n -> sort_of (D
713 + 2) <> sort_of (m + 2).
713 Proof.
714   intros D m Hmd Hmn Heq.
715   assert (Hidx : index_of (sort_of (D + 2)) = index_of (sort_of (m +
716     2))) by (rewrite Heq; reflexivity).
717   rewrite (sort_of_correct (m + 2) ltac:(lia) ltac:(lia)) in Hidx.
718   destruct (index_of_sort_of_any (D + 2)) as [H | H]; lia.
719 Qed.
720
721 Lemma rho_min_tier_noop : forall M D m v N',
722   m < D -> m + 2 <= n ->
723   (rho D M) [ retag v (sort_of (m + 2)) = N' ] = rho D M.
723 Proof.
724   induction M as [s | s_k k | sy y | P IHP Q IHQ | s A IHA M' IHM' | s
725     A IHA B IHB];
726   intros D m v N' Hmd Hmn.
727   - (* t_sort s *)
728     destruct D as [| D']; [lia | simpl; reflexivity].

```

```

728 - (* t_bvar k *)
729   simpl; reflexivity.
730 - (* t_fvar sy y *)
731   simpl.
732   destruct (eq_var_dec (retag y (sort_of (D + 2))) (retag v (sort_of
    (m + 2))))
733     as [Heq | Hne]; [ | reflexivity].
734   exfalso. apply retag_inj in Heq as [_ Hsort].
735   exact (sort_of_tier_neq D m Hmd Hmn Hsort).
736 - (* t_app P Q *)
737   simpl.
738   destruct (le_lt_dec (D + 1) (deg Q)) as [Hle | Hlt].
739   + generalize (deg Q - 1 - D) as k.
740     induction k as [| k' IHk]; intros.
741     * simpl. f_equal.
742       -- apply IHP; lia.
743       -- apply IHQ; lia.
744     * simpl. f_equal.
745       -- exact IHk.
746       -- apply IHQ; lia.
747   + apply IHP; lia.
748 - (* t_lam s A M' *)
749   simpl.
750   destruct (le_lt_dec (D + 2) (deg A)) as [Hle | Hlt].
751   + generalize (deg A - 1 - (D + 1)) as k.
752     induction k as [| k' IHk]; intros.
753     * simpl. f_equal.
754       -- apply IHA; lia.
755       -- apply IHM'; lia.
756     * simpl. f_equal.
757       -- apply IHA; lia.
758       -- exact IHk.
759   + apply IHM'; lia.
760 - (* t_pi s A B *)
761   simpl.
762   destruct (le_lt_dec (D + 1) (deg A)) as [Hle | Hlt].
763   + generalize (deg A - 1 - D) as k.
764     induction k as [| k' IHk]; intros.
765     * simpl. f_equal.
766       -- apply IHA; lia.
767       -- apply IHB; lia.
768     * simpl. f_equal.
769       -- apply IHA; lia.
770       -- exact IHk.
771   + apply IHB; lia.
772 Qed.
773
774 Lemma eq_var_dec_refl : forall v : var, exists e, eq_var_dec v v =
left e.
775 Proof.

```



```

776   intros v. destruct (eq_var_dec v v) as [e | f].
777   - exists e. reflexivity.
778   - exfalso. apply f. reflexivity.
779 Qed.
780
781 Lemma subst_high_preserves_clean_low : forall P vhi R vlo,
782   vhi <> vlo ->
783   (forall N0, R [ vlo = N0 ] = R) ->
784   (forall N0, P [ vlo = N0 ] = P) ->
785   forall N1, (P [ vhi = R ]) [ vlo = N1 ] = P [ vhi = R ].
786 Proof.
787   induction P as [s | s_k k | sy y | P1 IHP1 P2 IHP2 | s A IHA M' IHM'
788   | s A IHA B IHB];
789   intros vhi R vlo Hne HRclean HPclean N1.
790   - reflexivity.
791   - reflexivity.
792   - rewrite subst_var. destruct (eq_var_dec y vhi) as [Heq | Hneq].
793     + apply HRclean.
794     + rewrite subst_var. destruct (eq_var_dec y vlo) as [Heq2 |
795       Hneq2].
796       * subst y. specialize (HPclean N1). rewrite subst_var in
797         HPclean.
798         destruct (eq_var_dec_refl vlo) as [e Heq]. rewrite Heq in
799         HPclean.
800         destruct (eq_var_dec vlo vhi) as [Hc | Hc]; [congruence |
801           exact HPclean].
802       * reflexivity.
803   - assert (Hclean1 : forall N0, P1 [ vlo = N0 ] = P1).
804     { intros N0. specialize (HPclean N0). rewrite subst_app in
805       HPclean. injection HPclean as H1 H2. exact H1. }
806   - assert (Hclean2 : forall N0, P2 [ vlo = N0 ] = P2).
807     { intros N0. specialize (HPclean N0). rewrite subst_app in
808       HPclean. injection HPclean as H1 H2. exact H2. }
809   - rewrite subst_app. rewrite subst_app. f_equal.
810     + apply IHP1; auto.
811     + apply IHP2; auto.
812   - assert (Hclean1 : forall N0, A [ vlo = N0 ] = A).
813     { intros N0. specialize (HPclean N0). rewrite subst_lam in
814       HPclean. injection HPclean as H1 H2. exact H1. }
815   - assert (Hclean2 : forall N0, M' [ vlo = N0 ] = M').
816     { intros N0. specialize (HPclean N0). rewrite subst_lam in
817       HPclean. injection HPclean as H1 H2. exact H2. }
818   - rewrite subst_lam. rewrite subst_lam. f_equal.
819     + apply IHA; auto.
820     + apply IHM'; auto.
821   - assert (Hclean1 : forall N0, A [ vlo = N0 ] = A).
822     { intros N0. specialize (HPclean N0). rewrite subst_pi in HPclean.
823       injection HPclean as H1 H2. exact H1. }
824   - assert (Hclean2 : forall N0, B [ vlo = N0 ] = B).

```

```

815 { intros N0. specialize (HPclean N0). rewrite subst_pi in HPclean.
      injection HPclean as H1 H2. exact H2. }
816 rewrite subst_pi. rewrite subst_pi. f_equal.
817 + apply IHA; auto.
818 + apply IHB; auto.
819 Qed.
820
821 Fixpoint only_tagged (x : var) (s : Sort) (t : term) : Prop :=
822   match t with
823   | t_sort _      => True
824   | t_bvar _ _    => True
825   | t_fvar s0 y   => y = x -> s0 = s
826   | t_app P Q     => only_tagged x s P /\ only_tagged x s Q
827   | t_pi _ A B    => only_tagged x s A /\ only_tagged x s B
828   | t_lam _ A M   => only_tagged x s A /\ only_tagged x s M
829   end.
830
831 Lemma deg_subst_tagged : forall C x s N,
832   only_tagged x s C -> lc N -> deg N = index_of s - 1 ->
833   deg (C [ x = N ]) = deg C.
834 Proof.
835   induction C as [s0 | s_k k | s0 y | P IHP Q IHQ | s1 A IHA M IHM |
      s1 A IHA B IHB];
836   intros x s N Htag HlcN HdegN; simpl in *.
837   - reflexivity.
838   - reflexivity.
839   - destruct (eq_var_dec y x) as [Heq | Hneq].
840     + subst y. rewrite (Htag eq_refl). exact HdegN.
841     + reflexivity.
842   - destruct Htag as [HtagP _]. apply (IHP x s N HtagP HlcN HdegN).
843   - destruct Htag as [_ HtagM]. apply (IHM x s N HtagM HlcN HdegN).
844   - destruct Htag as [_ HtagB]. apply (IHB x s N HtagB HlcN HdegN).
845   Qed.
846
847 Lemma rho_subst_tel_noop_generic : forall P N varidx D i0 K,
848   i0 + K + 2 <= n ->
849   (forall m v N0, m < D -> m + 2 <= n -> P [ retag v (sort_of (m + 2))
      = N0 ] = P) ->
850   i0 + K + 1 <= D ->
851   rho_subst_tel P N varidx i0 K = P.
852 Proof.
853   intros P N varidx D i0 K HKn HcleanP.
854   revert i0 HKn HcleanP. induction K as [| K' IHK]; intros i0 HKn
      HcleanP Htop.
855   - simpl. apply HcleanP; lia.
856   - simpl.
857     assert (HS1 : S (K' + i0) = (S K' + i0)) by lia. rewrite HS1.
858     assert (HS2 : S (K' + i0 + 2) = S K' + i0 + 2) by lia. rewrite
      HS2.

```

```

859     rewrite (HcleanP (S K' + i0) varidx (rho (S K' + i0) N)); [ | lia
      | lia].
860     apply IHK; [lia | exact HcleanP | lia].
861 Qed.
862
863 Lemma rho_subst_tel_stays_clean : forall P N varidx D i0 K,
864   D <= i0 ->
865   (forall m v N0, m < D -> m + 2 <= n -> P [ retag v (sort_of (m + 2))
    = N0 ] = P) ->
866   forall m v N0, m < D -> m + 2 <= n ->
867   (rho_subst_tel P N varidx i0 K) [ retag v (sort_of (m + 2)) = N0 ]
    = rho_subst_tel P N varidx i0 K.
868 Proof.
869   intros P N varidx D i0 K. revert P.
870   induction K as [| K' IHK]; intros P Hle HcleanP m v N0 Hmd Hmn.
871   - simpl. apply subst_high_preserves_clean_low.
872     + intro Hc. apply retag_inj in Hc as [_ Hsc]. apply
      (sort_of_tier_neq i0 m); [lia | lia | exact Hsc].
873     + intros N1. apply rho_min_tier_noop; lia.
874     + intros N1. apply HcleanP; auto.
875   - simpl. apply IHK.
876     + lia.
877     + intros m' v' N0' Hmd' Hmn'. apply
      subst_high_preserves_clean_low.
878       * intro Hc. apply retag_inj in Hc as [_ Hsc]. apply
        (sort_of_tier_neq (S K' + i0) m'); [lia | lia | exact Hsc].
879       * intros N1. apply rho_min_tier_noop; lia.
880       * intros N1. apply HcleanP; auto.
881     + exact Hmd.
882     + exact Hmn.
883 Qed.
884
885 Lemma rho_subst_tel_split : forall P N varidx i0 K D,
886   i0 < D -> D <= i0 + K ->
887   rho_subst_tel P N varidx i0 K
888   = rho_subst_tel (rho_subst_tel P N varidx D (i0 + K - D)) N varidx
    i0 (D - i0 - 1).
889 Proof.
890   intros P N varidx i0 K. revert P i0.
891   induction K as [| K' IHK]; intros P i0 D Hlt Hle.
892   - lia.
893   - destruct (Nat.eq_dec D (i0 + S K')) as [Heq | Hneq].
894     + subst D.
895       replace (i0 + S K' - (i0 + S K')) with 0 by lia.
896       simpl.
897       replace (i0 + S K' - i0 - 1) with K' by lia.
898       replace (i0 + S K') with (S K' + i0) by lia.
899       reflexivity.
900     + assert (HleK' : D <= i0 + K') by lia.

```

```

901     simpl.
902     assert (HS1 : S (K' + i0 + 2) = S K' + i0 + 2) by lia. rewrite
HS1.
903     assert (HS2 : S (K' + i0) = S K' + i0) by lia. rewrite HS2.
904     rewrite (IHK (P [ retag varidx (sort_of (S K' + i0 + 2)) = rho
(S K' + i0) N ]) i0 D Hlt HleK')).
905     replace (i0 + S K' - D) with (S (i0 + K' - D)) by lia.
906     simpl.
907     f_equal.
908     replace (i0 + K' - D + D + 2) with (K' + i0 + 2) by lia.
909     replace (i0 + K' - D + D) with (K' + i0) by lia.
910     reflexivity.
911 Qed.
912
913 Lemma rho_subst_tel_pi_commute : forall N s' A' B' varidx i k,
914   rho_subst_tel (t_pi s' A' B') N varidx i k
915   = t_pi s' (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N varidx
i k).
916 Proof.
917   intros N s' A' B' varidx i k.
918   revert A' B' s'.
919   induction k as [| k' IHk]; intros s' A' B'; simpl; auto.
920 Qed.
921
922 Lemma rho_subst_tel_pi_tel_commute : forall A B N varidx i0 k0 i1 k1,
923   rho_subst_tel (rho_pi_tel rho A B i1 k1) N varidx i0 k0
924   = rho_pi_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0) A
B i1 k1.
925 Proof.
926   intros A B N varidx i0 k0 i1.
927   induction k1 as [| k1' IHk1]; intros.
928   - simpl. apply rho_subst_tel_pi_commute.
929   - simpl. rewrite rho_subst_tel_pi_commute. f_equal. apply IHk1.
930 Qed.
931
932 Lemma rho_subst_tel_app_commute : forall N A' B' varidx i k,
933   rho_subst_tel (t_app A' B') N varidx i k
934   = t_app (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N varidx i
k).
935 Proof.
936   intros N A' B' varidx i k.
937   revert A' B'.
938   induction k as [| k' IHk]; intros A' B'; simpl; auto.
939 Qed.
940
941 Lemma rho_subst_tel_app_tel_commute : forall A B N varidx i0 k0 i1 k1,
942   rho_subst_tel (rho_app_tel rho A B i1 k1) N varidx i0 k0
943   = rho_app_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0)
A B i1 k1.

```

```

944 Proof.
945   intros A B N varidx i0 k0 i1.
946   induction k1 as [| k1' IHk1]; intros.
947   - simpl. apply rho_subst_tel_app_commute.
948   - simpl. rewrite rho_subst_tel_app_commute. f_equal. apply IHk1.
949 Qed.
950
951 Lemma rho_subst_tel_lam_commute : forall N s' A' B' varidx i k,
952   rho_subst_tel (t_lam s' A' B') N varidx i k
953   = t_lam s' (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N
954     varidx i k).
955 Proof.
956   intros N s' A' B' varidx i k.
957   revert A' B' s'.
958   induction k as [| k' IHk]; intros s' A' B'; simpl; auto.
959 Qed.
960
961 Lemma rho_subst_tel_lam_tel_commute : forall A B N varidx i0 k0 i1 k1,
962   rho_subst_tel (rho_lam_tel rho A B i1 k1) N varidx i0 k0
963   = rho_lam_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0)
964     A B i1 k1.
965 Proof.
966   intros A B N varidx i0 k0 i1.
967   induction k1 as [| k1' IHk1]; intros.
968   - simpl. apply rho_subst_tel_lam_commute.
969   - simpl. rewrite rho_subst_tel_lam_commute. f_equal. apply IHk1.
970 Qed.
971
972 Lemma rho_commutes_substitution :
973   forall i, 0 <= i <= n ->
974   forall x s, x <> zero_var -> 1 <= index_of s <= n ->
975   forall M, deg M >= i + 1 -> only_tagged x s M ->
976   forall N, deg N = index_of s - 1 -> lc N ->
977   rho i (M [ x = N ]) = rho_subst M N x s i.
978 Proof.
979   intros i.
980   induction i as [i IHouter] using
981     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n
982       - i))).
983   intros Hi x s Hxnz Hx M.
984   induction M as [s0 | s_m m | sy y | D IHD C IHC | s1 C IHC D IHD |
985     s1 C IHC D IHD];
986     intros Hdeg Honly N HdegN HlcN.
987   - (* M = t_sort s0 *)
988     rewrite subst_sort. unfold rho_subst.
989     destruct (lt_dec (index_of s) (i + 2)) as [E | E]; auto.
990     assert (Hconst : forall k M0,
991       k <= index_of s - i - 2 ->

```

```

989      (exists s0', M0 = t_sort s0') /\ (i = 0 /\ M0 =
990      t_fvar (sort_of 2) zero_var) ->
991      rho_subst_tel M0 N x i k = M0).
992 { induction k as [| k' IHk]; intros M0 Hkbound Hcase.
993   - simpl. destruct Hcase as [[s0' Heq] | [Hi0 Heq]]; subst M0.
994     + rewrite subst_sort. reflexivity.
995     + rewrite subst_var.
996       destruct (eq_var_dec zero_var (retag x (sort_of (i + 2))))
997       as [Heq2 | Hne2]; auto.
998       exfalso. apply (zero_var_retag_ne x (sort_of (i + 2))).
999       symmetry. exact Heq2.
1000   - simpl. destruct Hcase as [[s0' Heq] | [Hi0 Heq]]; subst M0.
1001     + rewrite subst_sort. apply IHk; [lia | left; exists s0';
1002       reflexivity].
1003     + rewrite subst_var.
1004       destruct (eq_var_dec zero_var (retag x (sort_of (k' + i +
1005       2)))) as [Heq2 | Hne2].
1006       * exfalso. apply (zero_var_retag_ne x (sort_of (k' + i +
1007       2))). symmetry. exact Heq2.
1008       * destruct (eq_var_dec zero_var (retag x (sort_of (S (k' + i
1009       + 2))))) as [Heq3 | Hne3].
1010       -- exfalso. apply (zero_var_retag_ne x (sort_of (S (k' + i
1011       + 2)))). symmetry. exact Heq3.
1012       -- apply IHk; [lia | right; split; [exact Hi0 |
1013         reflexivity]].
1014 }
1015 symmetry. apply Hconst; auto.
1016 destruct i as [| i'].
1017   + right. split; auto.
1018   + left. exists (sort_of (S i')). reflexivity.
1019 - (* M = t_bvar m *)
1020 rewrite subst_bvar. unfold rho_subst.
1021 destruct (lt_dec (index_of s) (i + 2)) as [E | E].
1022 + reflexivity.
1023 + assert (Hconst : forall sy0 y0 k, t_bvar sy0 y0 = rho_subst_tel
1024   (t_bvar sy0 y0) N x i k)
1025   by (induction k as [| k' IHk]; auto).
1026   apply Hconst.
1027 - (* M = t_fvar sy y *)
1028 cbn in Hdeg. unfold rho_subst.
1029 destruct (lt_dec (index_of s) (i + 2)) as [E | E].
1030 + rewrite subst_var. destruct (eq_var_dec y x) as [Heq | Hneq].
1031   * exfalso. subst y. simpl in Honly. specialize (Honly eq_refl).
1032   rewrite Honly in Hdeg.
1033   pose proof (index_range s) as [Hsr1 Hsr2]. lia.
1034   * reflexivity.
1035 + rewrite subst_var. destruct (eq_var_dec y x) as [Heq | Hneq].

```

```

1027 * subst y. remember (retag x (sort_of (i + 2))) as z eqn:Heqz.
1028 assert (Hz : rho i N = (t_fvar (sort_of (i + 2)) z) [ z = rho
1029 i N ]).
1029 { rewrite subst_var. destruct (eq_var_dec z z) as [_ | Hc];
1030 [reflexivity | exfalso; apply Hc; reflexivity]. }
1030 rewrite Hz. simpl. rewrite <- Heqz.
1031 assert (Hconst : forall k, k <= index_of s - i - 2 ->
1032 rho_subst_tel (t_fvar (sort_of (i + 2)) z) N x i k = (t_fvar
1033 (sort_of (i + 2)) z) [ z = rho i N ]).
1033 { induction k as [| k' IHk]; intros Hk.
1034 - simpl. rewrite <- Heqz. reflexivity.
1035 - simpl.
1036 assert (Hlevel_ne : retag x (sort_of (S (k' + i + 2))) <>
1037 z).
1037 { rewrite Heqz. intro Hcontra.
1038 apply retag_inj in Hcontra as [_ Hsort].
1039 assert (Heq2 : S (k' + i + 2) = i + 2)
1040 by (apply (sort_of_inj (S (k' + i + 2)) (i + 2)); [lia
1041 | lia | lia | exact Hsort]).
1042 lia. }
1042 destruct (eq_var_dec z (retag x (sort_of (S (k' + i +
1043 2))))) as [Heq | Hne].
1043 + exfalso. apply Hlevel_ne. symmetry. exact Heq.
1044 + apply IHk. lia.
1045 }
1046 symmetry. apply Hconst. lia.
1047 * assert (Hconst : forall k, rho_subst_tel (t_fvar (sort_of (i +
1048 2)) (retag y (sort_of (i + 2)))) N x i k
1049 = t_fvar (sort_of (i + 2)) (retag y (sort_of
1050 (i + 2)))).
1050 { induction k as [| k' IHk].
1051 - simpl. destruct (eq_var_dec (retag y (sort_of (i + 2)))
1052 (retag x (sort_of (i + 2)))) as [Heq | Hne].
1053 + exfalso. apply retag_inj in Heq as [Heqxy _]. apply
1054 Hneq. exact Heqxy.
1055 + reflexivity.
1056 - simpl. destruct (eq_var_dec (retag y (sort_of (i + 2)))
1057 (retag x (sort_of (S (k' + i + 2))))) as [Heq | Hne].
1058 + exfalso. apply retag_inj in Heq as [Heqxy _]. apply
1059 Hneq. exact Heqxy.
1060 + exact IHk.
1061 }
1062 symmetry. apply Hconst.
1063 - (* M = t_app D C *)
1064 destruct Honly as [HonlyD HonlyC].
1065 assert (HdegEqC : deg (C [ x = N ]) = deg C)
1066 by (apply (deg_subst_tagged C x s N); auto).
1067 assert (HdegEqD : deg (D [ x = N ]) = deg D)

```



```

1064 by (apply (deg_subst_tagged D x s N); auto).
1065
1066 destruct (le_lt_dec (i + 1) (deg C)) as [HdegC | HdegC].
1067
1068 + (* deg C >= i+1 *)
1069   rewrite subst_app. unfold rho_subst in *.
1070   assert (E : i + 1 <= deg (C [ x = N ])) by lia.
1071   rewrite (rho_app_named i (D[x=N]) (C[x=N]) E).
1072   rewrite (rho_app_named i D C HdegC).
1073   destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1074
1075   ++ (* j < i+2 *)
1076     rewrite HdegEqC. remember (deg C - 1 - i) as k.
1077     assert (Htel : forall k', k' <= k -> rho_app_tel rho
1078       (D[x=N]) (C[x=N]) i k' = rho_app_tel rho D C i k').
1079     { induction k' as [| k'' IHk']; intros Hk'.
1080       - simpl. f_equal.
1081         + apply IHD; auto.
1082         + apply IHC; auto.
1083       - simpl. f_equal.
1084         + apply IHk'. lia.
1085         + assert (Hcle : deg C <= n + 1) by (apply deg_le_top).
1086           assert (HIH : rho (i + S k'') (C[x=N]) = rho_subst C N
1087             x s (i + S k''))
1088             by (apply IHouter; [unfold ltof; lia | lia | exact
1089               Hxnz | exact Hx | unfold deg in *; lia | exact
1090               HonlyC | exact HdegN | exact HlcN]).
1091           rewrite HIH. unfold rho_subst.
1092           destruct (lt_dec (index_of s) (i + S k'' + 2)) as [_ |
1093             Hcontra]; [reflexivity | lia]. }
1094     apply Htel; lia.
1095
1096   ++ (* j >= i+2 *)
1097     rewrite HdegEqC.
1098     assert (FD : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N
1099       x i (index_of s - i - 2))
1100     by (apply IHD; auto).
1101     assert (FC : rho i (C [ x = N ]) = rho_subst_tel (rho i C) N
1102       x i (index_of s - i - 2))
1103     by (apply IHC; auto).
1104
1105     remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1106     assert (Step1 : forall k, k <= deg C - 1 - i ->
1107       rho_app_tel rho (D[x=N]) (C[x=N]) i k =
1108       rho_app_tel f D C i k).
1109     { induction k as [| k' IHk]; intros Hk.
1110       - simpl. rewrite Heqf. f_equal.
1111         + unfold rho_subst.
1112         destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 |
1113           Hge0]; [lia | exact FD].

```



```

1105       + unfold rho_subst.
1106       destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 |
        Hge0]; [lia | exact FC].
1107   - simpl. rewrite Heqf. f_equal.
1108     + rewrite <- Heqf. apply IHk. lia.
1109     + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1110       apply IHouter; [unfold lt_of; lia | lia | exact Hxnz |
        exact Hx | unfold deg in *; lia | exact HonlyC | exact
        HdegN | exact HlcN].
1111   }
1112   rewrite Step1; auto.
1113
1114   assert (Step2 : forall k, k <= deg C - 1 - i ->
1115     rho_app_tel f D C i k = rho_subst_tel (rho_app_tel rho
1116       D C i k) N x i (index_of s - i - 2)).
1117   {
1118     intros k.
1119     rewrite (rho_subst_tel_app_tel_commute D C N x i (index_of
1120       s - i - 2) i k).
1121     induction k as [| k' IHk]; intros Hk.
1122     - simpl. rewrite Heqf. simpl. f_equal.
1123       + unfold rho_subst. destruct (lt_dec (index_of s) (i +
1124         2)) as [Hlt0 | Hge0]; [lia | reflexivity].
1125       + unfold rho_subst. destruct (lt_dec (index_of s) (i +
1126         2)) as [Hlt0 | Hge0]; [lia | reflexivity].
1127     - simpl. f_equal.
1128       + apply IHk. lia.
1129       + rewrite Heqf. simpl. unfold rho_subst.
1130         destruct (lt_dec (index_of s) (i + S k' + 2)) as [HjA
1131           | HjB].
1132         * assert (HCle : deg C <= n + 1) by (apply
1133           deg_le_top).
1134           symmetry.
1135           apply (rho_subst_tel_noop_generic (rho (i + S k') C)
1136             N x (i + S k') i (index_of s - i - 2)).
1137           -- lia.
1138           -- intros m v N0 Hm Hn. apply rho_min_tier_noop;
1139             lia.
1140           -- lia.
1141         * assert (HCle : deg C <= n + 1) by (apply
1142           deg_le_top).
1143           assert (Hsplit := rho_subst_tel_split (rho (i + S
1144             k') C) N x i (index_of s - i - 2) (i + S k')
1145             ltac:(lia) ltac:(lia)).
1146           replace (i + (index_of s - i - 2) - (i + S k'))
1147             with (index_of s - (i + S k') - 2) in Hsplit by
1148             lia.
1149           rewrite Hsplit.
1150           replace (i + S k' - i - 1) with k' by lia.

```

```

1140         symmetry.
1141         apply rho_subst_tel_noop_generic with (D := i + S
1142         k').
1143         -- lia.
1144         -- apply rho_subst_tel_stays_clean with (D := i + S
1145         k').
1146         ++ lia.
1147         ++ intros m v N0 Hm Hn. apply rho_min_tier_noop;
1148         lia.
1149         -- lia.
1150     }
1151     rewrite Step2; auto.
1152
1153 + (* deg C < i+1 *)
1154     assert (HdegC' : deg (C [ x = N ]) < i + 1) by lia.
1155     simpl.
1156     destruct (le_lt_dec (i + 1) (deg (C [ x = N ]))) as [E | E].
1157     lia.
1158     unfold rho_subst in *.
1159     destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1160
1161 * (* j < i+2 *)
1162     simpl. destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia.
1163     apply IHD; auto.
1164
1165 * (* j >= i+2 *)
1166     assert (F : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N x
1167     i (index_of s - i - 2))
1168     by (apply IHD; auto).
1169     rewrite F. f_equal. simpl.
1170     destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia. auto.
1171
1172 - (* M = t_lam s1 C D *)
1173     destruct Honly as [HonlyC HonlyD].
1174     assert (HdegEqC : deg (C [ x = N ]) = deg C)
1175     by (apply (deg_subst_tagged C x s N); auto).
1176     destruct (le_lt_dec (i + 2) (deg C)) as [HdegC | HdegC].
1177
1178 + (* deg C >= i+2 *)
1179     rewrite subst_lam. unfold rho_subst in *.
1180     assert (E : i + 2 <= deg (C [ x = N ])) by lia.
1181     assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1182     rewrite (rho_lam_named i s1 (C[x=N]) (D[x=N]) E).
1183     rewrite (rho_lam_named i s1 C D HdegC).
1184     destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1185
1186     ++ (* j < i+2 *)
1187     rewrite HdegEqC. remember (deg C - 1 - (i + 1)) as k.
1188     assert (Htel : forall k', k' <= k -> rho_lam_tel rho
1189     (C[x=N]) (D[x=N]) i k' = rho_lam_tel rho C D i k').

```

```

1184 { induction k' as [| k' IHk']; intros Hk'.
1185 - simpl. f_equal.
1186 + assert (HIH : rho (i + 1) (C[x=N]) = rho_subst C N x s
      (i + 1))
1187   by (apply IHouter; [unfold ltof; lia | lia | exact
      Hxnz | exact Hx | unfold deg in *; lia | exact
      HonlyC | exact HdegN | exact HlcN]).
1188   rewrite HIH. unfold rho_subst.
1189   destruct (lt_dec (index_of s) (i + 1 + 2)) as [_ |
      Hcontra]; [reflexivity | lia].
1190 + apply IHD; auto.
1191 - simpl. f_equal.
1192 + assert (HIH : rho (i + 1 + S k') (C[x=N]) = rho_subst
      C N x s (i + 1 + S k'))
1193   by (apply IHouter; [unfold ltof; lia | lia | exact
      Hxnz | exact Hx | unfold deg in *; lia | exact
      HonlyC | exact HdegN | exact HlcN]).
1194   rewrite HIH. unfold rho_subst.
1195   destruct (lt_dec (index_of s) (i + 1 + S k' + 2)) as
      [_ | Hcontra]; [reflexivity | lia].
1196 + apply IHk'. lia. }
1197 apply Htel; lia.
1198
1199 ++ (* j >= i+2 *)
1200 rewrite HdegEqC.
1201 assert (FD : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N
      x i (index_of s - i - 2))
1202 by (apply IHD; auto).
1203
1204 remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1205 assert (Step1 : forall k, k <= deg C - 1 - (i + 1) ->
      rho_lam_tel rho (C[x=N]) (D[x=N]) i k =
      rho_lam_tel f C D i k).
1206
1207 { induction k as [| k' IHk]; intros Hk.
1208 - simpl. rewrite Heqf. f_equal.
1209 + apply IHouter; [unfold ltof; lia | lia | exact Hxnz |
      exact Hx | unfold deg in *; lia | exact HonlyC | exact
      HdegN | exact HlcN].
      + unfold rho_subst.
      destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 |
      Hge0]; [lia | exact FD].
1210 + simpl. rewrite Heqf. f_equal.
1211 + apply IHouter; [unfold ltof; lia | lia | exact Hxnz |
      exact Hx | unfold deg in *; lia | exact HonlyC | exact
      HdegN | exact HlcN].
1212 + rewrite <- Heqf. apply IHk. lia.
1213 }
1214
1215 rewrite Step1; auto.
1216
1217

```

```

1218 assert (Step2 : forall k, k <= deg C - 1 - (i + 1) ->
1219   rho_lam_tel f C D i k = rho_subst_tel (rho_lam_tel rho
    C D i k) N x i (index_of s - i - 2)).
1220 {
1221   intros k.
1222   rewrite (rho_subst_tel_lam_tel_commute C D N x i (index_of
    s - i - 2) i k).
1223   induction k as [| k' IHk]; intros Hk.
1224   - simpl. rewrite Heqf. simpl. f_equal.
1225     + unfold rho_subst.
1226       destruct (lt_dec (index_of s) (i + 1 + 2)) as [HjA |
    HjB].
1227       * symmetry.
1228         apply (rho_subst_tel_noop_generic (rho (i + 1) C) N
    x (i + 1) i (index_of s - i - 2)).
1229         -- lia.
1230         -- intros m v N0 Hm Hn. apply rho_min_tier_noop;
    lia.
1231         -- lia.
1232       * assert (Hsplit := rho_subst_tel_split (rho (i + 1)
    C) N x i (index_of s - i - 2) (i + 1)
    ltac:(lia) ltac:(lia)).
1233         replace (i + (index_of s - i - 2) - (i + 1))
1234         with (index_of s - (i + 1) - 2) in Hsplit by lia.
1235         rewrite Hsplit.
1236         replace (i + 1 - i - 1) with 0 by lia.
1237         symmetry.
1238         apply rho_subst_tel_noop_generic with (D := i + 1).
1239         -- lia.
1240         -- apply rho_subst_tel_stays_clean with (D := i +
    1).
1241         ++ lia.
1242         ++ intros m v N0 Hm Hn. apply rho_min_tier_noop;
    lia.
1243         -- lia.
1244       + unfold rho_subst. destruct (lt_dec (index_of s) (i +
    2)) as [Hlt0 | Hge0]; [lia | reflexivity].
1245     - simpl. f_equal.
1246     + rewrite Heqf. simpl. unfold rho_subst.
1247       destruct (lt_dec (index_of s) (i + 1 + S k' + 2)) as
    [HjA | HjB].
1248       * symmetry.
1249         apply (rho_subst_tel_noop_generic (rho (i + 1 + S
    k') C) N x (i + 1 + S k') i (index_of s - i - 2)).
1250         -- lia.
1251         -- intros m v N0 Hm Hn. apply rho_min_tier_noop;
    lia.
1252         -- lia.
1253

```

```

1254      * assert (Hsplit := rho_subst_tel_split (rho (i + 1 +
1255      S k') C) N x i (index_of s - i - 2) (i + 1 + S k')
1256      ltac:(lia) ltac:(lia)).
1257      replace (i + (index_of s - i - 2) - (i + 1 + S k'))
1258      with (index_of s - (i + 1 + S k') - 2) in Hsplit
1259      by lia.
1260      rewrite Hsplit.
1261      replace (i + 1 + S k' - i - 1) with (S k') by lia.
1262      symmetry.
1263      apply rho_subst_tel_noop_generic with (D := i + 1 +
1264      S k').
1265      -- lia.
1266      -- apply rho_subst_tel_stays_clean with (D := i + 1
1267      + S k').
1268      ++ lia.
1269      ++ intros m v N0 Hm Hn. apply rho_min_tier_noop;
1270      lia.
1271      -- lia.
1272      + apply IHk. lia.
1273    }
1274    rewrite Step2; auto.
1275
1276  + (* deg C < i+2 *)
1277    assert (HdegC' : deg (C [ x = N ]) < i + 2) by lia.
1278    simpl.
1279    destruct (le_lt_dec (i + 2) (deg (C [ x = N ]))) as [E | E].
1280    lia.
1281    unfold rho_subst in *.
1282    destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1283
1284    * (* j < i+2 *)
1285      simpl. destruct (le_lt_dec (i + 2) (deg C)) as [E' | E']. lia.
1286      apply IHD; auto.
1287
1288    * (* j >= i+2 *)
1289      assert (F : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N x
1290      i (index_of s - i - 2))
1291      by (apply IHD; auto).
1292      rewrite F. f_equal. simpl.
1293      destruct (le_lt_dec (i + 2) (deg C)) as [E' | E']. lia. auto.
1294
1295  - (* M = t_pi s1 C D *)
1296    destruct Honly as [HonlyC HonlyD].
1297    assert (HdegEqC : deg (C [ x = N ]) = deg C)
1298    by (apply (deg_subst_tagged C x s N); auto).
1299    destruct (le_lt_dec (i + 1) (deg C)) as [HdegC | HdegC].
1300
1301  + (* deg C >= i+1 *)
1302    rewrite subst_pi. unfold rho_subst in *.

```

```

1296 assert (E : i + 1 <= deg (C [ x = N ])) by lia.
1297 rewrite (rho_pi_named i s1 (C[x=N]) (D[x=N]) E).
1298 rewrite (rho_pi_named i s1 C D HdegC).
1299 destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1300
1301 ++ (* j < i+2 *)
1302 rewrite HdegEqC. remember (deg C - 1 - i) as k.
1303 assert (Htel : forall k', k' <= k -> rho_pi_tel rho (C[x=N])
(D[x=N]) i k' = rho_pi_tel rho C D i k').
1304 { induction k' as [| k'' IHk']; intros Hk'.
1305   - simpl. f_equal.
1306     + apply IHC; auto.
1307     + apply IHD; auto.
1308   - simpl. f_equal.
1309     + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1310       assert (HIH : rho (i + S k'') (C[x=N]) = rho_subst C N
x s (i + S k''))
1311         by (apply IHouter; [unfold ltof; lia | lia | exact
Hxnz | exact Hx | unfold deg in *; lia | exact
HonlyC | exact HdegN | exact HlcN]).
1312       rewrite HIH. unfold rho_subst.
1313       destruct (lt_dec (index_of s) (i + S k'' + 2)) as [_ |
Hcontra]; [reflexivity | lia].
1314     + apply IHk'. lia. }
1315 apply Htel; lia.
1316
1317 ++ (* j >= i+2 *)
1318 rewrite HdegEqC.
1319 assert (FD : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N
x i (index_of s - i - 2))
1320 by (apply IHD; auto).
1321 assert (FC : rho i (C [ x = N ]) = rho_subst_tel (rho i C) N
x i (index_of s - i - 2))
1322 by (apply IHC; auto).
1323
1324 remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1325 assert (Step1 : forall k, k <= deg C - 1 - i ->
1326   rho_pi_tel rho (C[x=N]) (D[x=N]) i k = rho_pi_tel
f C D i k).
1327 { induction k as [| k' IHk]; intros Hk.
1328   - simpl. rewrite Heqf. f_equal.
1329     + unfold rho_subst.
1330       destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 |
Hge0]; [lia | exact FC].
1331     + unfold rho_subst.
1332       destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 |
Hge0]; [lia | exact FD].
1333   - simpl. rewrite Heqf. f_equal.
1334     + assert (HCle : deg C <= n + 1) by (apply deg_le_top).

```

```

1335         apply IHouter; [unfold lt_of; lia | lia | exact Hxnz |
        exact Hx | unfold deg in *; lia | exact HonlyC | exact
        HdegN | exact HlcN].
1336     + rewrite <- Heqf. apply IHk. lia.
1337 }
1338 rewrite Step1; auto.
1339
1340 assert (Step2 : forall k, k <= deg C - 1 - i ->
1341     rho_pi_tel f C D i k = rho_subst_tel (rho_pi_tel rho C
        D i k) N x i (index_of s - i - 2)).
1342 {
1343     intros k.
1344     rewrite (rho_subst_tel_pi_tel_commute C D N x i (index_of
        s - i - 2) i k).
1345     induction k as [| k' IHk]; intros Hk.
1346     - simpl. rewrite Heqf. simpl. f_equal.
1347       + unfold rho_subst. destruct (lt_dec (index_of s) (i +
        2)) as [Hlt0 | Hge0]; [lia | reflexivity].
1348       + unfold rho_subst. destruct (lt_dec (index_of s) (i +
        2)) as [Hlt0 | Hge0]; [lia | reflexivity].
1349     - simpl. f_equal.
1350       + rewrite Heqf. simpl. unfold rho_subst.
1351         destruct (lt_dec (index_of s) (i + S k' + 2)) as [HjA
        | HjB].
1352         * assert (HCle : deg C <= n + 1) by (apply
        deg_le_top).
1353           symmetry.
1354           apply (rho_subst_tel_noop_generic (rho (i + S k') C)
        N x (i + S k') i (index_of s - i - 2)).
1355           -- lia.
1356           -- intros m v N0 Hm Hn. apply rho_min_tier_noop;
        lia.
1357           -- lia.
1358         * assert (HCle : deg C <= n + 1) by (apply
        deg_le_top).
1359           assert (Hsplit := rho_subst_tel_split (rho (i + S
        k') C) N x i (index_of s - i - 2) (i + S k')
        ltac:(lia) ltac:(lia)).
1360           replace (i + (index_of s - i - 2) - (i + S k'))
        with (index_of s - (i + S k') - 2) in Hsplit by
        lia.
1361           rewrite Hsplit.
1362           replace (i + S k' - i - 1) with k' by lia.
1363           symmetry.
1364           apply rho_subst_tel_noop_generic with (D := i + S
        k').
1365           -- lia.
1366           -- apply rho_subst_tel_stays_clean with (D := i + S
        k').

```

```

1369         ++ lia.
1370         ++ intros m v N0 Hm Hn. apply rho_min_tier_noop;
           lia.
1371         -- lia.
1372         + apply IHk. lia.
1373     }
1374     rewrite Step2; auto.
1375
1376 + (* deg C < i+1 *)
1377   assert (HdegC' : deg (C [ x = N ]) < i + 1) by lia.
1378   simpl.
1379   destruct (le_lt_dec (i + 1) (deg (C [ x = N ]))) as [E | E].
1380   lia.
1381   unfold rho_subst in *.
1382   destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1383
1384 * (* j < i+2 *)
1385   simpl. destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia.
1386   apply IHD; auto.
1387
1388 * (* j >= i+2 *)
1389   assert (F : rho i (D [ x = N ]) = rho_subst_tel (rho i D) N x
1390     i (index_of s - i - 2))
1391   by (apply IHD; auto).
1392   rewrite F. f_equal. simpl.
1393   destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia. auto.
1394 Qed.
1395
1396 (* =====
1397 *)
1398 (* rho commutes with beta *)
1399
1400 Lemma brt_pi_A : forall s A A' B, A ->=b A' -> t_pi s A B ->=b t_pi s
1401 A' B.
1402 Proof.
1403   intros s A A' B H.
1404   induction H as [A | A A' HA | A A' A'' H1 IH1 H2 IH2].
1405   - apply brt_refl.
1406   - apply brt_step. apply beta_pi_A. exact HA.
1407   - apply brt_trans with (t_pi s A' B); auto.
1408 Qed.
1409
1410 Lemma brt_pi_B : forall s A B B', B ->=b B' -> t_pi s A B ->=b t_pi s
1411 A B'.
1412 Proof.
1413   intros s A B B' H.
1414   induction H as [B | B B' HB | B B' B'' H1 IH1 H2 IH2].
1415   - apply brt_refl.
1416   - apply brt_step. apply beta_pi_B. exact HB.

```



```

1412   - apply brt_trans with (t_pi s A B'); auto.
1413 Qed.
1414
1415 Lemma brt_pi_congr : forall s A A' B B',
1416   A ->>b A' -> B ->>b B' -> t_pi s A B ->>b t_pi s A' B'.
1417 Proof.
1418   intros s A A' B B' HA HB.
1419   apply brt_trans with (t_pi s A' B).
1420   - apply brt_pi_A. exact HA.
1421   - apply brt_pi_B. exact HB.
1422 Qed.
1423
1424 Lemma brt_lam_A : forall s A A' M, A ->>b A' -> t_lam s A M ->>b t_lam
s A' M.
1425 Proof.
1426   intros s A A' M H.
1427   induction H as [A | A A' HA | A A' A'' H1 IH1 H2 IH2].
1428   - apply brt_refl.
1429   - apply brt_step. apply beta_lam_A. exact HA.
1430   - apply brt_trans with (t_lam s A' M); auto.
1431 Qed.
1432
1433 Lemma brt_lam_M : forall s A M M', M ->>b M' -> t_lam s A M ->>b t_lam
s A M'.
1434 Proof.
1435   intros s A M M' H.
1436   induction H as [M | M M' HM | M M' M'' H1 IH1 H2 IH2].
1437   - apply brt_refl.
1438   - apply brt_step. apply beta_lam_M. exact HM.
1439   - apply brt_trans with (t_lam s A M'); auto.
1440 Qed.
1441
1442 Lemma brt_lam_congr : forall s A A' M M',
1443   A ->>b A' -> M ->>b M' -> t_lam s A M ->>b t_lam s A' M'.
1444 Proof.
1445   intros s A A' M M' HA HM.
1446   apply brt_trans with (t_lam s A' M).
1447   - apply brt_lam_A. exact HA.
1448   - apply brt_lam_M. exact HM.
1449 Qed.
1450
1451 Lemma brt_app_l : forall M M' N, M ->>b M' -> t_app M N ->>b t_app M'
N.
1452 Proof.
1453   intros M M' N H.
1454   induction H as [M | M M' HM | M M' M'' H1 IH1 H2 IH2].
1455   - apply brt_refl.
1456   - apply brt_step. apply beta_app_l. exact HM.
1457   - apply brt_trans with (t_app M' N); auto.
1458 Qed.

```

```

1459
1460 Lemma brt_app_r : forall M N N', N ->>b N' -> t_app M N ->>b t_app M
1461 N'.
1462 Proof.
1463   intros M N N' H.
1464   induction H as [N | N N' HN | N N' N'' H1 IH1 H2 IH2].
1465   - apply brt_refl.
1466   - apply brt_step. apply beta_app_r. exact HN.
1467   - apply brt_trans with (t_app M N'); auto.
1468 Qed.
1469
1470 Lemma brt_app_congr : forall M M' N N',
1471 M ->>b M' -> N ->>b N' -> t_app M N ->>b t_app M' N'.
1472 Proof.
1473   intros M M' N N' HM HN.
1474   apply brt_trans with (t_app M' N).
1475   - apply brt_app_l. exact HM.
1476   - apply brt_app_r. exact HN.
1477 Qed.
1478
1479 Lemma rho_pi_tel_dom_congr : forall C C' D i k,
1480 (forall m, i <= m <= i + k -> rho m C ->>b rho m C') ->
1481 rho_pi_tel rho C D i k ->>b rho_pi_tel rho C' D i k.
1482 Proof.
1483   induction k as [| k' IHk]; intros Hall; simpl.
1484   - apply brt_pi_A. apply Hall. lia.
1485   - apply brt_pi_congr.
1486     + apply Hall. lia.
1487     + apply IHk. intros m Hm. apply Hall. lia.
1488 Qed.
1489
1490 Lemma rho_pi_tel_body_congr : forall A B B' i k,
1491 rho i B ->>b rho i B' ->
1492 rho_pi_tel rho A B i k ->>b rho_pi_tel rho A B' i k.
1493 Proof.
1494   induction k as [| k' IHk]; intros Hb; simpl.
1495   - apply brt_pi_B. exact Hb.
1496   - apply brt_pi_B. apply IHk. exact Hb.
1497 Qed.
1498
1499 Lemma rho_lam_tel_dom_congr : forall C C' M' i k,
1500 (forall m, i + 1 <= m <= i + 1 + k -> rho m C ->>b rho m C') ->
1501 rho_lam_tel rho C M' i k ->>b rho_lam_tel rho C' M' i k.
1502 Proof.
1503   induction k as [| k' IHk]; intros Hall; simpl.
1504   - apply brt_lam_A. apply Hall. lia.
1505   - apply brt_lam_congr.
1506     + apply Hall. lia.
1507     + apply IHk. intros m Hm. apply Hall. lia.
1508 Qed.

```

```

1508
1509 Lemma rho_lam_tel_body_congr : forall A M' M'' i k,
1510   rho i M' ->>b rho i M'' ->
1511   rho_lam_tel rho A M' i k ->>b rho_lam_tel rho A M'' i k.
1512 Proof.
1513   induction k as [| k' IHk]; intros Hb; simpl.
1514   - apply brt_lam_M. exact Hb.
1515   - apply brt_lam_M. apply IHk. exact Hb.
1516 Qed.
1517
1518 Lemma rho_app_tel_func_congr : forall P P' Q i k,
1519   rho i P ->>b rho i P' ->
1520   rho_app_tel rho P Q i k ->>b rho_app_tel rho P' Q i k.
1521 Proof.
1522   induction k as [| k' IHk]; intros Hp; simpl.
1523   - apply brt_app_l. exact Hp.
1524   - apply brt_app_l. apply IHk. exact Hp.
1525 Qed.
1526
1527 Lemma rho_app_tel_arg_congr : forall P Q Q' i k,
1528   (forall m, i <= m <= i + k -> rho m Q ->>b rho m Q') ->
1529   rho_app_tel rho P Q i k ->>b rho_app_tel rho P Q' i k.
1530 Proof.
1531   induction k as [| k' IHk]; intros Hall; simpl.
1532   - apply brt_app_r. apply Hall. lia.
1533   - apply brt_app_congr.
1534     + apply IHk. intros m Hm. apply Hall. lia.
1535     + apply Hall. lia.
1536 Qed.
1537
1538 Axiom rho_commutates_beta_base : forall i, 0 <= i <= n ->
1539   forall s A M Q, deg M >= i + 1 ->
1540   rho i (t_app (t_lam s A M) Q) ->>b rho i (M ^^ Q).
1541
1542 Lemma rho_commutates_beta_aux :
1543   forall i, 0 <= i <= n ->
1544   forall M, deg M >= i + 1 ->
1545   forall N, M ->b N ->
1546   rho i M ->>b rho i N.
1547 Proof.
1548   intros i.
1549   induction i as [i IHouter] using
1550     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n
1551       - i))).
1551   intros Hi M.
1552   induction M as [s | s_m m | y | P IHP Q IHQ | s A IHA M' IHM' | s A
1553     IHA B IHB];
1554     intros Hdeg N Hstep.
1555   - (* t_sort *) inversion Hstep.

```

```

1556 - (* t_bvar *) inversion Hstep.
1557 - (* t_fvar *) inversion Hstep.
1558
1559 - (* t_app P Q *)
1560   simpl in Hdeg.
1561   inversion Hstep; subst; clear Hstep.
1562
1563 + (* beta_base *)
1564   apply rho_commutes_beta_base; [exact Hi | ].
1565   simpl in Hdeg. exact Hdeg.
1566
1567 + (* beta_app_l *)
1568   match goal with
1569   | Hs : P ->b ?P' | - _ =>
1570     assert (HdegP : deg P >= i + 1) by exact Hdeg;
1571     destruct (le_lt_dec (i + 1) (deg Q)) as [E | E];
1572     [ rewrite (rho_app_named i P Q E); rewrite (rho_app_named i P'
1573       Q E);
1574       apply rho_app_tel_func_congr;
1575       apply IHP; auto
1576     | assert (Hrho1 : rho i (t_app P Q) = rho i P) by (simpl;
1577       destruct (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);
1578       assert (Hrho2 : rho i (t_app P' Q) = rho i P') by (simpl;
1579       destruct (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);
1580       rewrite Hrho1, Hrho2;
1581       apply IHP; auto ]
1582   end.
1583
1584 + (* beta_app_r *)
1585   match goal with
1586   | Hs : Q ->b ?Q' | - _ =>
1587     assert (HdegQeq : deg Q = deg Q') by (apply deg_beta; apply
1588       brt_step; exact Hs);
1589     destruct (le_lt_dec (i + 1) (deg Q)) as [E | E];
1590     [ assert (E' : i + 1 <= deg Q') by lia;
1591       rewrite (rho_app_named i P Q E); rewrite (rho_app_named i P
1592       Q' E');
1593       assert (Hk : deg Q - 1 - i = deg Q' - 1 - i) by lia;
1594       rewrite Hk;
1595       apply rho_app_tel_arg_congr;
1596       intros mtier Hmtier;
1597       assert (HQle : deg Q <= n + 1) by (apply deg_le_top);
1598       assert (HdegQm : deg Q >= mtier + 1) by lia;
1599       destruct (Nat.eq_dec mtier i) as [Heqm | Hneqm];
1600       [ subst mtier; apply IHQ; [unfold deg in *; lia | exact Hs]
1601         | apply IHouter; [unfold ltof; lia | lia | unfold deg in *;
1602         lia | exact Hs] ]
1603     | assert (E' : ~ i + 1 <= deg Q') by lia;
1604       assert (Hrho1 : rho i (t_app P Q) = rho i P) by (simpl;
1605       destruct (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);

```

```

1599     assert (Hrho2 : rho i (t_app P Q') = rho i P) by (simpl;
1600     destruct (le_lt_dec (i+1) (deg Q')); [lia | reflexivity]);
1601     rewrite Hrho1, Hrho2; apply brt_refl ]
1602   end.
1603 - (* t_lam s A M' *)
1604   inversion Hstep; subst; clear Hstep.
1605
1606 + (* beta_lam_A *)
1607   match goal with
1608   | Hs : A ->b ?A1' |- _ =>
1609     assert (HdegAA' : deg A = deg A1') by (apply deg_beta; apply
1610     brt_step; exact Hs);
1611     destruct (le_lt_dec (i + 2) (deg A)) as [E | E];
1612     [ assert (E' : i + 2 <= deg A1') by lia;
1613       rewrite (rho_lam_named i s A M' E); rewrite (rho_lam_named i
1614       s A1' M' E');
1615       assert (Hk : deg A - 1 - (i + 1) = deg A1' - 1 - (i + 1)) by
1616       lia;
1617       rewrite Hk;
1618       apply rho_lam_tel_dom_congr;
1619       intros mtier Hmtier;
1620       assert (HAle : deg A <= n + 1) by (apply deg_le_top);
1621       assert (HdegAm : deg A >= mtier + 1) by lia;
1622       apply IHouter; [unfold lt_of; lia | lia | unfold deg in *;
1623       lia | exact Hs]
1624       | assert (E' : ~ i + 2 <= deg A1') by lia;
1625       assert (Hrho1 : rho i (t_lam s A M') = rho i M') by (simpl;
1626       destruct (le_lt_dec (i+2) (deg A)); [lia | reflexivity]);
1627       assert (Hrho2 : rho i (t_lam s A1' M') = rho i M') by
1628       (simpl; destruct (le_lt_dec (i+2) (deg A1')); [lia |
1629       reflexivity]);
1630       rewrite Hrho1, Hrho2; apply brt_refl ]
1631   end.
1632
1633 + (* beta_lam_M *)
1634   match goal with
1635   | Hs : M' ->b ?M1' |- _ =>
1636     assert (HdegM' : deg M' >= i + 1) by exact Hdeg;
1637     destruct (le_lt_dec (i + 2) (deg A)) as [E | E];
1638     [ rewrite (rho_lam_named i s A M' E); rewrite (rho_lam_named i
1639     s A M1' E);
1640       apply rho_lam_tel_body_congr;
1641       apply IHM'; auto
1642       | assert (Hrho1 : rho i (t_lam s A M') = rho i M') by (simpl;
1643       destruct (le_lt_dec (i+2) (deg A)); [lia | reflexivity]);
1644       assert (Hrho2 : rho i (t_lam s A M1') = rho i M1') by
1645       (simpl; destruct (le_lt_dec (i+2) (deg A)); [lia |
1646       reflexivity]);

```

```

1636         rewrite Hrho1, Hrho2;
1637         apply IHM'; auto ]
1638     end.
1639
1640 - (* t_pi s A B *)
1641     inversion Hstep; subst; clear Hstep.
1642
1643 + (* beta_pi_A *)
1644     match goal with
1645     | Hs : A ->b ?A1' |- _ =>
1646         assert (HdegAA' : deg A = deg A1') by (apply deg_beta; apply
1647         brt_step; exact Hs);
1648         destruct (le_lt_dec (i + 1) (deg A)) as [E | E];
1649         [ assert (E' : i + 1 <= deg A1') by lia;
1650           rewrite (rho_pi_named i s A B E); rewrite (rho_pi_named i s
1651           A1' B E');
1652           assert (Hk : deg A - 1 - i = deg A1' - 1 - i) by lia;
1653           rewrite Hk;
1654           apply rho_pi_tel_dom_congr;
1655           intros mtier Hmtier;
1656           assert (HAle : deg A <= n + 1) by (apply deg_le_top);
1657           assert (HdegAm : deg A >= mtier + 1) by lia;
1658           destruct (Nat.eq_dec mtier i) as [Heqm | Hneqm];
1659           [ subst mtier; apply IHA; [unfold deg in *; lia | exact Hs]
1660           | apply IHouter; [unfold ltof; lia | lia | unfold deg in *;
1661           lia | exact Hs] ]
1662           | assert (E' : ~ i + 1 <= deg A1') by lia;
1663           assert (Hrho1 : rho i (t_pi s A B) = rho i B) by (simpl;
1664           destruct (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1665           assert (Hrho2 : rho i (t_pi s A1' B) = rho i B) by (simpl;
1666           destruct (le_lt_dec (i+1) (deg A1')); [lia | reflexivity]);
1667           rewrite Hrho1, Hrho2; apply brt_refl ]
1668     end.
1669
1670 + (* beta_pi_B *)
1671     match goal with
1672     | Hs : B ->b ?B1' |- _ =>
1673         assert (HdegB : deg B >= i + 1) by exact Hdeg;
1674         destruct (le_lt_dec (i + 1) (deg A)) as [E | E];
1675         [ rewrite (rho_pi_named i s A B E); rewrite (rho_pi_named i s
1676         A B1' E);
1677           apply rho_pi_tel_body_congr;
1678           apply IHB; auto
1679           | assert (Hrho1 : rho i (t_pi s A B) = rho i B) by (simpl;
1680           destruct (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1681           assert (Hrho2 : rho i (t_pi s A B1') = rho i B1') by (simpl;
1682           destruct (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1683           rewrite Hrho1, Hrho2;
1684           apply IHB; auto ]

```

```

1677     end.
1678 Qed.
1679
1680 Lemma rho_commutes_beta :
1681   forall i, 0 <= i <= n ->
1682   forall M, deg M >= i + 1 ->
1683   forall N, M ->>b N ->
1684   rho i M ->>b rho i N.
1685 Proof.
1686   intros i Hi M Hdeg N Hbeta.
1687   induction Hbeta as [M | M N Hstep | M N P Hstep1 IH1 Hstep2 IH2].
1688   - apply brt_refl.
1689   - apply (rho_commutes_beta_aux i Hi M Hdeg N Hstep).
1690   - apply brt_trans with (rho i N); auto.
1691     apply IH2; auto.
1692     rewrite <- (deg_beta M N Hstep1); auto.
1693 Qed.
1694
1695 (* =====
1696 *)
1697 (* rho commutes with typing *)
1698
1699 Axiom n_at_least_two : n >= 2.
1700
1701 Axiom typing_star_weakening_incl : forall  $\Gamma$   $\Delta$  M A,
1702    $\Gamma \subseteq \Delta \rightarrow \Gamma \vdash^* M \in A \rightarrow \Delta \vdash^* M \in A$ .
1703
1704 Lemma subcontext_app_same : forall  $\Gamma_1$   $\Gamma_2$   $\Delta$ ,
1705    $\Gamma_1 \subseteq \Delta \rightarrow \Gamma_2 \subseteq \Delta \rightarrow \Gamma_1 ++ \Gamma_2 \subseteq \Delta$ .
1706 Proof.
1707   intros  $\Gamma_1$   $\Gamma_2$   $\Delta$  H1 H2 x T Hin.
1708   apply in_app_or in Hin. destruct Hin; [apply H1 | apply H2]; auto.
1709 Qed.
1710
1711 Lemma rho_ctx_tel_idx : forall y s T k v A,
1712   In (v, A) (rho_ctx_tel y s T k) -> exists s', v = retag y s'.
1713 Proof.
1714   intros y s T k.
1715   induction k as [| k' IHk]; intros v A Hin; simpl in Hin.
1716   - destruct Hin as [Heq | []]. injection Heq as Hv HA.
1717     exists (sort_of (index_of s)). symmetry. exact Hv.
1718   - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1719     + eapply IHk; eauto.
1720     + destruct Hin as [Heq | []]. injection Heq as Hv HA.
1721       exists (sort_of (index_of s - S k')). symmetry. exact Hv.
1722 Qed.
1723
1724 Lemma rho_ctx_tel_last : forall y s T k,
1725   In (retag y (sort_of (index_of s - k)),
1726     (sort_of (index_of s - k), rho (index_of s - k - 1) T))

```

```

1726     (rho_ctx_tel y s T k).
1727 Proof.
1728   intros y s T k.
1729   destruct k as [| k'].
1730   - simpl. left. replace (index_of s - 0) with (index_of s) by lia.
1731     replace (index_of s - 0 - 1) with (index_of s - 1) by lia.
1732     reflexivity.
1733   - simpl. apply in_or_app. right. left. reflexivity.
1734 Qed.
1735 Lemma rho_ctx_incl_old : forall  $\Gamma_0$  i x s T,
1736   rho_ctx i  $\Gamma_0 \subseteq$  rho_ctx i ( $\Gamma_0 ++ [(x, (s, T))]$ ).
1737 Proof.
1738   induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s T p q Hpq.
1739   - simpl in Hpq. destruct Hpq as [Hpq | []]. inversion Hpq; subst.
1740     simpl. apply in_or_app. right. left. reflexivity.
1741   - simpl in Hpq |- *. apply in_app_or in Hpq. apply in_or_app.
1742     destruct Hpq as [Hpq | Hpq].
1743     + left. exact Hpq.
1744     + right. apply (IH i x s T). exact Hpq.
1745 Qed.
1746 Lemma rho_ctx_incl_new : forall  $\Gamma_0$  i x s T,
1747   rho_ctx_tel x s T (index_of s - i - 1)  $\subseteq$  rho_ctx i ( $\Gamma_0 ++ [(x, (s,$ 
1748     T))]).
1749 Proof.
1750   induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s T p q Hpq.
1751   - simpl. apply in_or_app. left. exact Hpq.
1752   - simpl. apply in_or_app. right. apply (IH i x s T). exact Hpq.
1753 Qed.
1754 Lemma rho_ctx_fresh : forall  $\Gamma_0$  i x s,
1755   x <> zero_var -> is_fresh x  $\Gamma_0$  -> is_fresh (retag x s) (rho_ctx i
1756      $\Gamma_0$ ).
1757 Proof.
1758   induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s Hxnz Hfresh.
1759   - simpl. unfold is_fresh, dom. simpl. intro Hin.
1760     destruct Hin as [Heq | []]. apply (zero_var_retag_ne x s).
1761     symmetry. exact Heq.
1762   - assert (Hfresh' : is_fresh x  $\Gamma_0'$ ) by (intros Hin; apply Hfresh;
1763     simpl; right; exact Hin).
1764     assert (Hxy : x <> y)
1765     by (intro Heq; apply Hfresh; simpl; left; symmetry; exact Heq).
1766     unfold is_fresh, dom in *. simpl.
1767     intro Hin. apply in_map_iff in Hin. destruct Hin as [[v A] [Hveq
1768       Hin']]. simpl in Hveq. subst v.
1769     apply in_app_or in Hin'. destruct Hin' as [Hin' | Hin'].
1770     + pose proof (rho_ctx_tel_idx y sy Ty _ _ Hin') as [s' Hs'].
1771     + exact (retag_neq_of_atom_neq x s y s' Hxy Hs').

```



```

1769   + apply (IH i x s Hxnz Hfresh').
1770   apply in_map_iff. exists (retag x s, A). split; [reflexivity |
      exact Hin'].
1771 Qed.
1772
1773 Lemma subcontext_trans : forall  $\Gamma_1 \Gamma_2 \Gamma_3$ ,  $\Gamma_1 \subseteq \Gamma_2 \rightarrow \Gamma_2 \subseteq \Gamma_3 \rightarrow \Gamma_1 \subseteq \Gamma_3$ .
1774 Proof. intros  $\Gamma_1 \Gamma_2 \Gamma_3$  H12 H23 x T Hin. apply H23, H12, Hin. Qed.
1775
1776 Lemma rho_ctx_tel_mono : forall y s T k1 k2,  $k_1 \leq k_2 \rightarrow$ 
1777   rho_ctx_tel y s T  $k_1 \subseteq$  rho_ctx_tel y s T  $k_2$ .
1778 Proof.
1779   intros y s T k1 k2 Hle.
1780   induction k2 as [| k2' IH].
1781   - assert ( $k_1 = 0$ ) by lia. subst. intros p q Hpq. exact Hpq.
1782   - destruct (Nat.eq_dec k1 (S k2')) as [Heq | Hneq].
1783     + subst. intros p q Hpq. exact Hpq.
1784     + assert (Hle' :  $k_1 \leq k_2'$ ) by lia.
1785       intros p q Hpq. simpl. apply in_or_app. left. apply (IH Hle').
1786       exact Hpq.
1787 Qed.
1788
1789 Lemma app_subset_app : forall ( $\Gamma_1 \Gamma_2 \Delta_1 \Delta_2$  : context),
1790    $\Gamma_1 \subseteq \Delta_1 \rightarrow \Gamma_2 \subseteq \Delta_2 \rightarrow \Gamma_1 ++ \Gamma_2 \subseteq \Delta_1 ++ \Delta_2$ .
1791 Proof.
1792   intros  $\Gamma_1 \Gamma_2 \Delta_1 \Delta_2$  H1 H2 p q Hpq.
1793   apply in_app_or in Hpq. apply in_or_app.
1794   destruct Hpq as [Hpq | Hpq]; [left; apply H1 | right; apply H2];
1795   exact Hpq.
1796 Qed.
1797
1798 Lemma rho_ctx_mono : forall  $\Gamma$  i i',  $i \leq i' \rightarrow$  rho_ctx i'  $\Gamma \subseteq$  rho_ctx
1799 i  $\Gamma$ .
1800 Proof.
1801   induction  $\Gamma$  as [| [y [sy Ty]]  $\Gamma'$  IH]; intros i i' Hle.
1802   - simpl. intros p q Hpq. exact Hpq.
1803   - simpl. apply app_subset_app.
1804     + apply rho_ctx_tel_mono. lia.
1805     + apply (IH i i' Hle).
1806 Qed.
1807
1808 Axiom rho_pi_domain_erased_below :
1809   forall  $\Gamma_0$  x A0 B0 s1 T i,
1810   is_fresh x  $\Gamma_0 \rightarrow$ 
1811    $\Gamma_0 \vdash A_0 \in$  t_sort s1  $\rightarrow$ 
1812   index_of s1 < i + 1  $\rightarrow$ 
1813   rho_ctx (i + 1) ( $\Gamma_0 ++ [(x, (s1, A0))]$ )  $\vdash^*$  rho i (open_var B0 s1
1814 x)  $\in$  T  $\rightarrow$ 
1815   rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho i B0  $\in$  T.

```

```

1812
1813 Axiom rho_pi_tower :
1814   forall  $\Gamma_0$  A0 B0 s1 i L,
1815      $\Gamma_0 \vdash A0 \in t\_sort\ s1 \rightarrow$ 
1816     index_of s1  $\geq i + 1 \rightarrow$ 
1817     (forall x,  $\sim In\ x\ L \rightarrow$ 
1818       rho_ctx (i + 1) ( $\Gamma_0 \ ++\ [(x, (s1, A0))]$ )  $\vdash^*$  rho i (open_var B0
1819         s1 x)  $\in t\_sort\ (sort\_of\ (i + 1))$ )  $\rightarrow$ 
1820     rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho_pi_tel rho A0 B0 i (deg A0 - 1 - i)  $\in$ 
1821     t_sort (sort_of (i + 1)).
1822
1823 Axiom rho_lam_domain_erased_below :
1824   forall  $\Gamma_0$  x A0 M0 B0 s1 i,
1825     is_fresh x  $\Gamma_0 \rightarrow$ 
1826      $\Gamma_0 \vdash A0 \in t\_sort\ s1 \rightarrow$ 
1827     index_of s1  $< i + 2 \rightarrow$ 
1828     rho_ctx (i + 1) ( $\Gamma_0 \ ++\ [(x, (s1, A0))]$ )  $\vdash^*$  rho i (open_var M0 s1
1829       x)  $\in$  rho (i + 1) (open_var B0 s1 x)  $\rightarrow$ 
1830     rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho i M0  $\in$  rho (i + 1) B0.
1831
1832 Axiom rho_lam_tower :
1833   forall  $\Gamma_0$  A0 M0 B0 s1 i L,
1834      $\Gamma_0 \vdash A0 \in t\_sort\ s1 \rightarrow$ 
1835     index_of s1  $\geq i + 2 \rightarrow$ 
1836     (forall x,  $\sim In\ x\ L \rightarrow$ 
1837       rho_ctx (i + 1) ( $\Gamma_0 \ ++\ [(x, (s1, A0))]$ )  $\vdash^*$  rho i (open_var M0
1838         s1 x)  $\in$  rho (i + 1) (open_var B0 s1 x))  $\rightarrow$ 
1839     rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho_lam_tel rho A0 M0 i (deg A0 - 1 - (i +
1840       1))
1841        $\in$  rho_pi_tel rho A0 B0 (i + 1) (deg A0 - 1 -
1842         (i + 1)).
1843
1844 Axiom rho_app_tower :
1845   forall  $\Gamma_0$  M0 N0 A0 B0 sla i,
1846      $\Gamma_0 \vdash M0 \in t\_pi\ sla\ A0\ B0 \rightarrow$ 
1847      $\Gamma_0 \vdash N0 \in A0 \rightarrow$ 
1848     deg N0  $\geq i + 1 \rightarrow$ 
1849     rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho i M0  $\in$  rho (i + 1) (t_pi sla A0 B0)  $\rightarrow$ 
1850     rho_ctx (i + 1)  $\Gamma_0 \vdash^*$  rho i (t_app M0 N0)  $\in$  rho (i + 1) (B0 ^^
1851       N0).
1852
1853 Axiom rho_erased_subst_below : forall B N i,
1854   deg N  $< i + 1 \rightarrow$  rho (i + 1) (B ^^ N) = rho (i + 1) B.
1855
1856 Lemma deg_beq : forall M N, M =b N  $\rightarrow$  deg M = deg N.
1857 Proof.
1858   intros M N Heq.
1859   induction Heq as [M | M N Hstep | M N Heq IH | M N P Heq1 IH1 Heq2
1860     IH2].

```

```

1853 - reflexivity.
1854 - apply deg_beta. apply brt_step. exact Hstep.
1855 - symmetry. exact IH.
1856 - rewrite IH1. exact IH2.
1857 Qed.
1858
1859 Lemma rho_commutes_beta_eq : forall i, 0 <= i <= n ->
1860   forall M N, deg M >= i + 1 -> M =b N -> rho i M =b rho i N.
1861 Proof.
1862   intros i Hi M N Hdeg Heq.
1863   revert Hdeg.
1864   induction Heq as [M | M N Hstep | M N Heq IH | M N P Heq1 IH1 Heq2
1865     IH2]; intros Hdeg.
1866   - apply beq_refl.
1867   - apply beq_from_rtrans. apply (rho_commutes_beta i Hi M Hdeg N
1868     (brt_step M N Hstep)).
1869   - assert (HdegM : deg M >= i + 1) by (rewrite (deg_beq M N Heq);
1870     exact Hdeg).
1871     apply beq_sym. apply (IH HdegM).
1872   - assert (HdegN : deg N >= i + 1) by (rewrite <- (deg_beq M N Heq1);
1873     exact Hdeg).
1874     apply beq_trans with (rho i N).
1875     + apply (IH1 Hdeg).
1876     + apply (IH2 HdegN).
1877   Qed.
1878
1879 Lemma rho_commutes_typing :
1880   forall i, 0 <= i <= n ->
1881   forall Γ M N, Γ ⊢ M ∈ N ->
1882   deg M >= i + 1 ->
1883   rho_ctx (i + 1) Γ ⊢* (rho i M) ∈ (rho (i + 1) N).
1884 Proof.
1885   intros i.
1886   induction i as [i IHouter] using
1887     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n
1888       - i))).
1889   intros Hi Γ M N Htype.
1890   induction Htype as
1891     [ s s' HA
1892     | Γ0 x A0 s0 Hfresh HA IHA
1893     | Γ0 x B0 M0 A0 s0 Hfresh HM IHM HB IHB
1894     | Γ0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
1895     | Γ0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
1896     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
1897     | Γ0 M0 A0 B0 s HM IHM Heq HB IHB ];
1898   intros Hdeg.
1899
1900   - (* typing_axiom *)
1901     simpl in Hdeg.

```

```

1897 apply A_spec in HA as [Hjn Hjs'].
1898 pose proof n_at_least_two as Hn2.
1899 assert (HzeroTyped : []  $\vdash^*$  t_sort (sort_of 1)  $\in$  t_sort (sort_of
1900 2)).
1900 { apply typing_star_axiom. apply A_spec.
1901   rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
1902   rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
1903   lia. }
1904 destruct i as [| i'].
1905 + (* i = 0 *)
1906   simpl.
1907   apply typing_star_var with ( $\Gamma$  := []).
1908   * unfold is_fresh, dom. simpl. auto.
1909   * exact HzeroTyped.
1910 + (* i = S i' *)
1911   assert (HiSn : S i' < n) by lia.
1912   replace (S i' + 1) with (S (S i')) by lia.
1913   simpl.
1914   apply typing_star_weak with ( $\Gamma$  := []) (x := zero_var) (B :=
1915     t_sort (sort_of 1)).
1916   * unfold is_fresh, dom. simpl. auto.
1917   * apply typing_star_axiom. apply A_spec.
1918     rewrite (sort_of_correct (S i') ltac:(lia) ltac:(lia)).
1919     rewrite (sort_of_correct (S (S i')) ltac:(lia) ltac:(lia)).
1920     lia.
1921   * exact HzeroTyped.
1922 - (* typing_var *)
1923   simpl in Hdeg.
1924   pose proof (index_range s0) as [Hxlo Hxhi].
1925   assert (HdegA0 : deg A0 = index_of s0).
1926   { pose proof (deg_typing_succ  $\Gamma_0$  A0 (t_sort s0) HA) as Hds.
1927     unfold deg in *. simpl in Hds. lia. }
1928   assert (Hj2 : index_of s0  $\geq$  i + 2) by lia.
1929   assert (Hlt : ltof nat (fun k => n - k) (i + 1) i) by (unfold
1930     ltof; lia).
1931   assert (Hij : 0  $\leq$  i + 1  $\leq$  n) by (pose proof (index_range s0);
1932     lia).
1933   assert (HdegA0' : deg A0  $\geq$  (i + 1) + 1) by (lia).
1934   pose proof (IHouter (i + 1) Hlt Hij  $\Gamma_0$  A0 (t_sort s0) HA HdegA0')
1935     as HIH.
1936   replace (i + 1 + 1) with (S (S i)) in HIH by lia.
1937   simpl in HIH.
1938   replace (S (S i)) with (i + 2) in HIH by lia.
1939   pose (x' := retag x (sort_of (i + 2))).
1940   assert (Hxnz : x  $\neq$  zero_var)
1941     by (apply (typing_avoids_zero_var ( $\Gamma_0$  ++ [(x, (s0, A0))]))
1942       (t_fvar s0 x) A0
1943       (typing_var  $\Gamma_0$  x A0 s0 Hfresh HA) x s0 A0);

```

```

1940     apply in_or_app; right; left; reflexivity).
1941 assert (Hfreshx' : is_fresh x' (rho_ctx (i + 2)  $\Gamma_0$ ))
1942   by (apply rho_ctx_fresh; [exact Hxnz | exact Hfresh]).
1943 assert (Hstep : rho_ctx (i + 2)  $\Gamma_0$  ++ [(x', (sort_of (i + 2), rho
1944   (i + 1) A0))])
1945    $\vdash^*$  t_fvar (sort_of (i + 2)) x'  $\in$  rho (i + 1) A0)
1946   by (apply typing_star_var; [exact Hfreshx' | exact HIH]).
1947 assert (Hmem : In (x', (sort_of (i + 2), rho (i + 1) A0))
1948   (rho_ctx_tel x s0 A0 (index_of s0 - (i + 1) -
1949   1))).
1950 { replace (index_of s0 - (i + 1) - 1)
1951   with (index_of s0 - i - 2) by lia.
1952   pose proof (rho_ctx_tel_last x s0 A0 (index_of s0 - i - 2)) as
1953   Htel.
1954   replace (index_of s0 - (index_of s0 - i - 2) - 1)
1955   with (i + 1) in Htel by lia.
1956   replace (index_of s0 - (index_of s0 - i - 2))
1957   with (i + 2) in Htel by lia.
1958   unfold x'. exact Htel.
1959 }
1960 assert (Hincl : rho_ctx (i + 2)  $\Gamma_0$  ++ [(x', (sort_of (i + 2), rho
1961   (i + 1) A0))])
1962    $\subseteq$  rho_ctx (i + 1) ( $\Gamma_0$  ++ [(x, (s0, A0))])).
1963 { apply subcontext_app_same.
1964   - apply (subcontext_trans _ (rho_ctx (i + 1)  $\Gamma_0$ )).
1965   + apply rho_ctx_mono. lia.
1966   + apply rho_ctx_incl_old.
1967   - intros p q Hpq. destruct Hpq as [Heq | []]. inversion Heq;
1968     subst.
1969     apply (rho_ctx_incl_new  $\Gamma_0$  (i + 1) x s0 A0). exact Hmem.
1970 }
1971 simpl. unfold x' in Hstep, Hinc1.
1972 apply (typing_star_weakening_incl _ _ _ _ Hinc1 Hstep).
1973
1974 - (* typing_weak *)
1975 exact (typing_star_weakening_incl _ _ _ _ (rho_ctx_incl_old  $\Gamma_0$  (i
1976   + 1) x s0 B0)
1977   (IHM Hdeg)).
1978
1979 - (* typing_pi *)
1980 simpl in Hdeg.
1981 assert (Hs32 : s3 = s2) by (symmetry; apply (R_shape s1 s2 s3
1982   HR)).
1983 subst s3.
1984 assert (HdegA0 : deg A0 = index_of s1) by (apply (deg_sort_typed
1985    $\Gamma_0$  A0 s1 HA)).
1986 set (x0 := fresh (dom  $\Gamma_0$  ++ L)).
1987 assert (Hx0dom : is_fresh x0  $\Gamma_0$ ).
1988 { intro Hc. apply (fresh_notin (dom  $\Gamma_0$  ++ L)).

```

```

1981   apply in_or_app. left. exact Hc. }
1982 assert (Hx0L : ~ In x0 L).
1983 { intro Hc. apply (fresh_notin (dom Γ0 ++ L)).
1984   apply in_or_app. right. exact Hc. }
1985 assert (HBx0 : Γ0 ++ [(x0, (s1, A0))] ⊢ open_var B0 s1 x0 ∈ t_sort
s2) by (apply HB; exact Hx0L).
1986 assert (Hdegopen : deg (open_var B0 s1 x0) ≥ i + 1)
1987   by (rewrite <- (deg_open B0 x0 s1 HTagB); exact Hdeg).
1988 assert (Hcast : rho (i + 1) (t_sort s2) = t_sort (sort_of (i +
1))) .
1989 { replace (i + 1) with (S i) by lia. reflexivity. }
1990 destruct (le_lt_dec (i + 1) (deg A0)) as [HJge | HJlt].
1991
1992 + (* j ≥ i+1 *)
1993   rewrite (rho_pi_named i s1 A0 B0 HJge).
1994   rewrite Hcast.
1995   apply (rho_pi_tower Γ0 A0 B0 s1 i L HA).
1996   * lia.
1997   * intros x Hx.
1998     assert (HBx : Γ0 ++ [(x, (s1, A0))] ⊢ open_var B0 s1 x ∈
t_sort s2) by (apply HB; exact Hx).
1999     assert (Hd : deg (open_var B0 s1 x) ≥ i + 1)
2000       by (rewrite <- (deg_open B0 x s1 HTagB); exact Hdeg).
2001     pose proof (IHB x Hx Hd) as HIH.
2002     rewrite Hcast in HIH. exact HIH.
2003
2004 + (* j < i+1 *)
2005   assert (Hcollapse : rho i (t_pi s1 A0 B0) = rho i B0)
2006     by (simpl; destruct (le_lt_dec (i + 1) (deg A0)); [lia |
reflexivity]).
2007   rewrite Hcollapse, Hcast.
2008   assert (Hjlt' : index_of s1 < i + 1) by lia.
2009   apply (rho_pi_domain_erased_below Γ0 x0 A0 B0 s1 (t_sort
(sort_of (i + 1))) i
Hx0dom HA Hjlt').
2010   pose proof (IHB x0 Hx0L Hdegopen) as HIH.
2011   rewrite Hcast in HIH. exact HIH.
2012
2013 - (* typing_lam *)
2014   simpl in Hdeg.
2015   assert (HdegA0 : deg A0 = index_of s1) by (apply (deg_sort_typed
Γ0 A0 s1 HA)).
2016   set (x0 := fresh (dom Γ0 ++ L)).
2017   assert (Hx0dom : is_fresh x0 Γ0).
2018   { intro Hc. apply (fresh_notin (dom Γ0 ++ L)).
2019     apply in_or_app. left. exact Hc. }
2020   assert (Hx0L : ~ In x0 L).
2021   { intro Hc. apply (fresh_notin (dom Γ0 ++ L)).
2022     apply in_or_app. right. exact Hc. }
2023

```

```

2024  assert (HMx0 :  $\Gamma_0 \vdash [(x_0, (s_1, A_0))] \vdash \text{open\_var } M_0 \ s_1 \ x_0 \in$   

open_var B0 s1 x0) by (apply HM; exact Hx0L).  

2025  assert (Hdegopen : deg (open_var M0 s1 x0) >= i + 1)  

2026    by (rewrite <- (deg_open M0 x0 s1 HTagM); exact Hdeg).  

2027  destruct (le_lt_dec (i + 2) (deg A0)) as [HJge | HJlt].  

2028  

2029  + (* deg C >= i+2 *)  

2030    assert (Hlam : rho i (t_lam s1 A0 M0) = rho_lam_tel rho A0 M0 i  

(deg A0 - 1 - (i + 1)))  

2031      by (apply (rho_lam_named i s1 A0 M0 HJge)).  

2032    assert (HJge' : i + 1 + 1 <= deg A0) by lia.  

2033    assert (Hpi : rho (i + 1) (t_pi s1 A0 B0) = rho_pi_tel rho A0 B0  

(i + 1) (deg A0 - 1 - (i + 1)))  

2034      by (apply (rho_pi_named (i + 1) s1 A0 B0 HJge')).  

2035    rewrite Hlam, Hpi.  

2036    apply (rho_lam_tower  $\Gamma_0$  A0 M0 B0 s1 i L HA).  

2037    * lia.  

2038    * intros x Hx.  

2039    assert (HMx :  $\Gamma_0 \vdash [(x, (s_1, A_0))] \vdash \text{open\_var } M_0 \ s_1 \ x \in$   

open_var B0 s1 x) by (apply HM; exact Hx).  

2040    assert (Hd : deg (open_var M0 s1 x) >= i + 1)  

2041      by (rewrite <- (deg_open M0 x s1 HTagM); exact Hdeg).  

2042    apply (IHM x Hx Hd).  

2043  

2044  + (* deg C < i+2 *)  

2045    assert (Hcollapse1 : rho i (t_lam s1 A0 M0) = rho i M0)  

2046      by (simpl; destruct (le_lt_dec (i + 2) (deg A0)); [lia |  

reflexivity]).  

2047    assert (Hcollapse2 : rho (i + 1) (t_pi s1 A0 B0) = rho (i + 1)  

B0)  

2048      by (simpl; destruct (le_lt_dec (i + 1 + 1) (deg A0)); [lia |  

reflexivity]).  

2049    rewrite Hcollapse1, Hcollapse2.  

2050    assert (HJlt' : index_of s1 < i + 2) by lia.  

2051    apply (rho_lam_domain_erased_below  $\Gamma_0$  x0 A0 M0 B0 s1 i Hx0dom HA  

HJlt').  

2052    apply (IHM x0 Hx0L Hdegopen).  

2053  

2054  - (* typing_app *)  

2055    simpl in Hdeg.  

2056    assert (HdegC : deg A0 = deg N0 + 1)  

2057      by (pose proof (deg_typing_succ  $\Gamma_0$  N0 A0 HN) as Hh; lia).  

2058    destruct (le_lt_dec (i + 1) (deg N0)) as [HNge | HNlt].  

2059  

2060  + (* deg N >= i+1 *)  

2061    apply (rho_app_tower  $\Gamma_0$  M0 N0 A0 B0 sla i HM HN HNge).  

2062    exact (IHM Hdeg).  

2063  

2064  + (* deg N < i+1 *)

```



```

2065   assert (Hcollapse_app : rho i (t_app M0 N0) = rho i M0)
2066     by (simpl; destruct (le_lt_dec (i + 1) (deg N0)); [lia |
        reflexivity]).
2067   rewrite Hcollapse_app.
2068   assert (Hcol2 : rho (i + 1) (t_pi sla A0 B0) = rho (i + 1) B0)
2069     by (simpl; destruct (le_lt_dec (i + 1 + 1) (deg A0)); [lia |
        reflexivity]).
2070   assert (Htgt : rho (i + 1) (B0 ^^ N0) = rho (i + 1) (t_pi sla A0
        B0)).
2071   { rewrite Hcol2. apply (rho_erased_subst_below B0 N0 i HNlt). }
2072   rewrite Htgt.
2073   exact (IHM Hdeg).
2074
2075 - (* typing_conv *)
2076   simpl in Hdeg.
2077   destruct (typing_lc _ _ HM) as [HlcM _].
2078   assert (HdegM0 : deg M0 >= i + 1) by (lia).
2079   assert (HdegA0 : deg A0 = deg M0 + 1)
2080     by (pose proof (deg_typing_succ Γ0 M0 A0 HM); lia).
2081   assert (HdegAB : deg A0 = deg B0)
2082     by (apply (deg_conv_invariant Γ0 M0 A0 B0 s HM Heq HB)).
2083   assert (HdegBs : deg B0 = index_of s) by (apply (deg_sort_typed Γ0
        B0 s HB)).
2084   pose proof (index_range s) as [Hslo Hshi].
2085   assert (HsGe : index_of s >= i + 2) by lia.
2086   assert (Hlt : ltof nat (fun k => n - k) (i + 1) i) by (unfold
        ltof; lia).
2087   assert (Hij : 0 <= i + 1 <= n) by lia.
2088   assert (HdegA0' : deg A0 >= (i + 1) + 1) by lia.
2089   assert (HeqRho : rho (i + 1) A0 =b rho (i + 1) B0)
2090     by (apply (rho_commutates_beta_eq (i + 1) Hij A0 B0 HdegA0' Heq)).
2091   assert (HIH1 : rho_ctx (i + 1) Γ0 ⊢* rho i M0 ∈ rho (i + 1) A0)
2092     by (exact (IHM Hdeg)).
2093   assert (HdegB0' : deg B0 >= (i + 1) + 1) by (lia).
2094   pose proof (IHouter (i + 1) Hlt Hij Γ0 B0 (t_sort s) HB HdegB0')
        as HIH2.
2095   replace (i + 1 + 1) with (S (S i)) in HIH2 by lia.
2096   simpl in HIH2.
2097   replace (S (S i)) with (i + 2) in HIH2 by lia.
2098   assert (HIH2' : rho_ctx (i + 1) Γ0 ⊢* rho (i + 1) B0 ∈ t_sort
        (sort_of (i + 2))).
2099   { apply (typing_star_weakening_incl (rho_ctx (i + 2) Γ0)
        (rho_ctx_mono Γ0 (i + 1) (i + 2) ltac:(lia)) HIH2). }
2100   apply (typing_star_conv (rho_ctx (i + 1) Γ0) (rho i M0) (rho (i +
        1) A0)
2101     (rho (i + 1) B0) (sort_of (i + 2)) HIH1 HeqRho HIH2')).
2102
2103 Qed.
2104
2105 (* =====
*)

```



```

2106 (* Term-Level Translation *)
2107 (*  $\gamma : T \rightarrow T \ 0$  *)
2108
2109 Definition bullet_var : var := n + 2.
2110 Definition prod_var : var := 2 * (n + 2).
2111
2112 Lemma bullet_var_retag_ne : forall x s, retag x s <> bullet_var.
2113 Proof.
2114   intros x s. unfold bullet_var. replace (n + 2) with (1 * (n + 2)) by
2115     lia.
2116   apply (retag_neq_mult 1).
2117 Qed.
2118
2119 Lemma prod_var_retag_ne : forall x s, retag x s <> prod_var.
2120 Proof. intros x s. unfold prod_var. apply (retag_neq_mult 2). Qed.
2121
2122 Lemma prod_var_ne_bullet_var : prod_var <> bullet_var.
2123 Proof. unfold prod_var, bullet_var. pose proof n_range. lia. Qed.
2124
2125 Lemma prod_var_ne_zero_var : prod_var <> zero_var.
2126 Proof. unfold prod_var, zero_var. pose proof n_range. lia. Qed.
2127
2128 Lemma bullet_var_ne_zero_var : bullet_var <> zero_var.
2129 Proof. unfold bullet_var, zero_var. pose proof n_range. lia. Qed.
2130
2131 Fixpoint gamma_pi_tel (A body : term) (k : nat) : term :=
2132   match k with
2133   | 0      => t_pi (sort_of 0) (rho 0 A) body
2134   | S k'  => t_pi (sort_of (S k')) (rho (S k') A) (gamma_pi_tel A body k')
2135   end.
2136
2137 Fixpoint gamma_lam_tel (A body : term) (k : nat) : term :=
2138   match k with
2139   | 0      => t_lam (sort_of 0) (rho 0 A) body
2140   | S k'  => t_lam (sort_of (S k')) (rho (S k') A) (gamma_lam_tel A body k')
2141   end.
2142
2143 Fixpoint gamma_app_tel (M' N : term) (k : nat) : term :=
2144   match k with
2145   | 0      => t_app M' (rho 0 N)
2146   | S k'  => gamma_app_tel (t_app M' (rho (S k') N)) N k'
2147   end.
2148
2149 Fixpoint gamma (M : term) : term :=
2150   match M with
2151   | t_sort s    => t_fvar (sort_of 1) bullet_var
2152   | t_bvar s0 k => t_bvar s0 k
2153   | t_fvar s0 x => t_fvar (sort_of 1) (retag x (sort_of 1))

```

```

2153 | t_pi s A B =>
2154   t_app (t_app (t_app (t_fvar (sort_of 1) prod_var)
2155             (gamma_pi_tel A (t_fvar (sort_of 2) zero_var)
2156                           (deg A - 1)))
2157             (gamma A))
2158   (gamma_lam_tel A (gamma B) (deg A - 1))
2159 | t_lam s A M' =>
2160   t_app (t_lam (sort_of 1) (t_fvar (sort_of 2) zero_var)
2161         (gamma_lam_tel A (gamma M') (deg A - 1)))
2162   (gamma A)
2163 | t_app M' N =>
2164   t_app (gamma_app_tel (gamma M') N (deg N - 1)) (gamma N)
2165 end.
2166
2167 (* =====
2168 *)
2169 (** * gamma commutes with typing *)
2170
2171 (* prod :  $\prod A s1 . \emptyset \rightarrow A \rightarrow \emptyset$ , requires the rule (s2,s1), which we
2172    assume appear in  $\lambda S$ . *)
2173 Axiom R_s2_s1 : R (sort_of 2) (sort_of 1) (sort_of 1).
2174
2175 Lemma sort_of_1_ne_2 : sort_of 1 <> sort_of 2.
2176 Proof.
2177   pose proof n_at_least_two as Hn2.
2178   intro Heq.
2179   assert (H1 : index_of (sort_of 1) = 1) by (apply sort_of_correct;
2180     lia).
2181   assert (H2 : index_of (sort_of 2) = 2) by (apply sort_of_correct;
2182     lia).
2183   rewrite Heq in H1. lia.
2184 Qed.
2185
2186 Lemma R_s1_s1_s1 : R (sort_of 1) (sort_of 1) (sort_of 1).
2187 Proof.
2188   pose proof n_at_least_two as Hn2.
2189   pose proof (is_full _ _ _ R_s2_s1) as Hfw.
2190   unfold full_with in Hfw.
2191   apply Hfw.
2192   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2193   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2194   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2195     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2196     lia.
2197 Qed.
2198
2199 Definition T_prod : term :=
2200   t_pi (sort_of 2) (t_sort (sort_of 1))
2201   (t_pi (sort_of 1) (t_fvar (sort_of 2) zero_var)

```

```

2197      (t_pi (sort_of 1) (t_bvar (sort_of 2) 1) (t_fvar (sort_of 2)
2198      zero_var))).
2199
2200 Lemma subcontext_app_l : forall  $\Gamma$   $\Delta'$ ,  $\Gamma \subseteq \Gamma ++ \Delta'$ .
2201 Proof. intros  $\Gamma$   $\Delta'$  x T Hin. apply in_or_app. left. exact Hin. Qed.
2202
2203 Lemma subcontext_app_r : forall  $\Gamma$   $\Delta'$ ,  $\Gamma \subseteq \Delta' ++ \Gamma$ .
2204 Proof. intros  $\Gamma$   $\Delta'$  x T Hin. apply in_or_app. right. exact Hin. Qed.
2205
2206 Lemma zero_var_typed :
2207   [(zero_var, (sort_of 2, t_sort (sort_of 1)))]  $\vdash^*$  t_fvar (sort_of 2)
2208   zero_var  $\in$  t_sort (sort_of 1).
2209 Proof.
2210   pose proof n_at_least_two as Hn2.
2211   apply (typing_star_var [] zero_var (t_sort (sort_of 1)) (sort_of
2212   2)).
2213   - unfold is_fresh, dom. simpl. auto.
2214   - apply typing_star_axiom. apply A_spec.
2215     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2216     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2217     lia.
2218 Qed.
2219
2220 Lemma Tprod_typed :
2221   [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var, (sort_of
2222   1, t_fvar (sort_of 2) zero_var))]
2223    $\vdash^*$  T_prod  $\in$  t_sort (sort_of 1).
2224 Proof.
2225   pose proof n_at_least_two as Hn2.
2226   pose proof zero_var_typed as Hz1.
2227   assert (HtsortA : [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2228   (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))]
2229    $\vdash^*$  t_sort (sort_of 1)  $\in$  t_sort (sort_of 2)).
2230   { apply (typing_star_weakening_incl []).
2231     - intros x T [].
2232     - apply typing_star_axiom. apply A_spec.
2233       rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2234       rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2235       lia. }
2236   apply (typing_star_pi
2237     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2238     (sort_of 1, t_fvar (sort_of 2) zero_var))]
2239     (t_sort (sort_of 1))
2240     (t_pi (sort_of 1) (t_fvar (sort_of 2) zero_var)
2241     (t_pi (sort_of 1) (t_bvar (sort_of 2) 1) (t_fvar
2242     (sort_of 2) zero_var)))
2243     (sort_of 2) (sort_of 1) (sort_of 1)
2244     (dom [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2245     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))])).

```

```

2238 - exact HtsortA.
2239 - intros x Hx.
2240 unfold open_var. simpl.
2241 assert (Hx_typed :
2242   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
    (x, (sort_of 2, t_sort (sort_of 1)))]
2243    $\vdash$  t_fvar (sort_of 2) x  $\in$  t_sort (sort_of 1)).
2244 { apply (typing_star_var
2245   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))]
2246   x (t_sort (sort_of 1)) (sort_of 2)).
2247   - exact Hx.
2248   - exact HtsortA. }
2249 apply (typing_star_pi
2250   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2251   (x, (sort_of 2, t_sort (sort_of 1)))]
2252   (t_fvar (sort_of 2) zero_var)
2253   (t_pi (sort_of 1) (t_fvar (sort_of 2) x) (t_fvar (sort_of
2254   2) zero_var))
2255   (sort_of 1) (sort_of 1) (sort_of 1) []).
2256 + apply (typing_star_weakening_incl [(zero_var, (sort_of 2, t_sort
2257   (sort_of 1)))]).
2258   * apply (subcontext_app_l [(zero_var, (sort_of 2, t_sort
2259   (sort_of 1)))]).
2260   * exact Hz1.
2261 + intros y Hy.
2262 unfold open_var. simpl.
2263 apply (typing_star_pi
2264   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2265   (x, (sort_of 2, t_sort (sort_of 1))); (y, (sort_of 1,
2266   t_fvar (sort_of 2) zero_var))]
2267   (t_fvar (sort_of 2) x) (t_fvar (sort_of 2) zero_var)
2268   (sort_of 1) (sort_of 1) (sort_of 1) []).
2269 * apply (typing_star_weakening_incl
2270   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2)
2271   zero_var));
2272   (x, (sort_of 2, t_sort (sort_of 1)))]).
2273 -- apply (subcontext_app_l
2274   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
    (bullet_var, (sort_of 1, t_fvar (sort_of 2)
2275   zero_var));
2276   (x, (sort_of 2, t_sort (sort_of 1)))]).
2277 -- exact Hx_typed.
2278 * intros z Hz.
2279 unfold open_var. simpl.

```

```

2274     apply (typing_star_weakening_incl [(zero_var, (sort_of 2,
2275       t_sort (sort_of 1)))]).
2276     -- apply (subcontext_app_l [(zero_var, (sort_of 2, t_sort
2277       (sort_of 1)))]).
2278     -- exact Hz1.
2279     * split; [exact R_s1_s1_s1 |].
2280     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2281 + split; [exact R_s1_s1_s1 |].
2282     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2283     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2284     lia.
2285 Qed.
2286
2287 Lemma bullet_typed :
2288   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2289     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2290     (prod_var, (sort_of 1, T_prod))] ⊢* t_fvar (sort_of 1) bullet_var ∈
2291     t_fvar (sort_of 2) zero_var.
2292 Proof.
2293   pose proof n_at_least_two as Hn2.
2294   pose proof zero_var_typed as Hz1.
2295   assert (Hb1 : [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2296     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))]
2297     ⊢* t_fvar (sort_of 1) bullet_var ∈ t_fvar (sort_of 2)
2298     zero_var).
2299   { apply (typing_star_var [(zero_var, (sort_of 2, t_sort (sort_of
2300     1)))] bullet_var (t_fvar (sort_of 2) zero_var) (sort_of 1)).
2301     - unfold is_fresh, dom. simpl.
2302     intros [Heq | []].
2303     apply bullet_var_ne_zero_var. symmetry. exact Heq.
2304     - exact Hz1. }
2305   apply (typing_star_weak
2306     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2307       (sort_of 1, t_fvar (sort_of 2) zero_var))]
2308     prod_var T_prod (t_fvar (sort_of 1) bullet_var) (t_fvar
2309       (sort_of 2) zero_var) (sort_of 1)).
2310   - unfold is_fresh, dom. simpl.
2311     intros [Heq | [Heq | []]].
2312     + apply prod_var_ne_zero_var. symmetry. exact Heq.
2313     + apply prod_var_ne_bullet_var. symmetry. exact Heq.
2314     - exact Hb1.
2315     - exact Tprod_typed.
2316 Qed.
2317
2318 Lemma zero_var_typed_Δ :
2319   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2320     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));

```

```

2315   (prod_var, (sort_of 1, T_prod))] ⊢* t_fvar (sort_of 2) zero_var ∈
      t_sort (sort_of 1).
2316 Proof.
2317   apply (typing_star_weakening_incl [(zero_var, (sort_of 2, t_sort
      (sort_of 1))))].
2318   - apply (subcontext_app_l [(zero_var, (sort_of 2, t_sort (sort_of
      1))))].
2319   - exact zero_var_typed.
2320 Qed.
2321
2322 Lemma prod_var_typed_Δ :
2323   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2324    (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2325    (prod_var, (sort_of 1, T_prod))] ⊢* t_fvar (sort_of 1) prod_var ∈
      T_prod.
2326 Proof.
2327   apply (typing_star_var [(zero_var, (sort_of 2, t_sort (sort_of 1)));
      (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))]
2328     prod_var T_prod (sort_of 1)).
2329   - unfold is_fresh, dom. simpl.
2330     intros [Heq | [Heq | []]].
2331     + apply prod_var_ne_zero_var. symmetry. exact Heq.
2332     + apply prod_var_ne_bullet_var. symmetry. exact Heq.
2333   - exact Tprod_typed.
2334 Qed.
2335
2336 Axiom gamma_pi_tower :
2337   forall Δ0 Γ0 A0 s1,
2338     Δ0 ⊢* t_fvar (sort_of 2) zero_var ∈ t_sort (sort_of 1) ->
2339     Γ0 ⊢ A0 ∈ t_sort s1 ->
2340     Δ0 ++ rho_ctx 0 Γ0 ⊢* gamma_pi_tel A0 (t_fvar (sort_of 2)
      zero_var) (deg A0 - 1) ∈ t_sort (sort_of 1).
2341
2342 Axiom gamma_lam_tower :
2343   forall Δ0 Γ0 A0 B0 s1 L,
2344     Δ0 ⊢* t_fvar (sort_of 2) zero_var ∈ t_sort (sort_of 1) ->
2345     Γ0 ⊢ A0 ∈ t_sort s1 ->
2346     (forall x, ~ In x L ->
2347       Δ0 ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))]) ⊢* gamma (open_var B0
      s1 x) ∈ t_fvar (sort_of 2) zero_var) ->
2348     Δ0 ++ rho_ctx 0 Γ0 ⊢* gamma_lam_tel A0 (gamma B0) (deg A0 - 1)
2349       ∈ gamma_pi_tel A0 (t_fvar (sort_of 2)
      zero_var) (deg A0 - 1).
2350
2351 Lemma gamma_pi_tel_eq_rho_pi_tel : forall A B k,
2352   gamma_pi_tel A (rho 0 B) k = rho_pi_tel rho A B 0 k.
2353 Proof.
2354   intros A B k. induction k as [| k' IH].
2355   - reflexivity.

```

```

2356   - simpl. rewrite IH. reflexivity.
2357 Qed.
2358
2359 Lemma rho0_pi_eq_gamma_pi_tel : forall  $\Gamma$ 0 A0 B0 s1,
2360    $\Gamma$ 0  $\vdash$  A0  $\in$  t_sort s1 ->
2361   rho 0 (t_pi s1 A0 B0) = gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1).
2362 Proof.
2363   intros  $\Gamma$ 0 A0 B0 s1 HA.
2364   pose proof (deg_sort_typed  $\Gamma$ 0 A0 s1 HA) as HdegA0.
2365   pose proof (index_range s1) as [Hlo Hhi].
2366   assert (Hge : 0 + 1 <= deg A0) by lia.
2367   rewrite (rho_pi_named 0 s1 A0 B0 Hge).
2368   replace (deg A0 - 1 - 0) with (deg A0 - 1) by lia.
2369   symmetry. apply gamma_pi_tel_eq_rho_pi_tel.
2370 Qed.
2371
2372 Axiom gamma_lam_tower_D :
2373   forall  $\Delta$ 0  $\Gamma$ 0 A0 M0 B0 s1 L,
2374    $\Gamma$ 0  $\vdash$  A0  $\in$  t_sort s1 ->
2375   (forall x, ~ In x L ->
2376      $\Delta$ 0 ++ rho_ctx 0 ( $\Gamma$ 0 ++ [(x, (s1, A0))])  $\vdash^*$  gamma (open_var M0
2377       s1 x)  $\in$  rho 0 (open_var B0 s1 x)) ->
2378    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  gamma_lam_tel A0 (gamma M0) (deg A0 - 1)
2379      $\in$  gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1).
2380
2381 Axiom gamma_lam_applied :
2382   forall  $\Delta$ 0  $\Gamma$ 0 A0 lamTerm piType,
2383    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  lamTerm  $\in$  piType ->
2384    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  piType  $\in$  t_sort (sort_of 1) ->
2385    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  gamma A0  $\in$  t_fvar (sort_of 2) zero_var ->
2386   t_app (t_lam (sort_of 1) (t_fvar (sort_of 2) zero_var) lamTerm)
2387     (gamma A0)  $\in$  piType.
2388
2389 Axiom gamma_app_tower :
2390   forall  $\Delta$ 0  $\Gamma$ 0 M0 N0 A0 B0 s1a,
2391    $\Gamma$ 0  $\vdash$  M0  $\in$  t_pi s1a A0 B0 ->
2392    $\Gamma$ 0  $\vdash$  N0  $\in$  A0 ->
2393    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  gamma M0  $\in$  rho 0 (t_pi s1a A0 B0) ->
2394    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  gamma N0  $\in$  rho 0 A0 ->
2395    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  t_app (gamma_app_tel (gamma M0) N0 (deg N0 -
2396     1)) (gamma N0)
2397      $\in$  rho 0 (B0 ^^ N0).
2398
2399 Axiom gamma_prod_applied :
2400   forall  $\Delta$ 0  $\Gamma$ 0 A0 lamterm,
2401    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  t_fvar (sort_of 1) prod_var  $\in$  T_prod ->
2402    $\Delta$ 0 ++ rho_ctx 0  $\Gamma$ 0  $\vdash^*$  gamma_pi_tel A0 (t_fvar (sort_of 2)
2403     zero_var) (deg A0 - 1)  $\in$  t_sort (sort_of 1) ->

```



```

2401  $\Delta 0 \text{ ++ rho\_ctx } 0 \text{ } \Gamma 0 \vdash^* \text{ gamma } A0 \in \text{ t\_fvar (sort\_of 2) zero\_var } \rightarrow$ 
2402  $\Delta 0 \text{ ++ rho\_ctx } 0 \text{ } \Gamma 0 \vdash^* \text{ lamterm } \in \text{ gamma\_pi\_tel } A0 \text{ (t\_fvar (sort\_of$ 
2403  $\Delta 0 \text{ ++ rho\_ctx } 0 \text{ } \Gamma 0 \vdash^*$ 
2404  $\text{ t\_app (t\_app (t\_app (t\_fvar (sort\_of 1) prod\_var) (gamma\_pi\_tel$ 
 $\text{ A0 (t\_fvar (sort\_of 2) zero\_var) (deg A0 - 1)))}$ 
2405  $\text{ (gamma A0))}$ 
2406  $\text{ lamterm}$ 
2407  $\in \text{ t\_fvar (sort\_of 2) zero\_var.}$ 
2408
2409 Lemma gamma_commutes_typing :
2410 forall  $\Gamma \text{ M N, } \Gamma \vdash \text{ M } \in \text{ N } \rightarrow$ 
2411  $[(\text{zero\_var, (sort\_of 2, t\_sort (sort\_of 1)))};$ 
2412  $(\text{bullet\_var, (sort\_of 1, t\_fvar (sort\_of 2) zero\_var)});$ 
2413  $(\text{prod\_var, (sort\_of 1, T\_prod)})] \text{ ++ rho\_ctx } 0 \text{ } \Gamma \vdash^* \text{ gamma M } \in \text{ rho } 0$ 
 $\text{ N.}$ 
2414 Proof.
2415 intros  $\Gamma \text{ M N Htype.}$ 
2416 remember  $[(\text{zero\_var, (sort\_of 2, t\_sort (sort\_of 1)))};$ 
2417  $(\text{bullet\_var, (sort\_of 1, t\_fvar (sort\_of 2) zero\_var)});$ 
2418  $(\text{prod\_var, (sort\_of 1, T\_prod)})] \text{ as } \Delta.$ 
2419 induction Htype as
2420  $[ \text{ s s' HA}$ 
2421  $| \text{ } \Gamma 0 \text{ x A0 s0 Hfresh HA IHA}$ 
2422  $| \text{ } \Gamma 0 \text{ x B0 M0 A0 s0 Hfresh HM IHM HB IHB}$ 
2423  $| \text{ } \Gamma 0 \text{ A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR}$ 
2424  $| \text{ } \Gamma 0 \text{ A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi}$ 
2425  $| \text{ } \Gamma 0 \text{ M0 N0 A0 B0 s1a HM IHM HN IHN}$ 
2426  $| \text{ } \Gamma 0 \text{ M0 A0 B0 s HM IHM Heq HB IHB } ].$ 
2427
2428 - (* typing_axiom *)
2429 simpl.
2430 apply (typing_star_weakening_incl  $\Delta$ ).
2431 + apply (subcontext_app_l  $\Delta$ ).
2432 + rewrite Heq $\Delta$ . exact bullet_typed.
2433
2434 - (* typing_var *)
2435 pose proof (index_range s0) as [Hxlo Hxhi].
2436 pose proof (deg_sort_typed  $\Gamma 0 \text{ A0 s0 HA}$ ) as HdegA0.
2437 assert (Hdeg1 : deg A0  $\geq 0 + 1$ ) by lia.
2438 pose proof (rho_commutes_typing 0 (ltac:(lia))  $\Gamma 0 \text{ A0 (t\_sort s0)}$ 
 $\text{ HA Hdeg1}$ ) as HB1.
2439 simpl in HB1.
2440 assert (HB0 : rho_ctx 0  $\Gamma 0 \vdash^* \text{ rho } 0 \text{ A0 } \in \text{ t\_sort (sort\_of 1)}$ )
2441 by (apply (typing_star_weakening_incl (rho_ctx 1  $\Gamma 0$ ) (rho_ctx 0
 $\Gamma 0$ )));
2442  $[ \text{ apply rho\_ctx\_mono; lia | exact HB1} ]$ .
2443 pose (x' := retag x (sort_of 1)).
2444 assert (Hxnz : x  $\neq$  zero_var)

```



```

2445     by (apply (typing_avoids_zero_var (Γ0 ++ [(x, (s0, A0))]))
2446         (t_fvar s0 x) A0
2447         (typing_var Γ0 x A0 s0 Hfresh HA) x s0 A0);
2448     apply in_or_app; right; left; reflexivity).
2449 assert (Hfreshx' : is_fresh x' (rho_ctx 0 Γ0))
2450     by (apply rho_ctx_fresh; [exact Hxnz | exact Hfresh]).
2451 assert (Hstep : rho_ctx 0 Γ0 ++ [(x', (sort_of 1, rho 0 A0))] ⊢*
2452     t_fvar (sort_of 1) x' ∈ rho 0 A0)
2453     by (apply typing_star_var; [exact Hfreshx' | exact HB0]).
2454 assert (Hmem : In (x', (sort_of 1, rho 0 A0))
2455     (rho_ctx_tel x s0 A0 (index_of s0 - 0 - 1))).
2456 { pose proof (rho_ctx_tel_last x s0 A0 (index_of s0 - 0 - 1)) as
2457   Htel.
2458   replace (index_of s0 - (index_of s0 - 0 - 1))
2459       with 1 in Htel by lia.
2460   simpl in Htel.
2461   unfold x'. exact Htel.
2462 }
2463 assert (Hincl : rho_ctx 0 Γ0 ++ [(x', (sort_of 1, rho 0 A0))]
2464     ⊆ rho_ctx 0 (Γ0 ++ [(x, (s0, A0))])).
2465 { apply subcontext_app_same.
2466   - apply rho_ctx_incl_old.
2467   - intros p q Hpq. destruct Hpq as [Heq | []]. inversion Heq;
2468     subst.
2469     apply (rho_ctx_incl_new Γ0 0 x s0 A0). exact Hmem.
2470 }
2471 assert (Hfinal : rho_ctx 0 (Γ0 ++ [(x, (s0, A0))])) ⊢* t_fvar
2472     (sort_of 1) x' ∈ rho 0 A0)
2473     by (apply (typing_star_weakening_incl _ _ _ _ Hincl Hstep)).
2474 simpl. unfold x' in Hfinal.
2475 apply (typing_star_weakening_incl (rho_ctx 0 (Γ0 ++ [(x, (s0,
2476     A0))]))
2477     (Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s0, A0))]))).
2478 + apply (subcontext_app_r (rho_ctx 0 (Γ0 ++ [(x, (s0, A0))])) Δ).
2479 + exact Hfinal.
2480
2481 - (* typing_weak *)
2482 apply (typing_star_weakening_incl (Δ ++ rho_ctx 0 Γ0)
2483     (Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s0, B0))]))).
2484 + apply app_subset_app.
2485   * intros p q Hpq. exact Hpq.
2486   * apply rho_ctx_incl_old.
2487 + exact IHM.
2488
2489 - (* typing_pi *)
2490 simpl.
2491 assert (HzeroΔ0 : Δ ⊢* t_fvar (sort_of 2) zero_var ∈ t_sort
2492     (sort_of 1))
2493     by (rewrite HeqΔ; exact zero_var_typed_Δ).

```

```

2487 assert (HProdTyped :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* t\_fvar \ (sort\_of \ 1)$ 
prod_var  $\in T\_prod$ ).
2488 { apply (typing_star_weakening_incl  $\Delta$ ).
2489   - apply (subcontext_app_l  $\Delta$ ).
2490   - rewrite Heq $\Delta$ . exact prod_var_typed $\Delta$ . }
2491 assert (HpiTel :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* gamma\_pi\_tel \ A_0 \ (t\_fvar$ 
(sort_of 2) zero_var) (deg  $A_0 - 1$ )
 $\in t\_sort \ (sort\_of \ 1)$ )
2492   by (apply (gamma_pi_tower  $\Delta \ \Gamma_0 \ A_0 \ s1 \ Hzero\Delta_0 \ HA$ )).
2493 assert (HgammaA :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* gamma \ A_0 \in t\_fvar \ (sort\_of$ 
2) zero_var)
2494   by (simpl in IHA; exact IHA).
2495 assert (HBcofin : forall x, ~ In x L ->
2496    $\Delta \rightarrow \rho\_ctx \ 0 \ (\Gamma_0 \rightarrow [(x, (s1, A_0))]) \vdash^* gamma$ 
(open_var B0 s1 x)  $\in t\_fvar \ (sort\_of \ 2) \ zero\_var$ ).
2497 { intros x Hx. pose proof (IHB x Hx) as H. simpl in H. exact H. }
2498 assert (HlamTel :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* gamma\_lam\_tel \ A_0 \ (gamma \ B_0)$ 
(deg  $A_0 - 1$ )
 $\in gamma\_pi\_tel \ A_0 \ (t\_fvar \ (sort\_of \ 2)$ 
zero_var) (deg  $A_0 - 1$ ))
2500   by (apply (gamma_lam_tower  $\Delta \ \Gamma_0 \ A_0 \ B_0 \ s1 \ L \ Hzero\Delta_0 \ HA \ HBcofin$ )).
2501 apply (gamma_prod_applied  $\Delta \ \Gamma_0 \ A_0 \ (gamma\_lam\_tel \ A_0 \ (gamma \ B_0)$ 
(deg  $A_0 - 1$ ))
HProdTyped HpiTel HgammaA HlamTel).
2502
2503 - (* typing_lam *)
2504 rewrite (rho0_pi_eq_gamma_pi_tel  $\Gamma_0 \ A_0 \ B_0 \ s1 \ HA$ ).
2505 assert (HdegPi1 : deg (t_pi s1 A0 B0)  $\geq 0 + 1$ )
2506   by (pose proof (index_range s3) as [Hlo Hhi];
2507   pose proof (deg_sort_typed  $\Gamma_0 \ (t\_pi \ s1 \ A_0 \ B_0) \ s3 \ HPi$ ) lia).
2508 pose proof (rho_commutes_typing 0 (ltac:(lia))  $\Gamma_0 \ (t\_pi \ s1 \ A_0 \ B_0)$ 
(t_sort s3) HPi HdegPi1)
2509   as HpiTelD1.
2510 simpl in HpiTelD1.
2511 assert (HpiTelD0 :  $\rho\_ctx \ 0 \ \Gamma_0 \vdash^* \rho \ 0 \ (t\_pi \ s1 \ A_0 \ B_0) \in t\_sort$ 
(sort_of 1))
2512   by (apply (typing_star_weakening_incl ( $\rho\_ctx \ 1 \ \Gamma_0$ ) ( $\rho\_ctx \ 0$ 
 $\Gamma_0$ )));
2513   [apply rho_ctx_mono; lia | exact HpiTelD1]].
2514 rewrite (rho0_pi_eq_gamma_pi_tel  $\Gamma_0 \ A_0 \ B_0 \ s1 \ HA$ ) in HpiTelD0.
2515 assert (HpiTelD :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* gamma\_pi\_tel \ A_0 \ (\rho \ 0 \ B_0)$ 
(deg  $A_0 - 1$ )
 $\in t\_sort \ (sort\_of \ 1)$ )
2516   by (apply (typing_star_weakening_incl ( $\rho\_ctx \ 0 \ \Gamma_0$ ) ( $\Delta \rightarrow$ 
 $\rho\_ctx \ 0 \ \Gamma_0$ )));
2517   [apply (subcontext_app_r ( $\rho\_ctx \ 0 \ \Gamma_0$ )  $\Delta$ ) | exact
HpiTelD0]].
2518 assert (HgammaA :  $\Delta \rightarrow \rho\_ctx \ 0 \ \Gamma_0 \vdash^* gamma \ A_0 \in t\_fvar \ (sort\_of$ 
2) zero_var)

```

```

2522   by (simpl in IHA; exact IHA).
2523   assert (HMcofin : forall x, ~ In x L ->
2524     Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))]) ⊢* gamma
      (open_var M0 s1 x) ∈ rho 0 (open_var B0 s1 x)).
2525 { intros x Hx. exact (IHM x Hx). }
2526   assert (HlamTelD : Δ ++ rho_ctx 0 Γ0 ⊢* gamma_lam_tel A0 (gamma
      M0) (deg A0 - 1)
      ∈ gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1))
2527   by (apply (gamma_lam_tower_D Δ Γ0 A0 M0 B0 s1 L HA HMcofin)).
2528   apply (gamma_lam_applied Δ Γ0 A0
2529     (gamma_lam_tel A0 (gamma M0) (deg A0 - 1))
2530     (gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1))
2531     HlamTelD HpiTelD HgammaA).
2532
2533 - (* typing_app *)
2534   apply (gamma_app_tower Δ Γ0 M0 N0 A0 B0 s1a HM HN IHM IHN).
2535
2536 - (* typing_conv *)
2537   assert (HdegA0 : deg A0 ≥ 0 + 1)
2538     by (pose proof (deg_typing_succ Γ0 M0 A0 HM); lia).
2539   assert (HeqRho : rho 0 A0 =b rho 0 B0)
2540     by (apply (rho_commutes_beta_eq 0 (ltac:(lia)) A0 B0 HdegA0
      Heq)).
2541   assert (HdegB1 : deg B0 ≥ 0 + 1)
2542     by (pose proof (index_range s) as [Hlo Hhi];
      pose proof (deg_sort_typed Γ0 B0 s HB); lia).
2543   pose proof (rho_commutes_typing 0 (ltac:(lia)) Γ0 B0 (t_sort s) HB
      HdegB1) as HrhoB1.
2544   simpl in HrhoB1.
2545   assert (HrhoB0 : rho_ctx 0 Γ0 ⊢* rho 0 B0 ∈ t_sort (sort_of 1))
2546     by (apply (typing_star_weakening_incl (rho_ctx 1 Γ0) (rho_ctx 0
      Γ0)));
2547     [apply rho_ctx_mono; lia | exact HrhoB1]].
2548   assert (HrhoB0Δ : Δ ++ rho_ctx 0 Γ0 ⊢* rho 0 B0 ∈ t_sort (sort_of
      1))
2549     by (apply (typing_star_weakening_incl (rho_ctx 0 Γ0) (Δ ++
      rho_ctx 0 Γ0)));
2550     [apply (subcontext_app_r (rho_ctx 0 Γ0) Δ) | exact HrhoB0]].
2551   apply (typing_star_conv (Δ ++ rho_ctx 0 Γ0) (gamma M0) (rho 0 A0)
      (rho 0 B0)
      (sort_of 1) IHM HeqRho HrhoB0Δ).
2552
2553 Qed.
2554
2555 (* =====
2556 *)
2557 (* gamma commutes with substitution *)
2558
2559 Fixpoint gamma_subst_tel (M B : term) (x : var) (K : nat) : term :=
2560   match K with
2561

```

```

2562 | 0      => M
2563 | S K'   => gamma_subst_tel (M [ retag x (sort_of (K' + 2)) = rho K' B
    ] ) B x K'
2564 end.
2565
2566 Definition gamma_subst (A B : term) (x : var) (s : Sort) : term :=
2567   (gamma_subst_tel (gamma A) B x (index_of s - 1)) [ retag x (sort_of
    1) = gamma B ].
2568
2569 Lemma sort_of_1_ne_sort_of_m : forall m, 2 <= m -> m < n + 1 ->
    sort_of 1 <> sort_of m.
2570 Proof.
2571   intros m Hm2 Hmn Heq.
2572   assert (H1 : index_of (sort_of 1) = 1) by (apply sort_of_correct;
    lia).
2573   assert (H2 : index_of (sort_of m) = m) by (apply sort_of_correct;
    lia).
2574   rewrite Heq in H1. lia.
2575 Qed.
2576
2577 Lemma gamma_subst_tel_fvar_const : forall sC C x B K,
2578   (forall m, 2 <= m -> m <= K + 1 -> retag x (sort_of m) <> C) ->
2579   gamma_subst_tel (t_fvar sC C) B x K = t_fvar sC C.
2580 Proof.
2581   intros sC C x B K.
2582   induction K as [| K' IH]; intros Hm.
2583   - reflexivity.
2584   - simpl.
2585     destruct (eq_var_dec C (retag x (sort_of (K' + 2)))) as [Heq |
    Hneq].
2586     + exfalso. apply (Hm (K' + 2) ltac:(lia) ltac:(lia)). symmetry.
    exact Heq.
2587     + apply IH. intros m Hm2 Hm3. apply Hm; lia.
2588 Qed.
2589
2590 Lemma gamma_subst_tel_bvar_const : forall s0 k0 x B K,
2591   gamma_subst_tel (t_bvar s0 k0) B x K = t_bvar s0 k0.
2592 Proof.
2593   intros s0 k0 x B K.
2594   induction K as [| K' IH].
2595   - reflexivity.
2596   - simpl. exact IH.
2597 Qed.
2598
2599 Lemma gamma_subst_tel_app_commute : forall P Q x B K,
2600   gamma_subst_tel (t_app P Q) B x K
2601   = t_app (gamma_subst_tel P B x K) (gamma_subst_tel Q B x K).
2602 Proof.
2603   intros P Q x B K. revert P Q.

```

```

2604 induction K as [| K' IH]; intros P Q.
2605 - reflexivity.
2606 - simpl. apply IH.
2607 Qed.
2608
2609 Lemma gamma_subst_tel_lam_commute : forall s' A' M' x B K,
2610   gamma_subst_tel (t_lam s' A' M') B x K
2611   = t_lam s' (gamma_subst_tel A' B x K) (gamma_subst_tel M' B x K).
2612 Proof.
2613   intros s' A' M' x B K. revert A' M'.
2614   induction K as [| K' IH]; intros A' M'.
2615   - reflexivity.
2616   - simpl. apply IH.
2617 Qed.
2618
2619 Lemma gamma_subst_sentinel_const : forall sC C B x s,
2620   (forall y sy, retag y sy <> C) ->
2621   (gamma_subst_tel (t_fvar sC C) B x (index_of s - 1))
2622   [ retag x (sort_of 1) = gamma B ]
2623   = t_fvar sC C.
2624 Proof.
2625   intros sC C B x s Hsent.
2626   assert (Hconst : gamma_subst_tel (t_fvar sC C) B x (index_of s - 1)
2627     = t_fvar sC C).
2628   { apply gamma_subst_tel_fvar_const.
2629     intros m Hm2 Hm3 Heq. apply (Hsent x (sort_of m)). exact Heq. }
2629   rewrite Hconst. simpl.
2630   destruct (eq_var_dec C (retag x (sort_of 1))) as [Heq | Hneq].
2631   - exfalso. apply (Hsent x (sort_of 1)). symmetry. exact Heq.
2632   - reflexivity.
2633 Qed.
2634
2635 Axiom gamma_pi_tel_commutes_subst : forall C B x s KC,
2636   x <> zero_var -> 1 <= index_of s <= n ->
2637   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->
2638   (gamma_subst_tel (gamma_pi_tel C (t_fvar (sort_of 2) zero_var) KC) B
2639     x (index_of s - 1))
2640   [ retag x (sort_of 1) = gamma B ]
2641   = gamma_pi_tel (C [ x = B ]) (t_fvar (sort_of 2) zero_var) KC.
2642
2643 Axiom gamma_lam_tel_commutes_subst : forall C D B x s KC,
2644   x <> zero_var -> 1 <= index_of s <= n ->
2645   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->
2646   (gamma_subst_tel (gamma_lam_tel C D KC) B x (index_of s - 1))
2647   [ retag x (sort_of 1) = gamma B ]
2648   = gamma_lam_tel (C [ x = B ])
2649     ((gamma_subst_tel D B x (index_of s - 1))
2650      [ retag x (sort_of 1) = gamma B ])
2650   KC.
2651

```

```

2652 Axiom gamma_app_tel_commutates_subst : forall MG C B x s KC,
2653   x <> zero_var -> 1 <= index_of s <= n ->
2654   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->
2655   (gamma_subst_tel (gamma_app_tel MG C KC) B x (index_of s - 1))
2656   [ retag x (sort_of 1) = gamma B ]
2657   = gamma_app_tel
2658     ((gamma_subst_tel MG B x (index_of s - 1))
2659      [ retag x (sort_of 1) = gamma B ])
2660     (C [ x = B ]) KC.
2661
2662 Lemma gamma_commutates_substitution :
2663   forall x s, x <> bullet_var -> x <> zero_var -> x <> prod_var ->
2664   1 <= index_of s <= n ->
2665   forall A, only_tagged x s A ->
2666   forall B, deg B = index_of s - 1 -> lc B ->
2667   gamma (A [ x = B ]) = gamma_subst A B x s.
2668 Proof.
2669   intros x s Hxb Hxz Hxp Hxrange A.
2670   unfold gamma_subst.
2671   induction A as [s0 | s_m m | sy y | D IHD C IHC | s1 C IHC D IHD |
2672   s1 C IHC D IHD];
2673   intros HonlyA B HdegB HlcB.
2674
2675   - (* A = t_sort s0 *)
2676     rewrite subst_sort. simpl.
2677     rewrite (gamma_subst_sentinel_const (sort_of 1) bullet_var B x s
2678     bullet_var_retag_ne).
2679     reflexivity.
2680
2681   - (* A = t_bvar s_m m *)
2682     rewrite subst_bvar. simpl.
2683     rewrite gamma_subst_tel_bvar_const, subst_bvar.
2684     reflexivity.
2685
2686   - (* A = t_fvar sy y *)
2687     rewrite subst_var.
2688     destruct (eq_var_dec y x) as [Heq | Hneq].
2689     + subst y.
2690       simpl.
2691       assert (Hconst : gamma_subst_tel (t_fvar (sort_of 1) (retag x
2692       (sort_of 1))) B x (index_of s - 1)
2693       = t_fvar (sort_of 1) (retag x (sort_of 1))).
2694       { apply gamma_subst_tel_fvar_const.
2695         intros m Hm2 Hm3 Heq2.
2696         apply retag_inj in Heq2 as [_ Hsc].
2697         apply (sort_of_1_ne_sort_of_m m Hm2 ltac:(lia)).
2698         symmetry. exact Hsc. }
2699       rewrite Hconst. simpl.
2700       destruct (eq_var_dec (retag x (sort_of 1)) (retag x (sort_of
2701       1))) as [_ | Hne2].

```

```

2698 * reflexivity.
2699 * ex falso. apply Hne2. reflexivity.
2700 + simpl.
2701 assert (Hconst : gamma_subst_tel (t_fvar (sort_of 1) (retag y
    (sort_of 1))) B x (index_of s - 1)
    = t_fvar (sort_of 1) (retag y (sort_of 1))).
2702 { apply gamma_subst_tel_fvar_const.
2703   intros m Hm2 Hm3 Heq2.
2704   apply retag_inj in Heq2 as [Hxeq _]. apply Hneq. symmetry.
2705   exact Hxeq. }
2706 rewrite Hconst. simpl.
2707 destruct (eq_var_dec (retag y (sort_of 1)) (retag x (sort_of
    1))) as [Heq2 | Hneq2].
2708 * ex falso. apply retag_inj in Heq2 as [Hc _]. apply Hneq. exact
    Hc.
2709 * reflexivity.
2710
2711 - (* A = t_app D C *)
2712 simpl in HonlyA. destruct HonlyA as [HonlyD HonlyC].
2713 assert (HdegC : deg (C [ x = B ]) = deg C)
2714   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2715 rewrite subst_app. simpl. rewrite HdegC.
2716 rewrite gamma_subst_tel_app_commute, subst_app.
2717 rewrite (gamma_app_tel_commutes_subst (gamma D) C B x s (deg C -
    1) Hxz Hxrange HdegB HlcB HonlyC).
2718 rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB
    HlcB).
2719 reflexivity.
2720
2721 - (* A = t_lam s1 C D *)
2722 simpl in HonlyA. destruct HonlyA as [HonlyC HonlyD].
2723 assert (HdegC : deg (C [ x = B ]) = deg C)
2724   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2725 rewrite subst_lam. simpl. rewrite HdegC.
2726 rewrite gamma_subst_tel_app_commute, subst_app.
2727 rewrite gamma_subst_tel_lam_commute, subst_lam.
2728 rewrite (gamma_subst_sentinel_const (sort_of 2) zero_var B x s
    zero_var_rettag_ne).
2729 rewrite (gamma_lam_tel_commutes_subst C (gamma D) B x s (deg C -
    1) Hxz Hxrange HdegB HlcB HonlyC).
2730 rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB
    HlcB).
2731 reflexivity.
2732
2733 - (* A = t_pi s1 C D *)
2734 simpl in HonlyA. destruct HonlyA as [HonlyC HonlyD].
2735 assert (HdegC : deg (C [ x = B ]) = deg C)
2736   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2737 rewrite subst_pi. simpl. rewrite HdegC.

```



```

2738   rewrite gamma_subst_tel_app_commute, subst_app.
2739   rewrite gamma_subst_tel_app_commute, subst_app.
2740   rewrite gamma_subst_tel_app_commute, subst_app.
2741   rewrite (gamma_subst_sentinel_const (sort_of 1) prod_var B x s
prod_var_retagn).
2742   rewrite (gamma_pi_tel_commutes_subst C B x s (deg C - 1) Hxz
Hxrange HdegB HlcB HonlyC).
2743   rewrite (gamma_lam_tel_commutes_subst C (gamma D) B x s (deg C -
1) Hxz Hxrange HdegB HlcB HonlyC).
2744   rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB
HlcB).
2745   reflexivity.
2746 Qed.
2747
2748
2749 (* =====
*)
2750 (* gamma commutes with beta *)
2751
2752 Lemma gamma_pi_tel_dom_congr : forall C C' body k,
2753   (forall m, m <= k -> rho m C ->>b rho m C') ->
2754   gamma_pi_tel C body k ->>b gamma_pi_tel C' body k.
2755 Proof.
2756   induction k as [| k' IHk]; intros Hall; simpl.
2757   - apply brt_pi_A. apply Hall. lia.
2758   - apply brt_pi_congr.
2759     + apply Hall. lia.
2760     + apply IHk. intros m Hm. apply Hall. lia.
2761 Qed.
2762
2763 Lemma gamma_lam_tel_dom_congr : forall C C' body k,
2764   (forall m, m <= k -> rho m C ->>b rho m C') ->
2765   gamma_lam_tel C body k ->>b gamma_lam_tel C' body k.
2766 Proof.
2767   induction k as [| k' IHk]; intros Hall; simpl.
2768   - apply brt_lam_A. apply Hall. lia.
2769   - apply brt_lam_congr.
2770     + apply Hall. lia.
2771     + apply IHk. intros m Hm. apply Hall. lia.
2772 Qed.
2773
2774 Lemma gamma_lam_tel_body_congr : forall C body body' k,
2775   body ->>b body' -> gamma_lam_tel C body k ->>b gamma_lam_tel C body'
k.
2776 Proof.
2777   induction k as [| k' IHk]; intros Hb; simpl.
2778   - apply brt_lam_M. exact Hb.
2779   - apply brt_lam_M. apply IHk. exact Hb.
2780 Qed.

```



```

2781
2782 Lemma gamma_app_tel_func_congr : forall M M' N k,
2783   M ->>b M' -> gamma_app_tel M N k ->>b gamma_app_tel M' N k.
2784 Proof.
2785   intros M M' N k. revert M M'.
2786   induction k as [| k' IH]; intros M M' Hstep; simpl.
2787   - apply brt_app_l. exact Hstep.
2788   - apply IH. apply brt_app_l. exact Hstep.
2789 Qed.
2790
2791 Lemma gamma_app_tel_arg_congr : forall MG N N' k,
2792   (forall j, j <= k -> rho j N ->>b rho j N') ->
2793   gamma_app_tel MG N k ->>b gamma_app_tel MG N' k.
2794 Proof.
2795   intros MG N N' k. revert MG.
2796   induction k as [| k' IH]; intros MG Hall; simpl.
2797   - apply brt_app_r. apply Hall. lia.
2798   - apply brt_trans with (gamma_app_tel (t_app MG (rho (S k') N'))) N
2799     k').
2800     + apply gamma_app_tel_func_congr. apply brt_app_r. apply Hall.
2801       lia.
2802     + apply IH. intros j Hj. apply Hall. lia.
2803 Qed.
2804
2805 Axiom rho_commutates_beta_below : forall i, 0 <= i <= n ->
2806   forall M N, deg M < i + 1 -> M ->>b N -> rho i M ->>b rho i N.
2807
2808 Lemma rho_commutates_beta_total :
2809   forall i, 0 <= i <= n -> forall M N, M ->>b N -> rho i M ->>b rho i
2810   N.
2811 Proof.
2812   intros i Hi M N Hbeta.
2813   destruct (le_lt_dec (i + 1) (deg M)) as [E | E].
2814   - apply rho_commutates_beta; [lia | lia | exact Hbeta].
2815   - apply rho_commutates_beta_below; [lia | lia | exact Hbeta].
2816 Qed.
2817
2818 Lemma bt_pi_A : forall s A A' B, A ->>+b A' -> t_pi s A B ->>+b t_pi s
2819   A' B.
2820 Proof.
2821   intros s A A' B H.
2822   induction H as [A A' Hs | A A'' A' H1 IH1 H2 IH2].
2823   - apply bt_step. apply beta_pi_A. exact Hs.
2824   - apply bt_trans with (t_pi s A'' B); auto.
2825 Qed.
2826
2827 Lemma bt_pi_B : forall s A B B', B ->>+b B' -> t_pi s A B ->>+b t_pi s
2828   A B'.
2829 Proof.

```

```

2825   intros s A B B' H.
2826   induction H as [B B' Hs | B B'' B' H1 IH1 H2 IH2].
2827   - apply bt_step. apply beta_pi_B. exact Hs.
2828   - apply bt_trans with (t_pi s A B''); auto.
2829 Qed.
2830
2831 Lemma bt_lam_A : forall s A A' M, A ->+b A' -> t_lam s A M ->+b
t_lam s A' M.
2832 Proof.
2833   intros s A A' M H.
2834   induction H as [A A' Hs | A A'' A' H1 IH1 H2 IH2].
2835   - apply bt_step. apply beta_lam_A. exact Hs.
2836   - apply bt_trans with (t_lam s A'' M); auto.
2837 Qed.
2838
2839 Lemma bt_lam_M : forall s A M M', M ->+b M' -> t_lam s A M ->+b
t_lam s A M'.
2840 Proof.
2841   intros s A M M' H.
2842   induction H as [M M' Hs | M M'' M' H1 IH1 H2 IH2].
2843   - apply bt_step. apply beta_lam_M. exact Hs.
2844   - apply bt_trans with (t_lam s A M''); auto.
2845 Qed.
2846
2847 Lemma bt_app_l : forall M M' N, M ->+b M' -> t_app M N ->+b t_app M'
N.
2848 Proof.
2849   intros M M' N H.
2850   induction H as [M M' Hs | M M'' M' H1 IH1 H2 IH2].
2851   - apply bt_step. apply beta_app_l. exact Hs.
2852   - apply bt_trans with (t_app M'' N); auto.
2853 Qed.
2854
2855 Lemma bt_app_r : forall M N N', N ->+b N' -> t_app M N ->+b t_app M
N'.
2856 Proof.
2857   intros M N N' H.
2858   induction H as [N N' Hs | N N'' N' H1 IH1 H2 IH2].
2859   - apply bt_step. apply beta_app_r. exact Hs.
2860   - apply bt_trans with (t_app M N''); auto.
2861 Qed.
2862
2863 Lemma brt_bt_trans : forall M K N, M ->b K -> K ->+b N -> M ->+b N.
2864 Proof.
2865   intros M K N Hmk.
2866   induction Hmk as [M | M K Hs | M Q K H1 IH1 H2 IH2]; intros Hkn.
2867   - exact Hkn.
2868   - apply bt_trans with K; [apply bt_step; exact Hs | exact Hkn].
2869   - apply IH1. apply IH2. exact Hkn.

```

```

2870 Qed.
2871
2872 Lemma bt_brt_trans : forall M K N, K ->+b N -> M ->+b K -> M ->+b N.
2873 Proof.
2874   intros M K N Hkn.
2875   induction Hkn as [K | K N Hs | K Q N H1 IH1 H2 IH2]; intros Hmk.
2876   - exact Hmk.
2877   - apply bt_trans with K; [exact Hmk | apply bt_step; exact Hs].
2878   - apply IH2. apply IH1. exact Hmk.
2879 Qed.
2880
2881 Lemma bt_app_congr_L : forall M M' N N',
2882   M ->+b M' -> N ->+b N' -> t_app M N ->+b t_app M' N'.
2883 Proof.
2884   intros M M' N N' HM HN.
2885   apply (bt_brt_trans (t_app M N) (t_app M' N) (t_app M' N')).
2886   - apply brt_app_r. exact HN.
2887   - apply bt_app_l. exact HM.
2888 Qed.
2889
2890 Lemma bt_app_congr_R : forall M M' N N',
2891   M ->+b M' -> N ->+b N' -> t_app M N ->+b t_app M' N'.
2892 Proof.
2893   intros M M' N N' HM HN.
2894   apply (brt_bt_trans (t_app M N) (t_app M' N) (t_app M' N')).
2895   - apply brt_app_l. exact HM.
2896   - apply bt_app_r. exact HN.
2897 Qed.
2898
2899 Lemma gamma_app_tel_func_congr_plus : forall M M' N k,
2900   M ->+b M' -> gamma_app_tel M N k ->+b gamma_app_tel M' N k.
2901 Proof.
2902   intros M M' N k. revert M M'.
2903   induction k as [| k' IH]; intros M M' Hstep; simpl.
2904   - apply bt_app_l. exact Hstep.
2905   - apply IH. apply bt_app_l. exact Hstep.
2906 Qed.
2907
2908 Lemma gamma_lam_tel_body_congr_plus : forall C body body' k,
2909   body ->+b body' -> gamma_lam_tel C body k ->+b gamma_lam_tel C
    body' k.
2910 Proof.
2911   induction k as [| k' IHk]; intros Hb; simpl.
2912   - apply bt_lam_M. exact Hb.
2913   - apply bt_lam_M. apply IHk. exact Hb.
2914 Qed.
2915
2916 Axiom gamma_commutates_beta_base : forall s A M N,
2917   gamma (t_app (t_lam s A M) N) ->+b gamma (M ^^ N).
2918

```

```

2919 Lemma gamma_commutes_beta_aux :
2920   forall M N, M ->b N -> gamma M ->>+b gamma N.
2921 Proof.
2922   induction M as [s | s_m m | y | P IHP Q IHQ | s A IHA M' IHM' | s A
    IHA B IHB];
2923     intros N Hstep.
2924
2925   - (* t_sort *) inversion Hstep.
2926   - (* t_bvar *) inversion Hstep.
2927   - (* t_fvar *) inversion Hstep.
2928
2929   - (* t_app P Q *)
2930     inversion Hstep; subst; clear Hstep.
2931
2932   + (* beta_base *)
2933     apply gamma_commutes_beta_base.
2934
2935   + (* beta_app_l *)
2936     match goal with
2937       | Hs : P ->b ?P' | - _ =>
2938         simpl; apply bt_app_l; apply gamma_app_tel_func_congr_plus;
          apply IHP; exact Hs
2939     end.
2940
2941   + (* beta_app_r *)
2942     match goal with
2943       | Hs : Q ->b ?Q' | - _ =>
2944         assert (HdegQeq : deg Q = deg Q') by (apply deg_beta; apply
          brt_step; exact Hs);
2945         assert (HQle : deg Q <= n + 1) by (apply deg_le_top);
2946         simpl; rewrite <- HdegQeq;
2947         apply bt_app_congr_R;
2948         [ apply gamma_app_tel_arg_congr;
          intros j Hj; apply rho_commutes_beta_total; [lia | apply
          brt_step; exact Hs]
2949         | apply IHQ; exact Hs ]
2950     end.
2951
2952   - (* t_lam s A M' *)
2953     inversion Hstep; subst; clear Hstep.
2954
2955   + (* beta_lam_A *)
2956     match goal with
2957       | Hs : A ->b ?A1' | - _ =>
2958         assert (HdegAeq : deg A = deg A1') by (apply deg_beta; apply
          brt_step; exact Hs);
2959         assert (HAle : deg A <= n + 1) by (apply deg_le_top);
2960         simpl; rewrite <- HdegAeq;
2961         apply bt_app_congr_R;
2962

```

```

2963     [ apply brt_lam_M; apply gamma_lam_tel_dom_congr;
2964       intros mtier Hmtier; apply rho_commutes_beta_total; [lia |
        apply brt_step; exact Hs]
2965     | apply IHA; exact Hs ]
2966   end.
2967
2968 + (* beta_lam_M *)
2969   match goal with
2970   | Hs : M' ->b ?M1' | - _ =>
2971     simpl; apply bt_app_l; apply bt_lam_M; apply
      gamma_lam_tel_body_congr_plus;
2972     apply IHM'; exact Hs
2973   end.
2974
2975 - (* t_pi s A B *)
2976   inversion Hstep; subst; clear Hstep.
2977
2978 + (* beta_pi_A *)
2979   match goal with
2980   | Hs : A ->b ?A1' | - _ =>
2981     assert (HdegAeq : deg A = deg A1') by (apply deg_beta; apply
      brt_step; exact Hs);
2982     assert (HAle : deg A <= n + 1) by (apply deg_le_top);
2983     simpl; rewrite <- HdegAeq;
2984     apply bt_app_congr_L;
2985     [ apply bt_app_congr_R;
2986       [ apply brt_app_r; apply gamma_pi_tel_dom_congr;
          intros mtier Hmtier; apply rho_commutes_beta_total; [lia |
            apply brt_step; exact Hs]
2987       | apply IHA; exact Hs ]
2988     | apply gamma_lam_tel_dom_congr;
2989       intros mtier Hmtier; apply rho_commutes_beta_total; [lia |
        apply brt_step; exact Hs] ]
2990   end.
2991
2992 + (* beta_pi_B *)
2993   match goal with
2994   | Hs : B ->b ?B1' | - _ =>
2995     simpl; apply bt_app_r; apply gamma_lam_tel_body_congr_plus;
2996     apply IHB; exact Hs
2997   end.
2998 Qed.
2999
3000 Lemma gamma_commutes_beta :
3001   forall M N, M ->>b N -> gamma M ->>b gamma N.
3002 Proof.
3003   intros M N Hbeta.
3004   induction Hbeta as [M | M N Hstep | M N P Hstep1 IH1 Hstep2 IH2].
3005   - apply brt_refl.

```

```

3006   - apply brt_from_trans. apply (gamma_commutes_beta_aux M N Hstep).
3007   - apply brt_trans with (gamma N); auto.
3008 Qed.
3009
3010 (* =====
3011 *)
3012 (** Main Theorem:  $\lambda S$  is strongly normalizing iff  $\lambda S^*$  is *)
3013
3014 Lemma bt_forward_acc : forall X Y, X ->+b Y ->
3015   (forall Z, X ->b Z -> Acc (fun N M => M ->+b N) Z) ->
3016   Acc (fun N M => M ->+b N) Y.
3017 Proof.
3018   intros X Y HXY.
3019   induction HXY as [X Y Hstep | X Z Y H1 IH1 H2 IH2]; intros Hbase.
3020   - apply Hbase. exact Hstep.
3021   - apply IH2.
3022     intros Z0 HstepZ.
3023     destruct (IH1 Hbase) as [f].
3024     apply f. apply bt_step. exact HstepZ.
3025 Qed.
3026
3027 Lemma Acc_atomic_implies_Acc_plus : forall X,
3028   strongly_normalizing X -> Acc (fun N M => M ->+b N) X.
3029 Proof.
3030   intros X HX.
3031   induction HX as [X _ IH].
3032   constructor.
3033   intros Y HXY.
3034   exact (bt_forward_acc X Y HXY IH).
3035 Qed.
3036
3037 Lemma sn_reflection_core :
3038   forall b, Acc (fun N M => M ->+b N) b ->
3039   forall M, b = gamma M -> strongly_normalizing M.
3040 Proof.
3041   intros b Hacc.
3042   induction Hacc as [b _ IH].
3043   intros M Heq.
3044   constructor.
3045   intros M' HstepM.
3046   apply (IH (gamma M')).
3047   - rewrite Heq. apply gamma_commutes_beta_aux. exact HstepM.
3048   - reflexivity.
3049 Qed.
3050
3051 Lemma sn_reflection : forall M,
3052   strongly_normalizing (gamma M) -> strongly_normalizing M.
3053 Proof.
3054   intros M Hsn.

```

```

3054   exact (sn_reflection_core (gamma M) (Acc_atomic_implies_Acc_plus
      (gamma M) Hsn) M eq_refl).
3055 Qed.
3056
3057 Lemma gamma_commutes_typing_derivable : forall M,
3058   derivable M -> derivable_star (gamma M).
3059 Proof.
3060   intros M [ $\Gamma$  [N HMN]].
3061   exists ([ (zero_var, (sort_of 2, t_sort (sort_of 1)));
3062           (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
3063           (prod_var, (sort_of 1, T_prod))] ++ rho_ctx 0  $\Gamma$ ), (rho 0
      N).
3064   exact (gamma_commutes_typing  $\Gamma$  M N HMN).
3065 Qed.
3066
3067 Axiom derivable_star_implies_derivable : forall M,
3068   derivable_star M -> derivable M.
3069
3070 Definition system_strongly_normalizing (P : term -> Prop) : Prop :=
3071   forall M, P M -> strongly_normalizing M.
3072
3073 Theorem lambdaS_SN_iff_lambdaS_star_SN :
3074   system_strongly_normalizing derivable <->
3075   system_strongly_normalizing derivable_star.
3076 Proof.
3077   split.
3078
3079   - (*  $\lambda S$  SN ->  $\lambda S^*$  SN *)
3080     intros HSN M HMstar.
3081     apply HSN.
3082     apply derivable_star_implies_derivable.
3083     exact HMstar.
3084
3085   - (*  $\lambda S^*$  SN ->  $\lambda S$  SN *)
3086     intros HSNstar M HM.
3087     apply sn_reflection.
3088     apply HSNstar.
3089     apply gamma_commutes_typing_derivable.
3090     exact HM.
3091 Qed.
3092 End TPTS.

```

Abstract

In this thesis, the dependency-erasure translation introduced by Nathan Mull for Tiered Pure Type Systems [10] is fully formalized and mechanically verified in the proof assistant Coq. The translation functions, together with all intermediate lemmas—including the substitution lemma, preservation of reduction paths, and lemmas related to injectivity and image—are proved in a fully machine-checked manner. The main theorem of Mull, stating that weak normalization implies strong normalization for all tiered pure type systems with specific rules, is mechanically verified for the first time.

As a result, an open and extensible library is developed, providing a solid foundation for extending this translation to more expressive dependent systems such as the Calculus of Constructions. The results of this work constitute a significant step toward the proof of the BGK conjecture¹ [7] in dependent type systems.

Keywords: type theory, tiered pure type systems, dependency-erasure translation, strong normalization, mechanized proof, Coq, BGK conjecture

¹Barendregt–Geuvers–Klop



University of Tehran
College of Science
School of Mathematics, Statistics, and Computer Science

Formalization of the Dependency-Eliminating Translation in Tiered Pure Type Systems

By
Hossein Madadi

Supervisor
Majid Alizadeh

A thesis submitted to graduate studies office in partial fulfillment of the
requirements for the degree of master of science in Computer Science

Summer 2026